

AI 제조 기술 관련 교육

『W2:』 실습

2025.12.08-2025.12.19(10일, 70시간)

Prepared by Daekyeong Kim

Ph.D.

교육 시간표

시간	1일차		2일차		3일차		4일차		5일차		Total
1교시	『1과목』 PRD 작성 Part 1. 오리엔테이션 및 목표 Part 2. Cursor.AI 사용 가이드		『2과목』 클린코드 + 테스트 기반 구현 Part 1. 오리엔테이션 및 목표 Part 2. 클린코드 핵심 원칙	『3과목』 최소구현 + 리뷰 Part 1. 오리엔테이션 Part 2. GREEN, Review Cursor AI 프롬프트 Part 3. SW Implementation	『4과목』 리팩토링 + 반복 Part 1. 오리엔테이션 및 목표 Part 2. 리팩토링 이론	『5과목』 Individual Prompt Project Part 1. 오리엔테이션 Part 2. 7단계 전체 Part 3. Toy-project 예시 ChatGPT 음성 기반 영어 학습 챗봇 만들기 Part 4. Toy-project 예시 “PE와 Cussor.AI 시스템 연계” 실습 워크숍					
	실습1 : AI 개발 환경 구축 (Hands-on Practice)										
2교시	Part 3. 효과적인 워크플로우 Part 4. 프로젝트 관리 및 팀 협업		실습 1: Clean Code 실무 예시	실습 1: HTML 예제 만들어 보기	실습 3의 법칙(The Rule of Three) 실습		• 개별 프롬프트 프로젝트 실습				
3교시	Part 5. 문제 해결 및 최적화 Part 6. AI-assisted TDD		Part 3. 테스트 기반 개발 TDD-Ex1 숫자 더하기 계산기	Part 4. GREEN 단계의 최소 구현	Part 2. 리팩토링 이론						
4교시	실습2 : 캡스톤 미니 프로젝트: 간단한 로그 분석기 만들기		• 실습2 : 파이썬 테스트 코드 작성해보기 • 실습3 : 파이썬 테스트 코드 나눠서 테스트 하기 • 실습4 : TDD-문자열 계산기	실습2 : 결제 시스템 구현	실습 Refactoring_practice						
5교시	Part 7. 아이디어 구체화		Project: Full-Stack TDD +RED+ AI Assist Cursor AI 개발	Project: Full-Stack GREEN + Review Cursor AI 개발	실습1 : Clean Code + UnitTest + 리팩토링 실습2 : 정적 분석 도구 flake8과 pylint 실습3 : GitHub Action						
6교시	실습3, 실습4, 실습5, 실습6, 실습7, 실습8, 실습9문제1, 문제2				Project: Full-Stack Refactor + Loop Cursor AI 개발						
7교시	Part 8. PRD 작성 실습 실습 10 Project: Full-Stack PRD 개발						Part 4. 프로젝트 발표 및 피드백 학습 평가 및 수료증 수여				
Total Time		7	7	7	7	7	7	7	7	35	

『1과목』 PRD 작성



문제

1. 실습 1: Cursor 설정 최적화

- Cursor 설치 및 초기 설정
- 개인화된 Cursor Rules 생성
- 단축키 및 설정 커스터마이징

2. 실습 2: AI 프로젝트 구조 설계

- 프로젝트 요구사항 분석
- 모듈화된 구조 설계
- Cursor를 활용한 코드 생성

3. 실습 3: 협업 환경 구축

- Git 연동 설정
- 팀 규칙 설정
- 코드 리뷰 프로세스

작업 요약

입력 명령	Cursor가 자동 수행하는 작업
tdd start	요구사항 → 테스트 명세 변환 (RED)
make tests	실패하는 테스트 생성
minimal implementation	테스트 통과용 최소 코드 작성 (GREEN)
refactor	테스트 유지하며 리팩토링
review	코드 리뷰 + 개선점 자동 제안
add feature	다음 기능 TDD 사이클로 이동

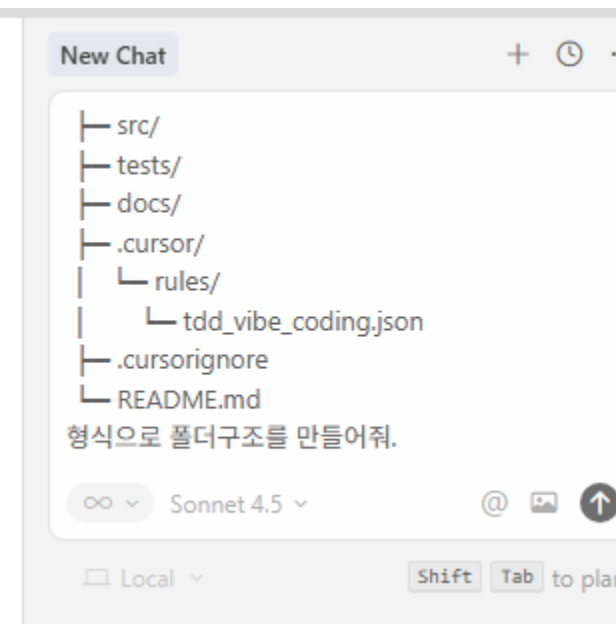
답

Cursor Rule: AI-Assisted TDD + Vibe Coding Workflow

아래 내용을 그대로 .cursor/rules/tdd_vibe_coding.json 파일로 저장하면 됩니다.

C:\DEV\SamsungDisplay\Week_2\Cursor_pro\CursorRule /

```
├── src/
├── tests/
├── docs/
├── .cursor/
│   └── rules/
│       └── tdd_vibe_coding.json
├── .cursorignore
└── README.md
```



실습1 : AI 개발 환경 구축 (Hands-on Practice)

답

빈 디렉토리네요. 요청하신 폴더 구조를 만들어드리겠습니다.

Explored 1 directory

Run command: mkdir

```
$ mkdir src, tests, docs, .cursor\rules
```

Skip Allowlist Run

Agents Editor File Edit Selection View Go Run ...

CURSORRULE

- .cursor\rules
 - tdd_vibe_coding.json
 - docs
 - src
 - tests
 - .cursorignore
 - README.md

tdd_vibe_coding.json

```
.cursor > rules > {} tdd_vibe_coding.json > [ ] rules
1 {
2   "name": "TDD Vibe Coding",
3   "description": "Test-Driven Development rules and guidelines",
4   "priority": 10,
5   "rules": [
6     {
7       "pattern": "^(?i)(tdd start|start tdd)",
8       "action": "StartTDD"
9     },
10    {
11      "pattern": "^(?i)(make tests|generate tests|write tests)",
12      "action": "GenerateTests"
13    },
14    {
15      "pattern": "^(?i)(minimal implementation|green|pass tests)",
16      "action": "GreenStage"
17    },
18    {
19      "pattern": "^(?i)(refactor|clean code|improve structure)",
20      "action": "RefactorCode"
21    },
22    {
23      "pattern": "^(?i)(review|check code|audit)",
24      "action": "ReviewCode"
25    },
26    {
27      "pattern": "^(?i)(add feature|next feature|continue)",
28      "action": "NextLoop"
29    }
30  ],
31  "actions": {
32    "StartTDD": {
33      "explanation": "Convert the requirement into TDD-ready test specifications.",
34      "prompt": [
35        "You are an AI TDD assistant. Convert the user's requirements into structured TDD test specifications.",
36        "Break down the feature into testable units.",
37        "Include edge cases, failure cases, and behavior-driven scenarios.",
38        "Output format: TEST SPEC + PRIORITY (MoSCoW).",
39      ]
40    },
41    "GenerateTests": {
42      "explanation": "Generate failing test cases using the specification.",
43      "prompt": [
```

폴더 구조 생성 요청

- src/
- tests/
- docs/

프로젝트 제목

프로젝트 설명을 여기에 작성하세요.

폴더 구조

Thought for 1s

완료되었습니다! 요청하신 폴더 구조가 성공적으로 생성되었습니다.

Ran command: tree

```
$ tree /F /A
PATH.
BED0-C858-1B0.
C:.
| .cursorignore
| README.md
|
+---.cursor
| | \---rules
| |       tdd_vibe_coding.json
|
+---docs
+---src
\---tests
```

Thought for 1s

완료되었습니다! 다음 구조로 프로젝트 폴더를 만
었습니다.

- src/ (소스 코드 폴더)
- tests/ (테스트 폴더)
- docs/ (문서 폴더)
- .cursor/
 - rules/
- .cursorignore (Cursor ignore 설정 파
- README.md (프로젝트 설명 파일)

실습1 : AI 개발 환경 구축 (Hands-on Practice)

```
{
  "name": "AI_TDD_VibeCoding",
  "description": "AI-assisted TDD + Vibe Coding workflow automation for Cursor",
  "priority": 10,
  "rules": [
    {
      "pattern": "^(?i)(tdd start|start tdd)",
      "action": "StartTDD"
    },
    {
      "pattern": "^(?i)(make tests|generate tests|write tests)",
      "action": "GenerateTests"
    },
    {
      "pattern": "^(?i)(minimal implementation|green|pass tests)",
      "action": "GreenStage"
    },
    {
      "pattern": "^(?i)(refactor|clean code|improve structure)",
      "action": "RefactorCode"
    },
    {
      "pattern": "^(?i)(review|check code|audit)",
      "action": "ReviewCode"
    },
    {
      "pattern": "^(?i)(add feature|next feature|continue)",
      "action": "NextLoop"
    }
  ]
},
```

실습1 : AI 개발 환경 구축 (Hands-on Practice)

```
"actions": {
  "StartTDD": {
    "explanation": "Convert the requirement into TDD-ready test specifications.",
    "prompt": [
      "You are an AI TDD assistant. Convert the user's requirements into structured TDD test specifications.",
      "Break down the feature into testable units.",
      "Include edge cases, failure cases, and behavior-driven scenarios.",
      "Output format: TEST SPEC + PRIORITY (MoSCoW).",
    ]
  },
  "GenerateTests": {
    "explanation": "Generate failing test cases using the specification.",
    "prompt": [
      "Generate failing tests according to the test specifications.",
      "Tests MUST fail initially. Do not write implementation.",
      "Follow the testing framework of the existing project.",
      "Ensure full coverage including edge cases."
    ]
  },
  "GreenStage": {
    "explanation": "Write minimum code to pass all tests.",
    "prompt": [
      "Write ONLY the minimal implementation required to make all current tests pass.",
      "Do NOT over-engineer.",
      "No refactoring at this stage."
    ]
  },
}
```

실습1 : AI 개발 환경 구축 (Hands-on Practice)

```
"RefactorCode": {
  "explanation": "Refactor the implementation while keeping tests green.",
  "prompt": [
    "Refactor the existing implementation.",
    "Improve structure, naming, modularization, reducing duplication, and apply SOLID where useful.",
    "All existing tests must remain green.",
    "If necessary, generate improved helper methods."
  ]
},
"ReviewCode": {
  "explanation": "AI code review for quality improvement.",
  "prompt": [
    "Perform a code review focusing on:",
    "- duplicated logic",
    "- naming issues",
    "- readability",
    "- missing edge cases",
    "- performance bottlenecks",
    "- security vulnerabilities",
    "Then propose recommended improvements."
  ]
},
"NextLoop": {
  "explanation": "Repeat TDD cycle for the next feature.",
  "prompt": [
    "Identify the next feature from the project context.",
    "Generate test specifications (RED).",
    "Then ask the user whether to generate failing tests now."
  ]
}
}
```

문제

구축한 환경을 활용하여, 앞으로 진행할 최종 캡스톤 프로젝트의 기반을 다지는 작은 실습을 진행해봅시다. 이번 미니 프로젝트의 목표는 여러 줄의 로그(log) 텍스트 파일을 읽고, 분석하여, 의미 있는 요약 정보를 출력하는 간단한 프로그램을 만드는 것입니다.

프로젝트 목표

주어진 형식의 로그 파일을 분석하여 다음과 같은 요약 리포트를 출력하는 Python 스크립트(log_analyzer.py)를 완성합니다.

샘플 로그 파일 (sample.log):

```
2023-10-27 10:00:00 INFO: User 'alice' logged in.  
2023-10-27 10:01:30 WARNING: Disk space is running low.  
2023-10-27 10:02:15 INFO: Data processing started.  
2023-10-27 10:05:00 ERROR: Failed to connect to database.  
2023-10-27 10:05:05 INFO: User 'bob' logged in.  
2023-10-27 10:06:00 ERROR: File not found: 'data.csv'.
```

예상 출력 결과:

Log Analysis Report

Total Logs: 6

Log Levels Count:

- INFO: 3
- WARNING: 1
- ERROR: 2

답 Capstone Mini Project – TDD 기반 Log Analyzer 개발 가이드

● TDD 기반 Capstone Mini Project: Log Analyzer 개발 절차

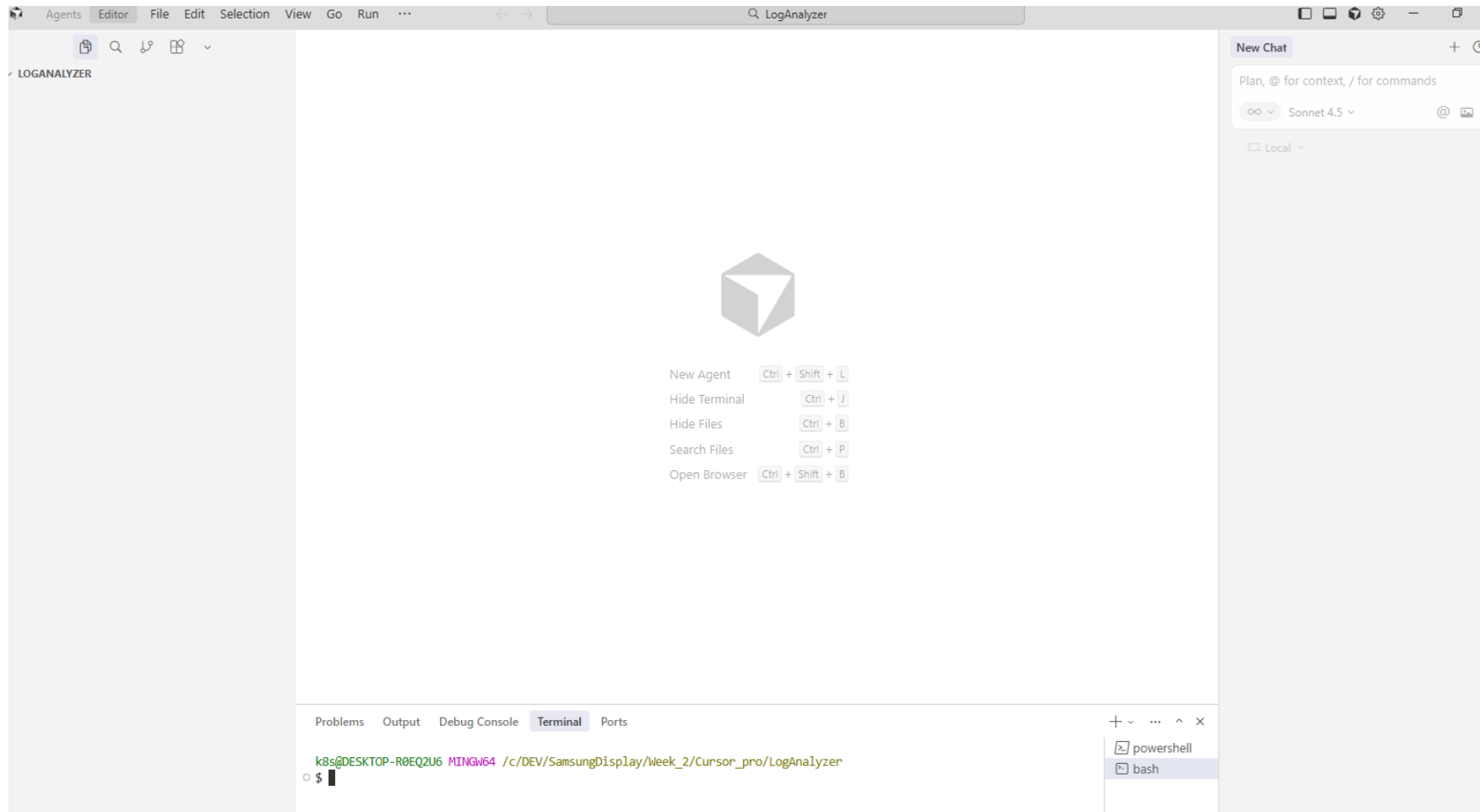
- 목표
 - log_analyzer.py를 TDD 방식으로 개발하여 로그 파일에서
 - 전체 로그 수
 - 로그 레벨별 카운트(INFO, WARNING, ERROR)를 정확하게 분석하고 출력하는 프로그램 제작.

● 프로젝트 진행 순서 전체 구조 (TDD + Cursor AI)

1. 요구사항 정리 → 2. 테스트 작성(RED) → 3. 최소 구현(GREEN) → 4. 리팩토링(마무리) → 5. 기능 확장(필요시)

실습2 : 캡스톤 미니 프로젝트: 간단한 로그 분석기 만들기

C:\DEV\SamsungDisplay\Week_2\Cursor_pro\LogAnalyzer



■ STEP 1. 요구사항 정리 (Planning)

Cursor에서 아래와 같이 먼저 입력하여 요구사항을 TDD-ready 사양으로 변환합니다.

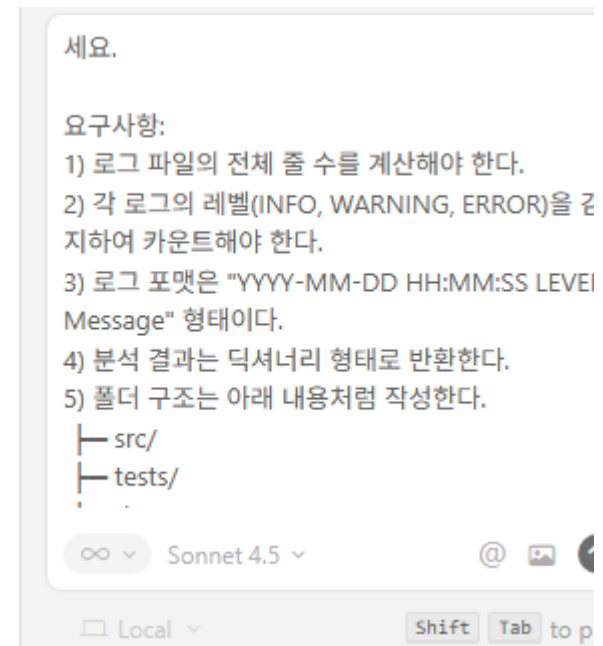
Cursor Prompt

아래 로그 분석 요구사항을 테스트 단위로 분해해주세요.

요구사항:

- 1) 로그 파일의 전체 줄 수를 계산해야 한다.
- 2) 각 로그의 레벨(INFO, WARNING, ERROR)을 감지하여 카운트해야 한다.
- 3) 로그 포맷은 "YYYY-MM-DD HH:MM:SS LEVEL: Message" 형태이다.
- 4) 분석 결과는 딕셔너리 형태로 반환한다.
- 5) 폴더 구조는 아래 내용처럼 작성한다.

```
├── src/
├── tests/
├── docs/
├── .cursor/
│   └── rules/
│       └── tdd_vibe_coding.json
├── .cursorignore
└── README.md
```



실습2 : 캡스톤 미니 프로젝트: 간단한 로그 분석기 만들기

Cursor는 아래와 같은 테스트 사양을 자동 생성하게 됨

test_total_log_count
test_log_level_count
test_invalid_format(확장 가능)
test_empty_file(확장 가능)

The screenshot displays the Cursor IDE interface for a project named 'LogAnalyzer'. The left sidebar shows the project structure with folders like 'docs', 'src', and 'tests'. The main editor window shows a JSON file named 'tdd_vibe_coding.json' containing a list of rules for TDD development. The rules include instructions on how to write tests, refactor code, and use the AAA pattern (Arrange-Act-Assert). The right sidebar shows a 'TDD 개발 순서' (TDD Development Order) section with four phases: Phase 1 - 핵심 기능부터 시작, Phase 2 - 통합 테스트, Phase 3 - 엡지 케이스 처리, and Phase 4 - 예외 처리. Below the phases, there is a note about the AAA pattern and a link to the 'docs/test_cases.md' file.

```
{
  "rules": [
    {
      "name": "TDD 사이클 준수",
      "description": "Red-Green-Refactor 사이클을 엄격히 따릅니다.",
      "steps": [
        "1. Red: 실패하는 테스트를 먼저 작성",
        "2. Green: 테스트를 통과하는 최소한의 코드 작성",
        "3. Refactor: 코드 개선 및 리팩토링"
      ]
    },
    {
      "name": "테스트 우선 작성",
      "description": "구현 코드보다 테스트 코드를 먼저 작성합니다."
    },
    {
      "name": "작은 단위로 개발",
      "description": "한 번에 하나의 기능만 구현하고 테스트합니다."
    },
    {
      "name": "테스트는 독립적",
      "description": "각 테스트는 서로 독립적이어야 하며, 실행 순서에 영향을 받지 않아야 합니다."
    },
    {
      "name": "명확한 테스트 이름",
      "description": "테스트 함수명은 테스트 목적을 명확히 나타내야 합니다. (예: test_count_info_logs)"
    },
    {
      "name": "AAA 패턴 사용",
      "description": "Arrange(준비) - Act(실행) - Assert(검증) 패턴으로 테스트를 작성합니다."
    },
    {
      "name": "최소한의 구현",
      "description": "테스트를 통과하는 최소한의 코드만 작성합니다. 필요하지 않은 기능은 추가하지 않습니다."
    }
  ]
}
```

TDD 개발 순서

- Phase 1 - 핵심 기능부터 시작
- Phase 2 - 통합 테스트
- Phase 3 - 엡지 케이스 처리
- Phase 4 - 예외 처리

각 테스트는 AAA(Arrange-Act-Assert) 패턴으로 작성되어 있으며, TDD Red-Green-Refactor 사이클을 따를 수 있도록 구조화했습니다.

[docs/test_cases.md](#) 파일에서 각 테스트의 상세 명세를 확인하실 수 있습니다!

STEP 2. 먼저 실패하는 테스트 작성 (RED)

Cursor Prompt

위 테스트 사양을 기반으로 pytest 테스트 코드를 작성해주세요.
테스트는 반드시 실패해야 합니다.
테스트 파일 이름: test_log_analyzer.py

예시로 생성될 테스트 코드(요약)

```
from log_analyzer import analyze_logs

def test_total_logs():
    logs = [
        "2023-10-27 10:00:00 INFO: User 'alice' logged in.",
        "2023-10-27 10:01:30 WARNING: Disk space is running low."
    ]
    result = analyze_logs(logs)
    assert result["total_logs"] == 2

def test_log_level_counts():
    logs = [
        "2023-10-27 10:00:00 INFO: User 'alice' logged in.",
        "2023-10-27 10:01:30 WARNING: Disk space is running low.",
        "2023-10-27 10:05:00 ERROR: Failed to connect to database."
    ]
    result = analyze_logs(logs)
    assert result["levels"]["INFO"] == 1
    assert result["levels"]["WARNING"] == 1
    assert result["levels"]["ERROR"] == 1
```

함.

위 테스트 사양을 기반으로 pytest 테스트 코드를
성해주세요.

테스트는 반드시 실패해야 합니다.

테스트 파일 이름: test_log_analyzer.py

실습2 : 캡스톤 미니 프로젝트: 간단한 로그 분석기 만들기

→ 이 시점에서는 `analyze_logs()`가 없기 때문에 테스트는 반드시 실패함.

```
Run command: pytest

$ pytest tests/
test_log_analyzer.py -v
```

Skip Allowlist Run

```
Ran command: pytest

$ pytest tests/
test_log_analyzer.py -v
pytest : 'pytest' cmdlet,
[Error]
C:\Users\k8s\AppData\Local\Temp\ps
-script-ee4ff692-96b5-4e89-9e7d-f1bb84fe
c0c4.ps1:7
+ pytest tests/test_log_analyzer.py -v
+ ~~~~~
+ CategoryInfo          : ObjectNotF
ound: (pytest:String) [], CommandNotFoun
dException
+ FullyQualifiedErrorId : CommandNot
FoundException

Thought for 2s
pytest가 설치되어 있지 않네요. 먼저 Python과
pytest를 설치하겠습니다.
```

```
Ran command: python

$ python --version

Python 3.13.7

Thought for 1s

Run command: pip install

$ pip install -r requirements.
txt
```

Skip Allowlist Run

```
Run command: python

$ python -m pytest tests/
test_log_analyzer.py -v
```

Skip Allowlist Run

실습2 : 캡스톤 미니 프로젝트: 간단한 로그 분석기 만들기

→ 이 시점에서는 `analyze_logs()`가 없기 때문에 테스트는 반드시 실패함.

좋습니다! pytest가 설치되었습니다. 이제 테스트를 실행해서 실패하는지 확인하겠습니다.

Ran command: python

```
$ python -m pytest tests/test_log_analyzer.py -v

===== test session starts =====
platform win32 -- Python 3.13.7, pytest-7.4.3, pluggy-1.6.0 -- C:\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\SamsungDisplay\Week_2\Cursor_pro\LogAnalyzer
plugins: anyio-4.11.0, cov-4.1.0
collecting ... collected 12 items

tests/test_log_analyzer.py::TestBasicFunctionality::test_empty_file FAILED [ 8%]
tests/test_log_analyzer.py::TestBasicFunctionality::test_count_total_lines FAILED [ 16%]
tests/test_log_analyzer.py::TestBasicFunctionality::test_return_dict_structure FAILED [ 25%]
tests/test_log_analyzer.py::TestBasicFunctionality::test_count_info_logs FAILED [ 33%]
```

실패한 테스트 (11개)

1. 기본 기능 테스트 (3개 실패)

- ✗ `test_empty_file` - TypeError: 'NoneType' object is not subscriptable
- ✗ `test_count_total_lines` - TypeError: 'NoneType' object is not subscriptable
- ✗ `test_return_dict_structure` - TypeError: argument of type 'NoneType' is not iterable

2. 로그 레벨 감지 테스트 (4개 실패)

- ✗ `test_count_info_logs` - TypeError: 'NoneType' object is not subscriptable

■ STEP 3. 최소한의 코드 작성하여 테스트 통과 (GREEN)

Cursor Prompt

test_log_analyzer.py의 모든 테스트가 통과하도록
최소한의 구현만 포함하는 log_analyzer.py를 생성해주세요.
리팩토링은 하지 마세요.

✓ 최소 구현 예시

```
def analyze_logs(log_lines):  
    total = len(log_lines)  
    levels = {"INFO": 0, "WARNING": 0, "ERROR": 0}  
  
    for line in log_lines:  
        if "INFO:" in line:  
            levels["INFO"] += 1  
        elif "WARNING:" in line:  
            levels["WARNING"] += 1  
        elif "ERROR:" in line:  
            levels["ERROR"] += 1  
  
    return {  
        "total_logs": total,  
        "levels": levels  
    }
```

→ 테스트 모두 GREEN.

test_log_analyzer.py의 모든 테스트가 통과하도록
최소한의 구현만 포함하는 log_analyzer.py를 생성해주세요.
리팩토링은 하지 마세요.

∞ Agent Sonnet 4.5



실습2 : 캡스톤 미니 프로젝트: 간단한 로그 분석기 만들기

→ 테스트 모두 GREEN.

The screenshot displays a code editor with the `log_analyzer.py` file open. The file defines a `LogAnalyzer` class with an `analyze` method. The `analyze` method reads a log file, counts the total lines, and counts the occurrences of INFO, WARNING, and ERROR levels. It also raises `FileNotFoundError` if the file does not exist and `PermissionError` if the file cannot be read.

```
10 class LogAnalyzer:
11     def __init__(self, log_file_path):
12         Args:
13             log_file_path: 분석할 로그 파일 경로
14         """
15         self.log_file_path = log_file_path
16
17     def analyze(self):
18         """
19         로그 파일을 분석하여 통계를 반환
20
21         Returns:
22             dict: 로그 분석 결과
23             - total_lines: 전체 줄 수
24             - INFO: INFO 레벨 로그 개수
25             - WARNING: WARNING 레벨 로그 개수
26             - ERROR: ERROR 레벨 로그 개수
27
28         Raises:
29             FileNotFoundError: 파일이 존재하지 않을 때
30             PermissionError: 파일 읽기 권한이 없을 때
31         """
32         try:
33             with open(self.log_file_path, 'r', encoding='utf-8') as f:
34                 lines = f.readlines()
35         except FileNotFoundError:
36             raise FileNotFoundError(f"파일을 찾을 수 없습니다: {self.log_file_path}")
37         except PermissionError:
38             raise PermissionError(f"파일을 읽을 권한이 없습니다: {self.log_file_path}")
39
40         total_lines = len(lines)
41         info_count = 0
42         warning_count = 0
43         error_count = 0
```

The right sidebar shows the test results for `test_log_analyzer.py`. It indicates that 11 tests passed and 1 test was skipped. The test results are as follows:

- 코드 커버리지: 93%
- 커버된 코드: 28줄 중 26줄
- 누락된 코드: 2줄 (42-43번 라인 - `PermissionError` 처리 부분)

The TDD status is shown as follows:

- Red-Green 사이클 완료
- 1. Red 단계 - 실패하는 테스트 작성 ☒
- 2. Green 단계 - 최소 구현으로 테스트 통과 ☒
- 3. Refactor 단계 - 요청에 따라 생략

The terminal shows the command `python test_log_analyzer.py` being executed, and the output shows the test results.

STEP 4. 리팩토링 (Refactor)

이제 테스트가 보호막 역할을 하므로 코드를 더 깨끗하게 개선할 수 있습니다.

Cursor Prompt

현재 log_analyzer.py를 리팩토링해 주세요.

조건:

- 모든 테스트는 유지
- 중복 제거
- 더 구조적인 코드로 개선

현재 log_analyzer.py를 리팩토링해 주세요.

조건:

- 모든 테스트는 유지
- 중복 제거
- 더 구조적인 코드로 개선

✓ 리팩토링 예시

import re

```
LOG_PATTERN = re.compile(r"\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2} (\w+):")
```

```
def analyze_logs(log_lines):  
    levels = {"INFO": 0, "WARNING": 0, "ERROR": 0}
```

```
    for line in log_lines:  
        match = LOG_PATTERN.search(line)  
        if match:  
            level = match.group(1)  
            if level in levels:  
                levels[level] += 1
```

```
    return {  
        "total_logs": len(log_lines),  
        "levels": levels  
    }
```

→ 구조적이고 유지보수 용이함.

🌀 STEP 5. 기능 확장 (Optional)

추가 테스트 → 추가 기능 확장하는 TDD 루프.

예시 확장 기능:

- 가장 많이 발생한 로그 레벨 찾기
- ERROR 로그만 필터링
- 시간대별 로그 요약
- 유효하지 않은 로그 포맷 카운트
- 파일 입출력 지원

문제

AI에게 브랜드의 특성을 설명하고, 이에 적합한 타겟 고객층과 페르소나를 생성하도록 요청합니다.

답

주식회사 케이지네트웍의 브랜드는 iBestie, iDay와 같은 생활밀착형 소확행 앱과 Erudite.AI라는 HTR-Rag AI 에이전트입니다. 이브랜드에 적합한 주요 타겟 고객층을 3가지 정도 제안해 주세요. 그리고 각 고객층별로 상세한 페르소나도 함께 만들어 줄 수 있을까요? 페르소나에는 나이, 직업, 라이프스타일, 가치관, 소비습관, 브랜드에 대한 니즈 등이 포함되어야 합니다.

주식회사 케이지네트웍의 브랜드는 iBestie, iDay와 같은 생활밀착형 소확행 앱과 Erudite.AI라는 HTR-Rag AI 에이전트입니다. 이브랜드에 적합한 주요 타겟 고객층을 3가지 정도 제안해 주세요. 그리고 각 고객층별로 상세한 페르소나도 함께 만들어 줄 수 있을까요? 페르소나에는 나이, 직업, 라이프스타일, 가치관, 소비습관, 브랜드에 대한 니즈 등이 포함되어야 합니다.

+



문제

생성된 페르소나를 바탕으로 고객 여정 맵을 작성하고, 각 단계별 고객의 생각과 감정, 브랜드와의 접점 등을 분석하도록 요청합니다.

답

앞서 만든 페르소나중 감성 루틴러, 유진(28세)의 고객 여정 맵을 작성해 주세요. 브랜드 인지부터 구매 후 경험까지의 과정을 단계별로 나누고, 각 단계에서 고객이느끼는 감정, 생각, 행동을 설명해 주세요. 또한 각 단계에서 브랜드가 어떤 방식으로 고객과 소통하고 가치를 전달할 수 있을지 제안해 주세요.

앞서 만든 페르소나중 감성 루틴러, 유진(28세)의 고객 여정 맵을 작성해 주세요. 브랜드 인지부터 구매 후 경험까지의 과정을 단계별로 나누고, 각 단계에서 고객이느끼는 감정, 생각, 행동을 설명해 주세요. 또한 각 단계에서 브랜드가 어떤 방식으로 고객과 소통하고 가치를 전달할 수 있을지 제안해 주세요.

+



문제

각 페르소나에 맞는 브랜드 스토리텔링 전략을 수립하고, 구체적인 메시지와 전달 방식을 제안하도록 요청합니다.

답

감성 루틴러, 유진(28세)을 타겟으로 한 브랜드 스토리텔링 전략을 수립해 주세요. 이 페르소나의 특성과 니즈를 고려하여, 어떤 내용의 스토리를 어떤 방식으로 전달하면 효과적일지 제안해 주세요. 스토리의 주요 내용, 톤앤매너, 전달 채널 등을 구체적으로 설명해 주시고, 이 전략이 왜 해당 페르소나에게 효과적일지 근거를 들어 설명해 주세요.

감성 루틴러, 유진(28세)을 타겟으로 한 브랜드 스토리텔링 전략을 수립해 주세요. 이 페르소나의 특성과 니즈를 고려하여, 어떤 내용의 스토리를 어떤 방식으로 전달하면 효과적일지 제안해 주세요. 스토리의 주요 내용, 톤앤매너, 전달 채널 등을 구체적으로 설명해 주시고, 이 전략이 왜 해당 페르소나에게 효과적일지 근거를 들어 설명해 주세요.

+



문제

각 페르소나의 특성에 맞는 구체적인 브랜드 메시지를 개발하고, 이를 다양한 접점에 적용하는 방안을 모색하도록 요청합니다.

답

앞서 분석한 3가지 페르소나 각각에 대해 맞춤형 브랜드 메시지를 5개씩 만들어 주세요. 각 메시지는 해당 페르소나의 특성, 니즈, 가치관을 반영해야 합니다. 또한 이 메시지들을 어떤 채널(SNS, 이메일 마케팅, 광고 등)을 통해 전달하면 좋을지, 그 이유와 함께 제안해 주세요.

앞서 분석한 3가지 페르소나 각각에 대해 맞춤형 브랜드 메시지를 5개씩 만들어 주세요. 각 메시지는 해당 페르소나의 특성, 니즈, 가치관을 반영해야 합니다. 또한 이 메시지들을 어떤 채널(SNS, 이메일 마케팅, 광고 등)을 통해 전달하면 좋을지, 그 이유와 함께 제안해 주세요.



문제

ChatGPT를 사용하여 브랜드에 맞는 인스타그램 바이오(프로필)을 작성합니다.

답

위 브랜드의 인스타그램 프로필을 작성해주세요. 다음 요소를 포함해주세요.

- 150자 이내의 브랜드 소개문
- 브랜드를 잘 표현하는이모지 3~5개
- 브랜드의 주요 특징이나 제품을 나타내는 키워드 3개
- 고객 행동을 유도하는 CTA(Call-to-Action)문구

프로필이 검색 최적화에 도움이 되도록 관련 키워드를 자연스럽게 포함시켜주세요.

위 브랜드의 인스타그램 프로필을 작성해주세요. 다음 요소를 포함해주세요.

-150자 이내의 브랜드 소개문

-브랜드를 잘 표현하는이모지 3~5개

-브랜드의 주요 특징이나 제품을 나타내는 키워드 3개

-고객 행동을 유도하는 CTA(Call-to-Action)문구

프로필이 검색 최적화에 도움이 되도록 관련 키워드를 자연스럽게 포함시켜 주세요.





실습 8: 브랜드의 랜딩 페이지를 기획하고, 핵심 요소들을 구성해 보세요.

문제

STP 분석을 바탕으로 랜딩페이지를 기획해 보세요. AI를 활용한STP 분석 및
유저 맵 작성

실습 8: 브랜드의 랜딩 페이지를 기획하고, 핵심 요소들을 구성해 보세요.

답

위 브랜드의 주요제품/서비스는 iBestie, iDay, Erudite.AI입니다. 이 정보를 바탕으로 STP분석을 진행해주세요.

1. Segmentation: 시장을 어떻게 세분화할 수 있을까요?
2. Targeting: 어떤 세그먼트를 타겟으로 삼아야 할까요?
3. Positioning: 타겟 세그먼트에서 어떻게 포지셔닝해야 할까요?

STP분석 결과를 바탕으로, 주요타겟 고객의 유저 맵도 작성해 주세요. 유저 맵에는 다음 내용을 포함해 주세요.

- 인구통계학적 특성
- 행동 패턴
- 니즈와 페인 포인트
- 구매 결정 요인
- 브랜드와의 접점

실습 9: 브랜드의 랜딩 페이지를 기획하고, 핵심 요소들을 구성해 보세요.

답

위 브랜드의 주요제품/서비스는 iBestie, iDay, Erudite.AI입니다. 이 정보를 바탕으로 STP분석을 진행해주세요.

1. Segmentation: 시장을 어떻게 세분화할 수 있을까요?
2. Targeting: 어떤 세그먼트를 타겟으로 삼아야 할까요?
3. Positioning: 타겟 세그먼트에서 어떻게 포지셔닝해야 할까요?

STP분석 결과를 바탕으로, 주요타겟 고객의 유저 맵도 작성해 주세요. 유저 맵에는 다음 내용을 포함해 주세요.

- 인구통계학적 특성
- 행동 패턴
- 니즈와 페인 포인트
- 구매 결정 요인
- 브랜드와의 접점

+





실습 9: 브랜드의 랜딩 페이지를 기획하고, 핵심 요소들을 구성해 보세요.

문제1

AI를 활용한 핏 메시지 및 랜딩페이지 메시지를 생성해 보세요.

실습 9: 브랜드의 랜딩 페이지를 기획하고, 핵심 요소들을 구성해 보세요.

답

앞서 진행한 STP 분석과 유저 맵을 바탕으로, Erudite.AI 의 핏 메시지를 작성해 주세요. 핏 메시지에는 다음 요소가 포함되어야 합니다.

1. 타깃 고개의 주요 문제점
2. Erudite.AI가 제공하는 솔루션
3. 기대할 수 있는 결과

이어서, 이 핏 메시지를 바탕으로 랜딩 페이지에 사용할 수 있는 다음 요소들을 작성해 주세요.

1. 헤드라인(5개 옵션)
2. 서브헤드라인(3개 옵션)
3. 주요 특징/혜택 설명(3-5개 항목)
4. CTA 문구(3개 옵션)

각 요소에 대해 간단히 설명과 함께, 왜 효과적일지 근거를 제시해 주세요.



문제 2

Cursor.AI를 사용하여 브랜드 홈페이지의 기본 구조를 설계합니다.

실습9 : 브랜드의 홈페이지를 디자인하고 제작해 보세요.

답

제 브랜드는 Erudite.AI입니다.

Erudite.AI - AI가 답하는 가장 스마트한 Intelligence Knowledge Agent System(지능형 에이전트 시스템) 플랫폼!

모든 정보를 한곳에서, 빠르고 정확하게! 임원들이 필요한 정보를 찾느라 시간을 낭비하고 있나요? Erudite.AI는 AI 기팀 지식 에이전트로 조직 내 모든 지식을 즉시 검색하고, 신뢰할 수 있는 답변을 제공합니다!

 이런 조직에 추천합니다!

- ✓ 팀복되는 질문과 답변으로 업무 생산성이 떨어지는 기업
- ✓ 사내 문서를 검색하는 데 시간이 오래 걸리는 팀
- ✓ 중요한 정보가 여러 시스템에 분산되어 있는 회사
- ✓ 보안이 중요한 조직 내 데이터 관리가 필요한 기업

"AI가 업무 속도를 높입니다!" 이제 Erudite.AI와 함께 지식 관리의 새로운 기준을 경험하세요! 지금 도입하고, 더 스마트한 조직을 만들어보세요!

실습9 : 브랜드의 홈페이지를 디자인하고 제작해 보세요.

답

보험사, 금융사, 사내 메뉴얼 등 문서를 참조하는 챗봇 을 만들어 보세요.

놀라운 인공지능 기술과 강력한 사전 구축된 LLM과 RAG 파이프라인을 통해 챗봇 개발을 가속화하도록 설계된 프리미엄 템플릿을 제공해 드립니다.

제작할 홈페이지의구조를 설계해 주세요. 다음 요소를 포함해 주세요.

1. 메인 페이지 구조(헤더,히어로션션, 주요 섹션들,푸터)
2. 필요한 하위 페이지 목록(예: About, Products/Services, Contact 등)
3. 각 페이지별 주요 콘텐츠 요소
4. 내비게이션구조

또한, 다음 사항도 고려해 주세요.

- 브랜드의 핵심 메시지를 효과적으로 전달할 방법
- UX를 개선할 수 있는 요소들
- 반응형 웹 지원

각 요소에 대해 간단한 설명과 함께, 왜 이런 구조가 효과적일지 근거를 제시해 주세요. 마지막으로, 수행 결과를 REPORT 폴더에 마크다운 파일로 저장해 주세요.

실습9 : 브랜드의 홈페이지를 디자인하고 제작해 보세요.

답

File Edit Selection View Go Run Terminal Help

EruditeAI - Cursor

ERUDI... [Icons]

New Chat

@ + Browser

Erudite.AI - AI가 답하는 가장 스마트한 Intelligence Knowledge Agent System(지능형 에이전트 시스템) 플랫폼!

모든 정보를 한곳에서, 빠르고 정확하게! 임원들이 필요한 정보를 찾느라 시간을 낭비하고 있나요?

Erudite.AI는 AI 기팀 지식 에이전트로 조직 내 모든 지식을 즉시 검색하고, 신뢰할 수 있는 답변을 제공합니다!

🎯 이런 조직에 추천합니다!

- ✓ 팀복되는 질문과 답변으로 업무 생산성이 떨어지는 기업
- ✓ 사내 문서를 검색하는 데 시간이 오래 걸리는 팀
- ✓ 중요한 정보가 여러 시스템에 분산되어 있는 회사
- ✓ 보안이 중요한 조직 내 데이터 관리가 필요한 기업

"AI가 업무 속도를 높입니다!" 이제 Erudite.AI와 함께 지식 관리의 새로운 기준을 경험하세요! 지금 도입

∞ Agent Ctrl+I Auto [Icons]

Shift Tab to pl

문제

Cursor.AI를 사용하여 브랜드 홈페이지를 상세히 디자인하고 제작합니다.

실습9 : 브랜드의 홈페이지를 디자인하고 제작해 보세요.

답

다음 작업을 수행하세요.

1. 메인 페이지 디자인:

- 히어로 섹션에 주요 헤드라인과 CTA버튼을 배치하세요.
- 제품/서비스 섹션을 추가하고, 주요 특징을 강조하세요.
- 고객 후기나 신뢰 요소(언론 보도, 수상 내역 등)을 포함하세요.

2. Aoubt 페이지 제작

- 브랜드 스토리를 간단히 소개하세요.
- 팀 멤버나 회사 연력을 추가하세요.

3. 제품/서비스 페이지 제작

- 각 제품/서비스에 대한 상세 정보를 제공하세요.
- 가격 정보나 구매/신청 버튼을 포함하세요.

4. 연락처 페이지 제작

- 연락처 정보와 위치(지도)를 표시하세요.
- 문의 폼을 추가하세요.

5. 전체적인 디자인 조정

- 브랜드 색상과 폰트를 일관되게 적용하세요.
- 모바일 버전을 확인하고 필요한 조정을 하세요.

각 단계를 마칠 때마다 작업 내용을 설명하고, "이 부분을 더 효과저금로 만들 방법이 있을까요?"라고 물어 조언을 구하세요. 완성된 결과를 Report라는 폴더에 마크다운 파일로 저장해주세요.

실습9 : 브랜드의 홈페이지를 디자인하고 제작해 보세요.

답

The screenshot displays a code editor interface with the 'about.html' file open. The file content is as follows:

```
1 <!DOCTYPE html>
2 <html lang="ko">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>회사소개 - Erudite.AI</title>
7   <link href="https://fonts.googleapis.com/css2?family=Noto+Sans+KR:wght@400;700">
8   <link rel="stylesheet" href="styles.css">
9 </head>
10 <body>
11   <!-- 헤더 -->
12   <header class="header">
13     <nav class="navbar">
14       <div class="nav-container">
15         <div class="nav-logo">
16           
17           <span class="brand-name">Erudite.AI</span>
18         </div>
19         <ul class="nav-menu">
20           <li class="nav-item">
21             <a href="index.html" class="nav-link">홈</a>
22           </li>
23           <li class="nav-item">
24             <a href="about.html" class="nav-link active">회사소개</a>
25           </li>
26           <li class="nav-item">
27             <a href="products.html" class="nav-link">제품/서비스</a>
28           </li>
29           <li class="nav-item">
30             <a href="#solutions" class="nav-link">솔루션</a>
31           </li>
32           <li class="nav-item">
33             <a href="#cases" class="nav-link">고객사례</a>
34           </li>
35           <li class="nav-item">
36             <a href="contact.html" class="nav-link">문의하기</a>
37           </li>
38         </ul>
39       </div>
40     </nav>
41   </header>
42 </body>
```

The sidebar on the left shows a project structure with files like 'about.html', 'contact.html', 'index.html', 'products.html', 'script.js', and 'styles.css'. The right sidebar contains an AI chat window titled 'Erudite.AI 홈페이지 구조 설계' with the following content:

제 브랜드는 Erudite.AI입니다.

Erudite.AI - AI가 답하는 가장 스마트한 Intelligence Knowledge Agent System(지능형 에이전트)

안녕하세요! 케이지네트워크의 QA 테스터로서 Erudite.AI 브랜드의 홈페이지 구조 설계를 도와드리겠습니다. 먼저 프로젝트 구조를 파악하고 체계적인 분석을 진행하겠습니다.

To-dos 5

- 현재 프로젝트 구조 분석 및 기존 파일 확인
- Erudite.AI 브랜드 분석 및 핵심 메시지 파악
- 홈페이지 구조 설계 (메인페이지, 하위페이지, 내비게이션)
- UX 개선 요소 및 반응형 웹 지원 방안 검토
- REPORT 폴더에 마크다운 파일로 결과 저장

Listed current directory

Searched web AI 지능형 에이전트 시스템 플랫폼 홈페이지 UX 디자인 트렌드 2024

1 of 5 Done View all

- 현재 프로젝트 구조 분석 및 기존 파일 확인
- Erudite.AI 브랜드 분석 및 핵심 메시지 파악

현재 프로젝트는 빈 상태이므로 처음부터 설계를 진행하겠습니다. 이제 Erudite.AI 브랜드 분석을 바탕으로 체계적인 홈페이지 구조를 설계하겠습니다.

Searched web 지능형 에이전트 시스템 플랫폼 홈페이지 UX 디자인 베스트 프랙티스 2024

Searched web RAG 파이프라인 챗봇 플랫폼 홈페이지 구조 설계 가이드

2 of 5 Done View all

- Erudite.AI 브랜드 분석 및 핵심 메시지 파악

8 Files Undo All Keep All

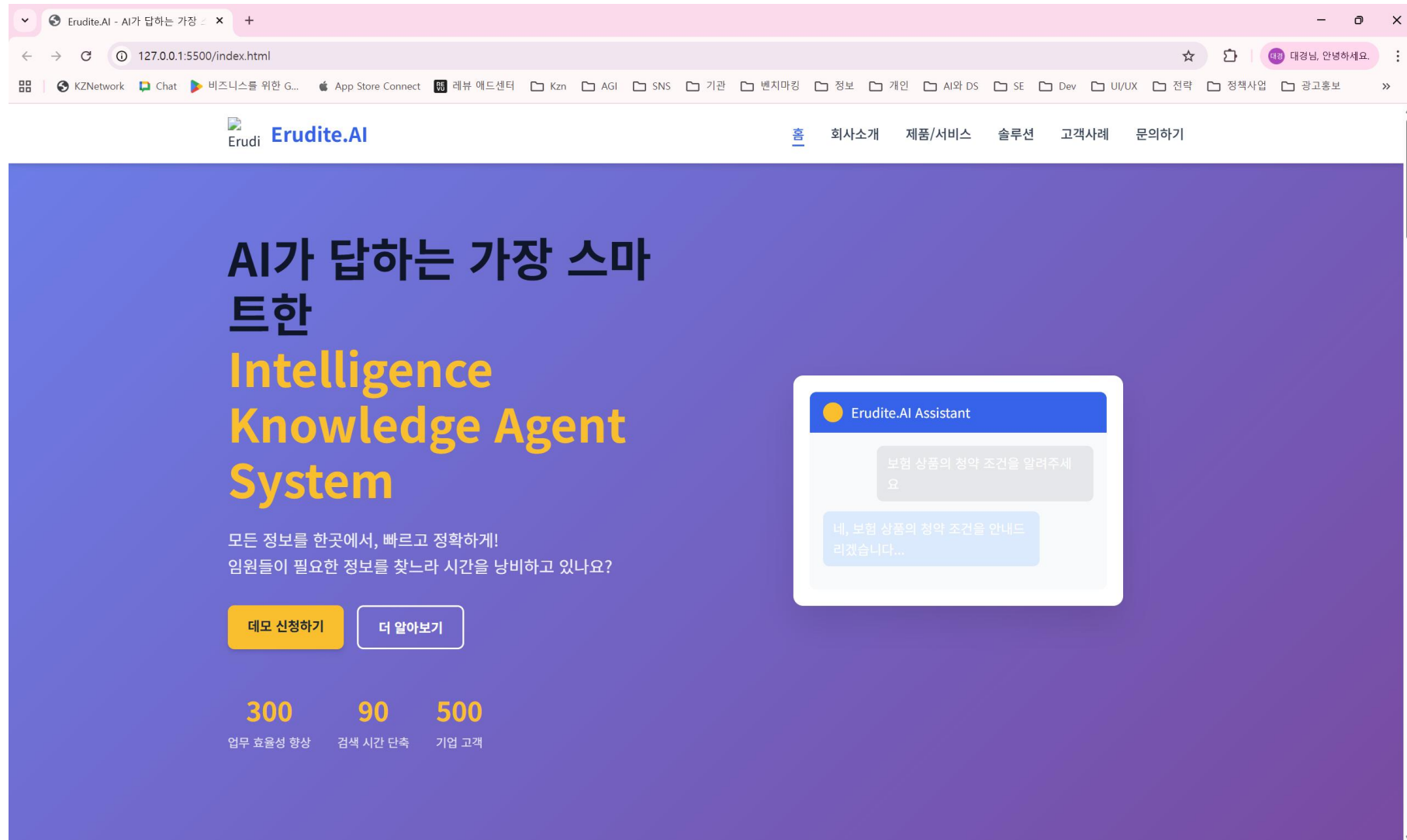
1 Tab 89.5%

Plan, search, build anything

Agent Ctrl+I Auto

실습9 : 브랜드의 홈페이지를 디자인하고 제작해 보세요.

결과





문제 :

WB1_3-OrderBean – PRD v1.0 참조해 PRD를 작성해 봅니다.

| 실습 1단계 - 아이디어 구상

목표: 커피 관련 서비스 아이디어 도출

Cursor.AI 프롬프트

```\n

실제 사용자 문제를 해결하는 커피 관련 웹 앱 아이디어  
3가지를 브레인스토밍하세요.

각 아이디어에 대해 다음을 설명하세요.

- 해결하는 문제
- 대상 사용자
- 주요 기능(3~5개) ```

예상 소요시간: 30분

## | 실습 1단계 - OrderBean 서비스 컨셉

선택한 아이디어: 간편 커피 주문 서비스

해결하는 문제

- 카페 대기 시간 낭비
- 복잡한 커스터마이징 주문
- 주문 히스토리 관리 어려움

목표 사용자

- 바쁜 직장인
- 단골 고객

## | 실습 2단계 - 문제 정의

목표: 한 문장으로 핵심 문제 표현

Cursor.AI 프롬프트

\\\

바쁜 직장인들이 시간을 절약할 수 있도록 돕는 커피 주문 웹 앱에 대한 한 문장으로 된 문제 진술문을 작성하세요

\\\

예상 출력

"바쁜 직장인들이 카페 대기 없이 빠르게 커피를 주문하고 픽업할 수 있는 서비스"

예상 소요시간: 15분

## | 실습 3단계 - 주요 기능 정의

목표: 5가지 핵심 기능 도출

Cursor.AI 프롬프트

```\n

OrderBean 웹 서비스의 5가지 주요 기능 요구 사항을
나열하세요.

고객 기능과 관리 기능을 모두 포함하세요.

간단한 설명과 함께 요점 형식으로 작성하세요.```\n

예상 소요시간: 45분

| 실습 3단계 - OrderBean 핵심 기능 의

고객 기능

1. 커피 메뉴 조회
2. 주문 생성 (옵션 선택)
3. 주문 내역 조회

관리자 기능

4. 메뉴 관리 (CRUD)
5. 주문 상태 관리

| 실습 4단계 - 사용자 시나리오

목표: Given-When-Then 시나리오 작성

Cursor.AI 프롬프트

\\\

5가지 기능적 요구사항을거킨 스타일 사용자 스토리로
변환하세요.Given-When-Then 형식을
사용하세요.구체적이고 테스트 가능하게 만드세요.

실습 4단계 - 사용자 시나리오 예시

Scenario: 고객이 커피를 주문한다

Given 고객이 메뉴 페이지에 접속하고

And "아메리카노" 메뉴가 존재하며

When 고객이 아메리카노를 장바구니에 추가하고

And "주문하기" 버튼을 클릭하면

Then 주문이 성공적으로 생성되고

And 주문 번호가 표시된다

| 실습 5단계 - 비기능 요구사항

목표: 성능, 보안, 확장성 정의

Cursor.AI 프롬프트

\\`

OrderBean에 대한 비기능적 요구 사항을 추가합니다.

- 성능 기대치
- 보안 요구 사항
- 확장성 고려 사항
- 사용성 표준

Toy Project용으로 현실적인 수준으로 유지하세요.

\\`

예상 소요시간: 30분

| 실습 6단계 - PRD 문서 구조화

이전 모든 출력을 구조화된 PRD 마크다운 문서로 결합합니다.

포함 항목:

- 개요
- 기능
- 데이터 모델(엔터티 및 관계)
- API 개요(엔드포인트)
- 사용자 스토리

| 실습 7단계 AI 검증

이전 모든 출력을 결합하여
PRD의 완전성, 일관성 및 테스트 가능성을 검토합니다.
개선 사항을 제안합니다.

1. Project_milestones.md 파일 만들기

가장 중요한 건 Cursor에게 프로젝트 전체 그림을 알려주는 거다.
Project_milestones.md 파일을 만들고 .cursorrules 파일에서 참조하도록 설정하자.

프로젝트 범위와 요구사항을 최대한 자세히 적어서 Cursor가 프로젝트 파일을 생성하도록 하고, 매 세션 끝에 (또는 진행 중에) 완료된 마일스톤과 배운 점들을 업데이트하라고 요청하자.

여기 실제 예시가 있다:

1. Project_milestones.md 파일 만들기

Phase 1: Manual Operations & Testing

- Basic API integration
- Database schema setup
- Basic OpenAI integration
- Manual data collection script
- Manual analysis trigger
- Manual review interface
- Testing and refinement

Learnings & Architecture Decisions

Worker Architecture

Current Setup: Single worker handling both API calls

Need to split into separate workers for better:

- Rate limiting
- Error handling
- Cost tracking
- Scaling

Next Steps Split workers:

- call-worker: API and data collection
- analysis-worker: OpenAI integration and analysis
- api-worker: Admin dashboard endpoints

2. Documentation.md 파일 관리하기

Documentation.md 파일도 만들어서 .cursorrules에서 참조하도록 하자. 새로운 함수나 스키마 등이 추가될 때마다 정기적으로 업데이트하거나 Cursor에게 업데이트하라고 요청하자.

.cursorrules 파일의 주요 내용

- 생성된 .mdc 파일에는 주로 다음과 같은 메타데이터와 규칙 내용이 포함됩니다:
- Description: 규칙에 대한 간단한 설명.
- Globs: 이 규칙이 적용될 파일 패턴 (예: `**/*.ts`, `src/**/*.py`).
- 본문: AI에게 전달할 구체적인 지침, 코딩 스타일, 프레임워크 사용법 등.
- 이 파일을 Git에 포함시키면 팀원 모두가 동일한 AI 지원을 받을 수 있어 코드 일관성 유지에 큰 도움이 됩니다

2. Documentation.md 파일 관리하기

.cursorrules 파일은 Cursor IDE에서 AI의 동작 방식(코드 스타일, 규칙, 프로젝트 구조 등)을 지정하는 데 사용되는 파일입니다.

.cursorrules 파일을 추가하는 방법은 다음과 같습니다.

방법 1: Cursor IDE 명령 팔레트 사용 (권장)

- 가장 쉬운 방법은 Cursor 내장 명령어를 사용하는 것입니다.
- 명령 팔레트 열기: Ctrl+Shift+P (Mac의 경우 Cmd+Shift+P)를 누릅니다.
- 명령어 검색: 검색창에 "File: New Cursor Rule" 또는 "New Cursor Rule"을 입력합니다.
- 규칙 파일 생성: 명령어를 선택하면 규칙 파일 이름(my-awesome-rule.mdc 등)을 입력하라는 메시지가 나타납니다.
- 파일 위치 확인: 지정된 이름으로 .mdc 확장자를 가진 파일이 프로젝트 폴더 내의 .cursor/rules/ 디렉토리에 자동으로 생성됩니다.

2. Documentation.md 파일 관리하기

방법 2: 프로젝트 루트 디렉토리에 수동 생성

- Cursor IDE의 초기 버전에서는 .cursorrules 파일을 프로젝트 루트에 직접 배치했지만, 최신 버전에서는 .cursor/rules/ 디렉토리 아래에 .mdc 형식의 파일을 사용하는 방식이 권장됩니다.
- 프로젝트 루트로 이동: 터미널이나 파일 탐색기를 사용하여 프로젝트의 최상위 디렉토리로 이동합니다.
- .cursor 디렉토리 생성: .cursor라는 이름의 숨겨진 디렉토리를 생성합니다.
- rules 디렉토리 생성: .cursor 디렉토리 안에 rules 디렉토리를 생성합니다.
- 규칙 파일 생성: rules 디렉토리 안에 원하는 이름으로 .mdc 확장자를 가진 파일을 생성합니다 (예: style_guide.mdc).
- 내용 작성: 이 파일 안에 AI에게 지시할 구체적인 규칙이나 지침을 마크다운 형식으로 작성합니다.

2. Documentation.md 파일 관리하기

.cursorrules 파일에 다음 내용들을 꼭 포함시키자:

Project Management:

- Reference PROJECT.md for all feature implementations
- Reference Documentation.md for all API endpoints and their request/response formats
- Ensure new code aligns with defined milestones
- Follow the established database schema
- Consider cost optimizations defined in metrics
- Maintain consistency with existing components

2. Documentation.md 파일 관리하기

프로젝트 관리:

- 모든 기능 구현에 대해 `PROJECT.md`를 참고하십시오.
- 모든 API 엔드포인트와 그 요청/응답 형식에 대해 `Documentation.md`를 참고하십시오.
- 새로운 코드가 정의된 마일스톤에 부합하도록 작성하십시오.
- 기존에 설정된 데이터베이스 스키마를 준수하십시오.
- 메트릭에서 정의된 비용 최적화를 고려하십시오.
- 기존 컴포넌트와의 일관성을 유지하십시오.

2. Documentation.md 파일 관리하기

상세 설명:

프로젝트의 기능 구현과 관련된 세부사항이 PROJECT.md 파일에 정의되어 있으므로, 이를 참고하여 작업을 진행해야 합니다.

Documentation.md 참조

API 엔드포인트와 요청/응답 형식에 대한 명세가 Documentation.md 파일에 정리되어 있으므로, 이를 기반으로 API를 설계하거나 확인해야 합니다.

마일스톤 준수

프로젝트에서 설정된 마일스톤(중요 목표나 단계)에 맞게 새로운 코드가 작성되도록 해야 합니다.

데이터베이스 스키마 준수

이미 정의된 데이터베이스 구조(스키마)를 따르고, 변경이 필요한 경우 기존 스키마와의 호환성을 고려해야 합니다.

비용 최적화 고려

메트릭에서 정의된 비용 최적화 기준(예: 서버 리소스, 데이터 처리 비용 등)을 염두에 두고 작업해야 합니다.

일관성 유지

기존에 작성된 컴포넌트(코드, 디자인, 구조 등)와의 일관성을 유지하여 프로젝트의 품질과 유지보수성을 높여야 합니다.

이 가이드는 개발자들이 프로젝트의 방향성과 규칙을 이해하고, 이를 따르면서 효율적으로 작업할 수 있도록 돕는 역할을 합니다.

3. 커뮤니티 규칙 활용하기

Awesome Cursor rules GitHub에 가서 본인 프로젝트 스택에 맞는 규칙들을 골라 사용하자.

<https://github.com/PatrickJS/awesome-cursorrules>

PatrickJS / awesome-cursorrules

Search: Type to search

<> Code Issues 37 Pull requests 8 Actions Projects Security Insights

awesome-cursorrules Public

Sponsor Watch 290 Fork 3k Star 35.6k

main 3 Branches 0 Tags

Go to file Add file <> Code

whysage and PatrickJS feat(python): prefer uv for Python dependency management; rena... fc2ce04 · last month 105 Commits

| | | |
|------------------------------|--|--------------|
| .github | refactor: wip lint | 5 months ago |
| rules-new | feat: Create rust.mdc (#129) | 3 months ago |
| rules | feat(python): prefer uv for Python dependency management... | last month |
| .cursorrules | feat: add .cursorrules to awesome-cursorrules | last year |
| .gitignore | feat: add Scala general cursorrules and with Kafka usage, clo... | 4 months ago |
| LICENSE | Create LICENSE | last year |
| LOCKUP_VERTICAL_25D_DARK.png | docs: new cursor logos | 2 months ago |

Monica Instant

저장소 요약

가장 진보된 모델을 지원하여 저장소 내용을 빠르게 이해할 수 있도록 도와줍니다

이 저장소 요약

About

Configuration files that enhance Cursor AI editor experience with custom



4. 점진적으로 작업하기

거대한 기능을 한 번에 달라고 하지 말자. project.md를 활용하거나 Cursor에게 "자, 다음 기능을 작업해보자"라고 말하면 거의 항상 마일스톤 순서를 지켜가며 작업한다.



실습 10: WB1_3-OrderBean – PRD v1.0 참조해 PRD를 작성해 봅니다.

답

WB1_3-OrderBean – PRD v1.0 참조

『 2과목 』 클린코드 + 테스트 기반 구현



문제 다음 예를, 좋은 예 (Clean Code)로 개선하세요.

✖ 나쁜 예 (Bad Code)

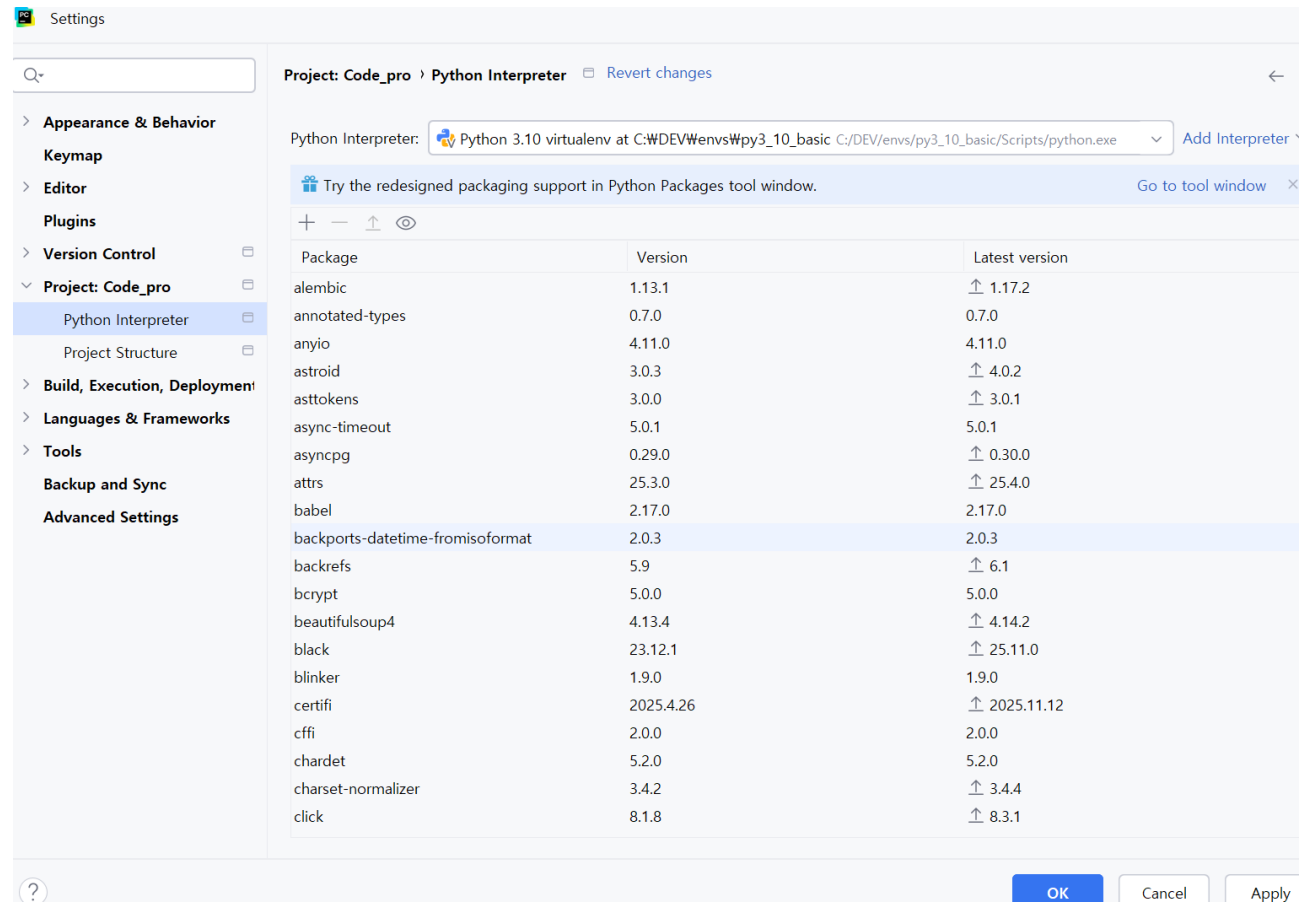
```
def calc(x, y, z):  
    if z == 1:  
        return x + y  
    elif z == 2:  
        return x - y  
    elif z == 3:  
        return x * y  
    elif z == 4:  
        return x / y  
    else:  
        return 0
```

문제점:

- 매직 넘버 (1, 2, 3, 4)
- 함수명이 모호 (calc가 무엇을 계산?)
- 하나의 함수가 너무 많은 책임
- 테스트하기 어려움

답
Week_2\Code_pro\python_test폴더를 열어, py3_10_basic
가상환경 설정을 합니다.

CleanCode.py



✓ 좋은 예 (Clean Code)

```
from enum import Enum
from typing import Callable
```

```
class Operation(Enum):
```

```
    ADD = "add"
    SUBTRACT = "subtract"
    MULTIPLY = "multiply"
    DIVIDE = "divide"
```

```
class Calculator:
```

```
    def __init__(self):
        self._operations: dict[Operation, Callable] = {
            Operation.ADD: self._add,
            Operation.SUBTRACT: self._subtract,
            Operation.MULTIPLY: self._multiply,
            Operation.DIVIDE: self._divide
        }
```

실습 1: Clean Code 실무 예시

```
def calculate(self, x: float, y: float, operation: Operation) -> float:
    """두 숫자에 대한 사칙연산을 수행합니다."""
    if operation not in self._operations:
        raise ValueError(f"지원하지 않는 연산: {operation}")

    return self._operations[operation](x, y)

def _add(self, x: float, y: float) -> float:
    return x + y

def _subtract(self, x: float, y: float) -> float:
    return x - y

def _multiply(self, x: float, y: float) -> float:
    return x * y

def _divide(self, x: float, y: float) -> float:
    if y == 0:
        raise ZeroDivisionError("0으로 나눌 수 없습니다")
    return x / y
```

사용 예시

```
calculator = Calculator()
result = calculator.calculate(10, 5, Operation.ADD) # 15
print(result)
```

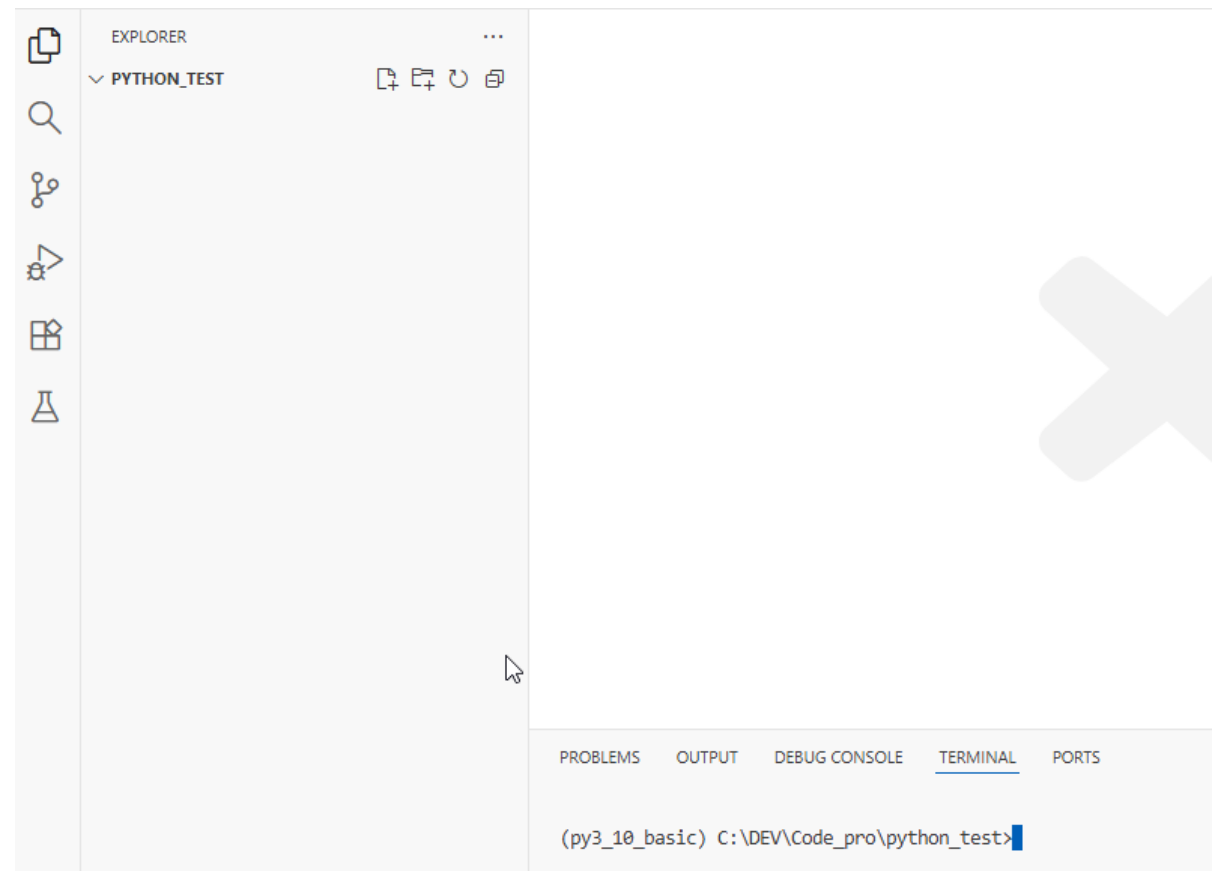
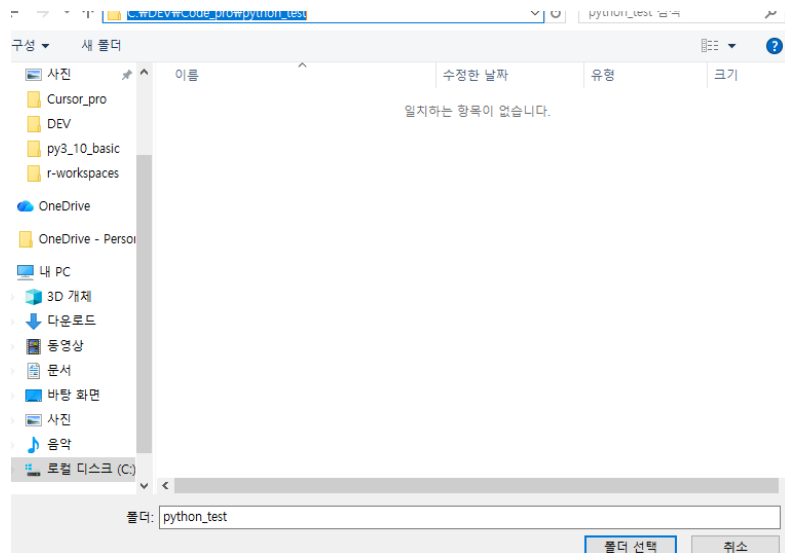
문제

파이썬 테스트 코드를 작성하려면 unittest 모듈이나 pytest 라이브러리를 사용합니다. unittest는 파이썬 내장 모듈로, unittest.TestCase를 상속받는 클래스를 만들어 assert 메소드를 사용해 테스트를 작성하고, setUp/tearDown으로 테스트 환경을 설정할 수 있습니다. pytest는 더 간결한 문법으로 테스트를 작성할 수 있으며, 더 많은 기능과 유연성을 제공합니다. 다음 add()메소드를 테스트 하세요.

```
def add(a, b):  
    return a + b
```

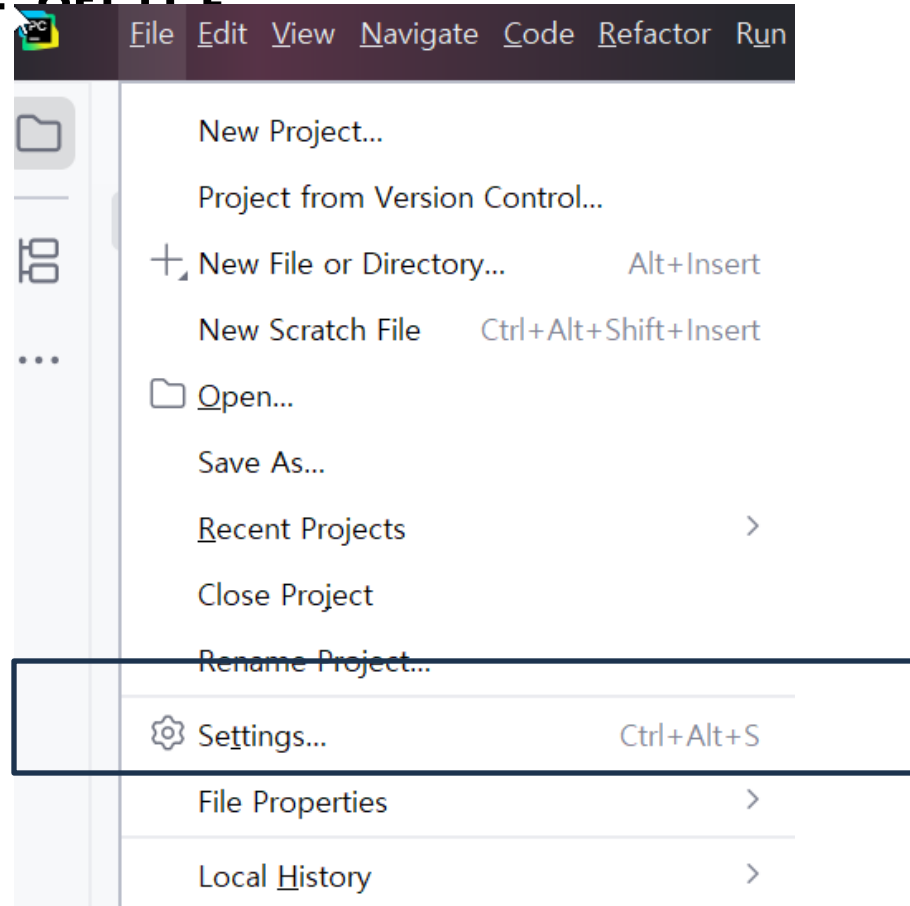
실습2 : 파이썬 테스트 코드 작성해보기

C:\DEV\SamsungDisplay\Python_test 폴더를 열어, py3_10_basic 가상환경 설정을 합니다.

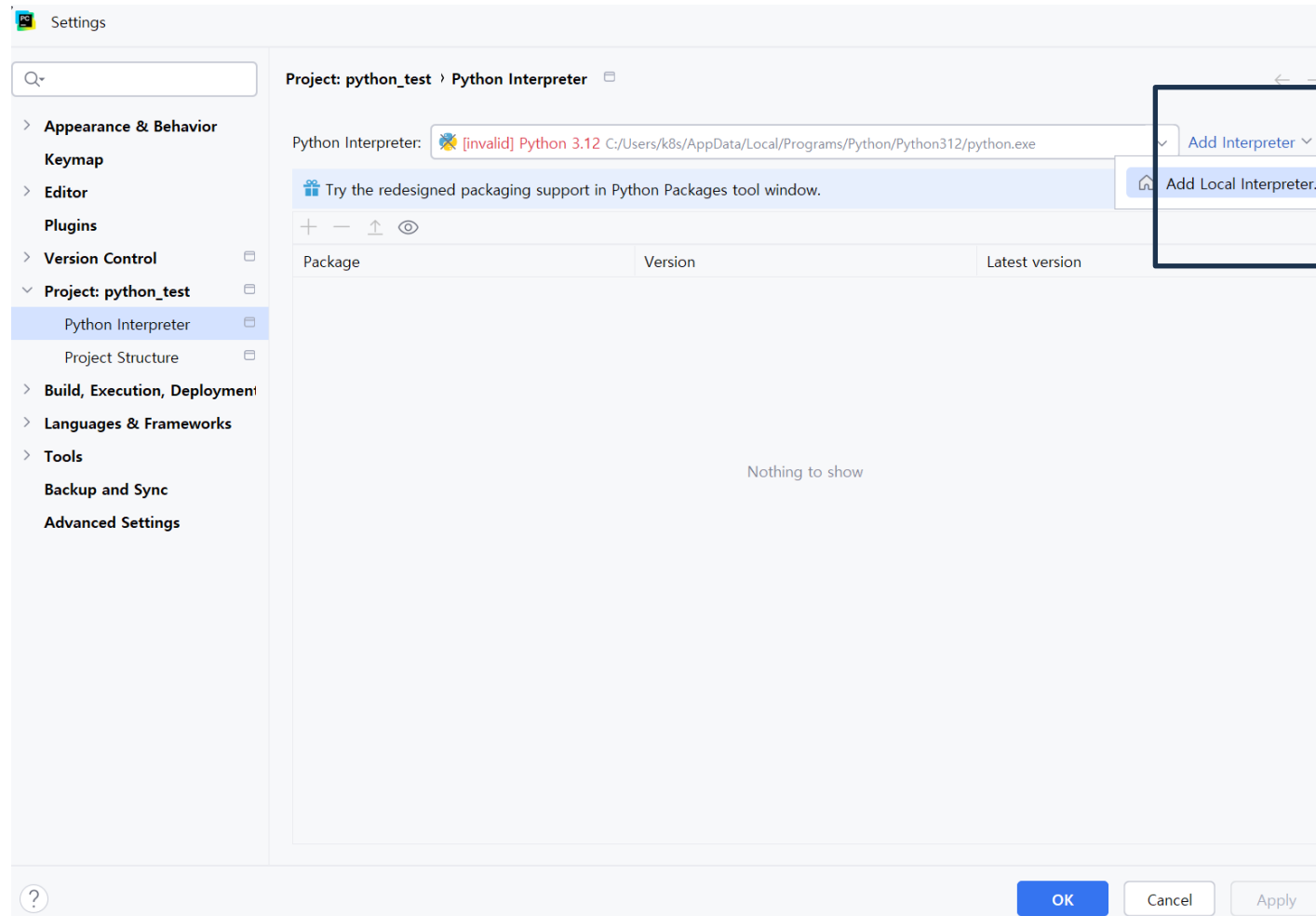


실습2 : 파이썬 테스트 코드 작성해보기

C:\DEV\SamsungDisplay\Python_test 폴더를 열어, py3_10_basic 가상환경 설정을 한다



Add Interpreter > Add Local Interpreter ...



실습2 : 파이썬 테스트 코드 작성해보기

Select existing

The screenshot shows the PyCharm Settings window with the 'Python Interpreter' section selected. The current interpreter is '[invalid] Python 3.12 C:/Users/k8s/AppData/Local/Programs/Python/Python312/python.exe'. A message提示 suggests trying the redesigned packaging support in the Python Packages tool window.

An 'Add Python Interpreter' dialog is open, showing the 'Select existing' option selected under 'Environment'. The 'Type' is set to 'Python' and the 'Python path' is 'C:\WDEV\envs\py3_10_basic\python.exe'.

A file explorer window titled 'Select Python Interpreter' is open, showing the contents of the 'Scripts' directory. The file 'python.exe' is selected.

| 이름 | 수정된 날짜 | 유형 | 크기 |
|-------------------------|---------------------|-------------------|-------|
| pip.exe | 2025-04-23 오후 5:03 | 응용 프로그램 | 106KB |
| pip3.10.exe | 2025-04-23 오후 5:03 | 응용 프로그램 | 106KB |
| pip3.exe | 2025-04-23 오후 5:03 | 응용 프로그램 | 106KB |
| py.test.exe | 2025-11-01 오전 9:31 | 응용 프로그램 | 106KB |
| pyftmerge.exe | 2025-04-25 오후 3:54 | 응용 프로그램 | 106KB |
| pyftsubset.exe | 2025-04-25 오후 3:54 | 응용 프로그램 | 106KB |
| pygmentize.exe | 2025-04-23 오후 5:02 | 응용 프로그램 | 106KB |
| pylupdate5.exe | 2025-05-10 오후 1:01 | 응용 프로그램 | 106KB |
| pyrcc5.exe | 2025-05-10 오후 1:01 | 응용 프로그램 | 106KB |
| pytest.exe | 2025-11-01 오전 9:31 | 응용 프로그램 | 106KB |
| python.exe | 2025-04-23 오후 4:51 | 응용 프로그램 | 263KB |
| pythonw.exe | 2025-04-23 오후 4:51 | 응용 프로그램 | 252KB |
| pyuic5.exe | 2025-05-10 오후 1:01 | 응용 프로그램 | 106KB |
| pywin32_postinstall.exe | 2025-04-23 오후 5:01 | 응용 프로그램 | 106KB |
| pywin32_postinstall.py | 2025-04-23 오후 5:01 | JetBrains PyCharm | 26KB |
| pywin32_testall.exe | 2025-04-23 오후 5:01 | 응용 프로그램 | 106KB |
| pywin32_testall.py | 2025-04-23 오후 5:01 | JetBrains PyCharm | 4KB |
| torchfrtrace.exe | 2025-05-30 오전 11:00 | 응용 프로그램 | 106KB |

실습2 : 파이썬 테스트 코드 작성해보기

Apply > Ok

Settings

Project: python_test > Python Interpreter [Revert changes](#)

Python Interpreter: [Python 3.10 virtualenv at C:\DEV\envs\py3_10_basic \(2\) C:/DEV/envs/py3_10_basic/Scripts/python.exe](#) [Add Interpreter](#)

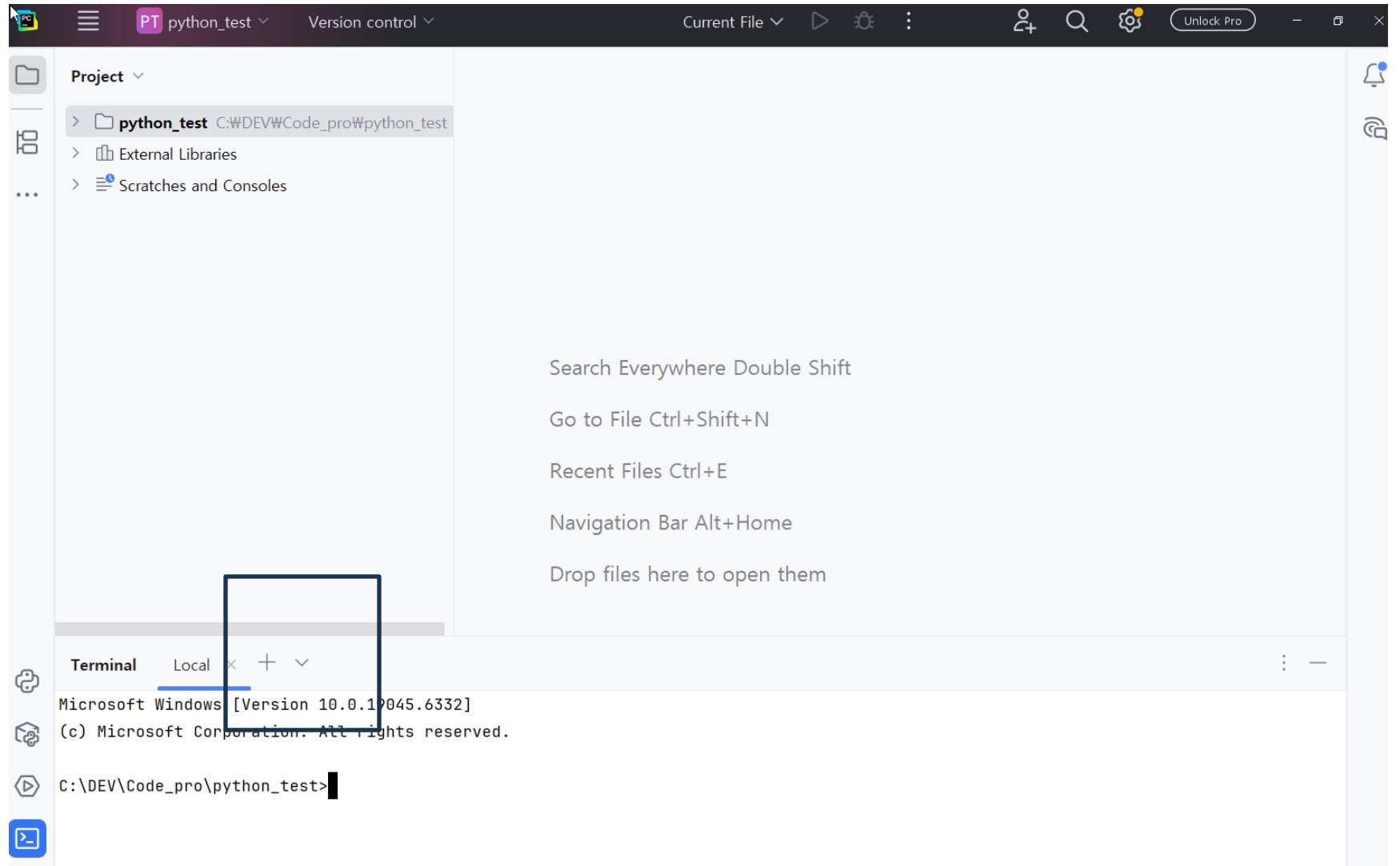
Try the redesigned packaging support in Python Packages tool window. [Go to tool window](#)

| Package | Version | Latest version |
|--------------------|-----------|---------------------------|
| asttokens | 3.0.0 | 3.0.0 |
| attrs | 25.3.0 | 25.4.0 |
| beautifulsoup4 | 4.13.4 | 4.14.2 |
| blinker | 1.9.0 | 1.9.0 |
| certifi | 2025.4.26 | 2025.10.5 |
| cfi | 1.17.1 | 2.0.0 |
| chardet | 5.2.0 | 5.2.0 |
| charset-normalizer | 3.4.2 | 3.4.4 |
| click | 8.1.8 | 8.3.0 |
| colorama | 0.4.6 | 0.4.6 |
| comm | 0.2.2 | 0.2.3 |
| contourpy | 1.3.2 | 1.3.3 |
| coverage | 7.11.0 | 7.11.0 |
| cycler | 0.12.1 | 0.12.1 |
| debugpy | 1.8.14 | 1.8.17 |
| decorator | 5.2.1 | 5.2.1 |
| et-xmlfile | 2.0.0 | 2.0.0 |
| exceptiongroup | 1.2.2 | 1.3.0 |
| executing | 2.2.0 | 2.2.1 |
| filelock | 3.18.0 | 3.20.0 |

OK Cancel Apply

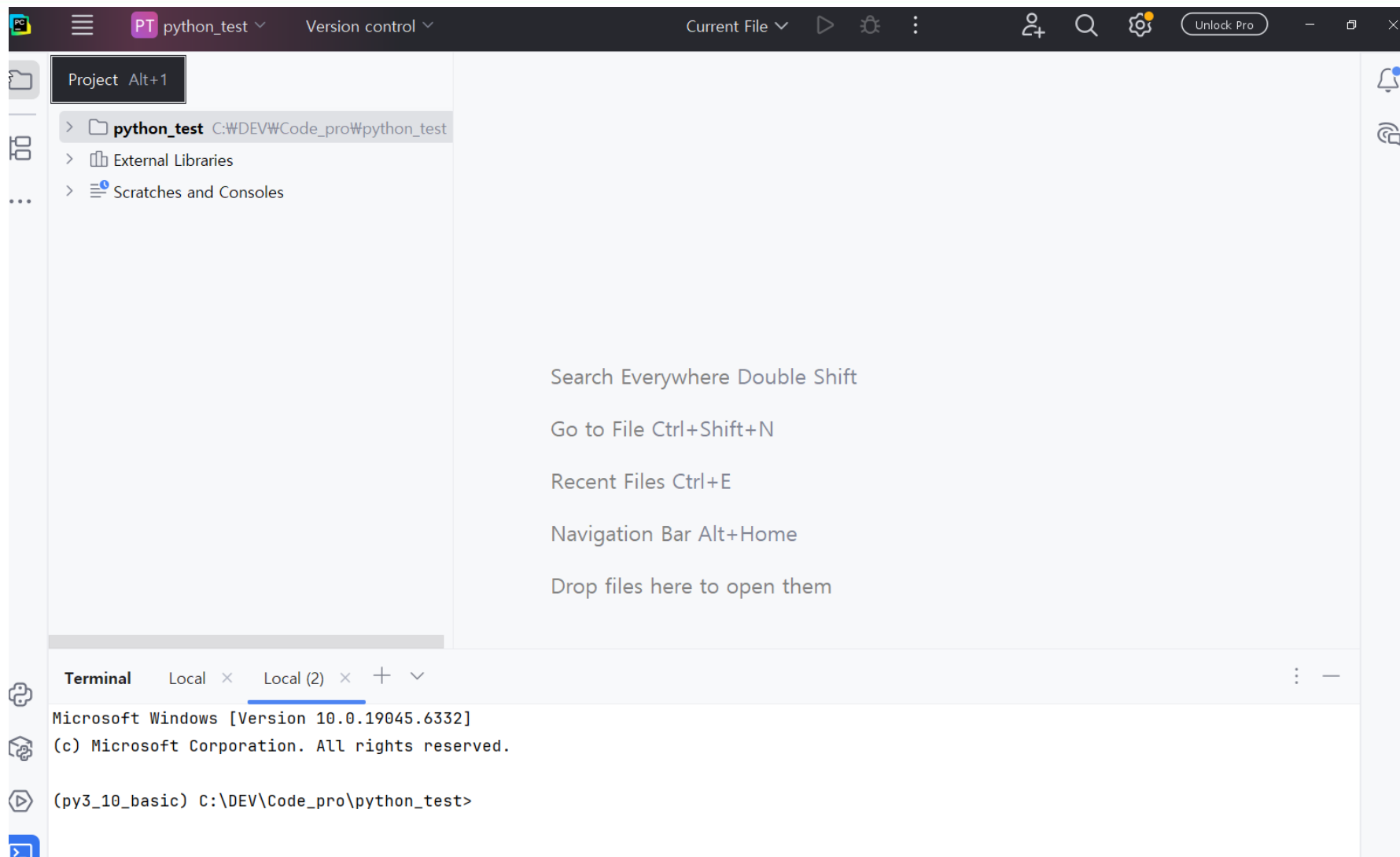
실습2 : 파이썬 테스트 코드 작성해보기

Local x + 클릭



실습2 : 파이썬 테스트 코드 작성해보기

환경설정 完!!!



unittest._add_function.py

```
import unittest

def add(a, b):
    return a + b

class TestAddFunction(unittest.TestCase):
    def test_add_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative_numbers(self):
        self.assertEqual(add(-1, -1), -2)

if __name__ == '__main__':
    unittest.main()
```

```
I
(py3_10_basic) C:\DEV\Code_pro\python_test>C:/DEV/envs/py3_10_basic/Scripts/python.exe c:/DEV/Code_pro/python_te
/unittest._add_function.py
..
-----
Ran 2 tests in 0.001s

OK

(py3_10_basic) C:\DEV\Code_pro\python_test>
```

실습2 : 파이썬 테스트 코드 작성해보기

pytest_add_function.py

```
def add(a, b):  
    return a + b
```

```
def test_add_positive_numbers():  
    assert add(2, 3) == 5
```

```
def test_add_negative_numbers():  
    assert add(-1, -1) == -2
```

```
(py3_10_basic) C:\DEV\Code_pro\python_test>pytest -v pytest_add_function.py  
'pytest'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는  
배치 파일이 아닙니다.
```

```
(py3_10_basic) C:\DEV\Code_pro\python_test>
```

```
(py3_10_basic) C:\DEV\SamsungDisplay\Python_test>pip install pytest  
pytest-cov
```

```
(py3_10_basic) C:\DEV\Code_pro\python_test>pytest -v pytest_add_function.py  
===== test session starts =====  
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe  
cachedir: .pytest_cache  
rootdir: C:\DEV\Code_pro\python_test  
plugins: cov-7.0.0  
collected 2 items  
  
pytest_add_function.py::test_add_positive_numbers PASSED [ 50%]  
pytest_add_function.py::test_add_negative_numbers PASSED [100%]  
  
===== 2 passed in 0.02s =====  
  
(py3_10_basic) C:\DEV\Code_pro\python_test>
```

pytest_add_function.py

```
(py3_10_basic) C:\DEV\Code_pro\python_test>pytest --cov=. -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\Code_pro\python_test
plugins: cov-7.0.0
collected 0 items
c:\DEV\envs\py3_10_basic\lib\site-packages\coverage\control.py:946: CoverageWarning: No data was collected. (no-data-collected); see https://coverage.readthedocs.io/en/7.11.0/messages.html#warning-no-data-collected
  self._warn("No data was collected.", slug="no-data-collected")

===== tests coverage =====
----- coverage: platform win32, python 3.10.11-final-0 -----

Name                               Stmts  Miss  Cover
-----
pytest_add_function.py              6      6     0%
TOTAL                               6      6     0%

===== no tests ran in 0.18s =====
```


pytest_add_function.py

```
(py3_10_basic) C:\DEV\Code_pro\python_test>pytest --cov=. --cov-report=html
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\DEV\Code_pro\python_test
plugins: cov-7.0.0
collected 0 items
c:\DEV\envs\py3_10_basic\lib\site-packages\coverage\control.py:946: CoverageWarning: No data was collected. (no-da
ta-collected); see https://coverage.readthedocs.io/en/7.11.0/messages.html#warning-no-data-collected
  self._warn("No data was collected.", slug="no-data-collected")

===== tests coverage =====
_____ coverage: platform win32, python 3.10.11-final-0 _____

Coverage HTML written to dir htmlcov
===== no tests ran in 0.31s =====
```

실습3 : 파이썬 테스트 코드 나눠서 테스트 하기

Python_test/

└─ add_func.py

└─ test_add_func.py

← 테스트 대상 코드

← 테스트 코드

add_func.py:

```
def add(a, b):  
    return a + b
```

test_add_func.py:

```
from add_func import add
```

```
def test_add_positive_numbers():  
    assert add(2, 3) == 5
```

```
def test_add_negative_numbers():  
    assert add(-1, -1) == -2
```

실습3 : 파이썬 테스트 코드 나눠서 테스트 하기

Python_test/

├── add_func.py

└── test_add_func.py

← 테스트 대상 코드

← 테스트 코드

```
(py3_10_basic) C:\DEV\Code_pro\python_test>pytest -v test_add_func.py
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\Code_pro\python_test
plugins: cov-7.0.0
collected 2 items

test_add_func.py::test_add_positive_numbers PASSED [ 50%]
test_add_func.py::test_add_negative_numbers PASSED [100%]

===== 2 passed in 0.04s =====
```

실습3 : 파이썬 테스트 코드 나눠서 테스트 하기

Python_test/

├── add_func.py

├── test_add_func.py

← 테스트 대상 코드

← 테스트 코드

```
(py3_10_basic) C:\DEV\Code_pro\python_test>pytest --cov=. --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\DEV\Code_pro\python_test
plugins: cov-7.0.0
collected 2 items

test_add_func.py .. [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.10.11-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
add_func.py                         2      0   100%
pytest_add_function.py              6      6     0%    1-8
test_add_func.py                    5      0   100%
-----
TOTAL                               13      6    54%
===== 2 passed in 0.11s =====

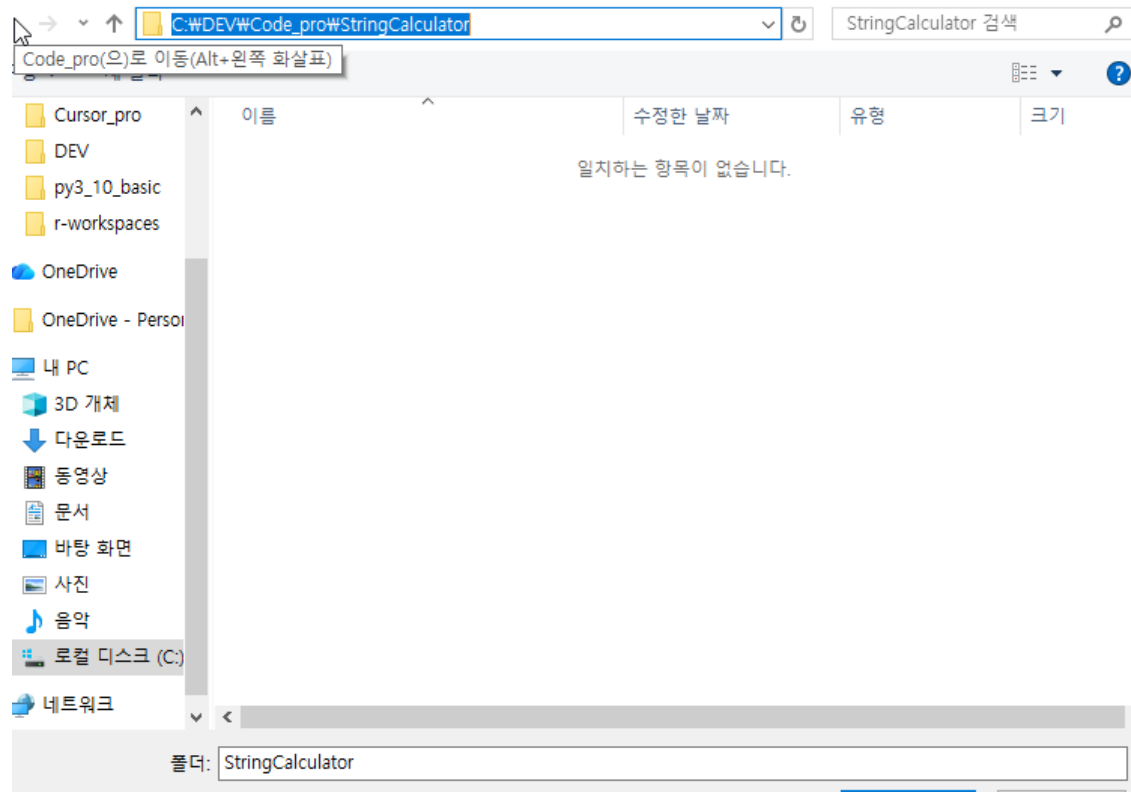
(py3_10_basic) C:\DEV\Code_pro\python_test>
```

요구사항

- 숫자 문자열을 입력받아 합계를 반환하는 계산기:
 - 빈 문자열 → 0 반환
 - 단일 숫자 → 숫자 자체 반환
 - 쉼표 구분 → 합계 반환

실습4 : TDD-문자열 계산기

C:\DEV\SamsungDisplay\StringCalculator폴더를 열고,
py3_10_basic에서 시작 ...



```
(py3_10_basic) C:\DEV\Code_pro\StringCalculator>
```

Step 1: Red (실패하는 테스트)

실행 결과: `NameError: name 'StringCalculator' is not defined` ← 실패!

```
# test_string_calculator.py
def test_empty_string_returns_zero():
    calculator = StringCalculator()
    assert calculator.add("") == 0
```

```
(py3_10_basic) C:\DEV\Code_pro\StringCalculator>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\Code_pro\StringCalculator
plugins: cov-7.0.0
collected 0 items / 1 error

===== ERRORS =====
_____ ERROR collecting test_string_calculator.py _____
ImportError while importing test module 'C:\DEV\Code_pro\StringCalculator\test_string_calculator.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
C:\Users\k8s\AppData\Local\Programs\Python\Python310\lib\importlib\__init__.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
test_string_calculator.py:1: in <module>
    from string_calculator import StringCalculator
E   ModuleNotFoundError: No module named 'string_calculator'

===== short test summary info =====
ERROR test_string_calculator.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.21s =====
```

Step 2: Green (통과하는 최소 코드)

실행 결과: 테스트 통과!

```
# string_calculator.py
class StringCalculator:
    def add(self, numbers: str) -> int:
        if not numbers:
            return 0
        return 0 # 일단 통과만 시킴
```

```
from string_calculator import
StringCalculator
```

```
def test_empty_string_returns_zero():
    calculator = StringCalculator()
    assert calculator.add("") == 0
```

```
(py3_10_basic) C:\DEV\Code_pro\StringCalculator>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\Code_pro\StringCalculator
plugins: cov-7.0.0
collected 1 item

test_string_calculator.py::test_empty_string_returns_zero PASSED [100%]

===== 1 passed in 0.02s =====
(py3_10_basic) C:\DEV\Code_pro\StringCalculator>
```


Step 3: 다음 테스트 추가

```
def test_single_number_returns_itself():  
    calculator = StringCalculator()  
    assert calculator.add("5") == 5
```

```
(py3_10_basic) C:\DEV\Code_pro\StringCalculator>pytest -v  
===== test session starts =====  
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe  
cachedir: .pytest_cache  
rootdir: C:\DEV\Code_pro\StringCalculator  
plugins: cov-7.0.0  
collected 2 items  
  
test_string_calculator.py::test_empty_string_returns_zero PASSED [ 50%]  
test_string_calculator.py::test_single_number_returns_itself FAILED [100%]  
  
===== FAILURES =====  
_____ test_single_number_returns_itself _____  
  
    def test_single_number_returns_itself():  
        calculator = StringCalculator()  
>     assert calculator.add("5") == 5  
E       AssertionError: assert None == 5  
E         + where None = add('5')  
E         +   where add = <string_calculator.StringCalculator object at 0x000001DF2095B280>.add  
  
test_string_calculator.py:9: AssertionError  
===== short test summary info =====  
FAILED test_string_calculator.py::test_single_number_returns_itself - AssertionError: assert None == 5  
===== 1 failed, 1 passed in 0.14s =====
```

Step 4: 코드 확장

```
class StringCalculator:
    def add(self, numbers: str) -> int:
        if not numbers:
            return 0
        return int(numbers)
```

```
(py3_10_basic) C:\DEV\Code_pro\StringCalculator>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\Code_pro\StringCalculator
plugins: cov-7.0.0
collected 2 items

test_string_calculator.py::test_empty_string_returns_zero PASSED [ 50%]
test_string_calculator.py::test_single_number_returns_itself PASSED [100%]

===== 2 passed in 0.04s =====

(py3_10_basic) C:\DEV\Code_pro\StringCalculator>
```

Step 5: 쉼표 구분 처리

```
def test_comma_separated_returns_sum():  
    calculator = StringCalculator()  
    assert calculator.add("1,2,3") == 6
```

최종 구현:

```
class StringCalculator:  
    def add(self, numbers: str) -> int:  
        if not numbers:  
            return 0  
  
        num_list = numbers.split(',')  
        return sum(int(n) for n in num_list)
```

```
(py3_10_basic) C:\DEV\Code_pro\StringCalculator>pytest -v  
===== test session starts =====  
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe  
cachedir: .pytest_cache  
rootdir: C:\DEV\Code_pro\StringCalculator  
plugins: cov-7.0.0  
collected 3 items  
  
test_string_calculator.py::test_empty_string_returns_zero PASSED [ 33%  
test_string_calculator.py::test_single_number_returns_itself PASSED [ 66%  
test_string_calculator.py::test_comma_separated_returns_sum PASSED [100%  
  
===== 3 passed in 0.06s =====
```

Step 6: Refactor

```
class StringCalculator:
    def add(self, numbers: str) -> int:
        """
        문자열 형태의 숫자들의 합을 반환

        Args:
            numbers: 쉼표로 구분된 숫자 문자열

        Returns:
            int: 숫자들의 합
        """
        if self._is_empty(numbers):
            return 0

        return self._calculate_sum(numbers)

    def _is_empty(self, numbers: str) -> bool:
        return not numbers or numbers.isspace()

    def _calculate_sum(self, numbers: str) -> int:
        num_list = numbers.split(',')
        return sum(int(n.strip()) for n in num_list)
```

```
(py3_10_basic) C:\DEV\Code_pro\StringCalculator>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\Code_pro\StringCalculator
plugins: cov-7.0.0
collected 3 items

test_string_calculator.py::test_empty_string_returns_zero PASSED [ 33%]
test_string_calculator.py::test_single_number_returns_itself PASSED [ 66%]
test_string_calculator.py::test_comma_separated_returns_sum PASSED [100%]

===== 3 passed in 0.02s =====
```

『3과목』 최소구현 + 리뷰





클라이언트 (Client)

클라이언트 (Client)

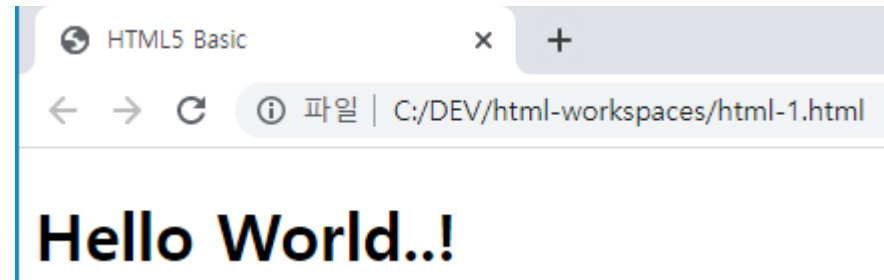
- 사용자가 직접 보고 조작하는 부분
- 웹 브라우저, 모바일 앱
- 역할: 사용자 입력 → 서버에 요청
- 구성 요소
 - HTML: 구조 (뼈대)
 - CSS: 디자인 (외관)
 - JavaScript: 동작 (기능)

실습 1: HTML 예제 만들어 보기

첫 번째 . HTML 예제

- **html-1.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Basic</title>
</head>
<body>
  <h1>Hello World..!</h1>
</body>
</html>
```



- **Week_2\html-workspaces**

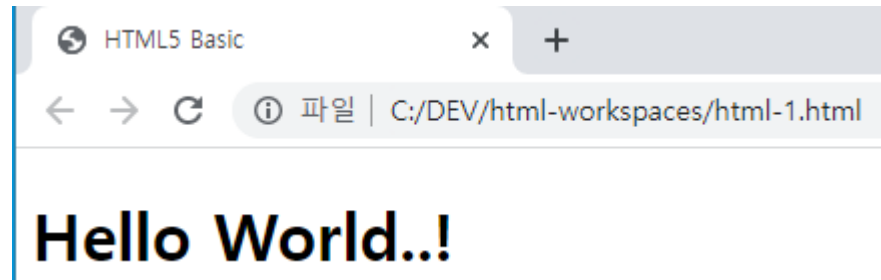
- **html-1.html** 파일을 입력 후, **Alt+B**를 클릭하면 구현된 웹페이지를 바로 볼 수 있습니다.

실습 1: HTML 예제 만들어 보기

첫 번째 . HTML 예제

- **html-1.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Basic</title>
</head>
<body>
  <h1>Hello World..!</h1>
</body>
</html>
```



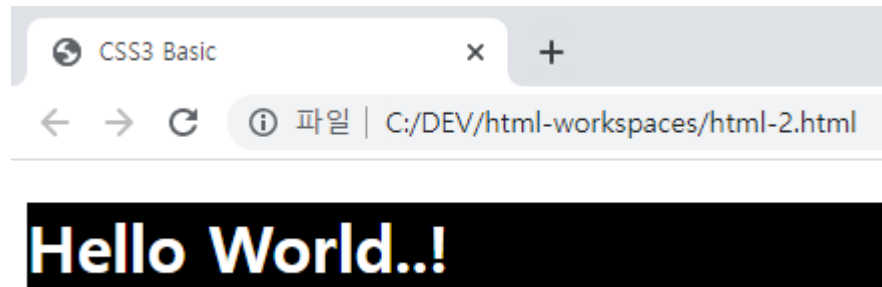
- **html-1.html** 파일을 입력 후, **Alt+B**를 클릭하면 구현된 웹페이지를 바로 볼 수 있습니다.

실습 1: HTML 예제 만들어 보기

첫 번째 HTML/CSS3 예제-방법 1

● html-2.html

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS3 Basic</title>
  <style>
    h1 {
      color: white;
      background: black;
    }
  </style>
</head>
<body>
  <h1>Hello World..!</h1>
</body>
</html>
```



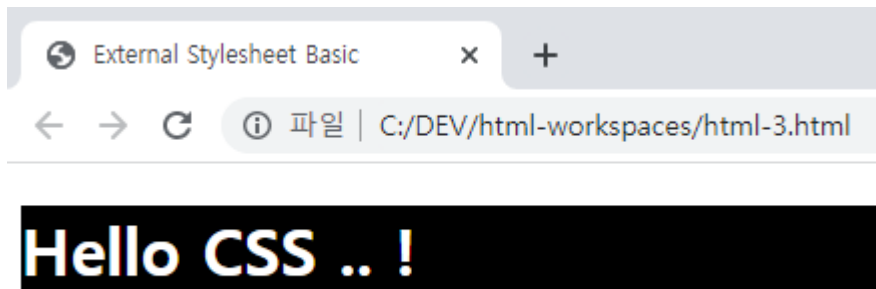
첫 번째 HTML/CSS3 예제-방법 2

● html-3.html

```
<!DOCTYPE html>
<html>
<head>
  <title>External Stylesheet Basic</title>
  <link rel="stylesheet" href="Style.css" />
</head>
<body>
  <h1>Hello CSS .. !</h1>
</body>
</html>
```

● Style.css

```
h1 {
  color: white;
  background: black;
}
```

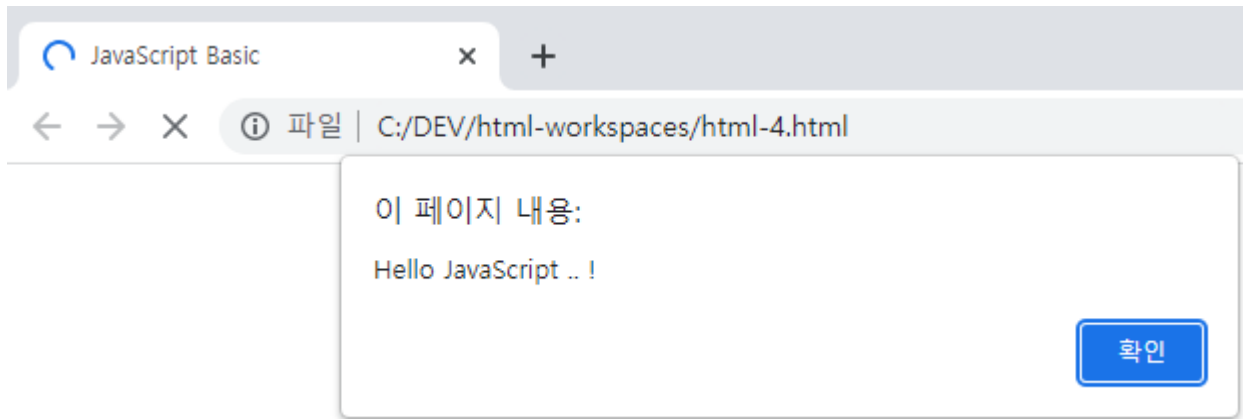


첫 번째 HTML/JavaScript 예제-방법 1

● html-4.html

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Basic</title>
  <script>
    // 경고창을 출력합니다.
    alert('Hello JavaScript .. !');
  </script>
</head>
<body>

</body>
</html>
```



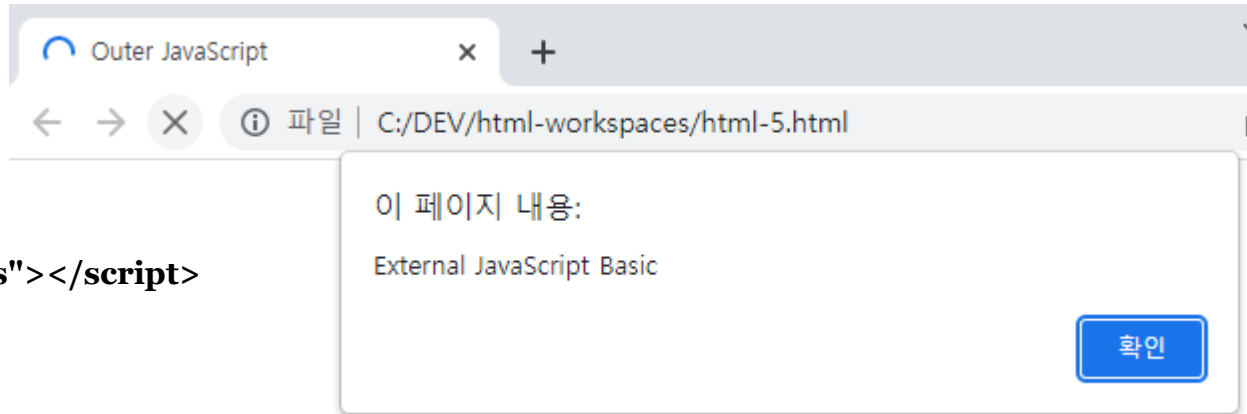
실습 1: HTML 예제 만들어 보기

첫 번째 HTML/JavaScript 예제-방법 1

● html-5.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Outer JavaScript</title>
  <script src="OuterJavaScript.js"></script>
</head>
<body>

</body>
</html>
```



● OuterJavaScript.js

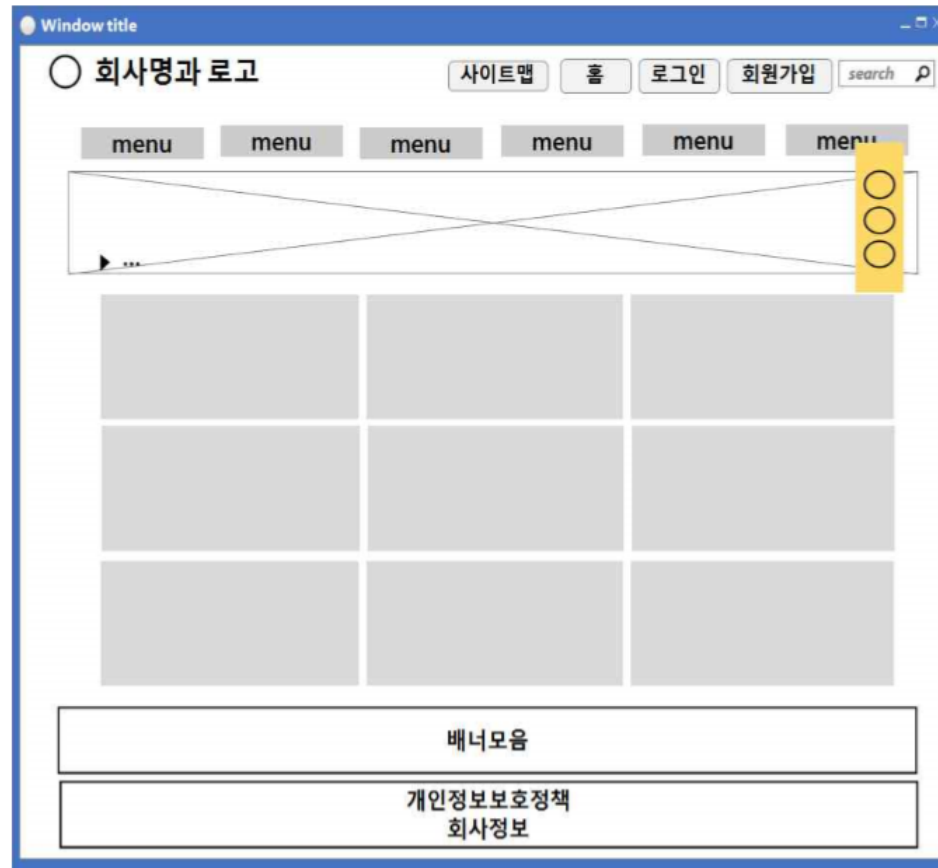
```
alert('External JavaScript Basic');
```



실습 1: HTML 예제 만들어 보기

구조를 파악

- 주어진 웹 페이지 화면 설계 예시 자료를 기준으로 전체적인 구조를 파악한다.





실습 1: HTML 예제 만들어 보기

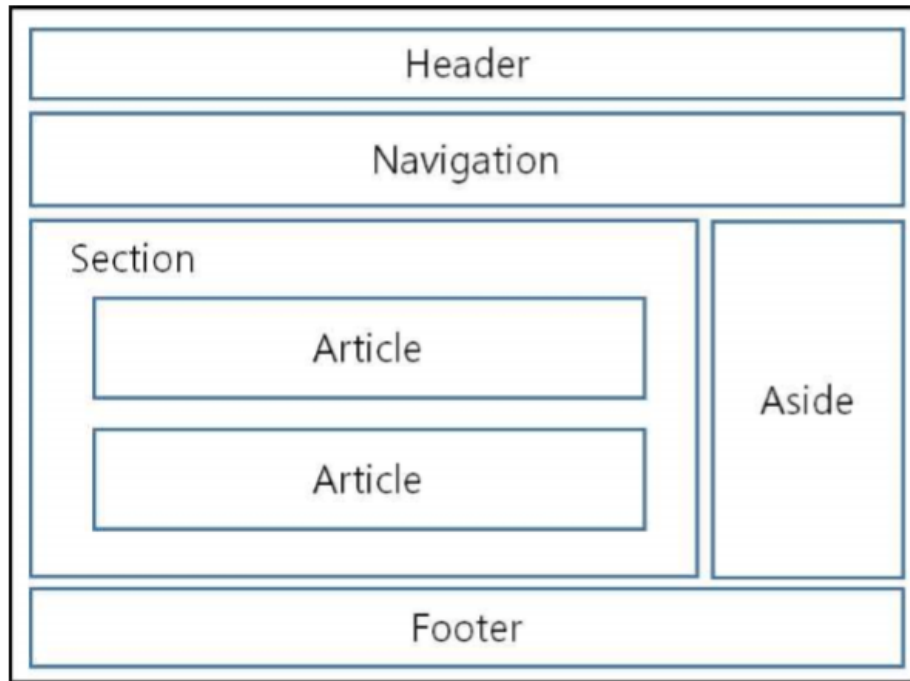
구조와 구성요소를 파악

- 주어진 웹 페이지 화면 설계 예시 자료를 기준으로 전체적인 구조와 구성요소를 파악한다.
- 웹페이지 화면 설계 예시



레이아웃 기본 개념

- 웹 페이지 구성요소를 기준으로 웹 페이지의 레이아웃을 확인한다.
- HTML5 레이아웃 관련 태그 위치



- **Header** - 해당 페이지의 헤더 영역을 지정하고 주로 로고나 회사명, 사이트 맵, 로그인/회원가입 버튼, 검색 버튼 등이 위치한다.
- **Navigation** - 본문의 주요 내비게이션(메인 메뉴) 영역을 지정한다.
- **Section** - 해당 페이지의 콘텐츠 영역을 지정할 때 사용하며 헤더, 푸터 태그와 비교해서 영역을 구분 지정할 때 사용한다.
- **Article** - 독립적인 콘텐츠 항목에 대한 영역을 지정할 때 사용한다.
- **Aside** - 본문 내용 이외에 표현하고자 하는 기타 내용이 있을 경우에 영역을 지정 할 때 사용하거나 서브 메뉴를 표시하고자 할 때 사용하기도 한다.
- **Footer** - 본문 내용의 아래에 위치하도록 지정하며 주로 개인정보 보호정책, 회사 주소 등을 작성한다.

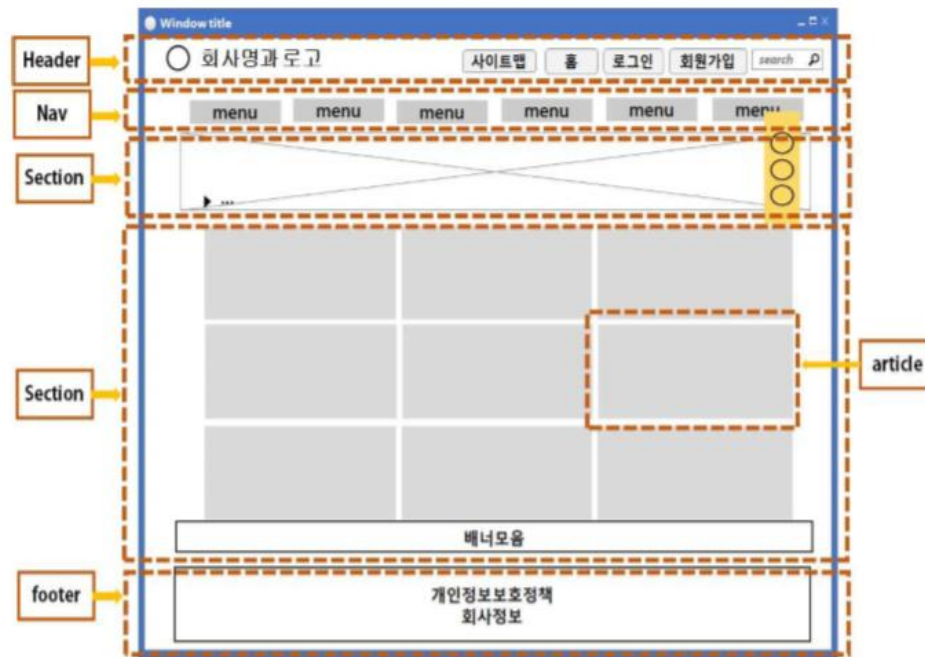
레이아웃 기본 개념과 공간 분할 작업

- 웹 페이지 구성 영역을 내용을 중심으로 하여 공간 분할 작업을 수행한다.
- 웹 페이지 전체를 3개의 영역(상단-가운데-하단)으로 분할한다.
- 공간 분할(레이아웃 구분)



레이아웃 기본 개념과 공간 분할 작업

- 웹 페이지 구성 영역을 내용을 중심으로 하여 공간 분할 작업을 수행한다.
- 분할된 영역을 구현하여야 할 내용을 기준으로 세분화한다.
- 공간 분할(레이아웃 세분화 결과물)





실습 1: HTML 예제 만들어 보기

태그 방식을 확인

- 파악된 구조 전체에 대한 전체적인 레이아웃 구현 위해 공간 분할 방식이나 시맨틱 태그 방식을 확인한다.
- 레이아웃 구현 예시(<div> 태그와 시맨틱 태그)

```
<body>
  <div>
    <h1>oo고등학교</h1>
  </div>
  <div>
    <ul>
      <li><a href="#">사이트맵</a></li>
      <li><a href="#">로그인</a></li>
      <li><a href="#">회원가입</a></li>
      <li><a href="#">검색창</a></li>
    </ul>
  </div>
```

```
<body>
  <header>
    <h1>oo고등학교</h1>
  </header>
  <nav>
    <ul>
      <li><a href="#">사이트맵</a></li>
      <li><a href="#">로그인</a></li>
      <li><a href="#">회원가입</a></li>
      <li><a href="#">검색창</a></li>
    </ul>
  </nav>
```

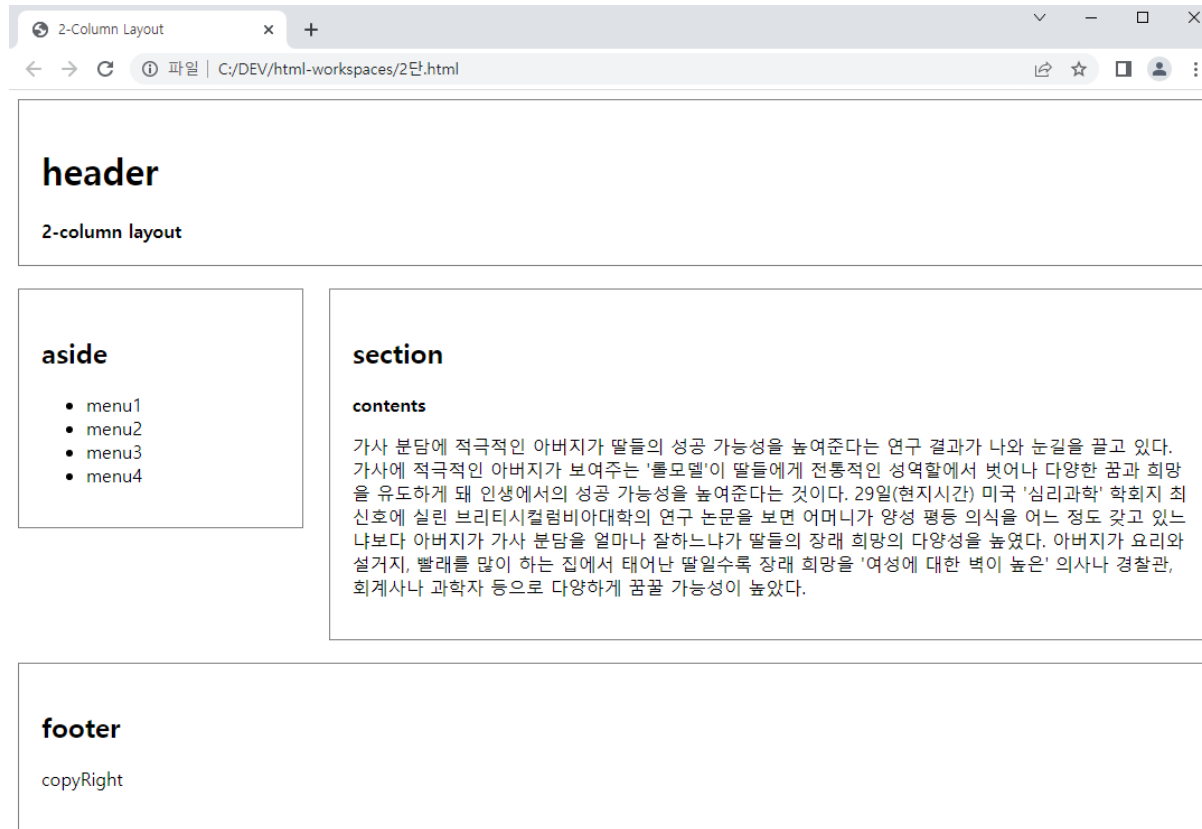


실습 1: HTML 예제 만들어 보기

2단

● 2단.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<title>2-Column Layout</title>
<style type="text/css">
#wrapper{
  width:940px;
  margin: 0px auto;
  padding: 20px;
  border: 1px solid #828282;
}
#header{
  margin-bottom: 20px;
  padding: 20px;
  border: 1px solid #828282;
}
#section{
  width: 70%;
  margin-bottom: 20px;
  padding: 20px;
  float: right;
  border: 1px solid #828282;
}
#aside{
  width: 20%;
  margin-bottom: 20px;
  padding: 20px;
  float: left;
  border: 1px solid #828282;
}
#footer{
  clear: both;
  padding: 20px;
  border: 1px solid #828282;
}
</style>
</head>
```





2단

● 2단.html

```
<body>
  <div id="wrapper">
    <div id="header">
      <h1>header</h1>
      <b>2-column layout</b>
    </div>
    <div id="section">
      <h2>section</h2>
      <b>contents</b>
      <p>가사 분담에 적극적인 아버지가 딸들의 성공 가능성을 높여준다는 연구 결과가 나와 눈길을 끌고 있다. 가사에
      적극적인 아버지가 보여주는 '롤모델'이 딸들에게 전통적인 성역할에서 벗어나 다양한 꿈과 희망을 유도하게 돼 인생에서의 성공
      가능성을 높여준다는 것이다. 29일(현지시간) 미국 '심리과학' 학회지 최신호에 실린 브리티시컬럼비아대학의 연구 논문을
      보면 어머니가 양성 평등 의식을 어느 정도 갖고 있느냐보다 아버지가 가사 분담을 얼마나 잘하느냐가 딸들의 장래 희망의
      다양성을 높였다. 아버지가 요리와 설거지, 빨래를 많이 하는 집에서 태어난 딸일수록 장래 희망을 '여성에 대한 벽이 높은'
      의사나 경찰관, 회계사나 과학자 등으로 다양하게 꿈꿀 가능성이 높았다.
      </p>
    </div>
    <div id="aside">
      <h2>aside</h2>
      <ul>
        <li>menu1</li>
        <li>menu2</li>
        <li>menu3</li>
        <li>menu4</li>
      </ul>
    </div>
    <div id="footer">
      <h2>footer</h2>
      <p>copyRight</p>
    </div>
  </div>
</body>
</html>
```



2단

● 2단.html

```
<body>
  <div id="wrapper">
    <div id="header">
      <h1>header</h1>
      <b>2-column layout</b>
    </div>
    <div id="section">
      <h2>section</h2>
      <b>contents</b>
      <p>가사 분담에 적극적인 아버지가 딸들의 성공 가능성을 높여준다는 연구 결과가 나와 눈길을 끌고 있다. 가사에
      적극적인 아버지가 보여주는 '롤모델'이 딸들에게 전통적인 성역할에서 벗어나 다양한 꿈과 희망을 유도하게 돼 인생에서의 성공
      가능성을 높여준다는 것이다. 29일(현지시간) 미국 '심리과학' 학회지 최신호에 실린 브리티시컬럼비아대학의 연구 논문을
      보면 어머니가 양성 평등 의식을 어느 정도 갖고 있느냐보다 아버지가 가사 분담을 얼마나 잘하느냐가 딸들의 장래 희망의
      다양성을 높였다. 아버지가 요리와 설거지, 빨래를 많이 하는 집에서 태어난 딸일수록 장래 희망을 '여성에 대한 벽이 높은'
      의사나 경찰관, 회계사나 과학자 등으로 다양하게 꿈꿀 가능성이 높았다.
      </p>
    </div>
    <div id="aside">
      <h2>aside</h2>
      <ul>
        <li>menu1</li>
        <li>menu2</li>
        <li>menu3</li>
        <li>menu4</li>
      </ul>
    </div>
    <div id="footer">
      <h2>footer</h2>
      <p>copyRight</p>
    </div>
  </div>
</body>
</html>
```



3단

● 3단.html

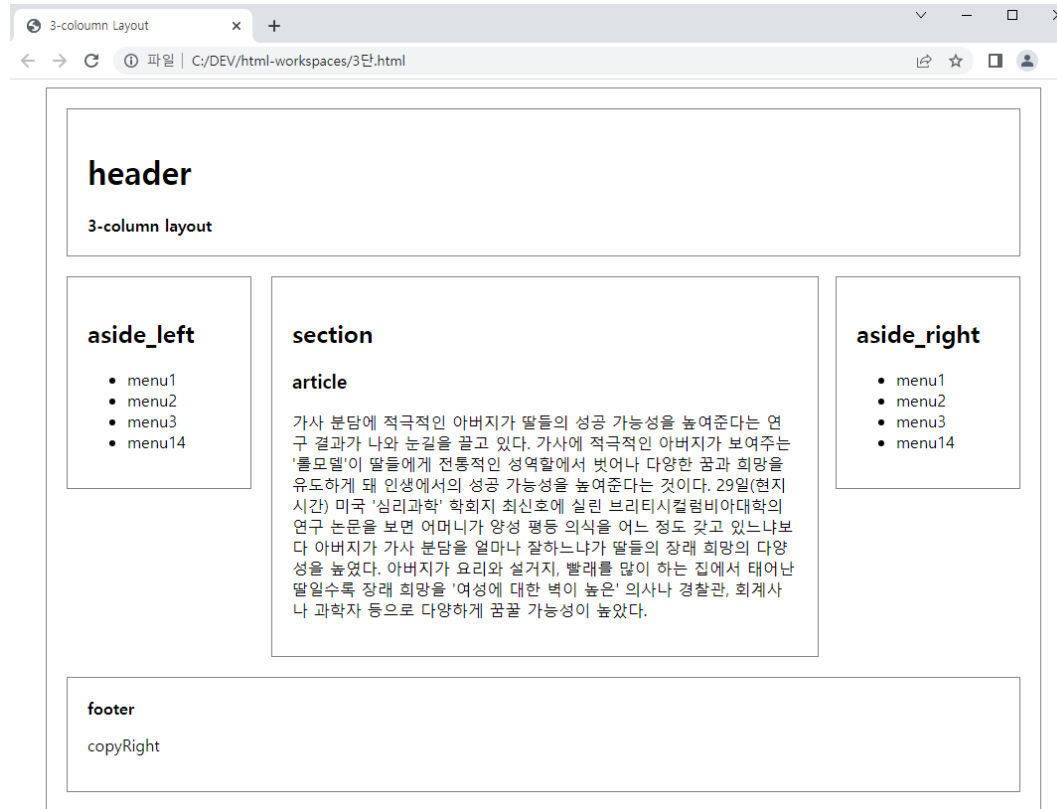
```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<title>3-coloumn Layout</title>
<style type="text/css">
#wrapper{
  width: 960px;
  margin: 0px auto;
  padding: 20px;
  border: 1px solid #828282;
}
#header{
  margin-bottom: 20px;
  padding: 20px;
  border: 1px solid #828282;
}
#aside_left{
  margin: 0 20px 20px 0;
  padding: 20px;
  width: 15%;
  float: left;
  border: 1px solid #828282;
}
#section{
  margin-bottom: 20px;
  padding: 20px;
  width: 53%;
  float:left;
  border: 1px solid #828282;
}
```



실습 1: HTML 예제 만들어 보기

3단

```
#aside_right{
  margin-bottom: 20px;
  padding: 20px;
  width: 15%;
  float: right;
  border: 1px solid #828282;
}
#footer{
  clear: both;
  padding: 20px;
  border: 1px solid #828282;
}
</style>
</head>
```





3단

```
<body>
  <div id="wrapper">

    <div id="header">
      <h1>header</h1>
      <b>3-column layout</b>
    </div>

    <div id="aside_left">
      <h2>aside_left</h2>
      <ul>
        <li>menu1</li>
        <li>menu2</li>
        <li>menu3</li>
        <li>menu14</li>
      </ul>
    </div>
    <div id="section">
      <h2>section</h2>
      <h3>article</h3>
      <p>가사 분담에 적극적인 아버지가 딸들의 성공 가능성을 높여준다는 연구 결과가 나와 눈길을 끌고 있다. 가사에
      적극적인 아버지가 보여주는 '롤모델'이 딸들에게 전통적인 성역할에서 벗어나 다양한 꿈과 희망을 유도하게 돼 인생에서의 성공
      가능성을 높여준다는 것이다. 29일(현지시간) 미국 '심리과학' 학회지 최신호에 실린 브리티시컬럼비아대학의 연구 논문을
      보면 어머니가 양성 평등 의식을 어느 정도 갖고 있느냐보다 아버지가 가사 분담을 얼마나 잘하느냐가 딸들의 장래 희망의
      다양성을 높였다. 아버지가 요리와 설거지, 빨래를 많이 하는 집에서 태어난 딸일수록 장래 희망을 '여성에 대한 벽이 높은'
      의사나 경찰관, 회계사나 과학자 등으로 다양하게 꿈꿀 가능성이 높았다.</p>
    </div>
    <div id="aside_right">
      <h2>aside_right</h2>
      <ul>
        <li>menu1</li>
        <li>menu2</li>
        <li>menu3</li>
        <li>menu14</li>
      </ul>
    </div>

    <div id="footer">
      <b>footer</b>
      <p>copyRight</p>
    </div>
  </div>
</body>
</html>
```

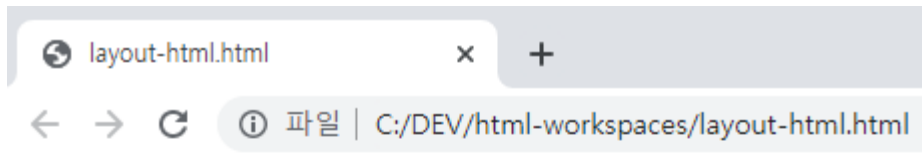



실습 1: HTML 예제 만들어 보기

HTML

● Layout-html.html

```
<!DOCTYPE html>
<html>
  <head>
    <title></title>
  </head>
  <body>
    <div>
      <header>
        <h1>상단 헤더 영역</h1>
      </header>
      <div>
        <h2>좌측 메뉴 영역</h2>
        <ul>
          <li><a href="#">홈</a></li>
          <li><a href="#">학교소개</a></li>
          <li><a href="#">Contact Us</a></li>
        </ul>
      </div>
      <div>
        <section>
          <h2>본문</h2>
          <p>
            첫 번째 단락 본문글입니다.
          </p>
          <p>
            두 번째 단락 본문글입니다.
          </p>
        </section>
      </div>
      <footer>
        <p>
          바닥글
        </p>
      </footer>
    </div>
  </body>
</html>
```



상단 헤더 영역

좌측 메뉴 영역

- [홈](#)
- [학교소개](#)
- [Contact Us](#)
-

본문

첫 번째 단락 본문글입니다.

두 번째 단락 본문글입니다.

바닥글



실습 1: HTML 예제 만들어 보기

HTML+CSS3

● layout-htmlcss3.html

```
<!DOCTYPE html>
<html>
<head>
  <title></title>
  <style>
    .container{
      border: 1px solid red;
      width: 940px;
      margin: 0px auto;
      padding: 20px;
    }

    header{
      margin-bottom: 20px;
      padding: 20px;
      border: 1px solid red;
    }

    #sidebar{
      padding: 20px;
      border: 1px solid #bcbebc;
      width: 260px;
      margin-bottom: 20px;
      float: left;
    }

    #sidebar > ul > li{
      font-size: 20pt;
      color: aqua;
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="header">
      <h1>HTML+CSS3</h1>
    </div>
    <div class="content">
      <div id="sidebar">
        <ul>
          <li>Home</li>
          <li>About</li>
          <li>Contact</li>
        </ul>
      </div>
      <div class="main">
        <h2>Hello World</h2>
      </div>
    </div>
  </div>
</body>
</html>
```



실습 1: HTML 예제 만들어 보기



HTML+CSS3

```
.content{  
  width: 580px;  
  padding: 20px;  
  margin-bottom: 20px;  
  border: 1px solid #bebebe;  
  float: right;  
}
```

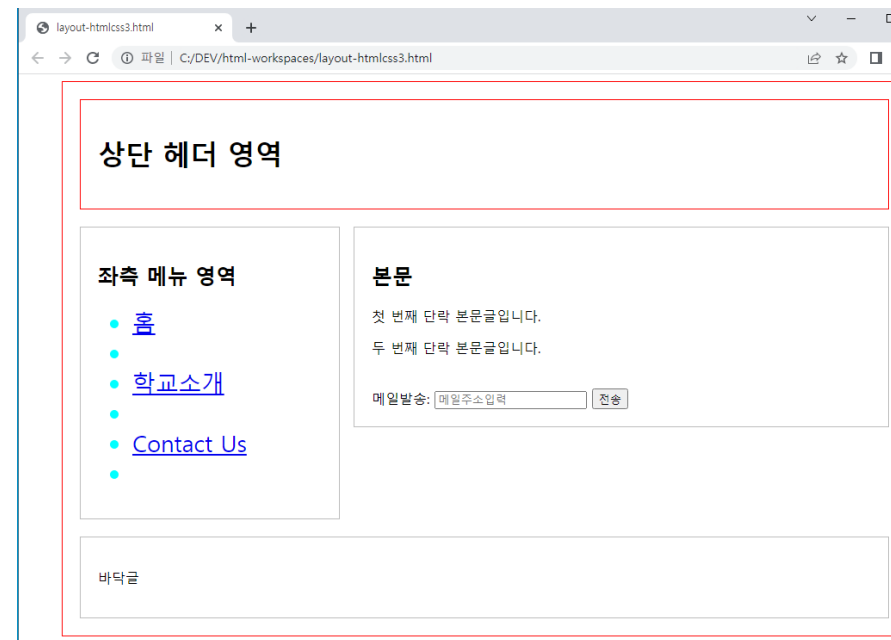
```
footer{  
  clear: both;  
  padding: 20px;  
  border: 1px solid #bebebe;  
}
```

```
</style>  
</head>
```



HTML+CSS3

```
<body>
<div class="container">
  <header>
    <h1>상단 헤더 영역</h1>
  </header>
  <div id="sidebar">
    <h2>좌측 메뉴 영역</h2>
    <ul>
      <li><a href="#">홈</a></li>
      <li><a href="#">학교소개</a></li>
      <li><a href="#">Contact Us</a></li>
    </ul>
  </div>
  <div class="content">
    <section>
      <h2>본문</h2>
      <p>
        첫 번째 단락 본문글입니다.
      </p>
      <p>
        두 번째 단락 본문글입니다.
      </p>
      <br />
      <form id="mailform" action="" method="post">
        메일발송: <input type="text" id="txtemail" placeholder="메일주소입력" />
        <input type="submit" id="btncsend" value="전송" onclick="returnfnSend();" />
      </form>
    </section>
  </div>
  <footer>
    <p>
      바닥글
    </p>
  </footer>
</div>
</body>
</html>
```





실습 1: HTML 예제 만들어 보기

HTML+CSS3+JavaScript

● layout-javascript.html

```
<!DOCTYPE html>
<html>
  <head>
    <title></title>
    <style>
      .container{
        border: 1px solid red;
        width: 940px;
        margin: 0px auto;
        padding: 20px;
      }

      header{
        margin-bottom: 20px;
        padding: 20px;
        border: 1px solid red;
      }

      #sidebar{
        padding: 20px;
        border: 1px solid #bcbcbc;
        width: 260px;
        margin-bottom: 20px;
        float: left;
      }

      #sidebar > ul > li{
        font-size: 20pt;
        color: aqua;
      }
    </style>
  </head>
  <body>
  </body>
</html>
```



실습 1: HTML 예제 만들어 보기

HTML+CSS3+JavaScript

```
.content{  
  width: 580px;  
  padding: 20px;  
  margin-bottom: 20px;  
  border: 1px solid #bcbcbc;  
  float: right;  
}  
  
footer{  
  clear: both;  
  padding: 20px;  
  border: 1px solid #bcbcbc;  
}  
  
</style>  
</head>
```



실습 1: HTML 예제 만들어 보기

HTML+CSS3+JavaScript

```
<body>
<div class="container">
  <header>
    <h1>상단 헤더 영역</h1>
  </header>
  <div id="sidebar">
    <h2>좌측 메뉴 영역</h2>
    <ul>
      <li><a href="#">홈</a></li>
      <li><a href="#">학교소개</a></li>
      <li><a href="#">Contact Us</a></li>
    </ul>
  </div>
  <div class="content">
    <section>
      <h2>본문</h2>
      <p>
        첫 번째 단락 본문글입니다.
      </p>
      <p>
        두 번째 단락 본문글입니다.
      </p>
      <br />
      <form id="mailform" action="" method="post">
        메일발송: <input type="text" id="txtemail" placeholder="메일주소입력" />
        <input type="submit" id="btntsend" value="전송" onclick="returnfnSend();" />
      </form>
    </section>
  </div>
  <footer>
    <p>
      바닥글
    </p>
  </footer>
</div>
```

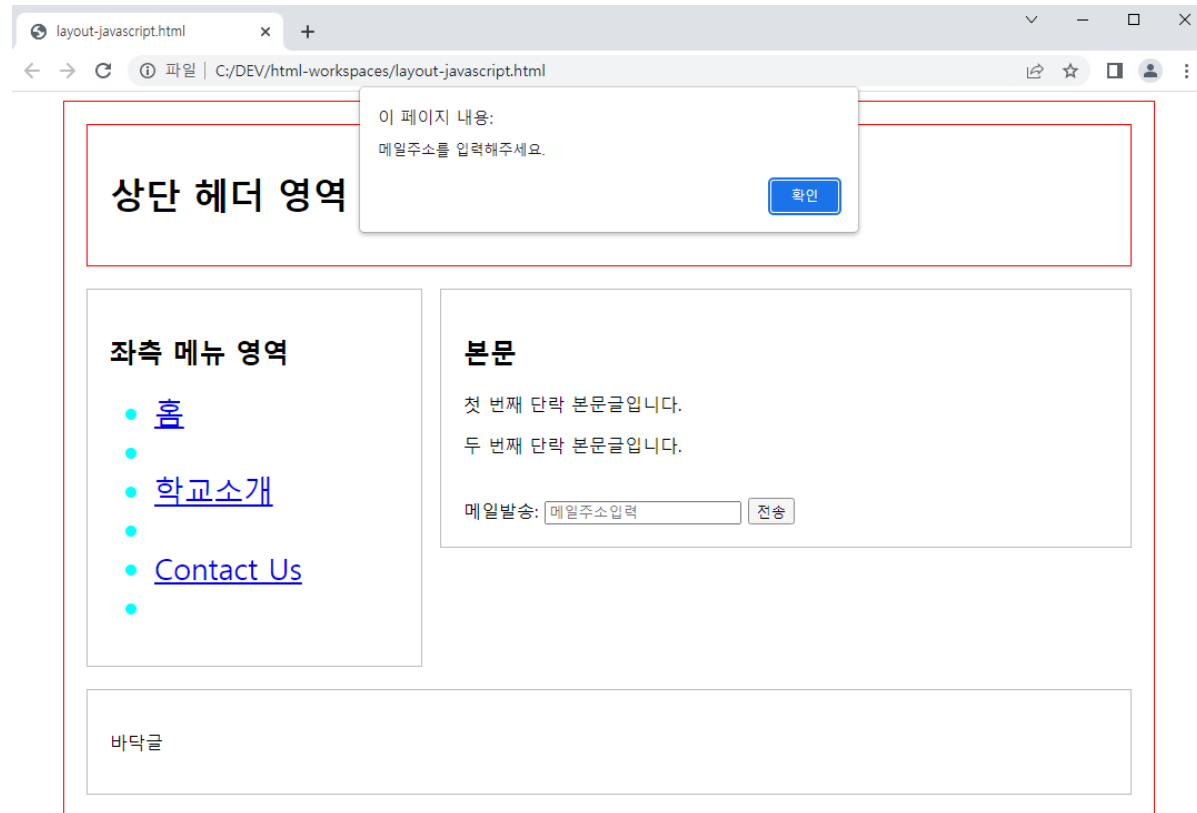


실습 1: HTML 예제 만들어 보기

HTML+CSS3+JavaScript

```
<!-- 스크립트 정의 영역 -->
<script type="text/javascript">
  function returnfnSend() {
    if (document.getElementById("txtemail").value == "") {
      alert("메일주소를 입력해주세요.");
      document.getElementById("txtemail").focus();
      return false;
    }
  }
</script>

</body>
</html>
```





실습 1: HTML 예제 만들어 보기

HTML+CSS3+jQuery

● layout-jQuery.html / jquery-3.3.1.js

```
<!DOCTYPE html>
<html>
  <head>
    <title></title>
    <style>
      .container{
        border: 1px solid red;
        width: 940px;
        margin: 0px auto;
        padding: 20px;
      }

      header{
        margin-bottom: 20px;
        padding: 20px;
        border: 1px solid red;
      }

      #sidebar{
        padding: 20px;
        border: 1px solid #bcbcbc;
        width: 260px;
        margin-bottom: 20px;
        float: left;
      }

      #sidebar > ul > li{
        font-size: 20pt;
        color: aqua;
      }
    </style>
  </head>
  <body>
  </body>
</html>
```



실습 1: HTML 예제 만들어 보기

HTML+CSS3+jQuery

```
.content{  
  width: 580px;  
  padding: 20px;  
  margin-bottom: 20px;  
  border: 1px solid #bcbcbc;  
  float: right;  
}  
  
footer{  
  clear: both;  
  padding: 20px;  
  border: 1px solid #bcbcbc;  
}  
  
</style>  
</head>
```



실습 1: HTML 예제 만들어 보기

HTML+CSS3+jQuery

```
<body>
<div class="container">
  <header>
    <h1>상단 헤더 영역</h1>
  </header>
  <div id="sidebar">
    <h2>좌측 메뉴 영역</h2>
    <ul>
      <li><a href="#">홈</a></li>
      <li><a href="#">학교소개</a></li>
      <li><a href="#">Contact Us</a></li>
    </ul>
  </div>
  <div class="content">
    <section>
      <h2>본문</h2>
      <p>
        첫 번째 단락 본문글입니다.
      </p>
      <p>
        두 번째 단락 본문글입니다.
      </p>
      <br />
      <form id="mailform" action="" method="post">
        메일발송: <input type="text" id="txtemail" placeholder="메일주소입력" />
        <input type="submit" id="btnsend" value="전송" />
      </form>
    </section>
  </div>
  <footer>
    <p>
      바닥글
    </p>
  </footer>
</div>
```

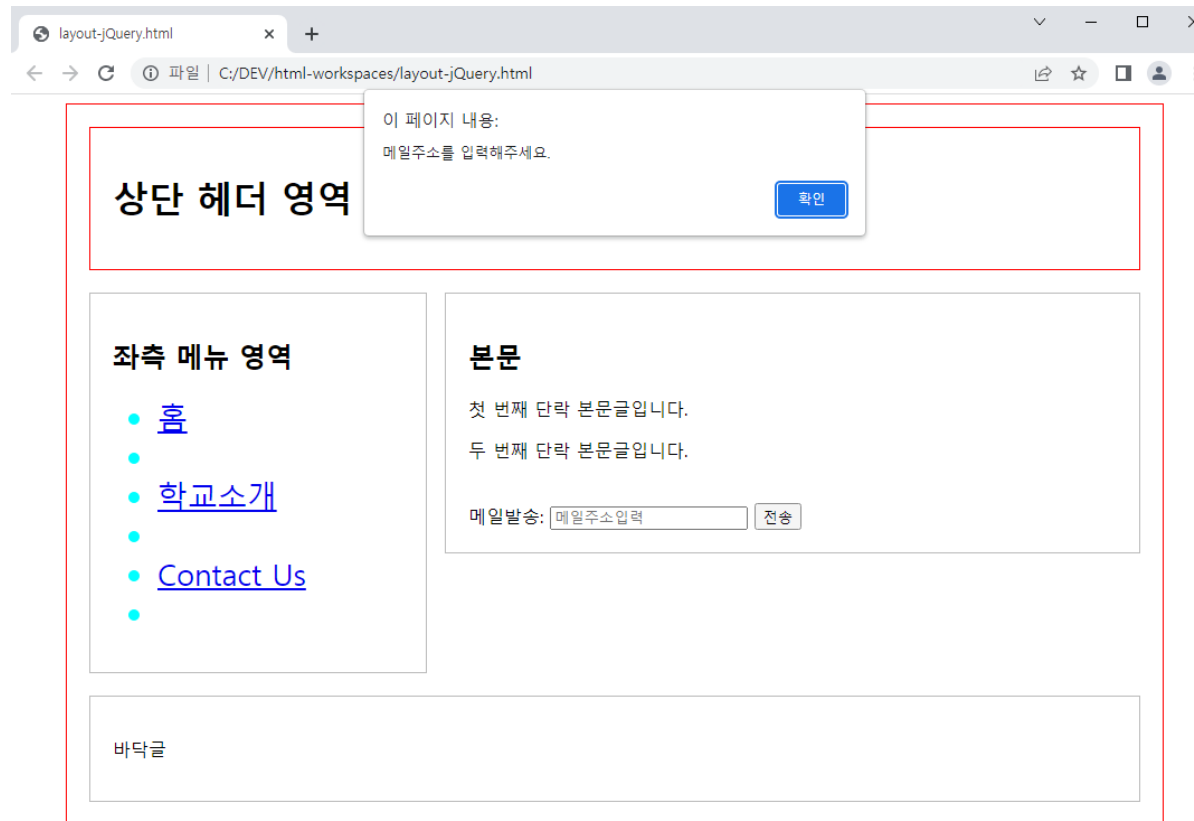


실습 1: HTML 예제 만들어 보기

HTML+CSS3+jQuery

```
<script src="jquery-3.3.1.js" type="text/javascript"></script>
<script type="text/javascript">
  $(document).ready(function () {
    $("#btnsend").click(function () {
      if ($("#txtemail").val() == "") {
        alert("메일주소를 입력해주세요.");
        $("#txtemail").focus();
        return false;
      }
    });
  });
</script>

</body>
</html>
```





실습 1: HTML 예제 만들어 보기

| jQuery 다운로드



jquery 다운로드



[전체](#) [이미지](#) [동영상](#) [뉴스](#) [지도](#) [더보기](#) [설정](#) [도구](#)

검색결과 약 428,000개 (0.33초)

Download jQuery | jQuery

<https://jquery.com/download/> ▼ 이 페이지 번역하기

You can also **download** a sourcemap file for use when debugging with a compressed file. The map file is not required for users to run **jQuery**, it just improves the ...

[Fixed dependencies in release ...](#) · [jQuery CDN](#) · [jQuery 1.11 and 2.1 Released](#)

.....



실습 1: HTML 예제 만들어 보기

| jQuery 다운로드


write less, do more.

[Download](#) [API Documentation](#) [Blog](#) [Plugins](#) [Browser Support](#)

Downloading jQuery

Compressed and uncompressed copies of jQuery files are available. The uncompressed file saves bandwidth and improves performance in production. You can use either a compressed or uncompressed file. The map file is *not* required for users to run jQuery, it just improves performance. In jQuery 1.11.0/2.1.0 the `/** sourceMappingURL` comment is **not included** in the compressed files.

To locally download these files, right-click the link and select "Save as..." from the menu.

jQuery

For help when upgrading jQuery, please see the [upgrade guide](#) most relevant to your version of jQuery.

[Download the compressed, production jQuery 3.3.1](#)

[Download the uncompressed, development jQuery 3.3.1](#)

[Download the map file for jQuery 3.3.1](#)

You can also use the slim build, which excludes the [ajax](#) and [effects](#) modules:

[Download the compressed, production jQuery 3.3.1 slim build](#)

[Download the uncompressed, development jQuery 3.3.1 slim build](#)

[Download the map file for the jQuery 3.3.1 slim build](#)

[jQuery 3.3.1 release notes](#)

Downloading jQuery using npm or Yarn



실습 1: HTML 예제 만들어 보기

| jQuery 다운로드

The screenshot shows the jQuery website with a blue header. The jQuery logo and tagline 'write less, do more.' are at the top left. A navigation bar contains links for 'Download', 'API Documentation', 'Blog', 'Plugins', and 'Browser Support'. The main content area features a 'Categories' sidebar with links to Events, Foundation, jQuery, jQuery UI, Projects, and Weekly News. The 'Recent Posts' sidebar lists three items, including 'jQuery 3.3.1 - fixed dependencies in release tag'. The main article is titled 'jQuery 3.3.1 – fixed dependen' and is dated 'January 20, 2018' by 'Timmy Willison'. The article text explains an issue with jQuery 3.3.0 and recommends version 3.3.1. A 'Download' section provides two URLs for the jQuery 3.3.1 files, which are highlighted in a blue box: <https://code.jquery.com/jquery-3.3.1.js> and <https://code.jquery.com/jquery-3.3.1.min.js>.

jQuery 3.3.1 – fixed dependen

Posted on [January 20, 2018](#) by [Timmy Willison](#)

We encountered an issue in the release for jQuery 3.3.0, so we've released a new version, 3.3.1. This version is identical, but our release dependencies (only used during release) were updated to match the new behavior of npm in version 5+.

jQuery 3.3.1 is now recommended if installing from npm or GitHub. Please see the [jQuery 3.3.0 blog post](#) for all relevant code changes.

Download

You can get the files from the jQuery CDN, or link to them directly:

<https://code.jquery.com/jquery-3.3.1.js>

<https://code.jquery.com/jquery-3.3.1.min.js>

다음 전통적인(Traditional)방식으로 구현된 코드를 TDD GREEN 단계의 최소 구현 코드(GREEN)로 바꿔보세요.

구분	Traditional	GREEN
초기 구조	모든 기능을 한 번에 구현	테스트가 요구한 기능만 구현
복잡도	처음부터 복잡함	매우 단순
결합도	높음	낮음
확장성	변경 어려움	점진적으로 성장
테스트	나중에 추가 / 어려움	처음부터 테스트 기반

Week_2\Code_pro\python_test

실습 2: 결제 시스템 구현

```
# traditional_payment.py
from decimal import Decimal

class PaymentResult:
    def __init__(self, success: bool, amount: Decimal):
        self.success = success
        self.amount = amount

    def __repr__(self):
        return f"PaymentResult(success={self.success}, amount={self.amount})"

class TraditionalPaymentProcessor:
    """초기부터 모든 결제 방식을 한 클래스에 넣는 전통적 방식"""

    def process_payment(
        self,
        amount: Decimal,
        method: str,
        card_number: str = None,
        bank_account: str = None,
        crypto_wallet: str = None,
        paypal_email: str = None,
        apple_token: str = None,
        google_token: str = None
    ) -> PaymentResult:

        if method == "card":
            if not card_number:
                raise ValueError("Card number required")
            # (여기 100줄의 카드 결제 검증 로직이 있다고 가정)
            return PaymentResult(True, amount)
```

실습 2: 결제 시스템 구현

```
elif method == "bank_transfer":
    if not bank_account:
        raise ValueError("Bank account required")
    # (여기 80줄의 은행 이체 로직이 있다고 가정)
    return PaymentResult(True, amount)

elif method == "crypto":
    if not crypto_wallet:
        raise ValueError("Crypto wallet required")
    return PaymentResult(True, amount)

elif method == "paypal":
    if not paypal_email:
        raise ValueError("PayPal email required")
    return PaymentResult(True, amount)

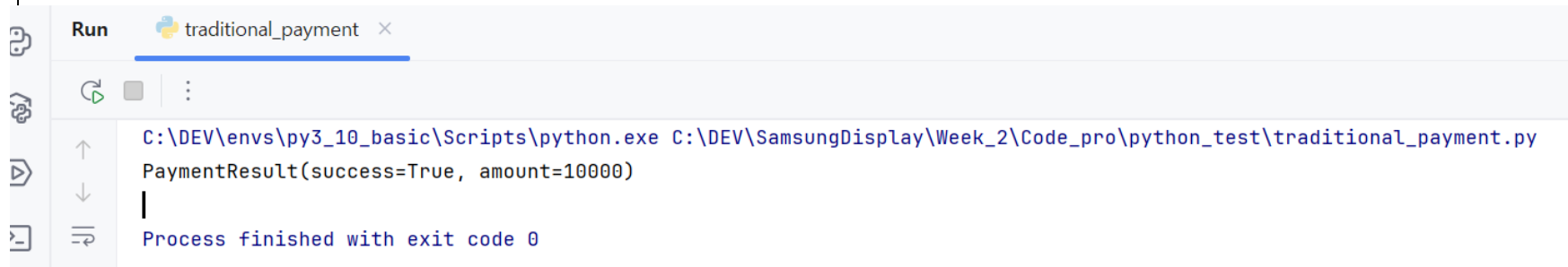
elif method == "apple_pay":
    if not apple_token:
        raise ValueError("ApplePay token required")
    return PaymentResult(True, amount)

elif method == "google_pay":
    if not google_token:
        raise ValueError("GooglePay token required")
    return PaymentResult(True, amount)

else:
    raise ValueError(f"Unknown payment method: {method}")
```

실습 2: 결제 시스템 구현

```
# -----  
# 실행 테스트  
# -----  
if __name__ == "__main__":  
    processor = TraditionalPaymentProcessor()  
    result = processor.process_payment(  
        amount=Decimal("10000"),  
        method="card",  
        card_number="1111-2222-3333-4444"  
    )  
    print(result)
```



The screenshot shows the 'Run' console of a Python IDE. The title bar indicates the file 'traditional_payment.py' is open. The console output shows the command to run the script, the resulting 'PaymentResult' object, and a confirmation that the process finished successfully.

```
Run traditional_payment ×  
C:\DEV\envs\py3_10_basic\Scripts\python.exe C:\DEV\SamsungDisplay\Week_2\Code_pro\python_test\traditional_payment.py  
PaymentResult(success=True, amount=10000)  
Process finished with exit code 0
```

답

TDD GREEN 단계 최소 구현 버전 (GREEN Version – Sprint 1 + Sprint 2)

GREEN – Sprint 1: 카드 결제만 구현 green_payment_step1.py

```
# green_payment_step1.py
from decimal import Decimal

class PaymentResult:
    def __init__(self, success: bool, amount: Decimal):
        self.success = success
        self.amount = amount

    def __repr__(self):
        return f"PaymentResult(success={self.success}, amount={self.amount})"

class PaymentProcessor:
    """GREEN 단계: 카드 결제만 필요한 만큼만 구현"""

    def process_card_payment(self, amount: Decimal, card_number: str) -> PaymentResult:
        return PaymentResult(success=True, amount=amount)

# -----
# 실행 예시
# -----
if __name__ == "__main__":
    processor = PaymentProcessor()
    result = processor.process_card_payment(
        amount=Decimal("10000"),
        card_number="1234-5678-9012-3456"
    )
    print(result)
```

✓ 특징

카드 결제 기능만 있음
다른 결제 기능은 존재하지 않음
테스트가 요구한 것 "만" 구현

● GREEN – Sprint 2: 은행 이체 추가 green_payment_step2.py

```
# green_payment_step2.py
from decimal import Decimal

class PaymentResult:
    def __init__(self, success: bool, amount: Decimal):
        self.success = success
        self.amount = amount

    def __repr__(self):
        return f"PaymentResult(success={self.success}, amount={self.amount})"

class PaymentProcessor:
    """GREEN 단계 확장: 카드 + 은행 이체만 최소 구현"""

    def process_card_payment(self, amount: Decimal, card_number: str) -> PaymentResult:
        return PaymentResult(success=True, amount=amount)

    def process_bank_transfer(self, amount: Decimal, account: str) -> PaymentResult:
        return PaymentResult(success=True, amount=amount)

# -----
# 실행 예시
# -----
if __name__ == "__main__":
    processor = PaymentProcessor()

    print(processor.process_card_payment(Decimal("10000"), "1234-5678"))
    print(processor.process_bank_transfer(Decimal("50000"), "110-123-456789"))
```

✓ 특징

카드 → 은행 이체로 기능이 점진적으로 추가
역시 최소한의 코드만 존재
다른 결제 방식은 아직 구현되어 있지 않음
테스트 추가 → 기능 추가 라는 TDD 흐름 유지

● GREEN – Sprint 2: 은행 이체 추가 green_payment_step2.py

```
# green_payment_step2.py
from decimal import Decimal

class PaymentResult:
    def __init__(self, success: bool, amount: Decimal):
        self.success = success
        self.amount = amount

    def __repr__(self):
        return f"PaymentResult(success={self.success}, amount={self.amount})"

class PaymentProcessor:
    """GREEN 단계 확장: 카드 + 은행 이체만 최소 구현"""

    def process_card_payment(self, amount: Decimal, card_number: str) -> PaymentResult:
        return PaymentResult(success=True, amount=amount)

    def process_bank_transfer(self, amount: Decimal, account: str) -> PaymentResult:
        return PaymentResult(success=True, amount=amount)

# -----
# 실행 예시
# -----
if __name__ == "__main__":
    processor = PaymentProcessor()

    print(processor.process_card_payment(Decimal("10000"), "1234-5678"))
    print(processor.process_bank_transfer(Decimal("50000"), "110-123-456789"))
```

✓ 특징

카드 → 은행 이체로 기능이 점진적으로 추가
역시 최소한의 코드만 존재
다른 결제 방식은 아직 구현되어 있지 않음
테스트 추가 → 기능 추가 라는 TDD 흐름 유지

『 4과목 』 리팩토링 + 코드 리뷰 + 배포



다음 코드를 리팩토링 합니다.

❌ 나쁜 코드 (Code Smell)

```
def p(u, m, q): # 의미 없는 변수명
    if u and m:
        t = m.price * q
        if t > 10000: # 매직 넘버
            t = t * 0.9
        return t
    return 0
```

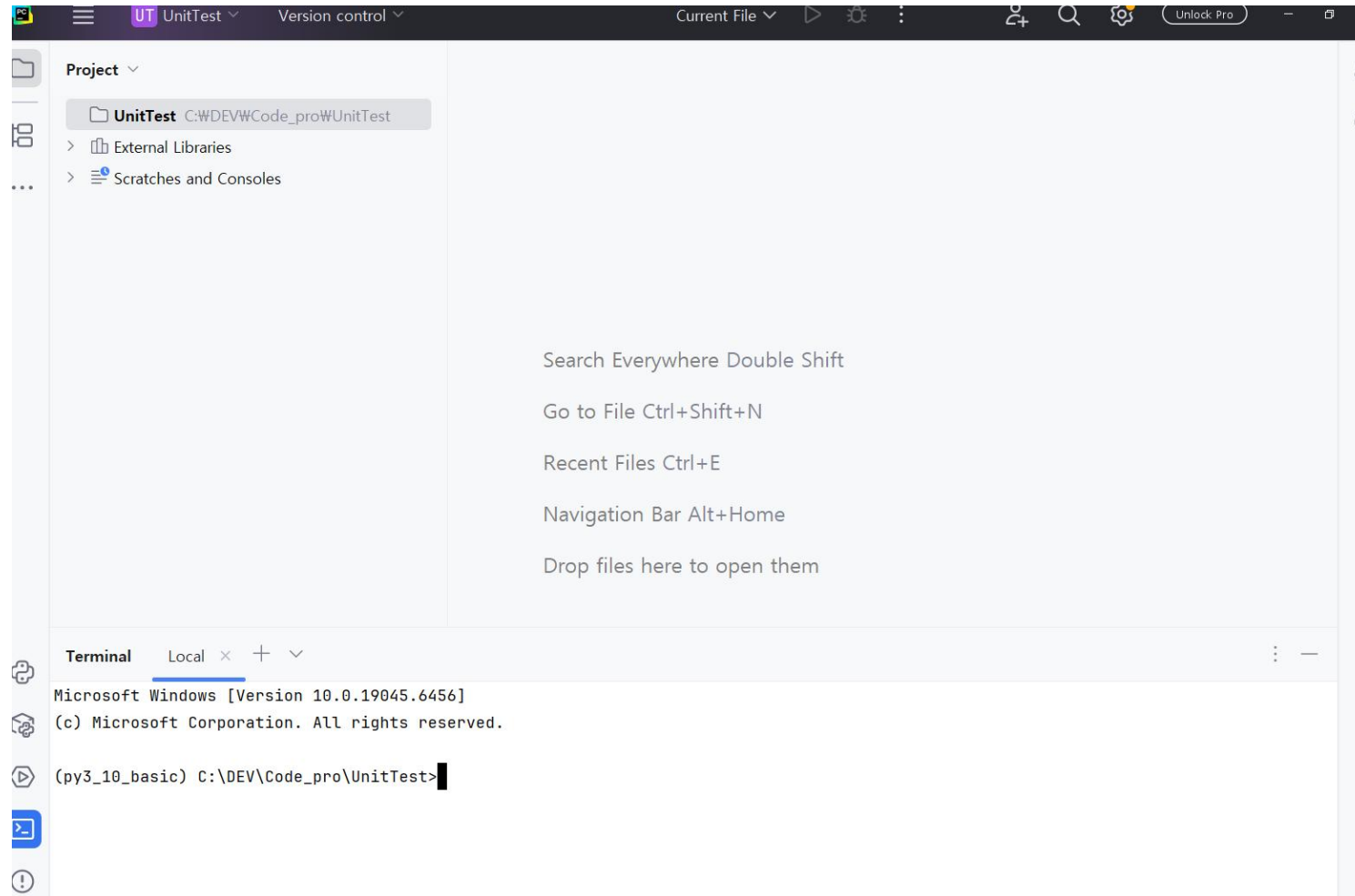
C:\DEV\SamsungDisplay\UnitTest 의 폴더 이름으로 시작...

The screenshot shows the 'New Project' dialog box with the following configuration:

- Location:** C:\DEV\Code_pro\UnitTest
- ☐ Create Git repository
- ☐ Create a welcome script
- Interpreter type:** Project venv, Base conda, Custom environment
- Environment:** ☐ Generate new, ☒ Select existing
- Type:** Python
- Python path:** Python 3.10.11 C:\DEV\envs\env3_10\basic\Scripts\python.exe

Buttons at the bottom: Create, Cancel

py3_10_basic 가상환경



CodeSmell.py

```
def p(u, m, q): # 의미 없는 변수명
    if u and m:
        t = m.price * q
        if t > 10000: # 매직 넘버
            t = t * 0.9
        return t
    return 0
```

TDD 실무 적용 (Red-Green-Refactor)

1. RED: 실패하는 테스트 작성 test_Order.py

```
def test_calculate_order_total_with_discount():
    user = User(id=1, name="홍길동")
    menu = Menu(id=1, name="아메리카노", price=4500)

    # 10,000원 이상 → 10% 할인 적용
    total = calculate_order_total(user, menu, quantity=3) # 13,500원

    assert total == 12150 # 13,500 * 0.9 = 12,150
```

```
.....

===== FAILURES =====
----- test_calculate_order_total_with_discount -----

def test_calculate_order_total_with_discount():
    user = User(id=1, name="홍길동")
    NameError: name 'User' is not defined

_Order.py:2: NameError
===== short test summary info =====
ID test_Order.py::test_calculate_order_total_with_discount - NameError: name 'User' is not defined
===== 1 failed in 0.12s =====

.10_basic) C:\DEV\Code_pro\UnitTest>
```

TDD 실무 적용 (Red-Green-Refactor)

1. RED: 실패하는 테스트 작성

```
def test_calculate_order_total_with_discount():
```

```
    user = User(id=1, name="홍길동")
```

```
    menu = Menu(id=1, name="아메리카노", price=4500)
```

```
    # 10,000원 이상 → 10% 할인 적용
```

```
    total = calculate_order_total(user, menu, quantity=3) # 13,500원
```

```
    assert total == 12150 # 13,500 * 0.9 = 12,150
```

```
    # ❌ 테스트 실패 (함수 미구현)
```

Step-by-step 단축키 순서

1) 함수 이름 바꾸기

커서: def p(u, m, q): 의 p 위

Shift + F6 → calculate_order_total 입력 → Enter

```
def calculate_order_total(u, m, q): # 의미 없는 변수명
    if u and m:
        t = m.price * q
        if t > 10000: # 매직 넘버
            t = t * 0.9
        return t
    return 0
```

Step-by-step 단축키 순서

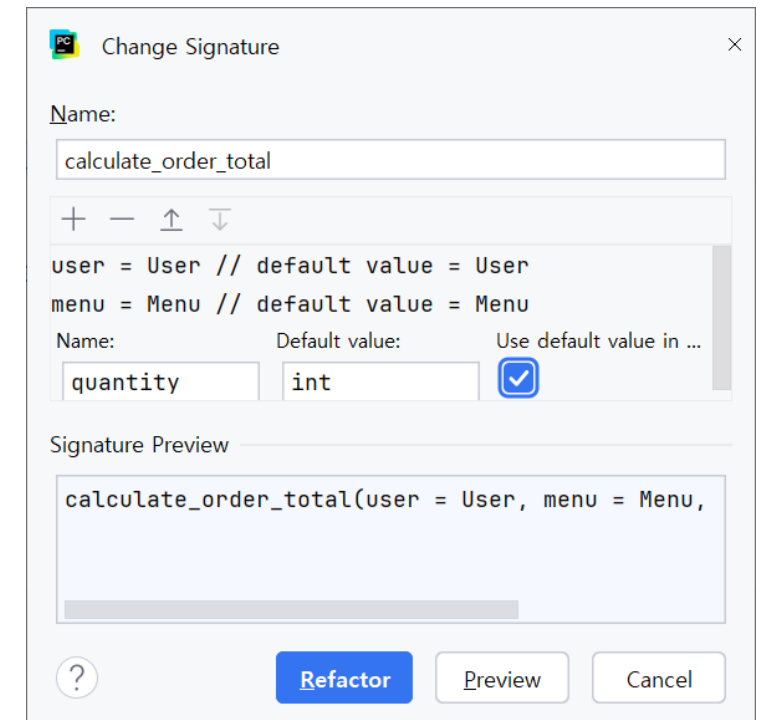
2) 시그니처(파라미터/반환형) 바꾸기

커서: 함수 선언부 어디든

Ctrl + F6 (Change Signature)

- 파라미터 이름을 u → user, m → menu, q → quantity 로 수정
- 타입: User, Menu, int 입력
- Return type: Decimal 입력 → OK
- (사용처 전체에 자동 반영)

```
=  
def calculate_order_total(user=User, menu=Menu, quantity=int)->Decimal:  
    if user and menu:  
        t = menu.price * quantity  
        if t > 10000: # 매직 넘버  
            t = t * 0.9  
        return t  
    return 0
```



def calculate_order_total(user: User, menu: Menu, quantity: int) -> Decimal:

Step-by-step 단축키 순서

3) Docstring 자동 생성

커서: 함수 헤더 바로 아래

""" 입력 후 Enter

또는 **Alt + Enter** → *Insert documentation string stub*

```
def calculate_order_total(user=User, menu=Menu, quantity=int)->Decimal: # 의
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
    if user and menu:
        t = menu.price * quantity
        if t > 10000: # 매직 넘버
            t = t * 0.9
        return t
    return 0
```

Step-by-step 단축키 순서

4) 임시 변수 이름 정리(가독성)

t 위에서 Shift + F6 → subtotal 로 변경

```
def calculate_order_total(user=User, menu=Menu, quantity=int)->Decimal: # 의미 없음
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
    if user and menu:
        subtotal = menu.price * quantity
        if subtotal > 10000: # 매직 넘버
            subtotal = subtotal * 0.9
        return subtotal
    return 0
```

Step-by-step 단축키 순서

5) 매직 넘버 추출(상수 도입)

10000 선택 → Ctrl + Alt + V (Introduce Variable) → DISCOUNT_THRESHOLD 입력

0.9 선택 → Ctrl + Alt + V → DISCOUNT_RATE 입력

두 상수를 함수 상단으로 이동(커서로 드래그)

모듈 상수로 빼고 싶으면 선택 후 Ctrl + Alt + V 대신 변수 만든 뒤 드래그하여 최상단으로 이동

```
def calculate_order_total(user=User, menu=Menu, quantity=int)->Decimal: # 의미 없음
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
    if user and menu:
        subtotal = menu.price * quantity
        DISCOUNT_THRESHOLD = 10000
        if subtotal > DISCOUNT_THRESHOLD: # 매직 넘버
            DISCOUNT_RATE = 0.9
            subtotal = subtotal * DISCOUNT_RATE
        return subtotal
    return 0
```

Unresolved reference 'Decimal'
Import this name Alt+Shift+Enter More actions



```
def calculate_order_total(user=User, menu=Menu, quantity=int)->Decimal: # 의미 없음
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
    DISCOUNT_THRESHOLD = 10000
    DISCOUNT_RATE = 0.9
    if user and menu:
        subtotal = menu.price * quantity

        if subtotal > DISCOUNT_THRESHOLD: # 매직 넘버
            subtotal = subtotal * DISCOUNT_RATE
        return subtotal
    return 0
```

Step-by-step 단축키 순서

6) Decimal로 금액 타입 명시

10000, 0.9 각각에 커서 → Alt + Enter → Convert to Decimal / Import 'Decimal' from decimal
상수 값을 Decimal("10000"), Decimal("0.9") 로 수정
: 지원 안 됨. 직접 수기로 작업해야 함.

from decimal import Decimal 자동 추가(또는 Alt + Enter)

```
10 """주문 총액 계산 (10,000원 이상 10% 할인)"""
11 DISCOUNT_THRESHOLD = Decimal("10000")
12
13 from decimal import Decimal
14
15 if user and menu:
16     subtotal = menu.price * quantity
17
18     if subtotal > DISCOUNT_THRESHOLD:
19         subtotal = subtotal * 0.9
20
21     return subtotal
22
23 return 0
```

- Import this name
- Import this name locally
- Create class 'Decimal' in module CodeSmell.py
- Create function 'Decimal'
- Create parameter 'Decimal'
- Rename reference
- Ignore all unresolved attributes of 'CodeSmell'
- Ignore unresolved references 'CodeSmell.Decimal'



```
def calculate_order_total(user=User, menu=Menu, quantity=int)->Decimal: # 의미 없는 변
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
    DISCOUNT_THRESHOLD = Decimal("10000")
    DISCOUNT_RATE = Decimal("0.9")

    if user and menu:
        subtotal = menu.price * quantity

        if subtotal > DISCOUNT_THRESHOLD: # 매직 넘버
            subtotal = subtotal * DISCOUNT_RATE

        return subtotal
```

Step-by-step 단축키 순서

7) 가드 절 추가(명확한 유효성 검사)

본문 첫 줄에 커서 → Ctrl + Alt + T (Surround With) → if 선택

조건을 if not user or not menu: 로 작성 후

내부에 raise ValueError("User and Menu are required") 입력

```
def calculate_order_total(user=User, menu=Menu)
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
```

Surround With

1. if
2. while

```
"10000")
)
```

```
def calculate_order_total(user=User, menu=Menu, quantity=int)->Decimal: # 의미 없는
    if not user or not menu:
        raise ValueError("User and Menu are required")
    |
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
    DISCOUNT_THRESHOLD = Decimal("10000")
    DISCOUNT_RATE = Decimal("0.9")

    if user and menu:
        subtotal = menu.price * quantity

        if subtotal > DISCOUNT_THRESHOLD: # 매직 넘버
            subtotal = subtotal * DISCOUNT_RATE
```

Step-by-step 단축키 순서

8) 조건식 정교화

if subtotal > DISCOUNT_THRESHOLD: 에 커서 → > 를 >= 로 교체
(수동 편집)

```
def calculate_order_total(user=User, menu=Menu, quantity=int)->Decimal: # !
    if not user or not menu:
        raise ValueError("User and Menu are required")

    """주문 총액 계산 (10,000원 이상 10% 할인)"""
    DISCOUNT_THRESHOLD = Decimal("10000")
    DISCOUNT_RATE = Decimal("0.9")

    if user and menu:
        subtotal = menu.price * quantity

    if subtotal >= DISCOUNT_THRESHOLD: # 매직 넘버
        subtotal = subtotal * DISCOUNT_RATE
```

Step-by-step 단축키 순서

9) 자동 정렬/정리

Ctrl + Alt + L : 코드 포맷 정리

Ctrl + Alt + O : import 정리

```
from decimal import Decimal

def calculate_order_total(user=User, menu=Menu, quantity=int) -> Decimal: # 의미 없
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
    DISCOUNT_THRESHOLD = Decimal("10000")
    DISCOUNT_RATE = Decimal("0.9")

    if not user or not menu:
        raise ValueError("User and Menu are required")
    subtotal = menu.price * quantity

    if subtotal >= DISCOUNT_THRESHOLD:
        return subtotal * DISCOUNT_RATE
    return 0
```

Step-by-step 단축키 순서

10) 타입 검사/힌트 보조

경고에 커서 → Alt + Enter 로 제안(미사용 import, 타입 힌트 누락 등) 처리

좋은 코드 (Clean Code)

```
def calculate_order_total(user: User, menu: Menu, quantity: int) -> Decimal:
```

```
    """주문 총액 계산 (10,000원 이상 10% 할인)"""
```

```
    DISCOUNT_THRESHOLD = Decimal("10000")
```

```
    DISCOUNT_RATE = Decimal("0.9")
```

```
    if not user or not menu:
```

```
        raise ValueError("User and Menu are required")
```

```
    subtotal = menu.price * quantity
```

```
    if subtotal >= DISCOUNT_THRESHOLD:
```

```
        return subtotal * DISCOUNT_RATE
```

```
    return subtotal
```


TDD 실무 적용 (Red-Green-Refactor)

1. RED: 실패하는 테스트 작성

```
def test_calculate_order_total_with_discount():  
    user = User(id=1, name="홍길동")  
    menu = Menu(id=1, name="아메리카노", price=4500)
```

10,000원 이상 → 10% 할인 적용

total = calculate_order_total(user, menu, quantity=3) # 13,500원

assert total == 12150 # 13,500 * 0.9 = 12,150

```
:test_Order.py::test_calculate_order_total_with_discount FAILED
```

```
===== FAILURES =====
```

```
----- test_calculate_order_total_with_discount -----
```

```
def test_calculate_order_total_with_discount():  
>     user = User(id=1, name="홍길동")  
:     NameError: name 'User' is not defined
```

```
:test_Order.py:2: NameError
```

```
===== short test summary info =====
```

TDD 실무 적용 (Red-Green-Refactor)

2. GREEN: 테스트 통과하는 최소 코드 작성 Models.py

```
from dataclasses import dataclass
from decimal import Decimal
```

```
@dataclass
class User:
    id: int
    name: str
```

```
@dataclass
class Menu:
    id: int
    name: str
    price: Decimal
```

CodeSmell.py

```
from decimal import Decimal
from Models import User, Menu
from CodeSmell import calculate_order_total # 실제 함수 import
```

```
def test_calculate_order_total_with_discount():
    user = User(id=1, name="홍길동")
    menu = Menu(id=1, name="아메리카노", price=Decimal("4500"))

    total = calculate_order_total(user, menu, quantity=3)

    assert total == Decimal("12150") # 13,500 * 0.9 = 12,150
```

TDD 실무 적용 (Red-Green-Refactor)

2. GREEN: 테스트 통과하는 최소 코드 작성

```
test_Order.py  from decimal import Decimal

                from Models import User
                from Models import Menu

                def calculate_order_total(user: User, menu: Menu, quantity: int) -> Decimal:
                    """주문 총액 계산 (10,000원 이상 10% 할인)"""
                    DISCOUNT_THRESHOLD = Decimal("10000")
                    DISCOUNT_RATE = Decimal("0.9")

                    if not user or not menu:
                        raise ValueError("User and Menu are required")
                    subtotal = menu.price * quantity

                    if subtotal >= DISCOUNT_THRESHOLD:
                        return subtotal * DISCOUNT_RATE
                    return 0
```

TDD 실무 적용 (Red-Green-Refactor)

2. GREEN: 테스트 통과하는 최소 코드 작성

```
[py3_10_basic] C:\DEV\Code_pro\UnitTest>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\Code_pro\UnitTest
plugins: cov-7.0.0
collected 1 item

test_order.py::test_calculate_order_total_with_discount PASSED [100%]

===== 1 passed in 0.04s =====

[py3_10_basic] C:\DEV\Code_pro\UnitTest>
```

TDD 실무 적용 (Red-Green-Refactor)

3. REFACTOR: 코드 개선

```
from decimal import Decimal
from typing import Optional

def calculate_order_total(
    user: User,
    menu: Menu,
    quantity: int,
    discount_threshold: Decimal = Decimal("10000"),
    discount_rate: Decimal = Decimal("0.9")
) -> Decimal:
    """주문 총액 계산

    Args:
        user: 주문 사용자
        menu: 주문 메뉴
        quantity: 수량
        discount_threshold: 할인 기준 금액
        discount_rate: 할인율

    Returns:
        Decimal: 최종 주문 금액

    Raises:
        ValueError: user 또는 menu가 None인 경우
    """
    if not user or not menu:
        raise ValueError("User and Menu are required")

    subtotal = menu.price * quantity

    if subtotal >= discount_threshold:
        return subtotal * discount_rate

    return subtotal
```

TDD 실무 적용 (Red-Green-Refactor)

```
✓ .....
===== 1 passed in 0.04s =====

(py3_10_basic) C:\DEV\Code_pro\UnitTest>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\Code_pro\UnitTest
plugins: cov-7.0.0
collected 1 item

test_order.py::test_calculate_order_total_with_discount PASSED [100%]

===== 1 passed in 0.02s =====
```

코드 커버리지

```
(py3_10_basic) C:\DEV\Code_pro\UnitTest>pytest --cov=. -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0 -- c:\DEV\envs\py3_10_basic\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\DEV\Code_pro\UnitTest
plugins: cov-7.0.0
collected 1 item

test_Order.py::test_calculate_order_total_with_discount PASSED [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.10.11-final-0 -----

Name            Stmts  Miss  Cover
-----
CodeSmell.py      10     2    80%
Models.py         11     0   100%
test_Order.py      8     0   100%
-----
TOTAL             29     2    93%

===== 1 passed in 0.06s =====
```

코드 커버리지

```
(py3_10_basic) C:\DEV\Code_pro\UnitTest>pytest --cov=. --cov-report=html
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\DEV\Code_pro\UnitTest
plugins: cov-7.0.0
collected 1 item

test_Order.py . [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.10.11-final-0 -----

Coverage HTML written to dir htmlcov
===== 1 passed in 0.14s =====
```


(py3_10_basic)

C:\DEV\SamsungDisplay\Week_2\Code_pro\python_test\StaticAnalysis.py>에서 다음 코드를 정적 분석하시오.

```
import sys
from collections import Counter

n = int(sys.stdin.readline())
cards = [int(sys.stdin.readline()) for x in range(n)]
count = Counter(sorted(cards))
print(count.most_common(1)[0][0])
```

(py3_10_basic)

C:\DEV\SamsungDisplay\Week_2\Code_pro\python_test>

pip install pylint

pip install flake8

StaticAnalysis.py

```
import sys
from collections import Counter

n = int(sys.stdin.readline())
cards = [int(sys.stdin.readline()) for x in range(n)]
count = Counter(sorted(cards))
print(count.most_common(1)[0][0])
```

분석 결과

```
(py3_10_basic) C:\DEV\SamsungDisplay\Week_2\Code_pro\python_test>python -m pylint StaticAnalysis.py
***** Module StaticAnalysis
StaticAnalysis.py:7:0: C0304: Final newline missing (missing-final-newline)
StaticAnalysis.py:1:0: C0114: Missing module docstring (missing-module-docstring)
StaticAnalysis.py:1:0: C0103: Module name "StaticAnalysis" doesn't conform to snake_case naming style (invalid-name)

-----
Your code has been rated at 5.00/10
```

- C0304 항목은 8번째 줄에 나타나 있는데, 파일의 마지막 줄에 공백 라인을 추가하라는 내용이고, C0114 항목은 첫 줄 주석에 한 줄 주석 대신 """ """를 사용한 주석을 권한다.
- 이 두 사항 모두 필수는 아니지만 권장 사항에 해당하는 권장 사항이다.

분석 결과

코드 스멜	해결 방법
Final newline missing	마지막 줄에 줄바꿈 추가
Missing module docstring	파일 최상단에 모듈 설명 주석 추가
Module name invalid-name	파일명을 snake_case로 변경 필요 (예: static_analysis.py)

해결

```
"""
```

카드 숫자의 최빈값을 출력하는 프로그램입니다.

입력:

- 첫 줄: 카드 개수 N
- 다음 N개의 줄: 각 카드 숫자

출력:

- 가장 많이 나타나는 카드 숫자

```
"""
```

```
import sys
```

```
from collections import Counter
```

```
def main():
```

```
    n = int(sys.stdin.readline().strip())
```

```
    cards = [int(sys.stdin.readline().strip()) for _ in range(n)]
```

```
    count = Counter(sorted(cards))
```

```
    print(count.most_common(1)[0][0])
```

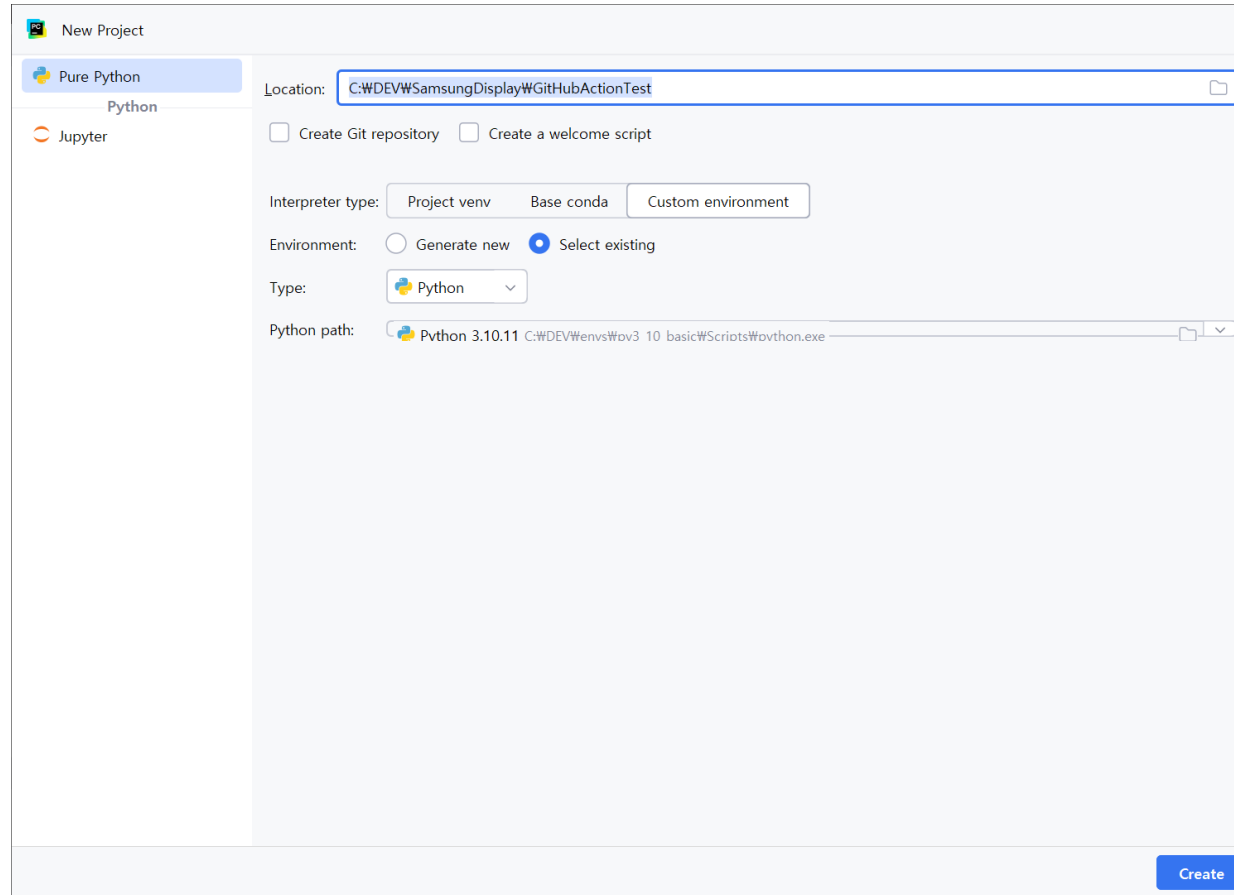
```
if __name__ == "__main__":
```

```
    main()
```

개선 내용

개선항목	설명
모듈 docstring 추가	Pylint C0114 해결
메인 함수 적용	구조적 코드로 유지보수 용이
변수명 _	의미 없는 루프변수는 _ 사용
.strip() 추가	입력 공백 처리 안전성 증가
파일명 static_analysis.py 추천	C0103 해결
마지막 줄 개행 추가	C0304 해결

C:\DEV\SamsungDisplay\Week_2\Code_pro\GitHubActionTest



The screenshot shows the 'New Project' dialog in Visual Studio Code. The 'Location' field is set to 'C:\DEV\SamsungDisplay\Week_2\Code_pro\GitHubActionTest'. The 'Interpreter type' is set to 'Project venv'. The 'Environment' is set to 'Select existing'. The 'Type' is set to 'Python'. The 'Python path' is set to 'Python 3.10.11 C:\DEV\envs\pov3_10\basic\Scripts\python.exe'. The 'Create' button is visible at the bottom right.

New Project

Pure Python
Python
Jupyter

Location: C:\DEV\SamsungDisplay\Week_2\Code_pro\GitHubActionTest

☐ Create Git repository ☐ Create a welcome script

Interpreter type: Project venv Base conda Custom environment

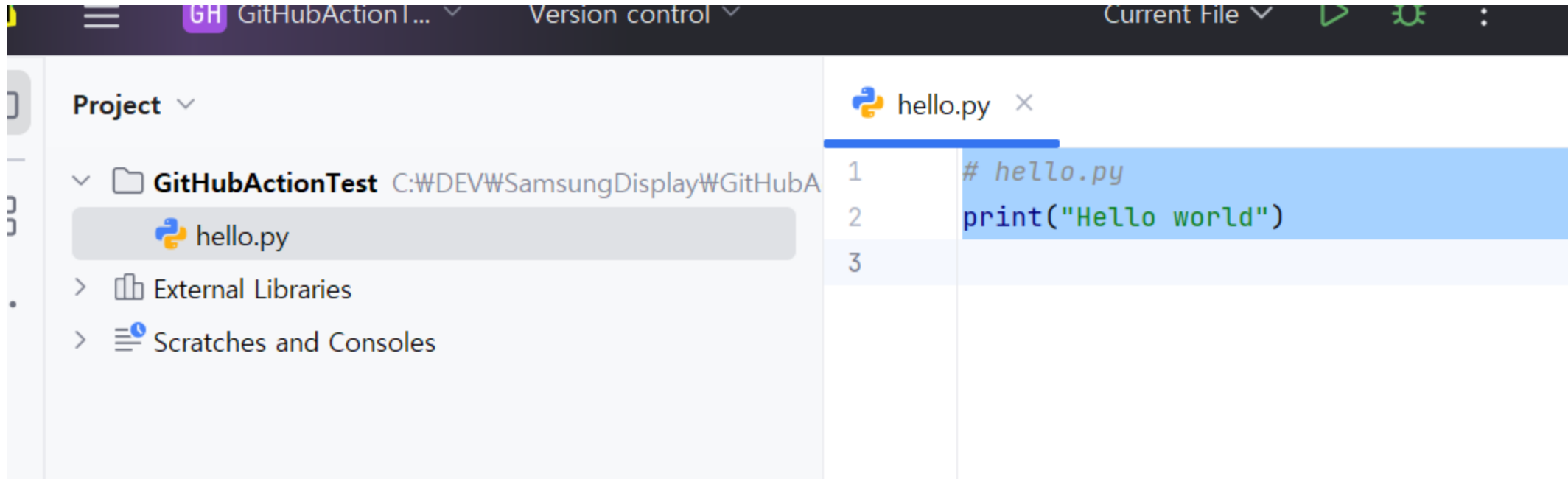
Environment: ☐ Generate new ☒ Select existing

Type: Python

Python path: Python 3.10.11 C:\DEV\envs\pov3_10\basic\Scripts\python.exe

Create


```
# hello.py  
print("Hello world")
```



실습3 : GitHub Action

<https://github.com/new>

The screenshot shows the GitHub 'New repository' page for the user 'onlookertozip'. The page title is 'GitHubActionTest' and it is marked as 'Public'. The page includes a search bar, navigation links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. Below the repository name, there are buttons for Pin, Watch (0), Fork (0), and Star (0). The main content area has two cards: 'Set up GitHub Copilot' and 'Add collaborators to this repository'. Below these is a 'Quick setup' section with options for 'Set up in Desktop', 'HTTPS', and 'SSH'. The 'SSH' option is selected, showing the repository URL: 'https://github.com/onlookertozip/GitHubActionTest.git'. Below the 'Quick setup' section, there are two sections for creating a new repository on the command line. The first section is titled '...or create a new repository on the command line' and shows the following commands:

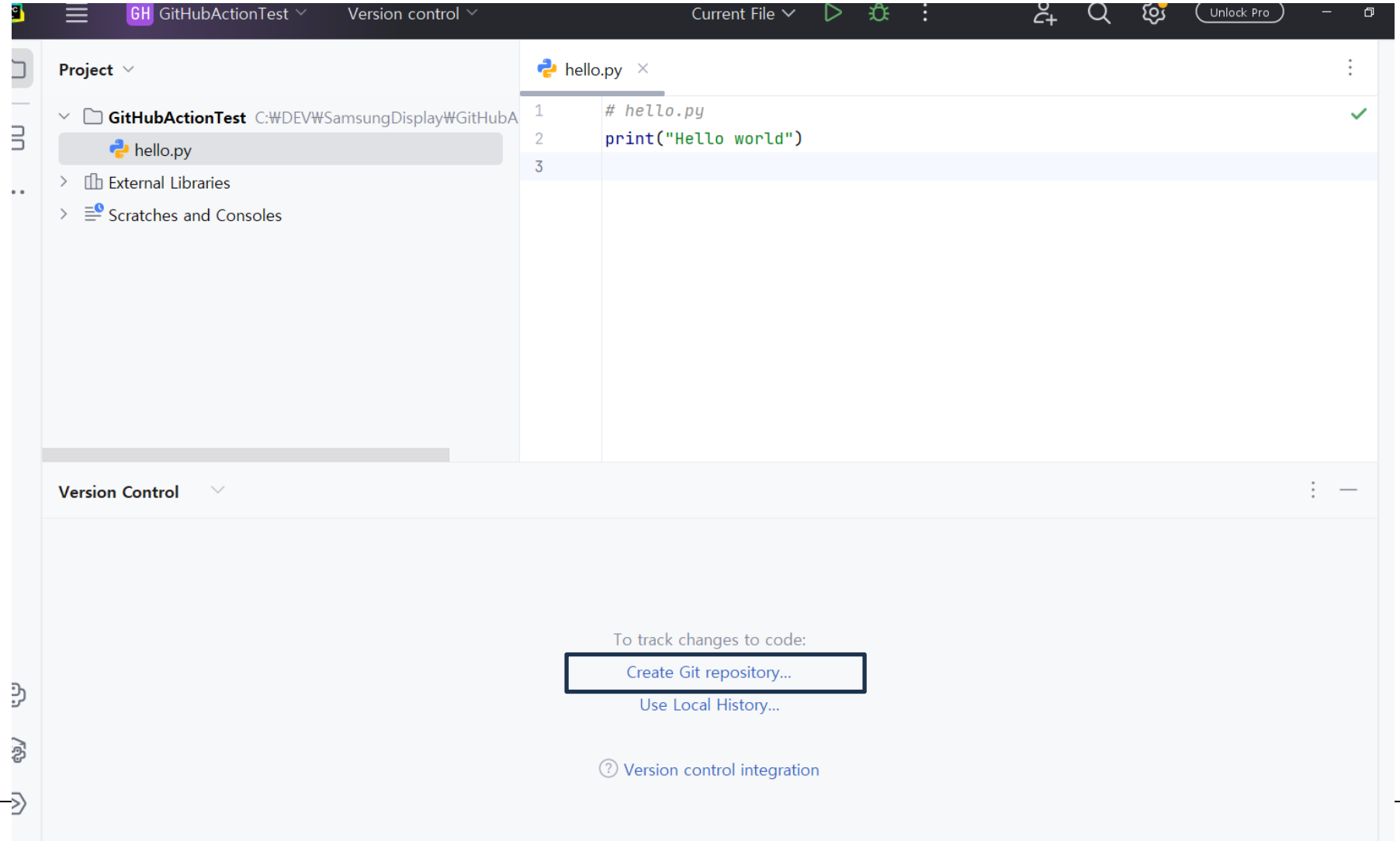
```
echo "# GitHubActionTest" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/onlookertozip/GitHubActionTest.git
git push -u origin main
```

 The second section is titled '...or push an existing repository from the command line' and shows the following commands:

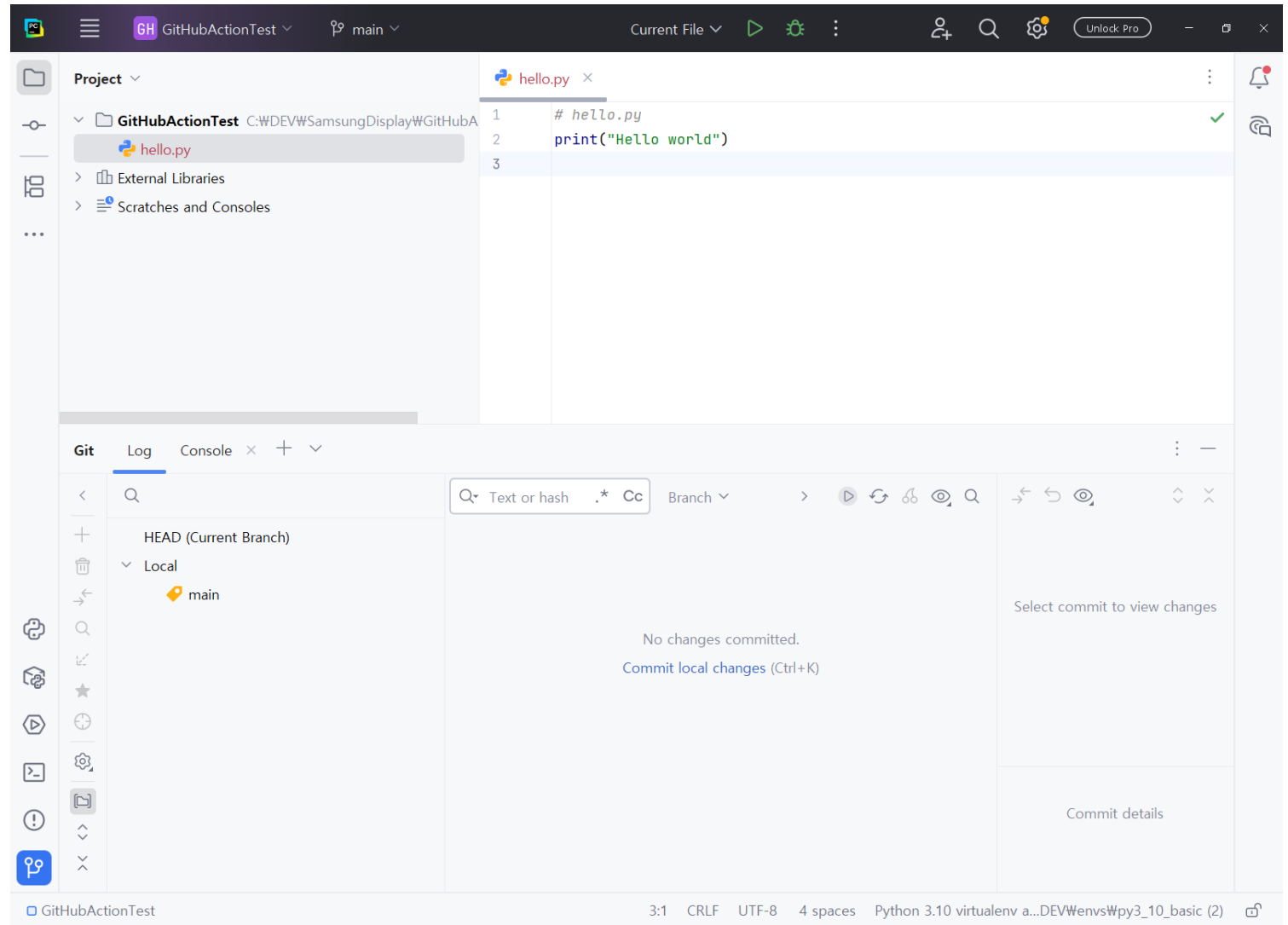
```
git remote add origin https://github.com/onlookertozip/GitHubActionTest.git
git branch -M main
git push -u origin main
```

 At the bottom of the page, there is a 'ProTip!' that says: 'Use the URL for this page when adding GitHub as a remote.'

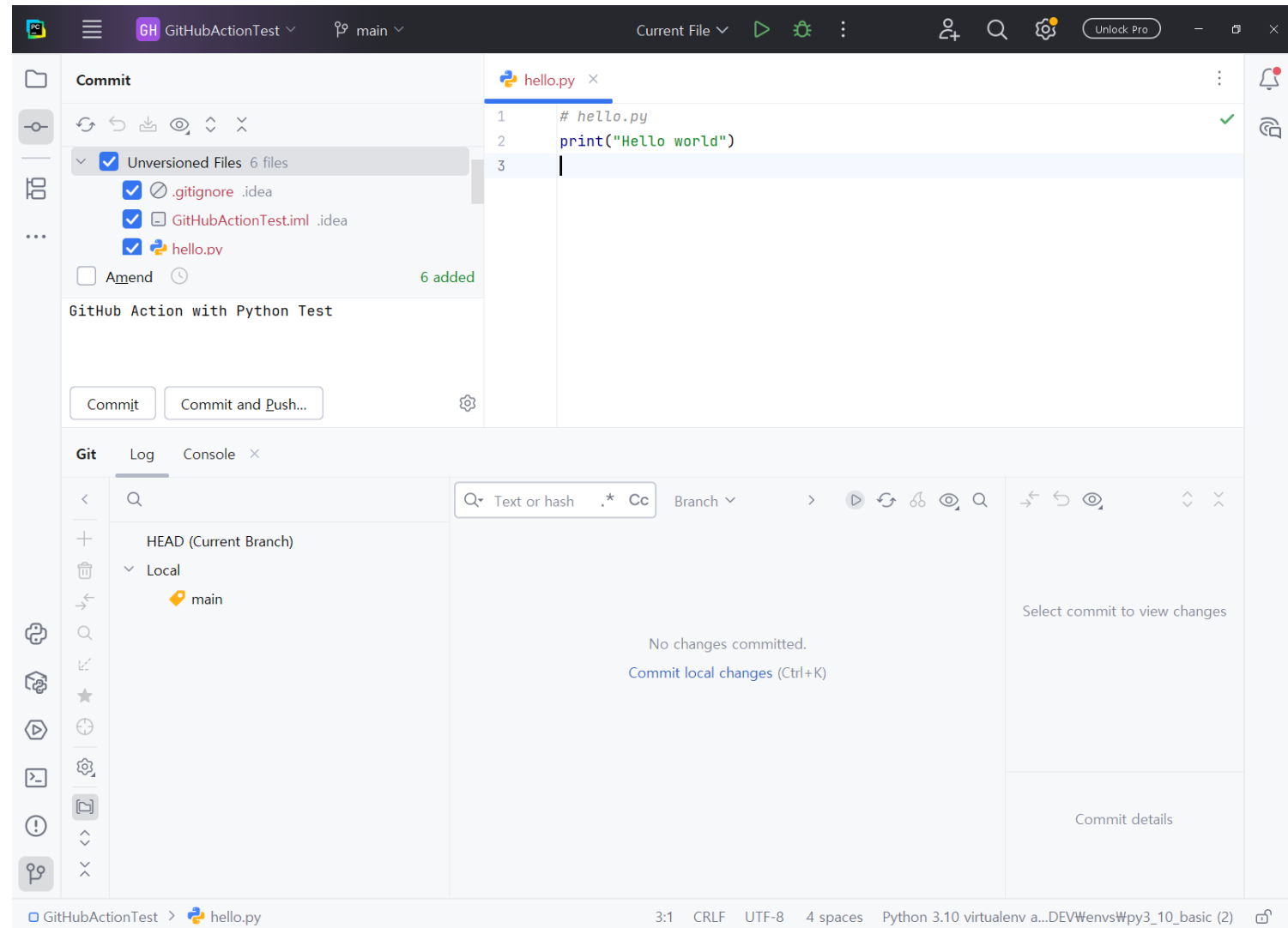
Create Git repository ...



Create Git repository ...

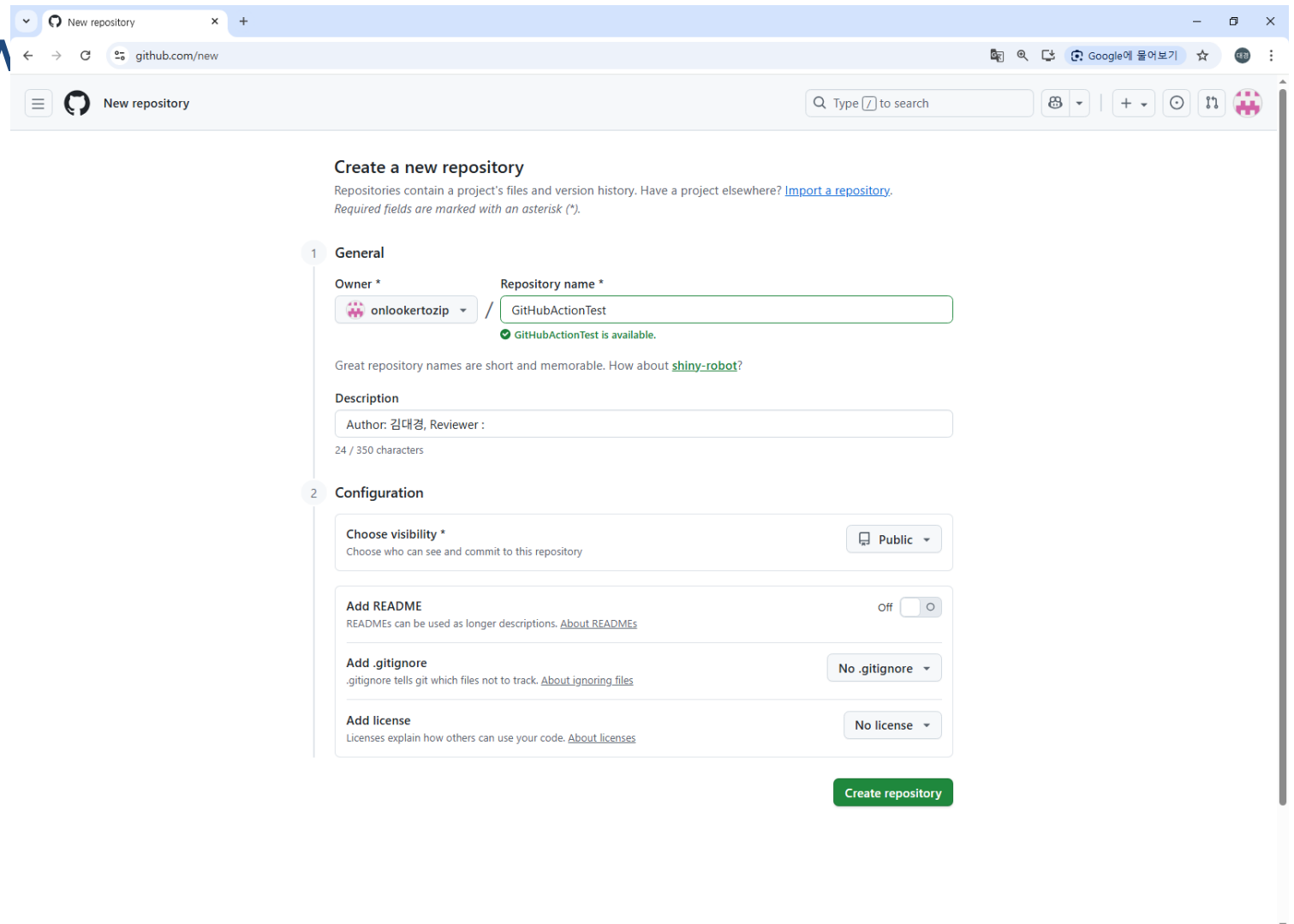


Commit



실습3 : GitHub Action

https://github.com/new



New repository

github.com/new

Google에 물어보기

Type / to search

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner * onlookertozip / Repository name *

✔ GitHubActionTest is available.

Great repository names are short and memorable. How about [shiny-robot](#)?

Description

Author: 김대경, Reviewer :

24 / 350 characters

2 Configuration

Choose visibility * Public

Choose who can see and commit to this repository

Add README Off

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore

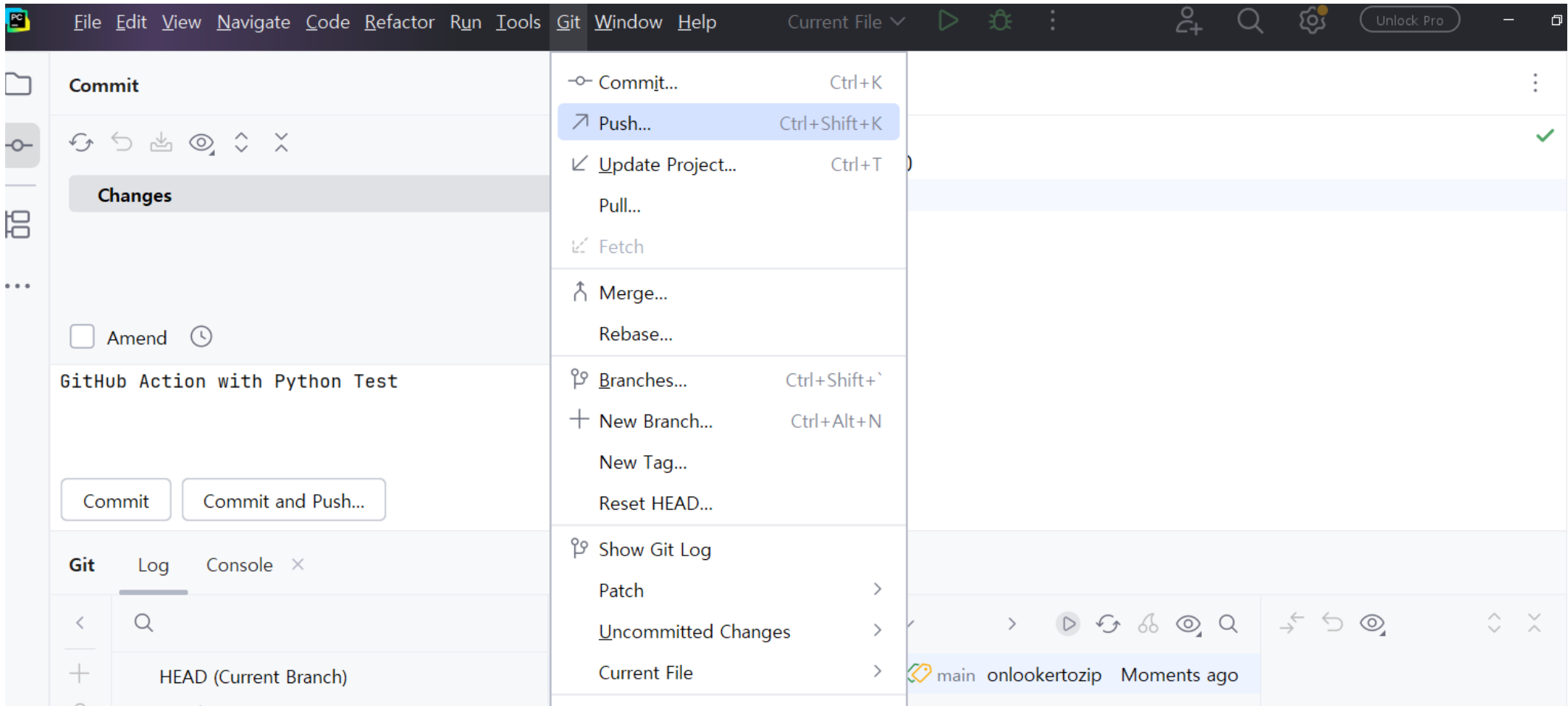
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license

Licenses explain how others can use your code. [About licenses](#)

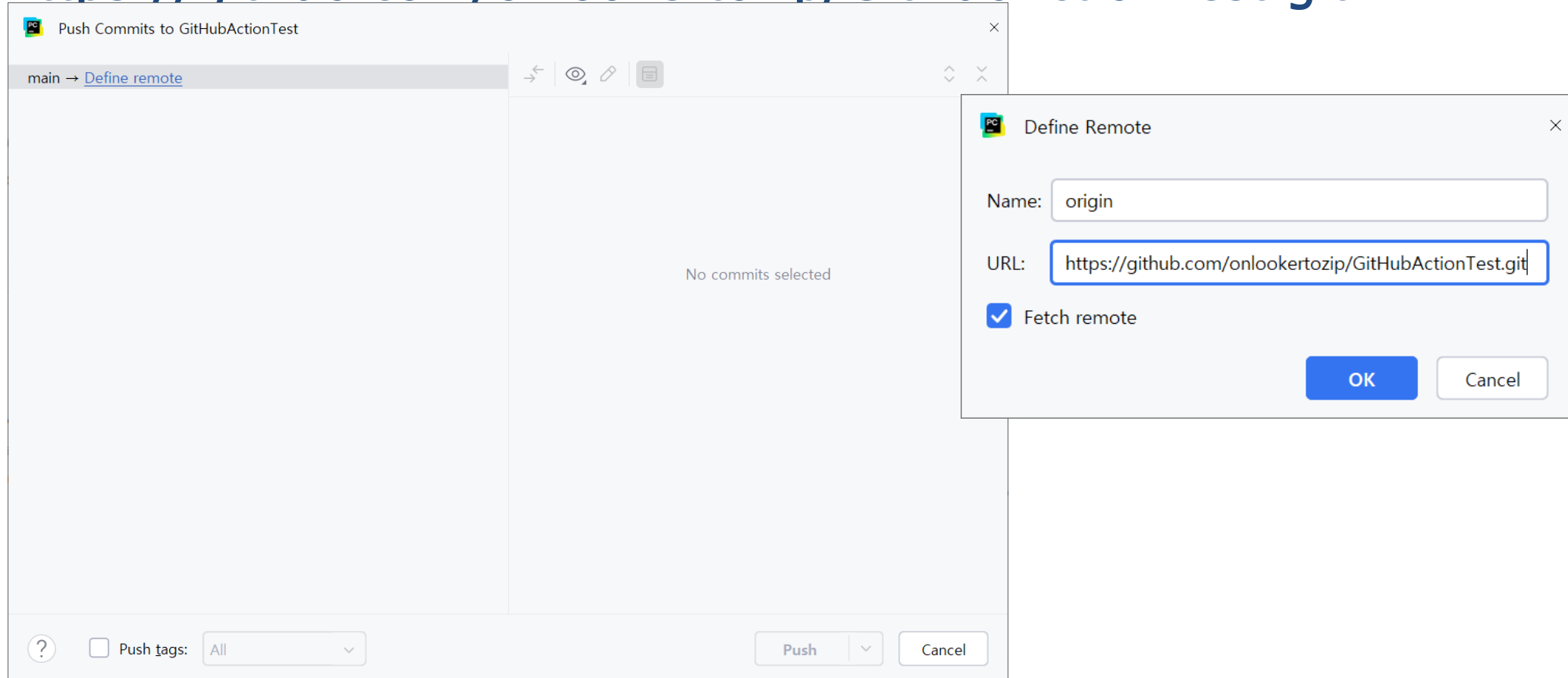
Create repository

push

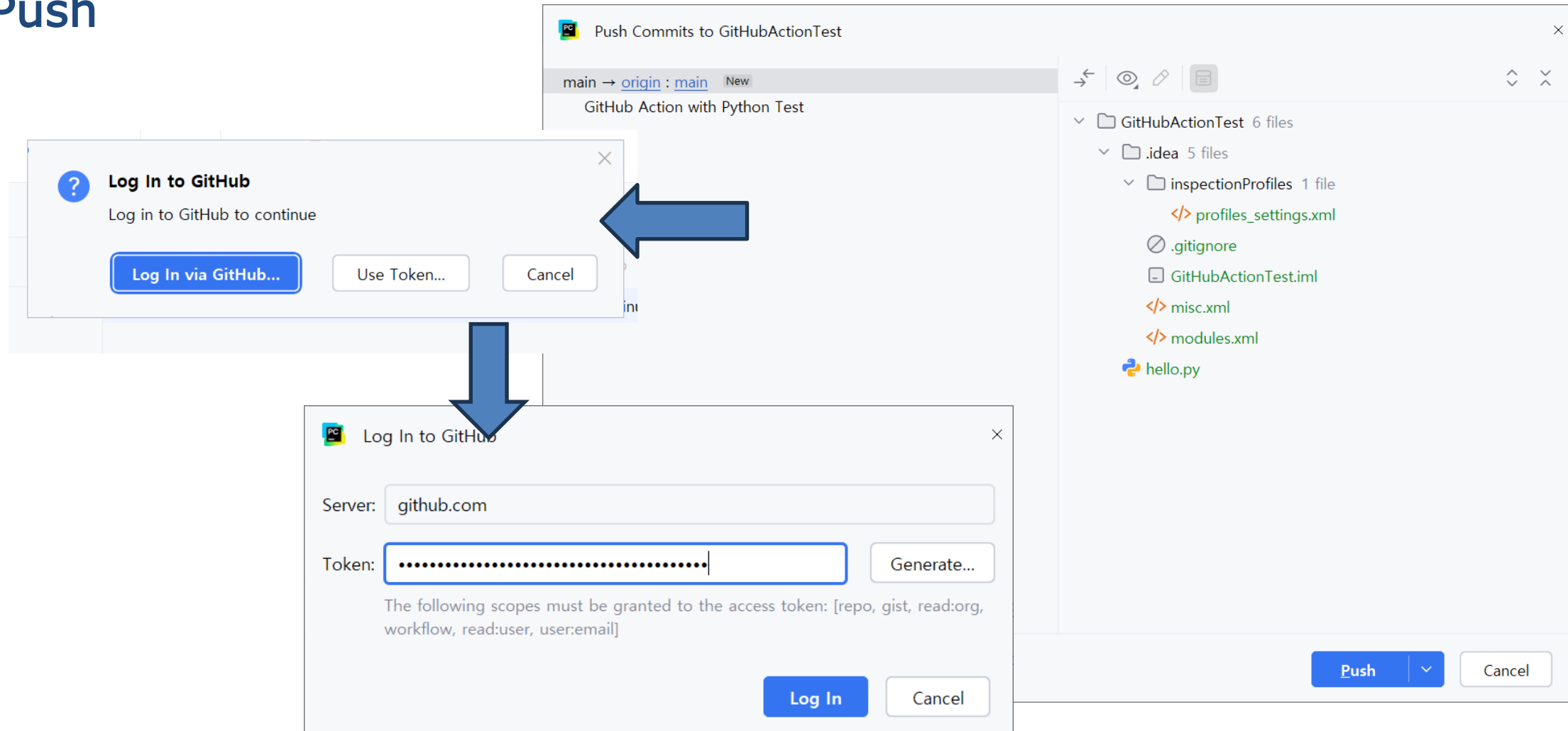


Define remote >

<https://github.com/onlookertozip/GitHubActionTest.git>



Push



Push

The screenshot shows the GitHub repository page for 'onlookertozip/GitHubActionTest'. The repository is public and has 1 branch (main) and 0 tags. It contains 1 commit and 2 files: '.idea' and 'hello.py', both committed 2 minutes ago. The repository has 0 stars, 0 forks, and 0 watchers. The 'About' section shows the author as '김대경' and the reviewer as 'Reviewer'. The 'Releases' section shows no releases published. The 'Packages' section shows no packages published. The 'Languages' section shows Python at 100.0%. The 'Suggested workflows' section shows two workflows: 'Python package' and 'Publish Python Package', both with 'Configure' buttons. The main content area shows a prompt to 'Add a README' with a button labeled 'Add a README'.

onlookertozip / GitHubActionTest

Type to search

<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings

GitHubActionTest (Public)

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file Add file <> Code

onlookertozip GitHub Action with Python Test 2673cf7 · 2 minutes ago 1 Commit

.idea	GitHub Action with Python Test	2 minutes ago
hello.py	GitHub Action with Python Test	2 minutes ago

README

Add a README

Help people interested in this repository understand your project by adding a README.

Add a README

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

Python 100.0%

Suggested workflows

Based on your tech stack

Python package Configure

Create and test a Python package on multiple Python versions.

Publish Python Package Configure

Choose a workflow > Pylint

Choose a workflow

Build, test, and deploy your code. Make code reviews, branch management, and issue triaging work the way you want. Select a workflow to get started.

Skip this and [set up a workflow yourself](#) →

Suggested for this repository

Python Package using Anaconda

By GitHub Actions



Create and test a Python package on multiple Python versions using Anaconda for package management.

Configure

Python

Publish Python Package

By GitHub Actions



Publish a Python Package to PyPI on release.

Configure

Python

Django

By GitHub Actions



Build and Test a Django Project

Configure

Python

Pylint

By GitHub Actions



Lint a Python application with pylint.

Configure

Python

Python application

By GitHub Actions



Create and test a Python application.

Configure

Python

Python package

By GitHub Actions



Create and test a Python package on multiple Python versions.

Configure

Python

실습3 : GitHub Action

name: Pylint

on: [push]

jobs:

build:

runs-on: ubuntu-latest

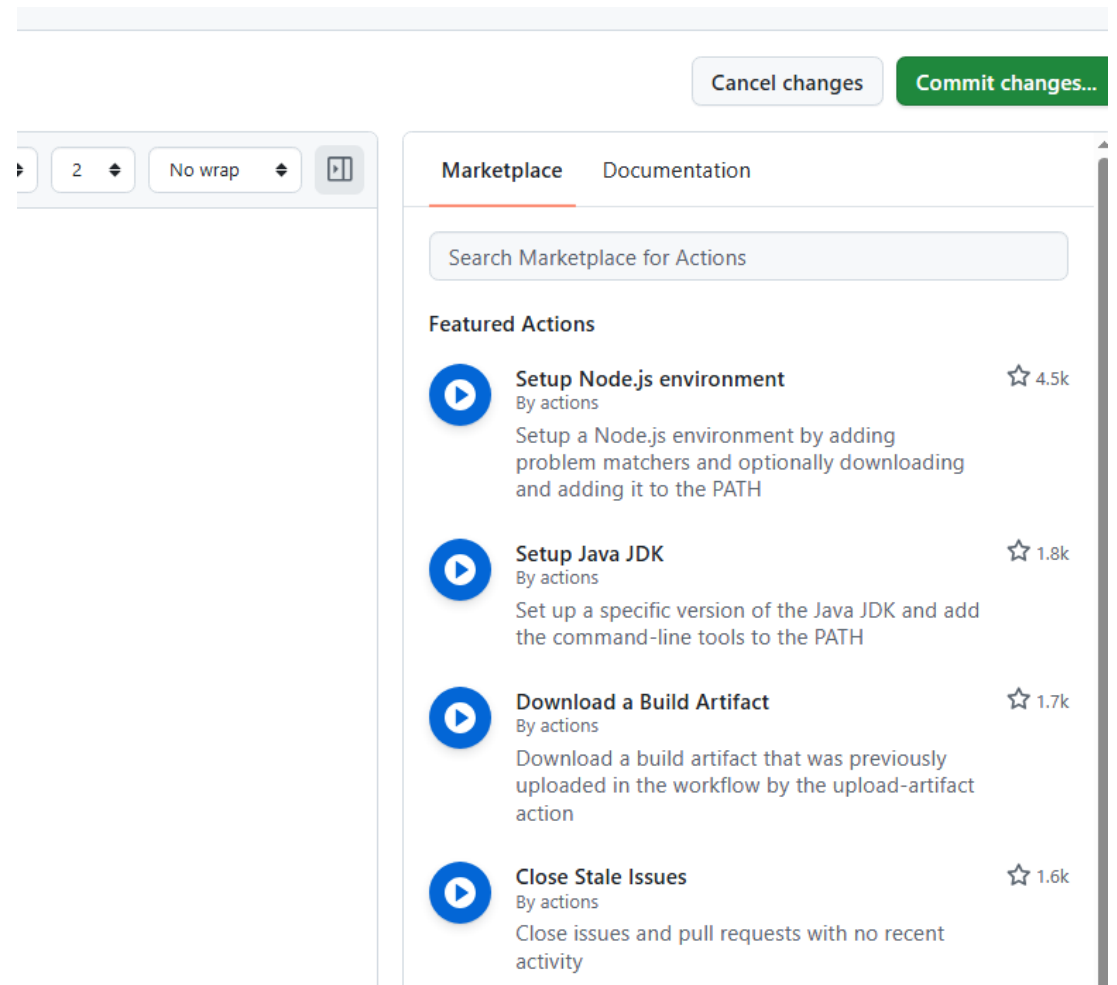
strategy:

matrix:

python-version: ["3.8", "3.9", "3.10"]

steps:

- uses: actions/checkout@v4
- name: Set up Python $\{\{\text{matrix.python-version}\}\}$
uses: actions/setup-python@v3
with:
python-version: $\{\{\text{matrix.python-version}\}\}$
- name: Install dependencies
run: |
python -m pip install --upgrade pip
pip install pylint
- name: Analysing the code with pylint
run: |
pylint $\$(git ls-files '*.py')$



Commit

The screenshot shows the GitHub Actions workflow editor for a workflow named `pylint.yml` in the `main` branch. The workflow is configured to run on `ubuntu-latest` and uses a strategy matrix for Python versions. The workflow steps include checking out the code, setting up Python, installing dependencies, and running Pylint. A commit dialog is open, showing the commit message `Create pylint.yml` and the extended description `Add an optional extended description...`. The dialog also shows the commit options: `Commit directly to the main branch` (selected) and `Create a new branch for this commit and start a pull request`. A blue arrow points to the `pylint.yml` file in the commit dialog. In the background, a list of actions is visible, including `Setup Java JDK`, `Download a Build Artifact`, `Close Stale Issues`, and `Setup .NET Core SDK`.

```
name: Pylint
on: [push]

jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.8", "3.9", "3.10"]
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python ${ matrix.python-version }}
        uses: actions/setup-python@v3
        with:
          python-version: ${ matrix.python-version }}
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pylint
      - name: Analysing the code with pylint
        run: |
          pylint $(git ls-files '*.py')
```

Commit changes

Commit message

Create pylint.yml

Extended description

Add an optional extended description...

☒ Commit directly to the main branch

☐ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)

Cancel Commit changes

File successfully deleted.

onlookertozip Delete .github/workflows/main.yml

Name	Last commit message
..	
pylint.yml	Create pylint.yml

Setup Java JDK

By actions

Set up a specific version of the Java JDK and add the command-line tools to the PATH

Download a Build Artifact

By actions

Download a build artifact that was previously uploaded in the workflow by the upload-artifact action

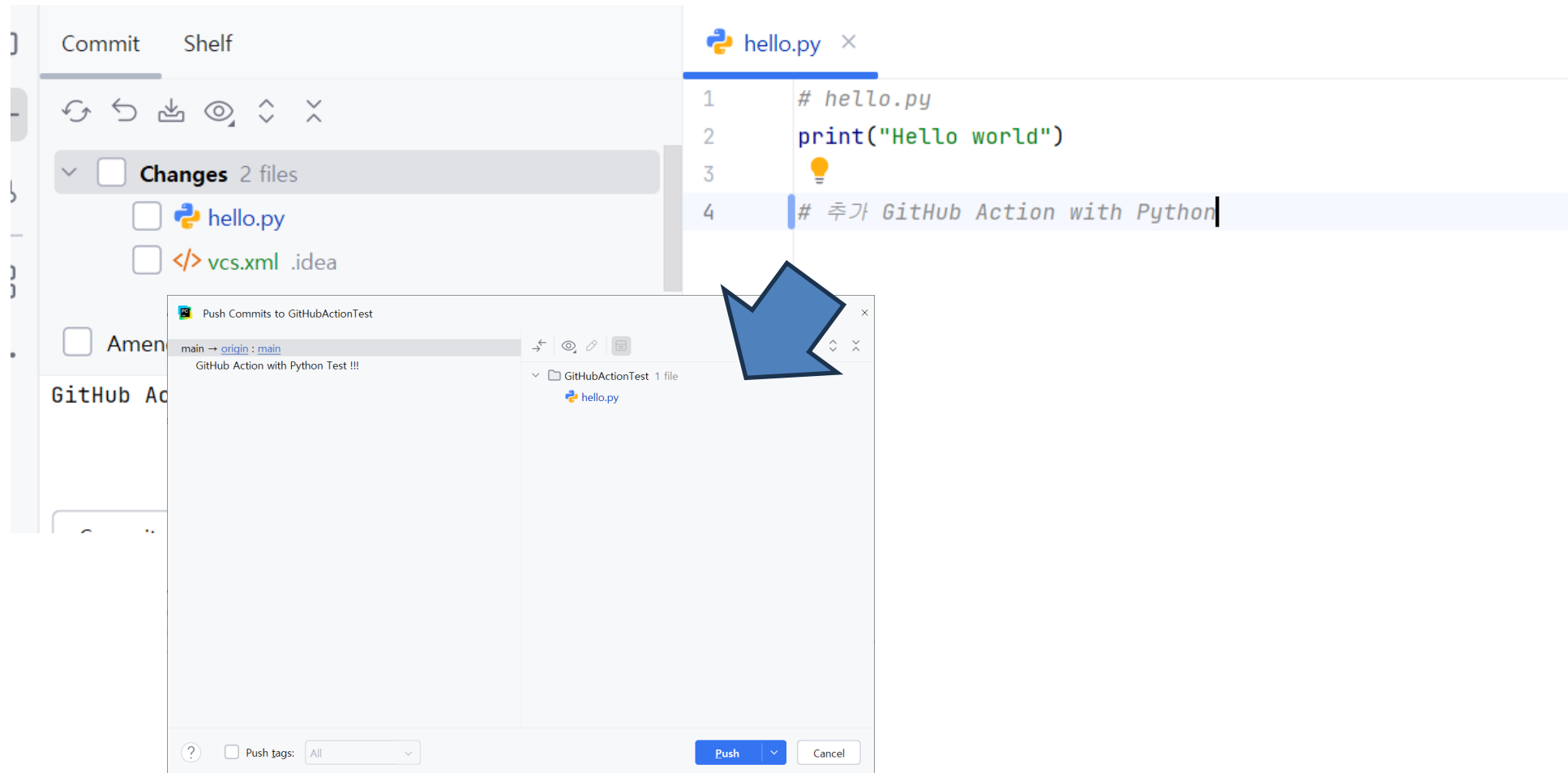
Close Stale Issues

By actions

Close issues and pull requests with no recent activity

Setup .NET Core SDK

내용 수정 후 Push



내용 수정 후 Push

The screenshot shows the GitHub interface for a repository named 'GitHubActionTest' by user 'onlookertozip'. The repository is public and has 0 stars, 0 forks, and 0 watchers. The main branch is 'main'. The repository contains three files: '.github/workflows', '.idea', and 'hello.py'. The 'README' section is currently empty, with a prompt to 'Add a README'. The right sidebar shows the 'About' section with the author '김대경' and reviewer, and the 'Releases' and 'Packages' sections, both indicating no published items.

onlookertozip / GitHubActionTest

ode Issues Pull requests Actions Projects Wiki Security Insights Settings

GitHubActionTest Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file Add file <> Code

onlookertozip GitHub Action with Python Test !!! 1b8520d · 3 minutes ago 8 Commits

.github/workflows	Delete .github/workflows/main.yml	6 minutes ago
.idea	GitHub Action with Python Test !!!	5 minutes ago
hello.py	GitHub Action with Python Test !!!	3 minutes ago

README

Add a README

Help people interested in this repository understand your project by adding a README.

About

Author: 김대경, Reviewer :

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

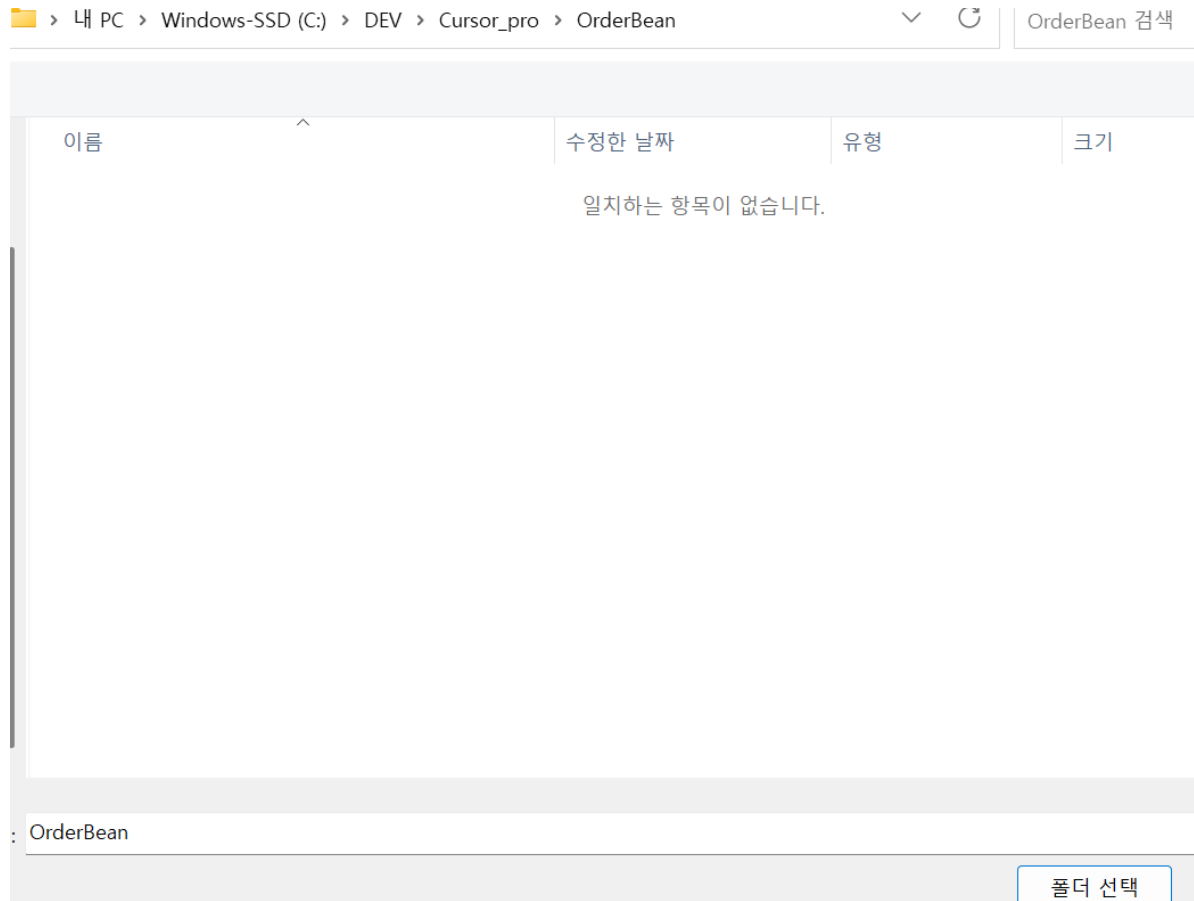
No packages published

[Publish your first package](#)

풀스택 웹 개발 - OrderBean Toy Project



C:\DEV\Cursor_pro\OrderBean 프로젝트 폴더 만들기



OrderBean 저장소 만들기

OrderBean Public



Start coding with Codespaces

Add a README file and start coding in a secure, configurable, and dedicated development environment.

Create a codespace



Quick setup — if you've done this kind of thing before

Set up in Desktop

or

HTTPS

SSH

<https://github.com/kznetwork/OrderBean.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# OrderBean" >> README.md
```

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *

kznetwork

Repository name *

OrderBean

✓ OrderBean is available.

Great repository names are short and memorable. How about **refactored-octo-garbanzo**?

Description

Author: 김대경, Reviewer : 전체 팀원 명

31 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

@OrderBean 나는 지금 OrderBean Toy Project를 시작하려고 하는 데, README 파일을 하나 만들고, 깃허브 저장소 @<https://github.com/kznetwork/OrderBean.git>에 업로드해줘. 그리고 그 결과를 Report 폴더에 마크다운 파일로 저장해줘.

New Chat



 OrderBean 나는 지금 OrderBean Toy Project를 시작하려고 하는 데, README 파일을 하나 만들고, 깃허브 저장소 @<https://github.com/kznetwork/OrderBean.git>에 업로드해줘. 그리고 그 결과를 Report 폴더에 마크다운 파일로 저장해줘.

현재 프로젝트 구조

```
OrderBean/
├── .git/
├── README.md
├── Report/
│   └── project-initializati
```

GitHub 저장소에서 확인하실 수 있습니다:

<https://github.com/kznetwork/OrderBean>

Report 폴더의 마크다운 파일에서 상세한 작업 내역을 확인하실 수 있습니다! 🍷

<https://github.com/kznetwork/OrderBean>

The screenshot shows the GitHub repository page for 'OrderBean' by 'kznetwork'. The repository is public and has 1 branch (main) and 0 tags. The commit history shows an initial commit 'Add README.md' by 'kznetwork' 2 minutes ago. The README file is displayed, showing the project name 'OrderBean' with a logo, a description 'OrderBean은 주문 관리 시스템을 위한 Toy Project입니다.', and a list of features: '주문 생성 및 관리', '주문 상태 추적', '고객 정보 관리', and '주문 내역 조회'. The right sidebar shows the repository's statistics: 0 stars, 0 forks, and 0 watchers. The 'About' section mentions the author '김대경' and reviewer '전체 팀원 명'. The 'Releases' section indicates no releases are published.

OrderBean은 주문 관리 시스템을 위한 Toy Project입니다.

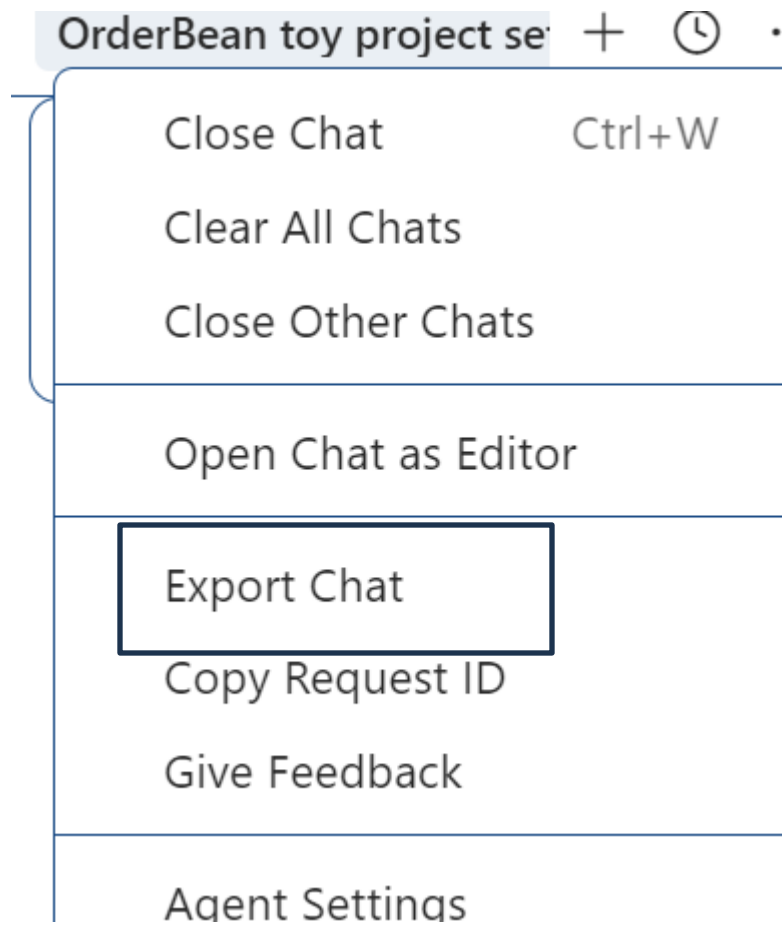
프로젝트 개요

OrderBean은 간단하고 효율적인 주문 관리를 위한 애플리케이션입니다. 이 프로젝트는 학습 및 포트폴리오 목적으로 개발되었습니다.

주요 기능

- 주문 생성 및 관리
- 주문 상태 추적
- 고객 정보 관리
- 주문 내역 조회

챗 내보내기



실습 1단계 - 아이디어 구상

목표: 커피 관련 서비스 아이디어 도출

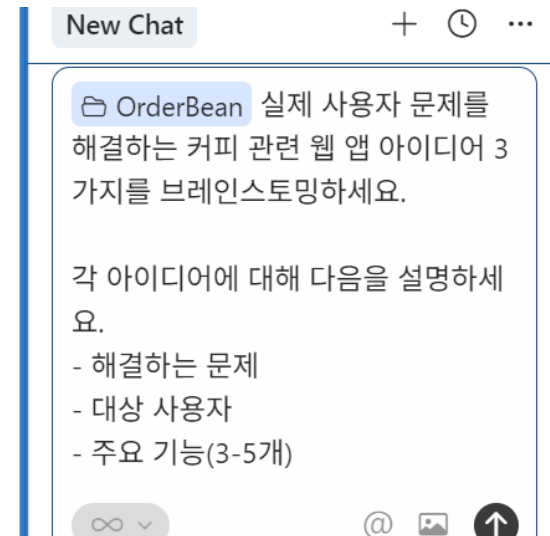
Cursor.AI 프롬프트

실제 사용자 문제를 해결하는 커피 관련 웹 앱 아이디어 3가지를 브레인스토밍하세요.

각 아이디어에 대해 다음을 설명하세요.

- 해결하는 문제
- 대상 사용자
- 주요 기능(3-5개)

예상 소요시간: 30분



실습 1단계 - 아이디어 구상

목표: 커피 관련 서비스 아이디어 도출

선택한 아이디어: 간편 커피 주문 서비스

해결하는 문제

- 카페 대기 시간 낭비
- 복잡한 커스터마이징 주문
- 주문 히스토리 관리 어려움

목표 사용자

- 바쁜 직장인
- 단골 고객

추천 우선순위

OrderBean의 "주문 관리" 개념과 가장 잘 맞는 아이디어는 1번 (스마트 커피 선주문 플랫폼) 입니다. 이미 주문 생성, 상태 추적, 고객 정보 관리 등의 기능이 프로젝트 개요에 포함되어 있어 자연스럽게 확장할 수 있습니다.

각 아이디어는 실제 사용자의 pain point를 해결하며, 커피 문화가 발달한 한국 시장에서 충분히 수요가 있는 서비스입니다. 어떤 방향으로 진행하고 싶으신가요? 😊

실습 2단계 - 문제 정의

목표: 한 문장으로 핵심 문제 표현

Cursor.AI 프롬프트

바쁜 직장인들이 시간을 절약할 수 있도록 돕는 커피 주문 웹 앱에 대한 한 문장으로 된 문제 진술문을 작성하세요. 사용자가 커피 메뉴를 주문하고, 관리자가 주문을 관리할 수 있는 간단한 풀스택 웹 앱입니다.

예상 소요시간: 15분

바쁜 직장인들이 시간을 절약할 수 있도록 돕는 커피 주문 웹 앱에 대한 한 문장으로 된 문제 진술문을 작성하세요. 사용자가 커피 메뉴를 주문하고, 관리자가 주문을 관리할 수 있는 간단한 풀스택 웹 앱입니다.

💡 더 간결한 버전 (선택)

필요하시다면 더 짧은 버전도 제공해 드립니다:

"바쁜 직장인이 대기 시간 없이 커피를 선주문하고, 카페 관리자가 효율적으로 주문을 관리할 수 있는 웹 앱이 필요하다."

어떤 버전이 더 마음에 드시나요? 또는 특정 부분을 수정하고 싶으신가요?

실습 3단계 - 주요 기능 정의

목표: 5가지 핵심 기능 도출

Cursor.AI 프롬프트
\\\

OrderBean 웹 서비스의 5가지 주요 기능 요구 사항을 나열하세요.

- 고객 기능과 관리 기능을 모두 포함하세요.
 - 간단한 설명과 함께 요점 형식으로 작성하세요.
- \\\

예상 소요시간: 45분

OrderBean 웹 서비스의 5가지 주요
기능 요구 사항을 나열하세요.

- 고객 기능과 관리 기능을 모두 포함하
세요.
- 간단한 설명과 함께 요점 형식으로 작
성하세요|

실습 3단계 - 주요 기능 정의

목표: 5가지 핵심 기능 도출

고객 기능

1. 간편 메뉴 주문 및 커스터마이징

- 커피 메뉴를 직관적으로 탐색하고 선택
- 옵션 변경 (사이즈, 샷 추가, 얼음량, 시럽 등)
- 장바구니에 담고 한 번에 주문
- 자주 주문하는 메뉴를 즐겨찾기로 저장

관리자 기능

4. 주문 관리 대시보드

- 실시간으로 접수되는 주문을 한눈에 확인
- 주문 상태 변경 (접수 → 제조 중 → 완료 → 픽업 완료)
- 주문 우선순위 조정 및 대기 시간 관리
- 완료된 주문과 대기 중인 주문을 구분하여 표시

실습 4단계 - 사용자 시나리오

목표: Given-When-Then 시나리오 작성

Cursor.AI 프롬프트

5가지 기능적 요구사항을 거킨 스타일 사용자 스토리(Gherkin-style user stories)로 변환하세요.
-Given-When-Then 형식을 사용하세요.
-구체적이고 테스트 가능하게 만드세요.

5가지 기능적 요구사항을 거킨 스타일
사용자 스토리(Gherkin-style user
stories)로 변환하세요.
-Given-When-Then 형식을 사용하세
요.
-구체적이고 테스트 가능하게 만드세
요|

실습 4단계 - 사용자 시나리오

목표: Given-When-Then 시나리오 작성

Feature 1: 간편 메뉴 주문 및 커스터마이징

Feature: 메뉴 주문 및 커스터마이징

As a 바쁜 직장인

I want to 커피 메뉴를 빠르게
So that 내 취향에 맞는 커피를

Scenario: 기본 메뉴 주문

Given 사용자가 메뉴 페이지에
When 사용자가 "아메리카노" 선택
And "장바구니 담기" 버튼을 클릭
Then 장바구니에 "아메리카노"
And 장바구니 아이콘에 "(1)"

Scenario: 커피 옵션 커스터마이징

Given 사용자가 "카페라떼" 메뉴
When 사용자가 사이즈를 "Gr

Feature 2: 실시간 주문 상태 추적

Feature: 실시간 주문 상태 추적

As a 주문한 고객

I want to 내 주문의 현재 상태
So that 픽업 시간을 예측하고

Scenario: 주문 완료 후 상태 확인

Given 사용자가 커피 2잔을 주문
When 주문이 성공적으로 완료
Then 주문 번호 "ORD-202511"
And 주문 상태가 "접수 완료"
And 예상 픽업 시간 "약 5분"

Scenario: 주문 상태 실시간 업데이트

Given 사용자가 주문 상태 페이지

Feature 3: 주문 내역 및 재주문

Feature: 주문 내역 조회 및 재주문

As a 단골 고객

I want to 이전 주문 내역을 확인
So that 매번 메뉴를 선택하는

Scenario: 주문 내역 조회

Given 사용자가 지난 30일간
When 사용자가 "주문 내역" 페이지
Then 최신 주문부터 역순으로
And 각 주문에는 날짜, 주문

Scenario: 한 번의 클릭으로 재주문

Given 사용자가 주문 내역에서
When 사용자가 "재주문" 버튼
Then 동일한 메뉴와 옵션이

실습 5단계 - 비기능 요구사항

목표: 성능, 보안, 확장성 정의

Cursor.AI 프롬프트

OrderBean에 대한 비기능적 요구 사항을 추가합니다.

- 성능 기대치
- 보안 요구 사항
- 확장성 고려 사항
- 사용성 표준

Toy project용으로는 현실적인 수준으로 유지하세요.

예상 소요시간: 30분

OrderBean에 대한 비기능적 요구 사항을 추가합니다.

- 성능 기대치
- 보안 요구 사항
- 확장성 고려 사항
- 사용성 표준

Toy project용으로는 현실적인 수준으로 유지하세요.

실습 5단계 - 비기능 요구사항

목표: 성능, 보안, 확장성 정의

요구사항	우선순위	구현 난이도	비고
기본 인증/권한	높음	중	필수 보안
반응형 디자인	높음	중	모바일 필수
페이지 로딩 2초 이내	중간	중	최적화 필요
HTTPS 적용	중간	낮	배포 시 필수
실시간 업데이트	중간	높	WebSocket/Polling
접근성 WCAG AA	낮음	중	시간 여유 시
100명 동시 접속	낮음	높	부하 테스트
다국어 지원	낮음	중	추후 확장

우선, 지금까지 작업한 내용을 마크다운 파일로 저장 ... 지금까지 작업한 내용을 Report 폴더에 마크다운 파일로 저장해줘.

지금까지 작업한 내용을 Report 폴더에 마크다운 파일로 저장해줘.

저장된 내용:

- ✓ 프로젝트 개요
- ✓ 문제 진술문
- ✓ 3가지 아이디어 브레인스토밍
- ✓ 5가지 기능적 요구사항 (고객 3개 + 관리자 2개)
- ✓ Gherkin 스타일 사용자 스토리 (Given-When-Then 형식)
- ✓ 비기능적 요구사항 (성능, 보안, 확장성, 사용성)
- ✓ 우선순위 매트릭스 및 검증 방법
- ✓ 다음 단계 가이드

이 문서는 OrderBean 프로젝트의 **요구사항 명세서**로 활용할 수 있으며, 향후 개발 과정에서 레퍼런스 문서로 이용하실 수 있습니다! 🚀

> 1 File Edited [Review](#)

✕ ✓

📁 project-requirements.md +518 -1

실습 6단계 - PRD 문서 구조화

목표: PRD 작성

Cursor.AI 프롬프트

```\n

이전 모든 출력을 구조화된 PRD 마크다운 문서로 결합 Docs 폴더에 PRD.md 파일로 저장해줘.

포함 항목:

- 개요
- 기능
- 데이터 모델(엔터티 및 관계)
- API 개요(엔드포인트)
- 사용자 스토리
- 기타 필요한 모든 것

```\n

예상 소요시간: 30분

이전 모든 출력을 구조화된 PRD 마크다운 문서로 결합
Docs 폴더에 PRD.md 파일로 저장해줘.

포함 항목:

- 개요
- 기능
- 데이터 모델(엔터티 및 관계)
- API 개요(엔드포인트)
- 사용자 스토리
- 기타 필요한 모든 것

| ORDERBEAN | | 📄 | 📁 | ↺ | 🗑 |
|-------------------------------------|--|---|---|---|---|
| ▼ Docs | | | | | ● |
| ↓ PRD.md | | | | | U |
| ▼ Prompting | | | | | ● |
| ↓ Orderbean_toy_project_setup_in... | | | | | U |
| ▼ Report | | | | | ● |
| ↓ project-initialization-report.md | | | | | U |
| ↓ project-requirements.md | | | | | U |
| 📘 README.md | | | | | |

지금까지 작업한 내용을 마크다운 파일로 저장 ...

지금까지 작업한 내용을 Report 폴더에 project-requirements.md 파일로 저장해줘.

지금까지 작업한 내용을 Report 폴더에 project-requirements.md 파일로 저장해줘.

∞ Agent ▾

Sonnet 4.5 ▾



지금까지 작업한 내용을 main 브랜치에서 requirements 브랜치를 새로 생성해 @<https://github.com/kznetwork/OrderBean.git> 깃허브에 업로드할 수 있을까?

지금까지 작업한 내용을 main 브랜치에서 requirements
브랜치를 새로 생성해
@<https://github.com/kznetwork/OrderBean.git> 깃허브
에 업로드할 수 있을까?

∞ Agent ▾

Sonnet 4.5 ▾



https://github.com/kznetwork/OrderBean

OrderBean / OrderBean

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

OrderBean Public

requirements had recent pushes 1 minute ago [Compare & pull request](#)

main 2 Branches 0 Tags

Go to file Add file Code

Switch branches/tags

Find or create a branch...

Branches Tags

✓ main default

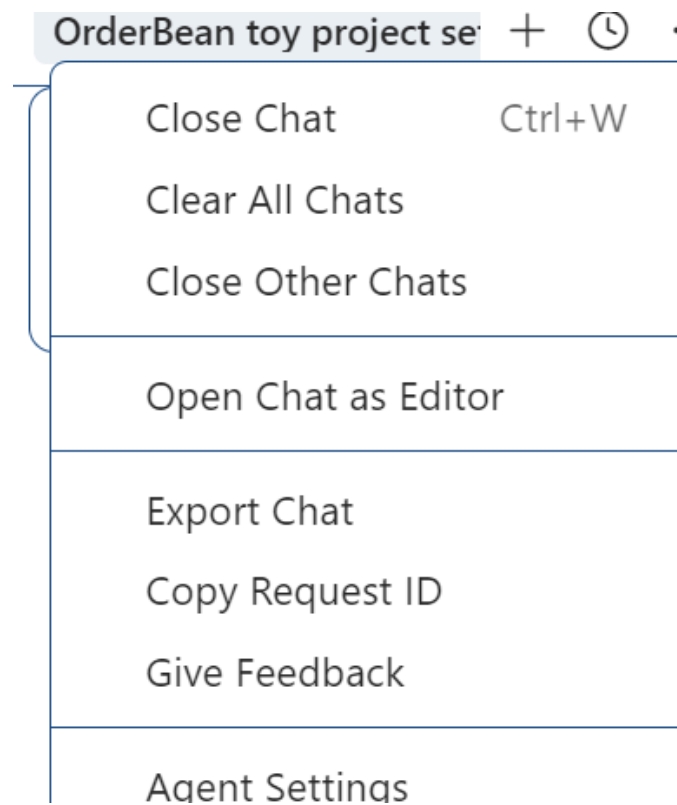
requirements

View all branches

9c2bf49 · 1 hour ago 1 Commit

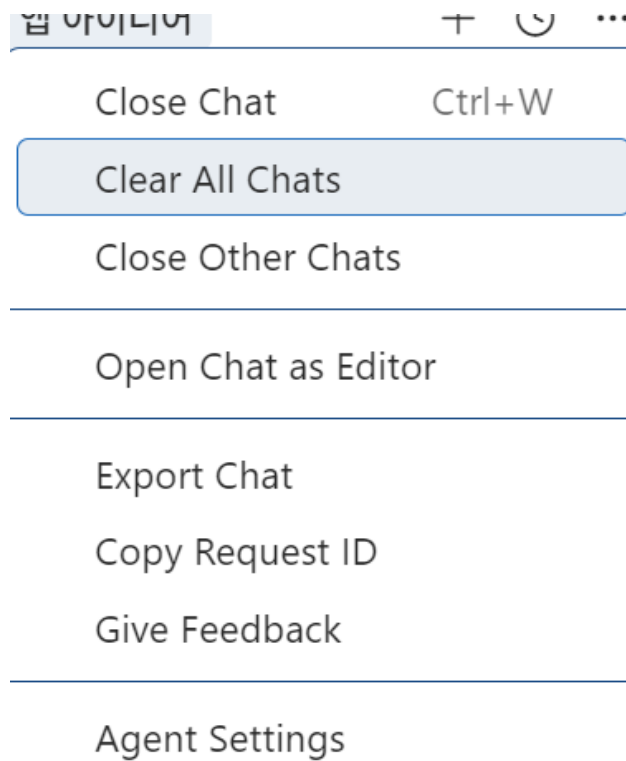
Initial commit: Add README.md 1 hour ago

지금까지 한 내용, Orderbean_toy_project-requirements.md 으로 챗 내보내기



실습 1단계 – 작업 초기화

Clear All Chats한 후, 작업 ...



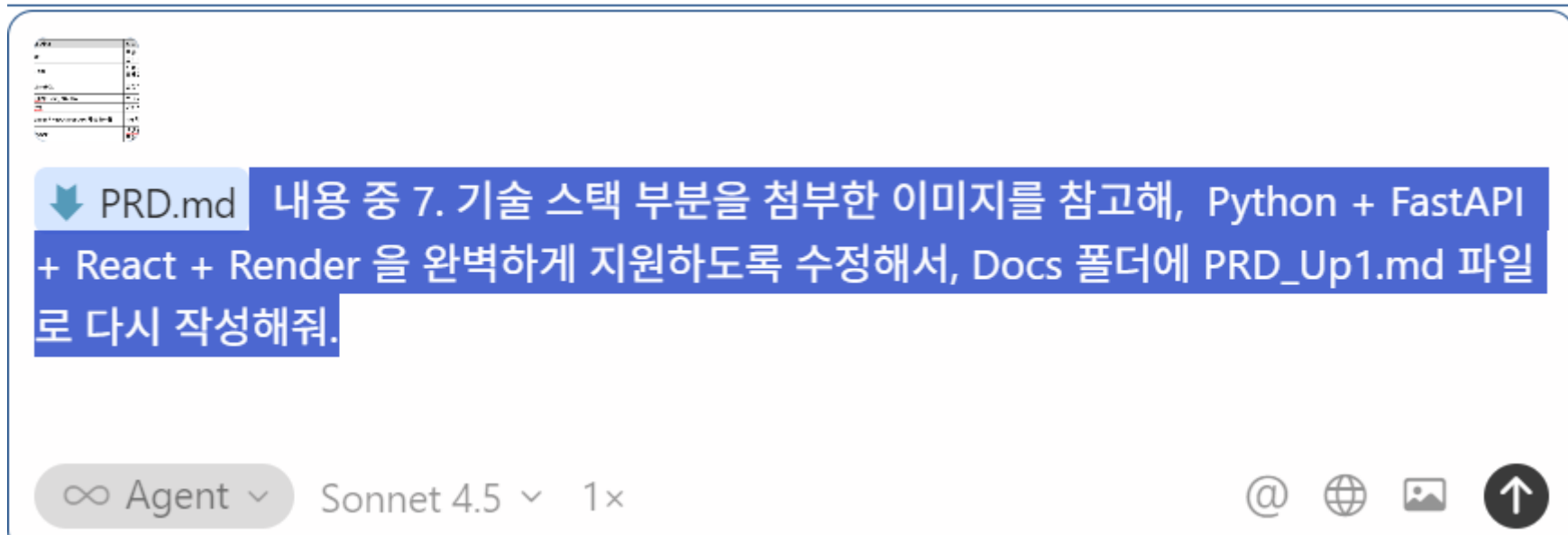
실습 2단계 - PRD 수정 및 추가

OrderBean 프로젝트를 Python 기반 풀스택 웹앱으로 개발할 때 적합한 개발 환경 검토 및 수정. wf-0.png 참조

| 계층(Layer) | 사용 기술 | 설명 |
|------------|---------------------------------|--------------------------|
| 프론트엔드 (UI) | React | 빠른 프로토타입 또는 완성형 웹 인터페이스 |
| 백엔드 (API) | FastAPI | 비동기 고성능 Python 웹 프레임워크 |
| 데이터베이스 | PostgreSQL | 안정적이고 ORM 기반의 DB 설계 |
| 인증/세션 | FastAPI Users, OAuth2 | 로그인, JWT 인증 |
| 테스트 | pytest | 단위/통합 테스트 |
| 문서화 | Swagger / ReDoc (FastAPI 자동 문서) | API 자동 문서 생성 |
| 배포/환경 | Render | 백엔드, DB, 프론트 통합 실행 환경 구성 |

실습 2단계 - PRD 수정 및 추가

...내용 중 7. 기술 스택 부분을 첨부한 이미지를 참고해, Python + FastAPI + React + Render 을 완벽하게 지원하도록 수정해서, Docs 폴더에 PRD_Up1.md 파일로 다시 작성해줘.



실습 2단계 – PRD 수정 및 추가

‘사용자’ 화면에 대한 PRD 작성. wf-1.png 첨부

Cursor.AI 프롬프트

```\n

프론트엔드 UI 관련한 PRD를 작성하려고 합니다.

현재 앱에는 ‘사용자’ 와 ‘관리자’ 이렇게 2개의 화면이 있습니다. 우선, ‘사용자’ 화면의 와이어프레임을 첨부할 테니 ‘사용자’ 화면에 대한 PRD를 작성해 Docs 폴더에 저장해 주세요. 그리고 작업한 결과를 Report 폴더에 마크다운 파일로 작성해 주세요.

```\n

예상 소요시간: 30분



프론트엔드 UI 관련한 PRD를 작성하려고 합니다.

현재 앱에는 ‘사용자’ 와 ‘관리자’ 이렇게 2개의 화면이 있습니다. 우선, ‘사용자’ 화면의 와이어프레임을 첨부할 테니 ‘사용자’ 화면에 대한 PRD를 작성해 Docs 폴더에 저장해 주세요. 그리고 작업한 결과를 Report 폴더에 마크다운 파일로 작성해 주세요.

|

실습 2단계 – PRD 수정 및 추가

‘관리자’ 화면에 대한 PRD 작성. wf-2.png 첨부

Cursor.AI 프롬프트
```\n

다음, ‘관리자 ’ 화면의 와이어프레임을 첨부할 테니 ‘관리자 ’ 화면에 대한 PRD를 작성해 Docs 폴더에 저장해 주세요. 그리고 작업한 결과를 Report 폴더에 마크다운 파일로 작성해 주세요.  
```\n

예상 소요시간: 30분



다음, ‘관리자 ’ 화면의 와이어프레임을 첨부할 테니 ‘관리자 ’ 화면에 대한 PRD를 작성해 Docs 폴더에 저장해 주세요. 그리고 작업한 결과를 Report 폴더에 마크다운 파일로 작성해 주세요.

실습 3단계 - '사용자'화면 만들기

'사용자' 화면 구현하기

Cursor.AI 프롬프트

```\n

그렇다면, 이제 Frontend\_UI\_PRD\_Customer.md를 참고해서 커피 주문하기 화면을 만들어 주세요.  
첨부한 와이어프레임 이미지를 참고해서 화면을 만들어 주세요. 다음 TDDD 방법으로 단계적으로  
구현해주세요.

1. RED 실패하는 테스트 작성

↓

2. GREEN 최소한의 코드로 통과

↓

3. REFACTOR 코드 개선

-네비게이션과 버튼의 색상을 임의의 색으로 맞춰 주세요.

-커피 메뉴는 임의로 넣어 주세요.

그리고 작업한 결과를 Report 폴더에 마크다운 파일로 작성해 주세요.

```\n

예상 소요시간: 30분

실습 3단계 - '사용자'화면 만들기 '사용자' 화면 구현하기

그렇다면, 이제 Frontend_UI_PRD_Customer.md를 참고해서 커피 주문하기 화면을 만들어 주세요. 첨부한 와이어프레임 이미지를 참고해서 화면을 만들어 주세요. 다음 TDDD 방법으로 단계적으로 구현해주세요.

1. RED 실패하는 테스트 작성

↓

2. GREEN 최소한의 코드로 통과

↓

3. REFACTOR 코드 개선

-네비게이션과 버튼의 색상을 임의의 색으로 맞춰 주세요.

-커피 메뉴는 임의로 넣어 주세요.

그리고 작업한 결과를 Report 폴더에 마크다운 파일로 작성해 주세요.

실습 3단계 - '사용자'화면 만들기

'사용자' 화면 구현하기

Run command: cd, npm create

```
cd c:\DEV\Cursor_pro\OrderBean && npm create  
vite@latest frontend -- --template react-ts
```

Use Allowlist ▾

Skip Allowlist 'npm c...' ↗

Run ↗

I thought for 1s

Run command: npm create

```
npm create vite@latest frontend -- --template react-ts
```

Use Allowlist ▾

Skip Allowlist 'npm c...' ↗

Run ↗

실습 3단계 - '사용자'화면 만들기 '사용자' 화면 구현하기

Thought for 2s

● TDD 사이클 1: CustomerHeader - RED 단계

실패하는 테스트부터 작성하겠습니다.

CustomerHeader.test.tsx (new) +37

```
import { describe, it, expect, vi } from 'vitest';
import { render, screen } from '@testing-library/react';
import { CustomerHeader } from '../CustomerHeader';

describe('CustomerHeader', () => {
  it('가 표시되어야 한다' () => {
```

Run command: npm test

npm test -- --run CustomerHeader.test

Use Allowlist ▾

Skip Allowlist 'npm test' ⬆⬇

Run ⬇

|| - 최소한의 코드로 테스트 통과

Run command: npm test

npm test -- --run CustomerHeader

Use Allowlist ▾

Skip Allowlist 'npm test' ⬆⬇

Run ⬇

실습 3단계 - '사용자'화면 만들기

'사용자' 화면 구현하기 ... 계속 작업합니다.

Run command: npm test

npm test -- --run MenuCard

Use Allowlist ▼ Skip Allowlist 'npm test' ⬆️ Run

> 11 Files Stop Ctrl+Shift+Q

실습 3단계 - '사용자'화면 만들기

테스트 커버리지

```
명령 프롬프트
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kz4ne>cd C:\DEV\Cursor_pro\OrderBean\frontend
C:\DEV\Cursor_pro\OrderBean\frontend>npm test -- --run

> frontend@0.0.0 test
> vitest --run

RUN v4.0.6 C:/DEV/Cursor_pro/OrderBean/frontend

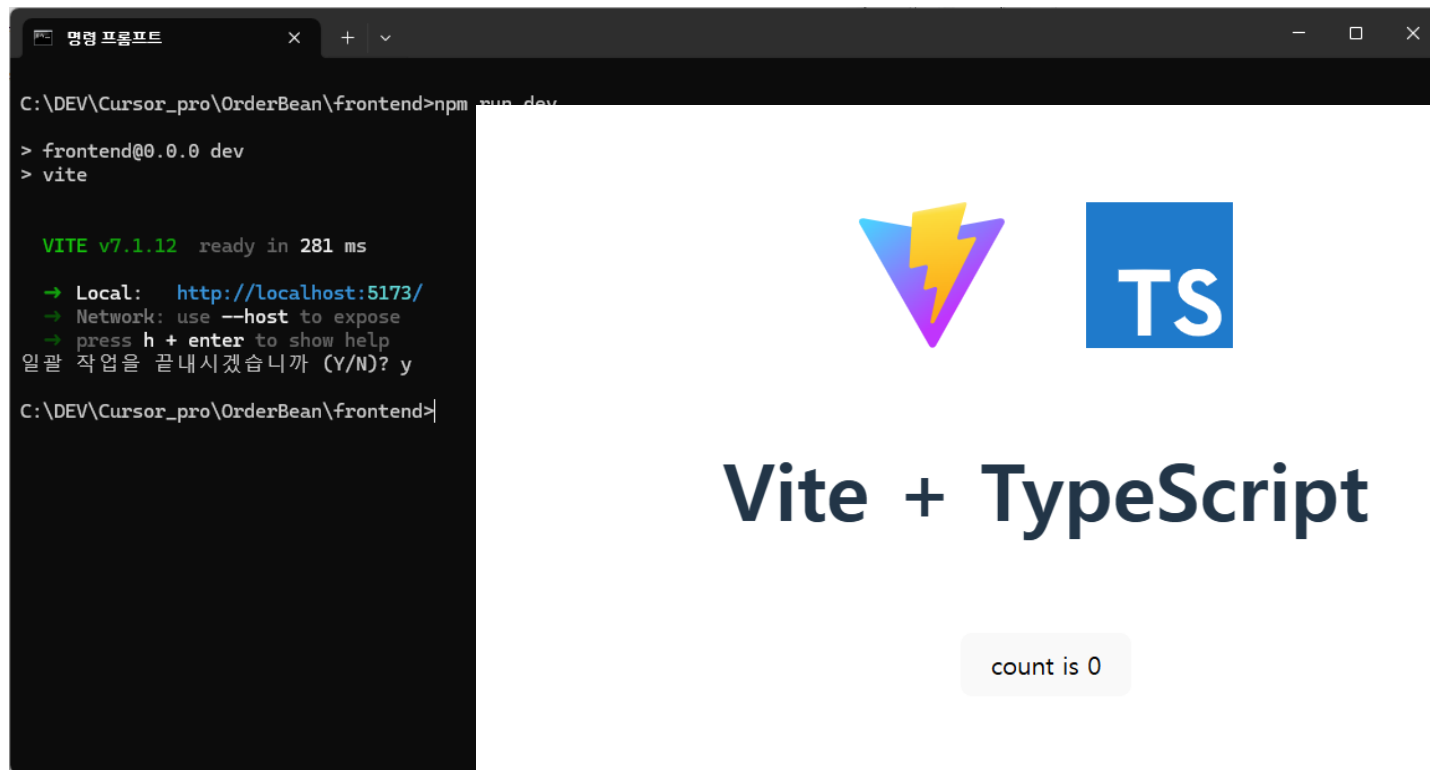
✓ src/components/customer/CustomerHeader.test.tsx (5 tests) 41ms
✓ src/components/customer/CartSection.test.tsx (8 tests) 253ms
✓ src/components/customer/MenuCard.test.tsx (7 tests) 314ms

Test Files 3 passed (3)
Tests 20 passed (20)
Start at 14:00:55
Duration 2.21s (transform 217ms, setup 928ms, collect 363ms, tests 607ms, environment 3.61s, prepare 56ms)

C:\DEV\Cursor_pro\OrderBean\frontend>
```

실습 3단계 - '사용자'화면 만들기

실행



실습 3단계 - '사용자'화면 만들기

Vite 기본 템플릿 화면이 보이고 있는데요. 제대로 된 커피 주문 화면이 표시되도록 수정해주세요.

....

문제가 계속 될 수 있습니다. 계속 수정

실습 3단계 - '사용자'화면 만들기

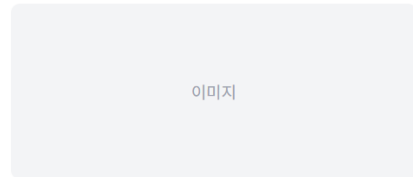
<http://localhost:5173/>

COZY

주문하기

관리자

재고 현황



이미지

아메리카노(ICE)

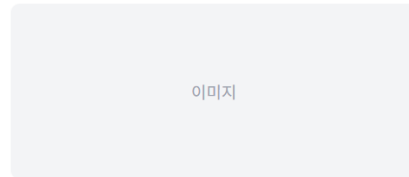
4,000원

간단한 설명...

☐ 샷 추가 (+500원)

☐ 시럽 추가 (+0원)

담기



이미지

아메리카노(HOT)

4,000원

간단한 설명...

☐ 샷 추가 (+500원)

☐ 시럽 추가 (+0원)

담기

장바구니

장바구니가 비어있습니다

총 금액

0원

주문하기

실습 3단계 - '사용자'화면 만들기

화면 수정

Cursor.AI 프롬프트

```\n

수정해야 할 부분은 다음과 같습니다.

- 제목 표시줄의 'COZY'를 'OrderBean - 커피 주문'으로 수정하세요.
- 장바구니 영역을 2개로 나누어서 왼쪽에는 주문 내역을 표시하고, 오른쪽에는 총 금액과 '주문하기' 버튼을 넣어 줘.
- 장바구니 영역의 메뉴별 금액을 보기 좋게 정렬하세요.
- 같은 메뉴를 여러 번 주문했을 때 장바구니에 반복해서 표시하지 말고 개수를 늘려주세요.

```\n

예상 소요시간: 30분

실습 3단계 - '사용자'화면 만들기

화면 수정

> 24 Files

Undo All Keep All ^↵

수정해야 할 부분은 다음과 같습니다.

- 제목 표시줄의 'COZY'를 'OrderBean - 커피 주문'으로 수정하세요.
- 장바구니 영역을 2개로 나누어서 왼쪽에는 주문 내역을 표시하고, 오른쪽에는 총 금액과 '주문하기' 버튼을 넣어 줘.
- 장바구니 영역의 메뉴별 금액을 보기 좋게 정렬하세요.
- 같은 메뉴를 여러 번 주문했을 때 장바구니에 반복해서 표시하지 말고 개수를 늘려주세요.

실습 3단계 - '사용자'화면 만들기

<http://localhost:5173/>

OrderBean - 커피 주문

주문하기

관리자

재고 현황

이미지

아메리카노(ICE)

4,000원

간단한 설명...

☐ 샷 추가 (+500원)

☐ 시럽 추가 (+0원)

담기

이미지

아메리카노(HOT)

4,000원

간단한 설명...

☐ 샷 추가 (+500원)

☐ 시럽 추가 (+0원)

담기

이미지

카페라떼

장바구니

주문 내역

장바구니가 비어있습니다

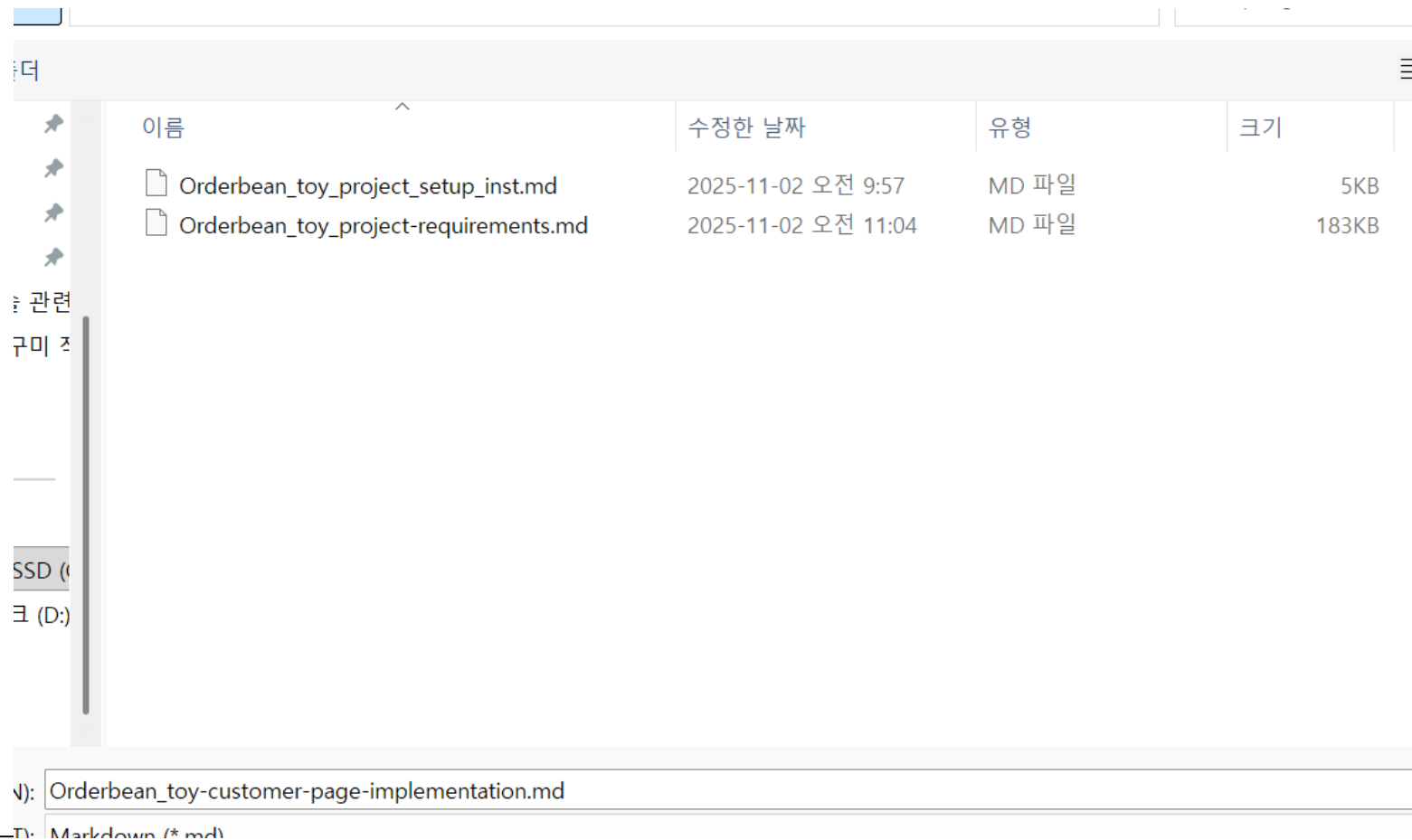
총 금액

0원

주문하기

실습 3단계 - '사용자'화면 만들기

Orderbean_toy-customer-page-implementation.md파일로 챗 내보내기



| 이름 | 수정한 날짜 | 유형 | 크기 |
|---------------------------------------|---------------------|-------|-------|
| Orderbean_toy_project_setup_inst.md | 2025-11-02 오전 9:57 | MD 파일 | 5KB |
| Orderbean_toy_project-requirements.md | 2025-11-02 오전 11:04 | MD 파일 | 183KB |

Orderbean_toy-customer-page-implementation.md

Markdown (*.md)

실습 4단계 - '관리자'화면 만들기

Cursor.AI 프롬프트

\\

관리자 화면을 만들어 주세요. Docs/Frontend_UI_PRD_Admin.md 문서와 첨부한 이미지를 참고해서 만들어 주세요. 다음 단계처럼 TDD방법론으로 작성해 주세요.

1. RED 실패하는 테스트 작성

↓

2. GREEN 최소한의 코드로 통과

↓

3. REFACTOR 코드 개선

-관리자 대시보드는4개의항목이 있고, 각 항목마다 몇 개가 있는 지 표시할 겁니다.

- '재고 현황'에는 메뉴 3개에 대한 재고 개수를 표시해 주세요. 5개 미만이면 '주의', 0개면 '품절', 그 외에는 '정상' 이라고 표시해 주세요.

- '재고 현황'에는 관리자가 재고개수를 늘리거나 줄일 수 있는 버튼을 함께 만들어 주세요.

- '주문 현황'에는 접수된 주문 정보가 표시되어야 합니다. 주문 접수 일자와 시간, 주문 메뉴, 금액 등을 함께 표시해 주세요.

-주문이 들어오면 처음에는 '주문 접수' 상태이고, 관리자가 '제조 시작' 버튼을 누르면 제조 과정으로 넘어가도록 해야 합니다.

마지막으로, 작업한 결과를 Report 폴더에 마크다운 파일로 작성해 주세요.

\\

예상 소요시간: 30분

실습 4단계 - '관리자'화면 만들기

Cursor.AI 프롬프트

\\

```
1. 관리  
2. 관리  
3. 관리
```

관리자 화면을 만들어 주세요. Docs/Frontend_UI_PRD_Admin.md 문서와 첨부한 이미지를 참고해서 만들어 주세요. 다음 단계처럼 TDD방법론으로 작성해 주세요.

1. RED 실패하는 테스트 작성

↓

2. GREEN 최소한의 코드로 통과

↓

3. REFACTOR 코드 개선

-관리자 대시보드는4개의항목이 있고, 각 항목마다 몇 개가 있는 지 표시할 겁니다.

-‘재고 현황’에는 메뉴 3개에 대한 재고 개수를 표시해 주세요. 5개 미만이면 ‘주의’, 0개면 ‘품질’, 그 외에는 ‘정상 ’ 이라고 표시해 주세요.

-‘재고 현황’에는 관리자가 재고개수를 늘리거나 줄일 수 있는 버튼을 함께 만들어 주세요.

∞ Agent Sonnet 4.5



실습 4단계 - '관리자'화면 만들기

실행

9단계: 테스트 실행

Run command: cd, npm test

```
cd frontend && npm test -- --run
```

Use Allowlist ▾

Skip Allowlist 'npm test' ⬆⬇

Run ⬇

Run command: cd, npm test

```
cd frontend; npm test -- --run
```

Use Allowlist ▾

Skip Allowlist 'npm test' ⬆⬇

Run ⬇

실습 4단계 - '관리자'화면 만들기

실행

COZY

주문하기

관리자

```
C:\WINDOWS\system32\cmd.exe
오 후 2:10:22 [vite] (client) page reload
C:\DEV\Cursor_pro\OrderBean\frontend>npm run dev

> frontend@0.0.0 dev
> vite

Port 5173 is in use, trying another one...
VITE v7.1.12 ready in 295 ms
  → Local:   http://localhost:5174/
  → Network: use --host to expose
  → press h + enter to show help
```

관리자 대시보드

총 주문 1 / 주문 접수 1 / 제조 중 0 / 제조 완료 0

재고 현황

아메리카노(ICE)

10개

정상

+ -

아메리카노(HOT)

10개

정상

+ -

카페라떼

10개

정상

+ -

주문 현황

7월 31일 13:00

아메리카노(ICE) x 1

4,000원

주문 접수

실습 4단계 - '관리자'화면 만들기

수정

Cursor.AI 프롬프트

\\

수정해야 할 부분은 다음과 같습니다.

-제목 표시줄의 'COZY'를 'OrderBean - 커피 주문'으로 수정하세요.

- '재고 현황' 에서 재고 수량과 재고 상태(정상, 주의, 품절)텍스트를 가로로 나란히 배치해 주세요. 그리고 마크 다운 파일을 업데이트 해 주세요.

\\

예상 소요시간: 30분

실습 4단계 - '관리자'화면 만들기

수정

OrderBean - 커피 주문

주문하기

관리자

관리자 대시보드

총 주문 1 / 주문 접수 1 / 제조 중 0 / 제조 완료 0

재고 현황

아메리카노(ICE)

10개 정상

+ -

아메리카노(HOT)

10개 정상

+ -

카페라떼

10개 정상

+ -

주문 현황

7월 31일 13:00

아메리카노(ICE) x 1

4,000원

주문 접수

실습 4단계 - '관리자'화면 만들기

Orderbean_toy-admin-page-implementation.md 이름으로 챗 내보내기

폴더

| 이름 | 수정한 날짜 | 유형 | 크기 |
|---|---------------------|-------|----|
| Orderbean_toy_project_setup_inst.md | 2025-11-02 오전 9:57 | MD 파일 | |
| Orderbean_toy_project-requirements.md | 2025-11-02 오전 11:04 | MD 파일 | |
| Orderbean_toy-customer-page-implementation... | 2025-11-02 오후 2:39 | MD 파일 | |

기술 관련
영 구미 적

ig

vs-SSD (C:
스크 (D:)

름(N): Orderbean_toy-admin-page-implementation.md

식(T): Markdown (*.md)

실습 5단계 -화면 깃 허브에 업로드하기

지금까지 작업한 내용을 requirements 브랜치에서 frontend 브랜치를 새로 생성해 @https://github.com/kznetwork/OrderBean.git 깃허브에 업로드할 수 있을까?

지금까지 작업한 내용을 requirements 브랜치에서 frontend 브랜치를 새로 생성해 @https://github.com/kznetwork/OrderBean.git 깃허브에 업로드할 수 있을까?

∞ Agent ▾ Sonnet 4.5 ▾



Local ▾

shift Tab to plan

실습 5단계 -화면 깃 허브에 업로드하기

깃 허브

The screenshot shows the GitHub interface for a repository named 'OrderBean' owned by 'kznetwork'. The repository is public and has 0 stars, 0 forks, and 0 watches. The main content area shows a commit history with two recent pushes: 'frontend' (2 minutes ago) and 'requirements' (57 seconds ago), both with 'Compare & pull request' buttons. A 'Switch branches/tags' dropdown menu is open, showing the 'main' branch as the default, with other branches 'frontend' and 'requirements' listed. The commit message for the latest commit is 'Initial commit: Add README.md'. On the right, there is a 'Monica' widget with a '저장소 요약' (Repository Summary) section and an 'About' section listing the author as '김대경' and the reviewer as '전체 팀원 명'.

실습 5단계 -화면 깃 허브에 업로드하기

Orderbean_toy_frontend-branch-deployment.md 이름으로 챗 내보내기

실습 6단계 -화면 QA 진행

프론트엔드 테스트 및 개선을 위한 QA를 각자 진행해 봅니다.

Cursor.AI 프롬프트 예시

\\`

지금까지 작성한 프론트엔드 코드를 테스트하고, 개선할 부분이 있는 지 분석하고 알려 주세요.

\\`

예상 소요시간: 45분

실습 1단계 -주문처리를 위한 PRD 작성

데이터 모델과 API 정보를 정리하는 PRD부터 작성한 후 백엔드 개발을 위한 개발 환경을 만들어봅니다.

Cursor.AI 프롬프트

Docs/PRD_Up1.md 파일을 살펴보고,백엔드 개발을 위한 PRD를 작성해 기존 Docs/에 저장해 주세요.

1.데이터 모델

- Menus: 커피 이름, 설명, 가격, 이미지, 재고 수량
- Options: 옵션 이름, 옵션 가격, 연결할 메뉴
- Orders: 주문 일시, 주문 내용(메뉴, 수량, 옵션, 금액)

실습 1단계 -주문처리를 위한 PRD 작성

2.데이터 스키마를 위한 사용자 흐름

- 1) Menus에 있는 내용을 가져와 브라우저 화면에 표시합니다. 이 때 Menus에 있는정보 중 재고 수량은 관리자 화면에 표시합니다.
- 2) 사용자가 앱 화면에서 커피 메뉴를선택해 담고, 선택 정보는 장바구니에 표시됩니다.
- 3) 장바구니에서 '주문하기'버튼을 클릭하면 주문정보를 Orders에 저장합니다. Orders에는 주문 시간과 주문 내용을(메뉴, 수량, 옵션, 금액)을 담습니다.
- 4) Orders에 있는 정보를 관리자 화면의 '주문 현황'표시합니다. 주문의 기본 상태를 '주문 접수'이고, '주문 접수'를 클릭하면 '제조 중'-'완료'로 상태가 변경됩니다.

3. API 설계

- '주문하기'메뉴를 클릭하면 데이터베이스에서 커피 메뉴 목록을 불러와서 보여 줍니다.
- 사용자가 커피를 선택하고 주문하기 버튼을 누르면, 주문 정보를 데이터베이스에 저장합니다
- 주문 정보에 따라 메뉴 목록의 재고도 수정합니다.
- 주문 ID를 전달하면 해당 주문 정보를 보여줍니다.

실습 1단계 -주문처리를 위한 PRD 작성

1.데이터 모델

- Menus: 커피 이름, 설명, 가격, 이미지, 재고 수량
- Options: 옵션 이름, 옵션 가격, 연결할 메뉴
- Orders: 주문 일시, 주문 내용(메뉴, 수량, 옵션, 금액)

2.데이터 스키마를 위한 사용자 흐름

- 1) Menus에 있는 내용을 가져와 브라우저 화면에 표시합니다. 이 때 Menus에 있는정보 중 재고 수량은 관리자 화면에 표시합니다.
- 2) 사용자가 앱 화면에서 커피 메뉴를선택해 담고, 선택 정보는 장바구니에 표시됩니다.
- 3) 장바구니에서 '주문하기'버튼을 클릭하면 주문정보를 Orders에 저장합니다. Orders에는 주문 시간과 주문 내용을(메뉴, 수량, 옵션, 금액)을 담습니다.
- 4) Orders에 있는 정보를 관리자 화면의 '주문 현황'표시합니다. 주문의 기본 상태를 '주문 접수'이고, '주문 접수'를 클릭하면 '제조 중'-'>'완료'로 상태가 변경됩니다.

∞ Agent ▾ Sonnet 4.5 ▾ 1×



실습 2단계 -주문처리를 위한 서버 환경 구성하기

Python 기반 백엔드 서버환경을 구성합니다.

(py3_10_basic) 가상환경으로 이동...

실습 2단계 -주문처리를 위한 서버 환경 구성하기

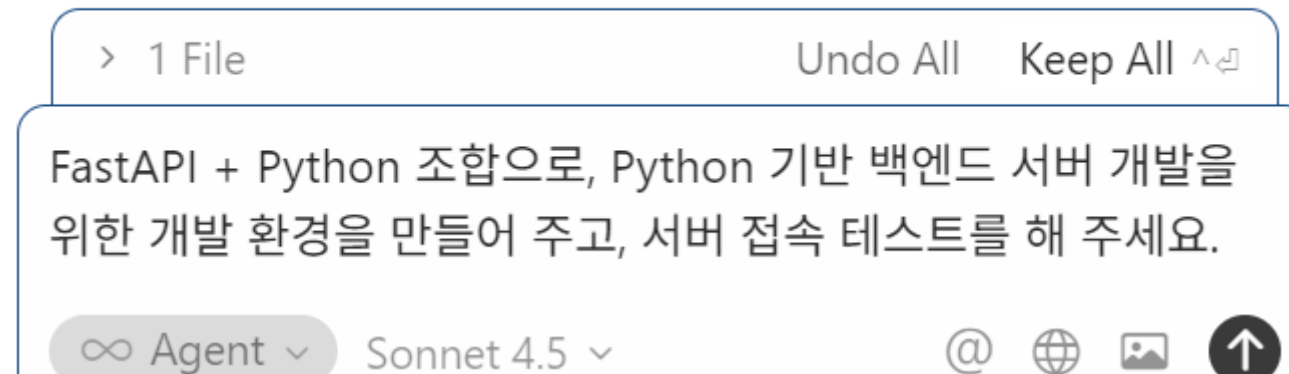
Python 기반 백엔드 서버환경을 구성합니다.

Cursor.AI 프롬프트

```\n

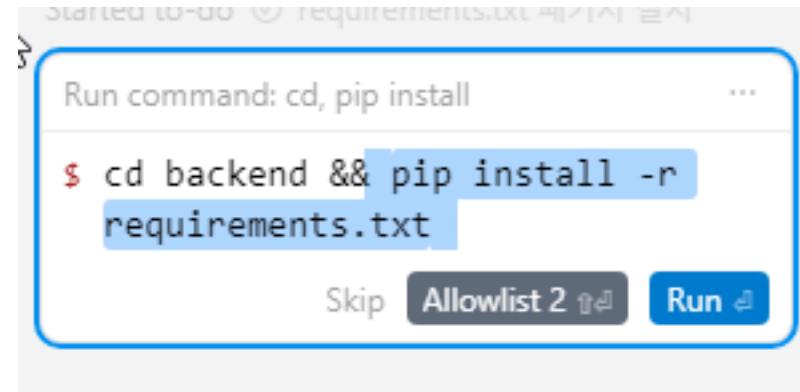
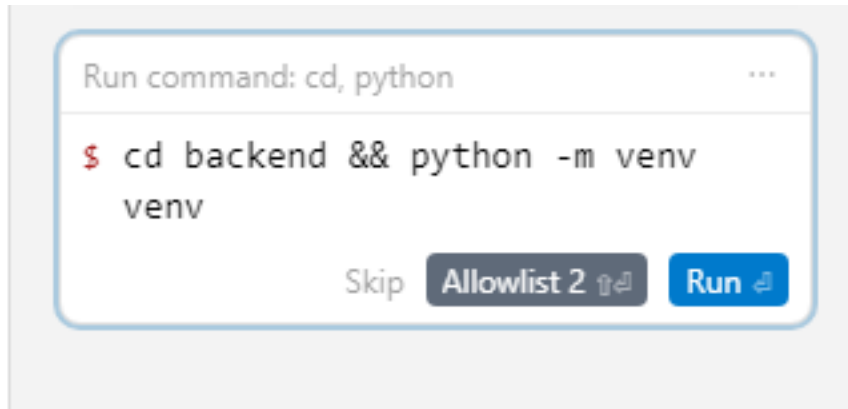
FastAPI + Python 조합으로, Python 기반 백엔드 서버 개발을 위한 개발 환경을 만들어 주고, 서버 접속 테스트를 해 주세요.

```\n



실습 2단계 -주문처리를 위한 서버 환경 구성하기

설치하지 않고, Skip... pip install -r requirements.txt 직접 실행



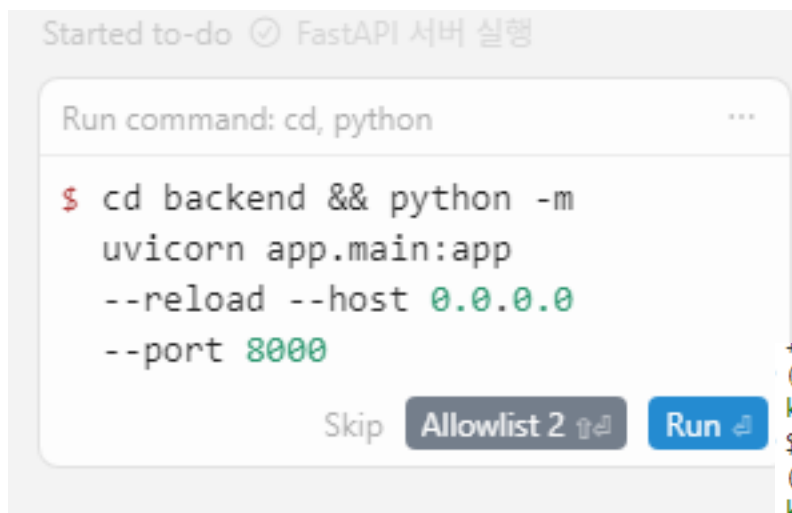
Problems Output Debug Console Terminal Ports

(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean/backend (backend)
\$ pip install -r requirements.txt
Collecting fastapi==0.109.0 (from -r requirements.txt (line 2))
Downloading fastapi-0.109.0-py3-none-any.whl.metadata (24 kB)

실습 2단계 -주문처리를 위한 서버 환경 구성하기

Skip

```
$ python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

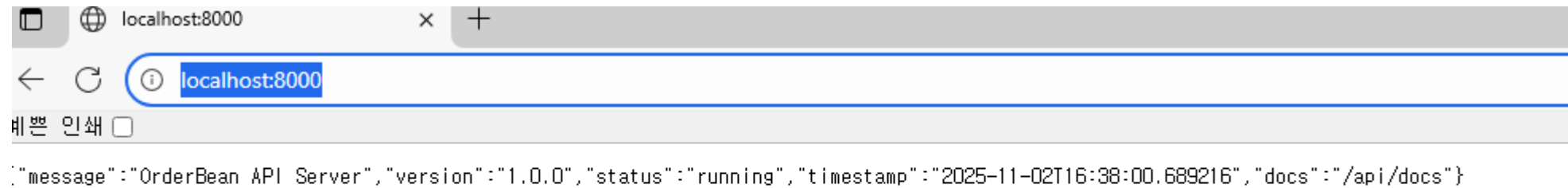


```
+ source C:/DEV/ENV/py3_10_basic/scripts/activate
(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
$ cd backend/
(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean/backend (backend)
$ python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
INFO: Will watch for changes in these directories: ['C:\\DEV\\Cursor_pro\\OrderBean\\backend']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [5948] using WatchFiles
INFO: Started server process [7784]
INFO: Waiting for application startup.
🚀 OrderBean API 서버 시작 중 ...
```


실습 2단계 -주문처리를 위한 서버 환경 구성하기

<http://localhost:8000/>

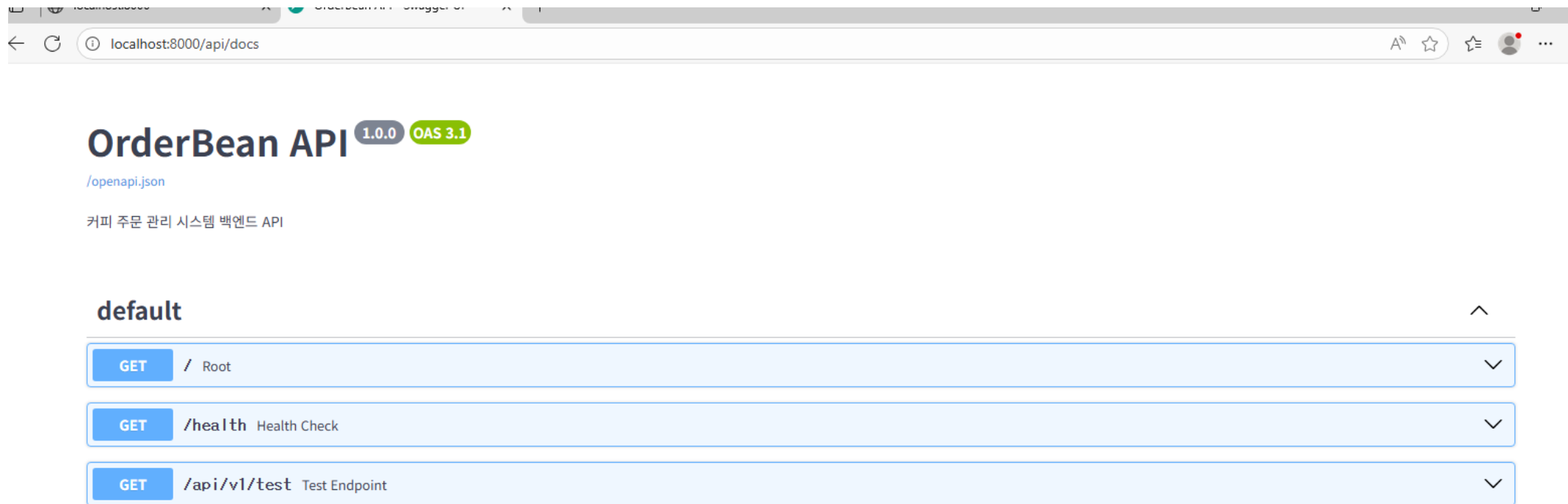
FastAPI 서버가 성공적으로 시작



실습 2단계 -주문처리를 위한 서버 환경 구성하기

OrderBean API - Swagger UI

<http://localhost:8000/api/docs>



실습 3단계 -주문처리를 위한 데이터베이스

개발 환경을 갖추었다면 데이터베이스를 설치하고, 데이터베이스 테이블을 만들어야 합니다.

Cursor.AI 프롬프트
```\n

컴퓨터에 PostgreSQL을 설치했습니다. 현재 프로젝트에 필요한 패키지를 설치하고 데이터베이스를 연결할 수 있을까요?

#데이터베이스 설정

DB\_HOST=localhost

DB\_PORT=5432

DB\_NAME=orderbean\_db

DB\_USER=postgres

DB\_PASSWORD=postgresql

```\n

```
컴퓨터에 PostgreSQL을 설치했습니다. 현재 프로젝트에 필요한 패키지를 설치하고 데이터베이스를 연결할 수 있을까요?
#데이터베이스 설정
DB_HOST=localhost
DB_PORT=5432
DB_NAME=orderbean_db
DB_USER=postgres
DB_PASSWORD=postgresql
```

∞ Sonnet 4.5



실습 3단계 -주문처리를 위한 데이터베이스

Run

```
started to-do PostgreSQL orderbean_db 네...  
Ran command: [?] [?] ...  
$env:PGPASSWORD='postgresql',  
psql  
  
$ $env:PGPASSWORD='postgresql';  
psql -U postgres -h  
localhost -c "CREATE  
DATABASE orderbean_db;"  
psql : 'psql' [?] [?] cmdlet, [?] [?], [?] [?]
```

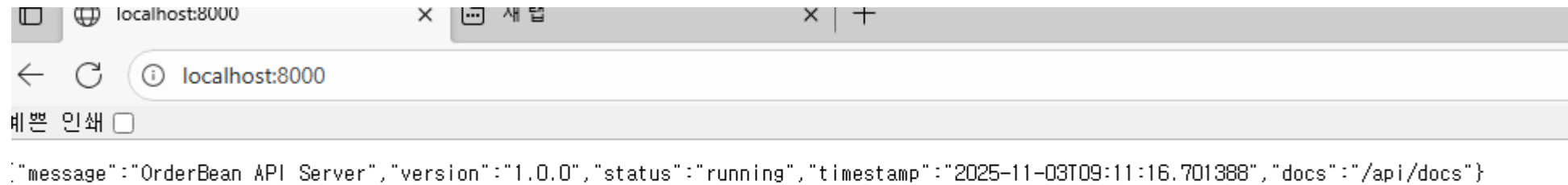
실습 3단계 -주문처리를 위한 데이터베이스

```
$ python -m uvicorn app.main:app --reload
```

```
(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean/backend (backend)
$ python -m uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\DEV\\Cursor_pro\\OrderBean\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [2072] using WatchFiles
INFO: Started server process [4172]
INFO: Waiting for application startup.
🚀 OrderBean API 서버 시작 중...
📊 데이터베이스: localhost:5432/orderbean_db
2025-11-03 09:23:08,312 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2025-11-03 09:23:08,312 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-11-03 09:23:08,313 INFO sqlalchemy.engine.Engine select current_schema()
2025-11-03 09:23:08,315 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-11-03 09:23:08,316 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2025-11-03 09:23:08,317 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-11-03 09:23:08,318 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-11-03 09:23:08,318 INFO sqlalchemy.engine.Engine SELECT 1
2025-11-03 09:23:08,318 INFO sqlalchemy.engine.Engine [generated in 0.00061s] ()
2025-11-03 09:23:08,322 INFO sqlalchemy.engine.Engine ROLLBACK
✅ 데이터베이스 연결 성공!
INFO: Application startup complete.
■
```

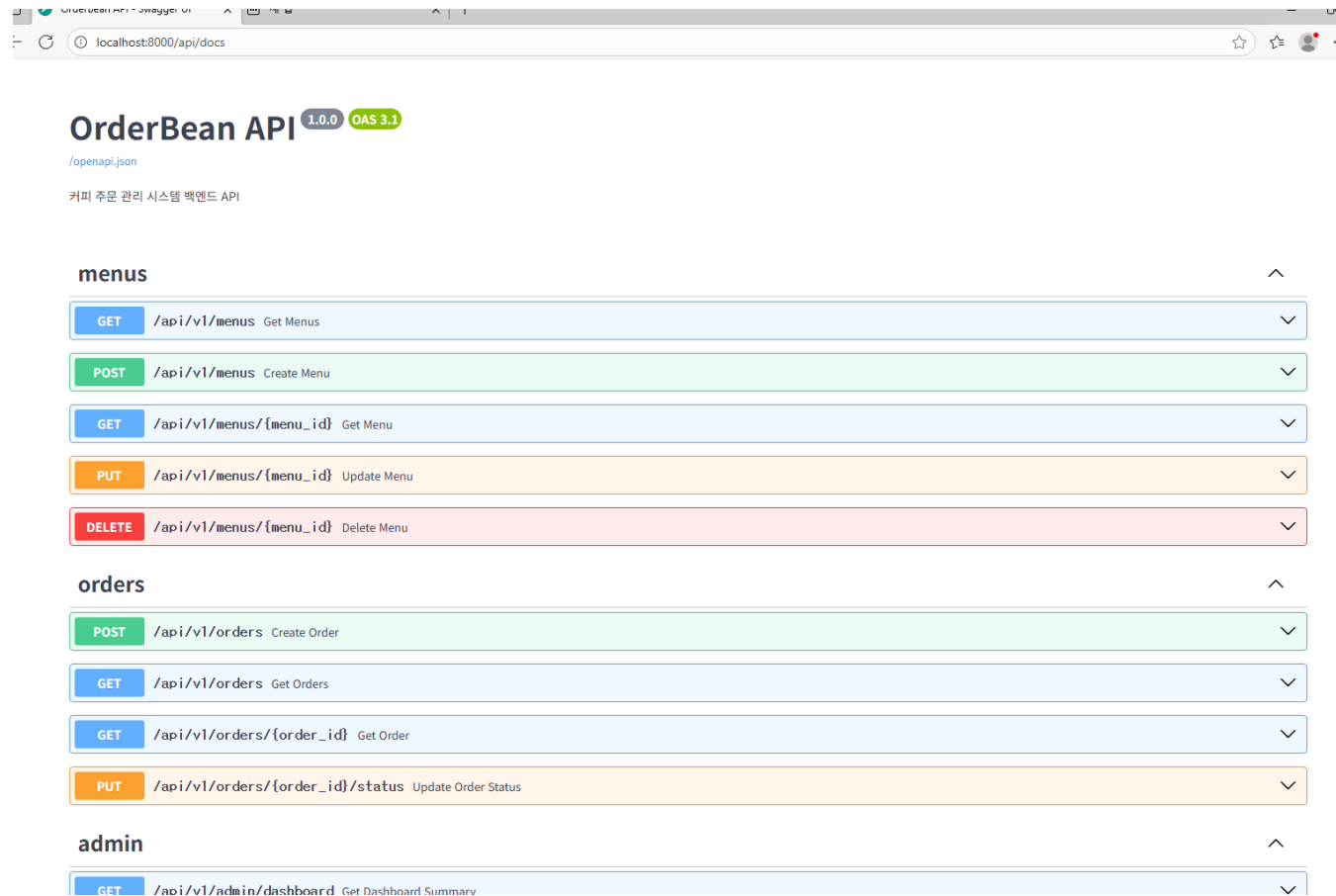
실습 3단계 -주문처리를 위한 데이터베이스

<http://localhost:8000/>



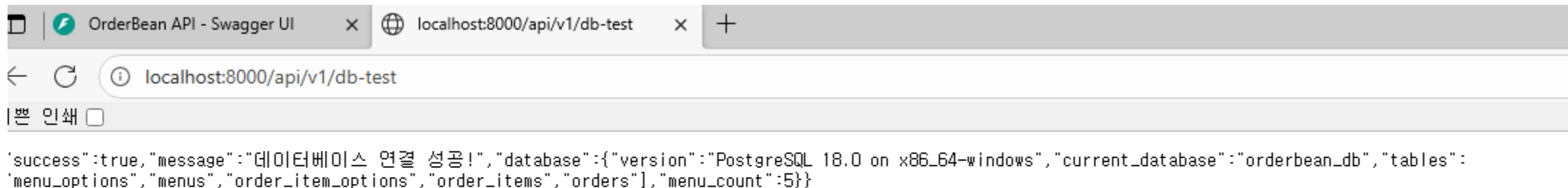
실습 3단계 -주문처리를 위한 데이터베이스

<http://localhost:8000/api/docs>



실습 3단계 -주문처리를 위한 데이터베이스

<http://localhost:8000/api/v1/db-test>



실습 3단계 -주문처리를 위한 데이터베이스

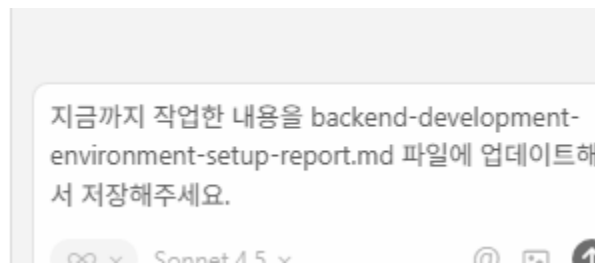
문서 정리를 중도에 한 번 합니다. Orderbean_toy_backend-development.md 챗 보내기 > 이 때 주의!!! 오버라이트 되 버림. 업데이트가 안되니 요령껏 저장. Orderbean_toy_backend-development_유.md로 저장하는 게 좋을 듯 ...

Cursor.AI 프롬프트

\\\

지금까지 작업한 내용을 backend-development-environment-setup-report.md 파일에 업데이트해서 저장해주세요.

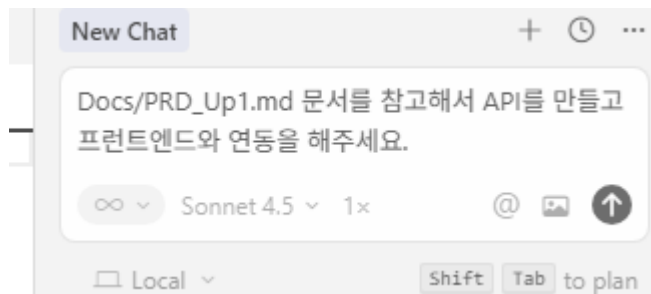
\\\



실습 4단계 -주문처리를 위한 API 프론트엔드와 백엔드를 연동합니다.

Cursor.AI 프롬프트

Docs/PRD_Up1.md 문서를 참고해서 API를 만들고 프론트엔드와 연동을 해주세요.



실습 4단계 -주문처리를 위한 API

백엔드 실행

```
C:\Users\k8s>cd c:\dev\envs
c:\DEV\envs>cd py3_10_basic
c:\DEV\envs\py3_10_basic>Scripts\activate
(py3_10_basic) c:\DEV\envs\py3_10_basic>cd C:\DEV\Cursor_pro\OrderBean
(py3_10_basic) C:\DEV\Cursor_pro\OrderBean>cd backend
(py3_10_basic) C:\DEV\Cursor_pro\OrderBean\backend> python -m uvicorn app.main:app -
-reload --host 0.0.0.0 --port 8000
```

실습 4단계 -주문처리를 위한 API

백엔드 실행

```
선택 명령 프롬프트 - python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

(py3_10_basic) C:\DEV\Cursor_pro\OrderBean\backend>python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
+32mINFO+0m: Will watch for changes in these directories: ['C:\DEV\Cursor_pro\OrderBean\backend']
+32mINFO+0m: Uvicorn running on +1mhttp://0.0.0.0:8000+0m (Press CTRL+C to quit)
+32mINFO+0m: Started reloader process [+36m+1m1608+0m] using +36m+1mWatchFiles+0m
+32mINFO+0m: Started server process [+36m2668+0m]
+32mINFO+0m: Waiting for application startup.
❖ OrderBean API 서버 시작 중...
❖ 데이터베이스: localhost:5432/orderbean_db
2025-11-03 09:44:35,640 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2025-11-03 09:44:35,640 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-11-03 09:44:35,671 INFO sqlalchemy.engine.Engine select current_schema()
2025-11-03 09:44:35,671 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-11-03 09:44:35,671 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2025-11-03 09:44:35,671 INFO sqlalchemy.engine.Engine [raw sql] ()
2025-11-03 09:44:35,671 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-11-03 09:44:35,671 INFO sqlalchemy.engine.Engine SELECT 1
2025-11-03 09:44:35,671 INFO sqlalchemy.engine.Engine [generated in 0.00058s] ()
2025-11-03 09:44:35,671 INFO sqlalchemy.engine.Engine ROLLBACK
✔ 데이터베이스 연결 성공!
+32mINFO+0m: Application startup complete.
```

실습 4단계 -주문처리를 위한 API

프론트 엔드 실행

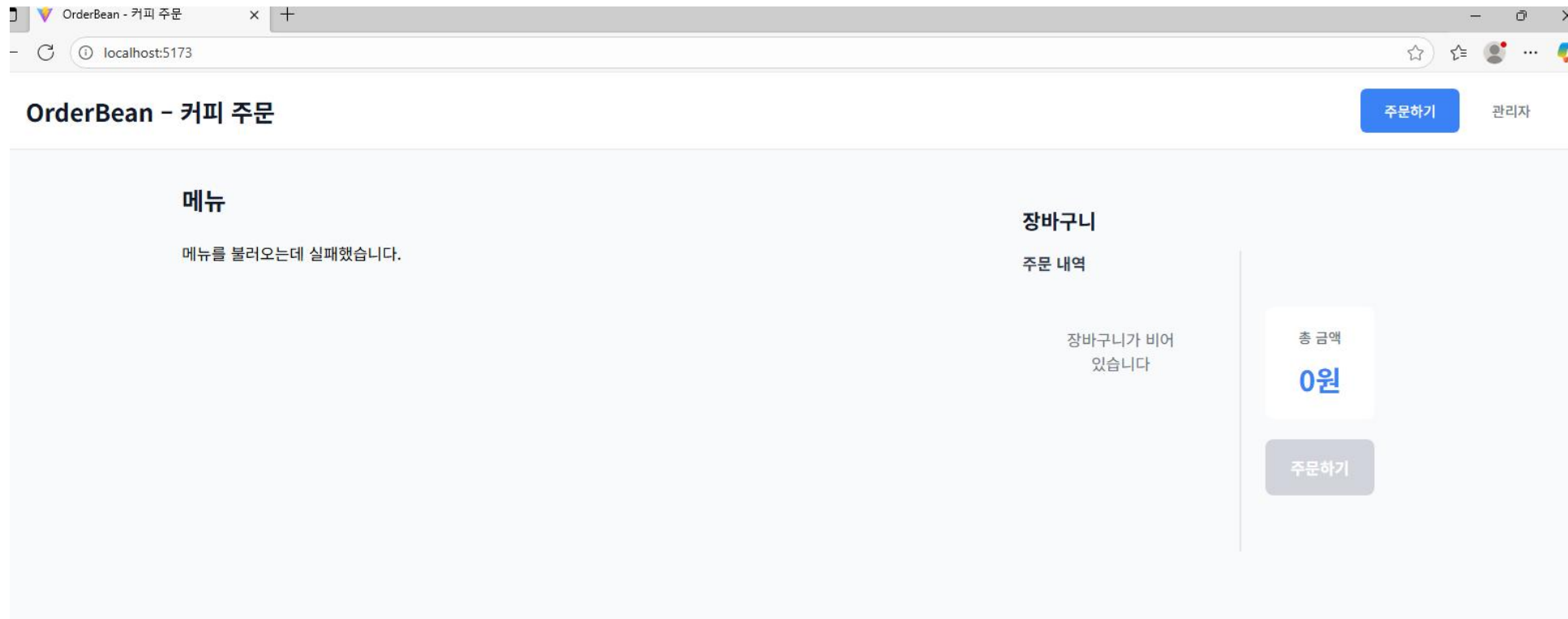
```
C:\Users\k8s>cd C:\DEV\Cursor_pro\OrderBean  
C:\DEV\Cursor_pro\OrderBean>cd frontend  
C:\DEV\Cursor_pro\OrderBean\frontend>npm run dev
```



```
선택 C:\Windows\system32\cmd.exe  
  
VITE v7.1.12 ready in 710 ms  
- Local:   http://localhost:5173/  
- Network: use --host to expose  
- press h + enter to show help
```

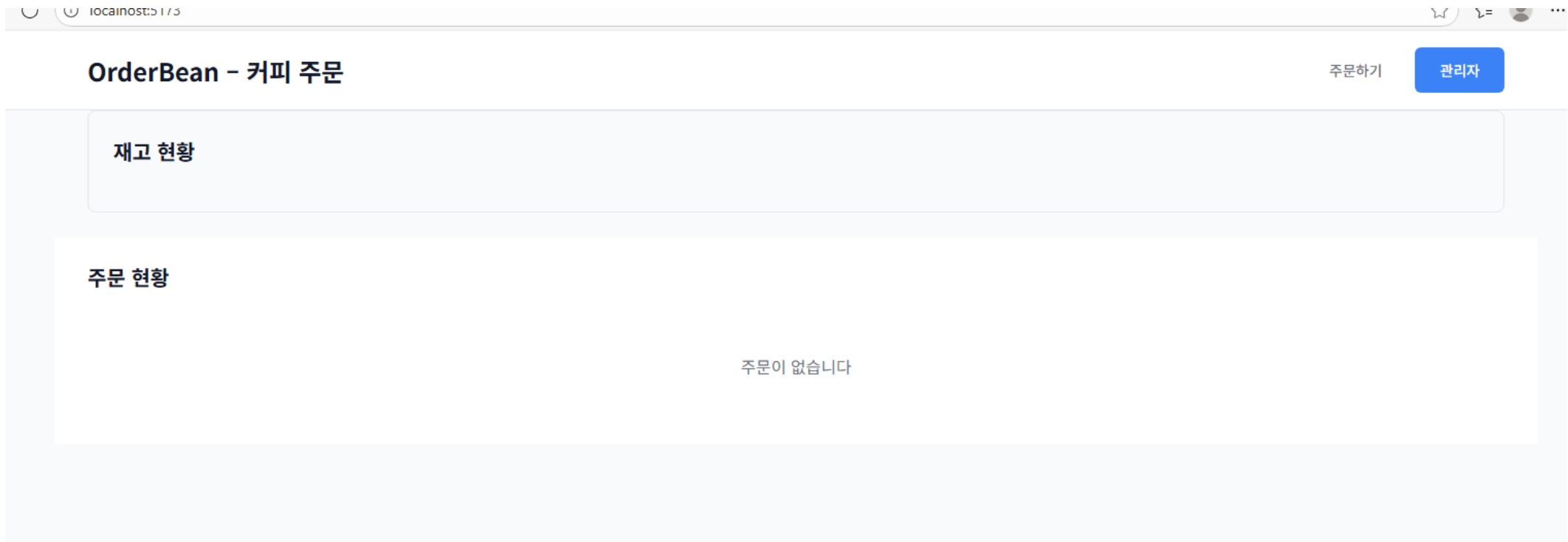
실습 4단계 -주문처리를 위한 API

주문하기 > QA 해야 함



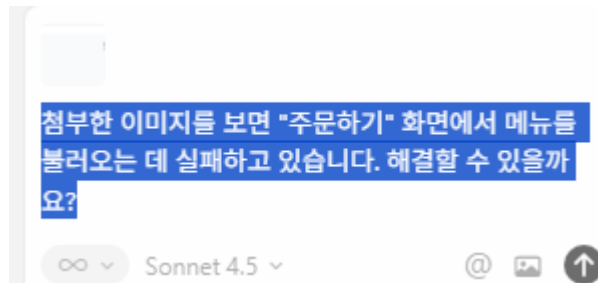
실습 4단계 -주문처리를 위한 API

관리자



실습 5단계 -QA

첨부한 이미지를 보면 "주문하기" 화면에서 메뉴를 불러오는 데 실패하고 있습니다. 해결할 수 있을까요?



실습 5단계 -QA

성공!!!

OrderBean - 커피 주문

localhost:5173

☆ ☆ 👤 ...

주문하기

관리자

OrderBean - 커피 주문

메뉴

이미지

Americano

4000.00원

Strong espresso with water

☐ Size: Regular (+0.00원)

☐ Size: Large (+500.00원)

☐ Extra Shot: 1 Shot (+500.00원)

☐ Extra Shot: 2 Shots (+1000.00원)

☐ Temperature: HOT (+0.00원)

☐ Temperature: ICE (+0.00원)

담기

이미지

Cafe Latte

4500.00원

Smooth milk with espresso

☐ Size: Regular (+0.00원)

☐ Size: Large (+500.00원)

☐ Extra Shot: 1 Shot (+500.00원)

☐ Extra Shot: 2 Shots (+1000.00원)

☐ Temperature: HOT (+0.00원)

☐ Temperature: ICE (+0.00원)

담기

장바구니

주문 내역

장바구니가 비어 있습니다

총 금액

0원

주문하기

실습 5단계 -QA

성공!!!

The screenshot shows a web browser window with the title "OrderBean - 커피 주문" and the address "localhost:5173". The page has a header with "주문하기" and "관리자" buttons. The main content area is divided into three sections:

- 관리자 대시보드**: Today's total orders: 1 (pending), 1 (processing), 0 (completed), 0 (cancelled). Today's sales: 0원.
- 재고 현황**: Inventory status for five coffee types, each with a quantity and a status (정상 - Normal):
 - Americano: 100개, 정상
 - Cafe Latte: 100개, 정상
 - Cappuccino: 100개, 정상
 - Vanilla Latte: 80개, 정상
 - Caramel Macchiato: 80개, 정상
- 주문 현황**: Current orders list showing a single order: 11월 3일 00:21, Americano x 2.

실습 5단계 -QA

인터넷에 접속해
Americano
Cafe Latte
Cappuccino
Vanilla Latte
Caramel Macchiato
이미지를 검색해
'주문하기' 화면에
적용해 봅니다.

The screenshot shows a web browser window with the URL localhost:5173. The page title is "OrderBean - 커피 주문". In the top right corner, there are two buttons: "주문하기" (Order) and "관리자" (Admin). The main content area is divided into two columns. The left column is titled "메뉴" (Menu) and contains two items: "Americano" and "Cafe Latte". Each item has a placeholder image labeled "이미지" (Image). Below the image, the item name is displayed in bold, followed by its price in blue text. For "Americano", the price is 4000.00원, and the description is "Strong espresso with water". For "Cafe Latte", the price is 4500.00원, and the description is "Smooth milk with espresso". Below the description, there are several checkboxes for customization: "Size: Regular (+0.00원)", "Size: Large (+500.00원)", "Extra Shot: 1 Shot (+500.00원)", "Extra Shot: 2 Shots (+1000.00원)", "Temperature: HOT (+0.00원)", and "Temperature: ICE (+0.00원)". At the bottom of each item's section is a blue button labeled "담기" (Add). The right column is titled "장바구니" (Shopping Cart) and has a subtitle "주문 내역" (Order History). It contains the text "장바구니가 비어 있습니다" (Shopping cart is empty). To the right of this, there is a box showing the total amount "총 금액" (Total Amount) as "0원" (0 won) and a "주문하기" (Order) button.

실습 5단계 -QA

인터넷에 접속해

Americano

Cafe Latte

Cappuccino

Vanilla Latte

Caramel Macchiato

이미지를 검색해

'주문하기' 화면에

적용해 봅니다.



🔍 Americano, Cafe Latte, Cappuccino, Vanilla Latte, Caramel Macchiato 이미지

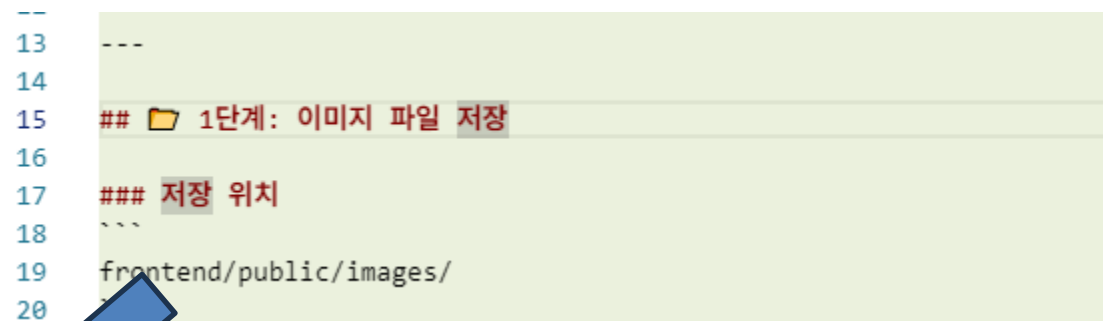
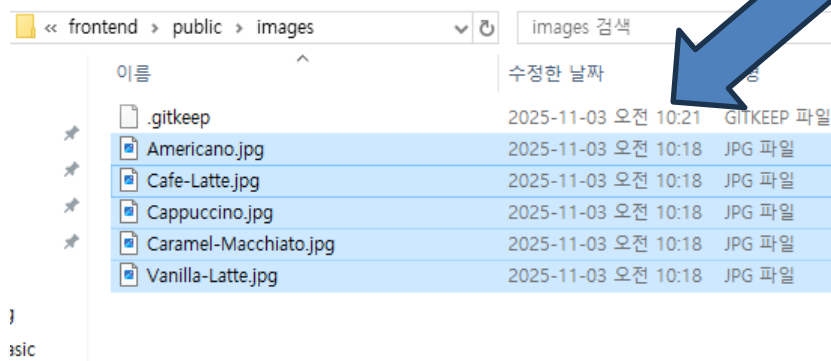
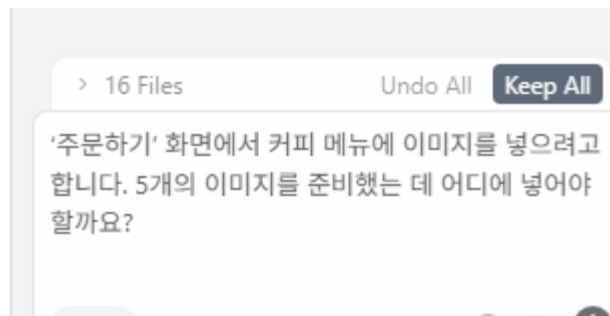
🔍 Americano, Cafe Latte, Cappuccino, Vanilla Latte, Caramel Macchiato 이미지 - Google 검색

실습 5단계 -QA

메뉴 이미지 지정하기

Cursor.AI 프롬프트

'주문하기' 화면에서 커피 메뉴에 이미지를 넣으려고 합니다. 5개의 이미지를 준비했는데 어디에 넣어야 할까요?



실습 5단계 -QA

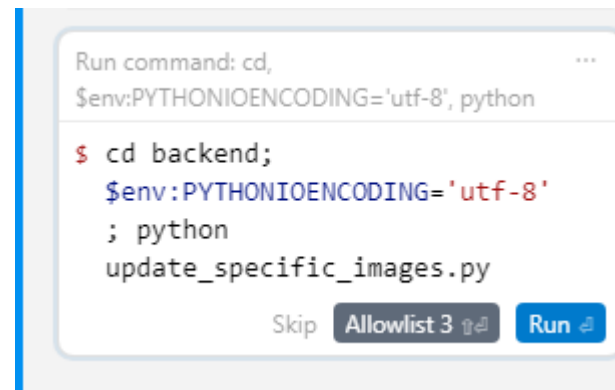
메뉴 이미지 지정하기

Cursor.AI 프롬프트

```\n

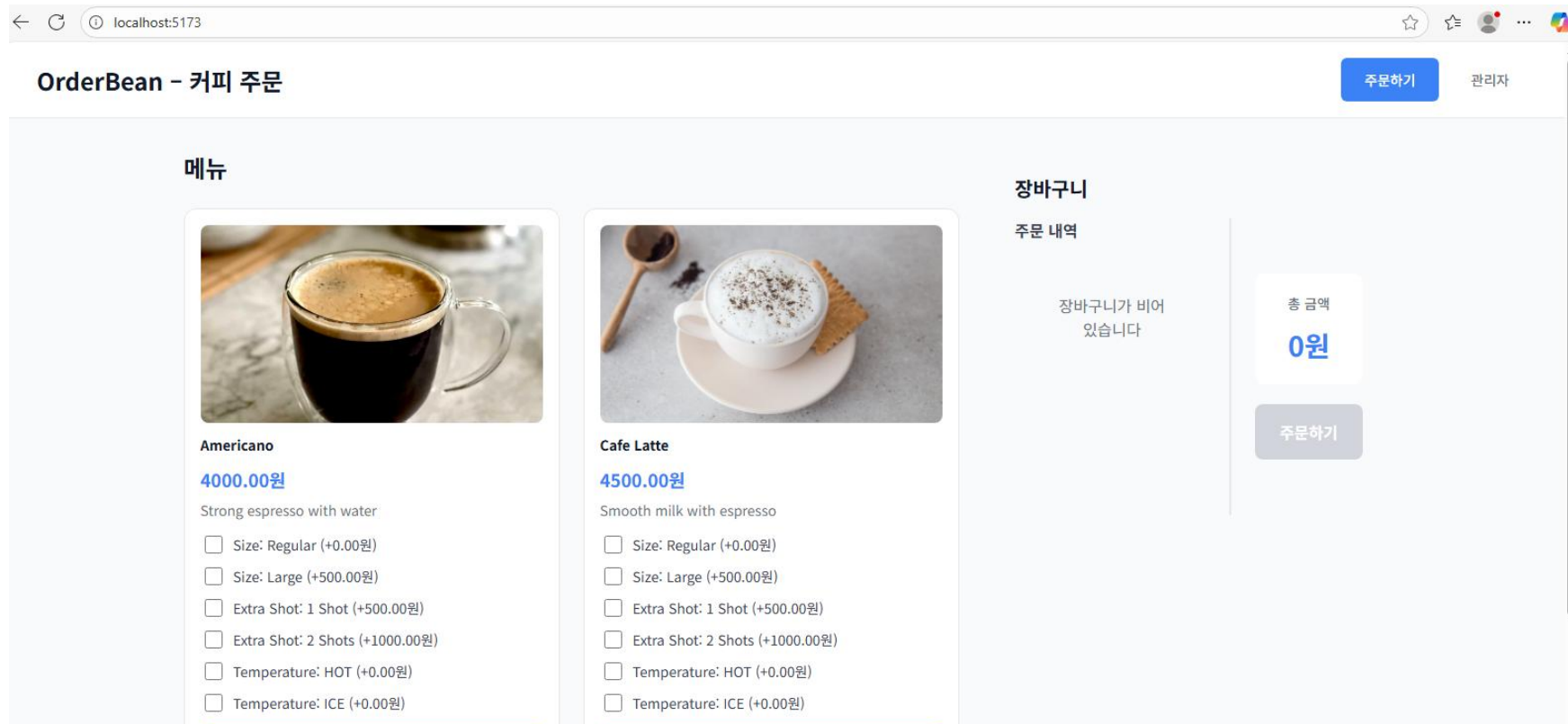
3개의 이미지를 C:\DEV\Cursor\_pro\OrderBean\frontend\public\images 폴더로 옮겼습니다. 데이터베이스에서 Americano는 Americano.jpg, Cafe Latte는 Cafe-Latte.jpg, Cappuccino는 Cappuccino.jpg, Vanilla Latte는 Vanilla-Latte.jpg, Caramel Macchiato는 Caramel-Macchiato.jpg를 넣어주세요.

```\n



실습 5단계 -QA

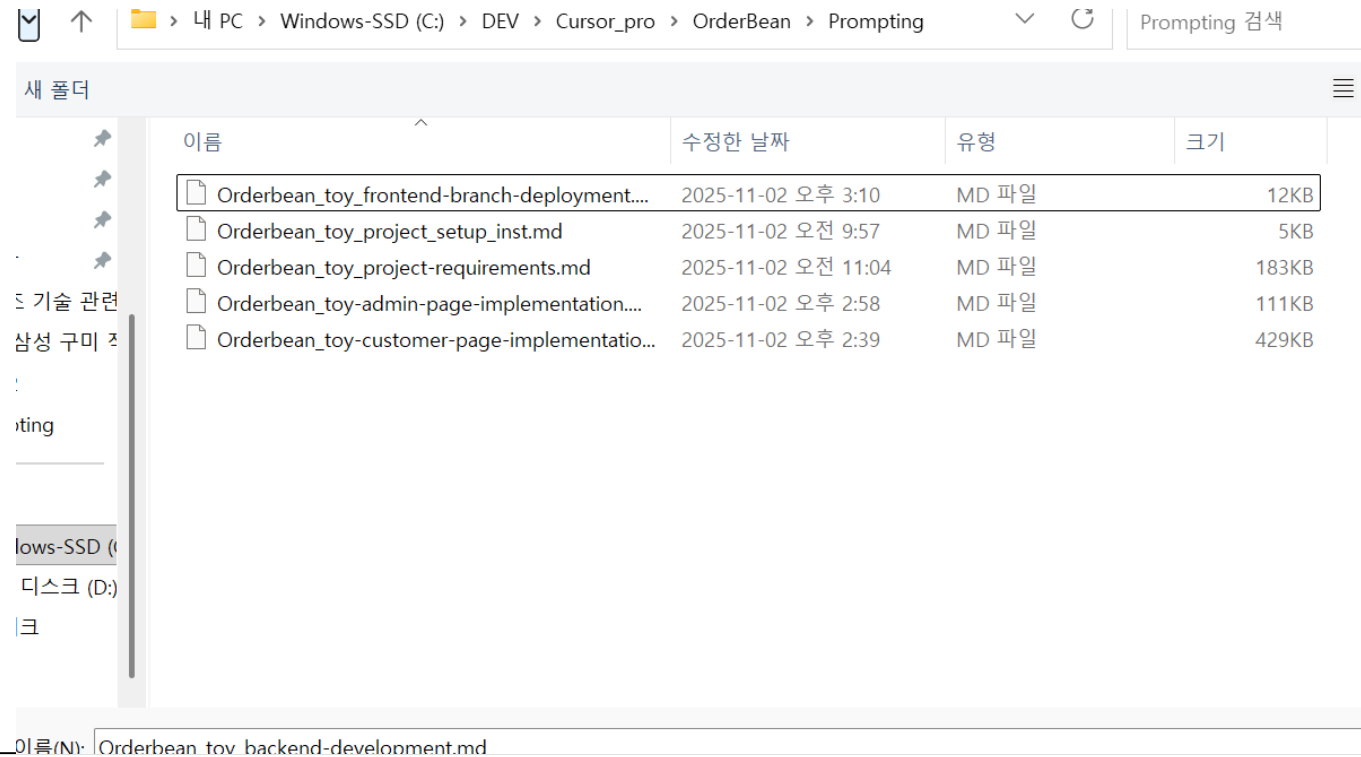
성공적으로 이미지가 적용된 것을 확인할 수 있습니다.



실습 5단계 -문서 관리

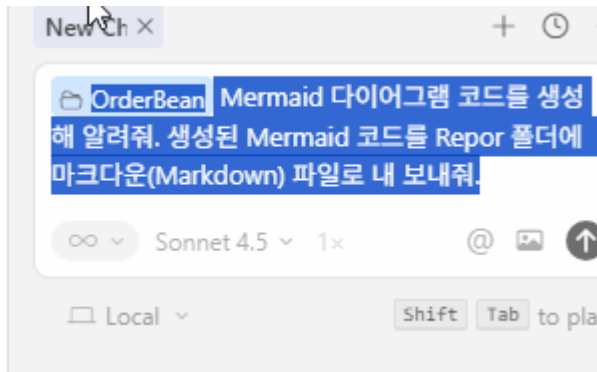
지금까지 작업한 내용을 Report 폴더에 backend-development_finish.md 파일로 저장해주세요.

Orderbean_toy_backend-development_finish.md 챗 내보내기



실습 5단계 -문서 관리

@OrderBean Mermaid 다이어그램 코드를 생성해 알려줘. 생성된 Mermaid 코드를 Repor 폴더에 마크다운(Markdown) 파일로 내 보내줘.



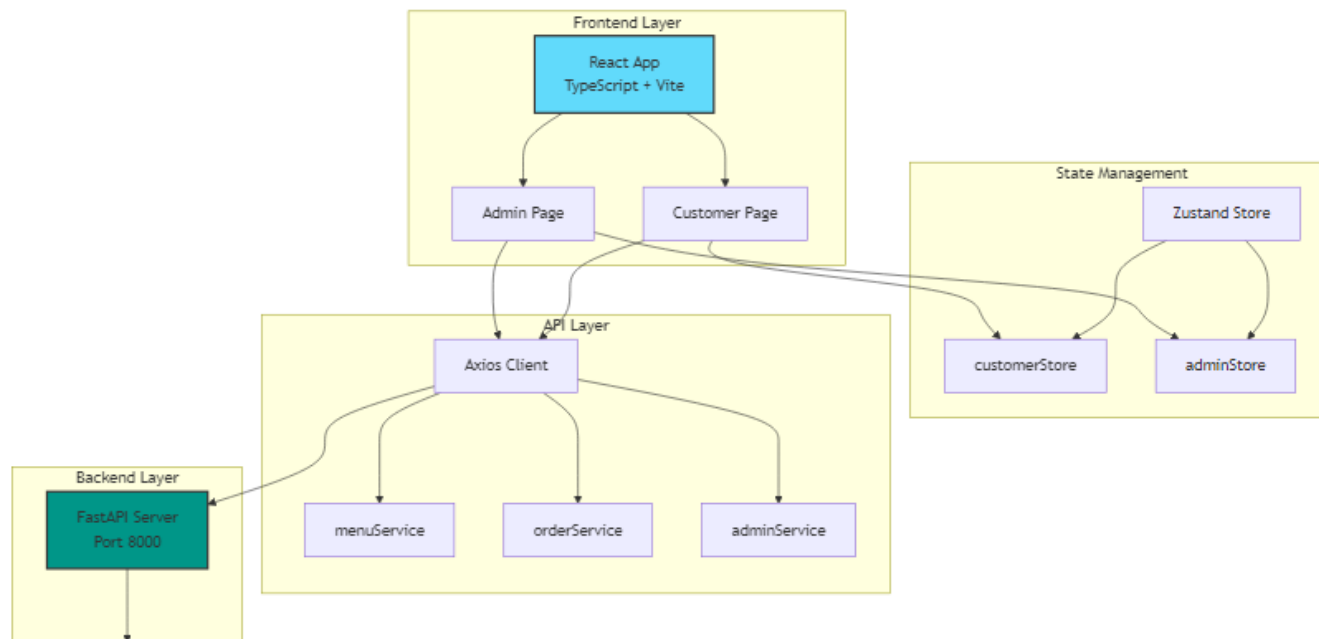
실습 5단계 -문서 관리

OrderBean_Mermaid_Diagrams.md 열고, Ctrl + Shift + V → 미리보기



1. 시스템 아키텍처

전체 시스템 구성도



실습 5단계 -문서 관리

MkDocs 설치와 사용법

1. 설치

- Python 설치: MkDocs는 Python이 필요하므로, Python.org에서 다운로드하여 설치합니다. 설치 시 "Add Python to PATH" 옵션을 선택하는 것이 좋습니다.
- pip 업그레이드: Python 설치 후, 터미널에서 `pip install --upgrade pip` 명령어로 pip를 최신 버전으로 업데이트합니다.
- MkDocs 설치: `pip install mkdocs` 명령어를 실행하여 MkDocs를 설치합니다.
- 테마 설치 (선택 사항): Material 테마를 사용하려면 `pip install mkdocs-material` 명령어를 실행합니다.

실습 5단계 -문서 관리

2. 사용법

- 새 프로젝트 생성: 문서를 만들고 싶은 디렉토리로 이동한 후, `mkdocs new <폴더명>` 명령어를 실행하여 새 프로젝트를 생성합니다.
- 개발 서버 실행: 생성된 폴더로 이동하여 `mkdocs serve` 명령어를 실행합니다. 터미널에 표시되는 로컬 주소(예: `http://127.0.0.1:8000/`)를 웹 브라우저에 입력하면 문서가 보입니다.
- 문서 작성: docs 폴더 안에 있는 .md 파일들을 Markdown 문법으로 작성하여 문서를 만듭니다.
- 설정 변경: 프로젝트 최상위 폴더의 `mkdocs.yml` 파일을 열어 테마, 네비게이션 메뉴 등의 설정을 수정합니다.
- 변경 사항 확인: `mkdocs.yml` 파일을 저장한 후, `mkdocs serve`가 실행 중인 터미널에서 `Ctrl+C`를 눌러 서버를 재시작합니다. 웹 브라우저를 새로고침하면 변경사항이 적용됩니다.
- 정적 사이트 빌드: 모든 개발이 완료되면 `mkdocs build` 명령어를 실행하여 site 폴더에 정적 HTML 파일들을 생성합니다. 이 파일들을 웹 호스팅에 배포할 수 있습니다.

실습 5단계 -문서 관리

2. 사용법

- 새 프로젝트 생성: 문서를 만들고 싶은 디렉토리로 이동한 후, `mkdocs new <폴더명>` 명령어를 실행하여 새 프로젝트를 생성합니다.
- 개발 서버 실행: 생성된 폴더로 이동하여 `mkdocs serve` 명령어를 실행합니다. 터미널에 표시되는 로컬 주소(예: `http://127.0.0.1:8000/`)를 웹 브라우저에 입력하면 문서가 보입니다.
- 문서 작성: docs 폴더 안에 있는 .md 파일들을 Markdown 문법으로 작성하여 문서를 만듭니다.
- 설정 변경: 프로젝트 최상위 폴더의 `mkdocs.yml` 파일을 열어 테마, 네비게이션 메뉴 등의 설정을 수정합니다.
- 변경 사항 확인: `mkdocs.yml` 파일을 저장한 후, `mkdocs serve`가 실행 중인 터미널에서 `Ctrl+C`를 눌러 서버를 재시작합니다. 웹 브라우저를 새로고침하면 변경사항이 적용됩니다.
- 정적 사이트 빌드: 모든 개발이 완료되면 `mkdocs build` 명령어를 실행하여 site 폴더에 정적 HTML 파일들을 생성합니다. 이 파일들을 웹 호스팅에 배포할 수 있습니다.

실습 5단계 -문서 관리

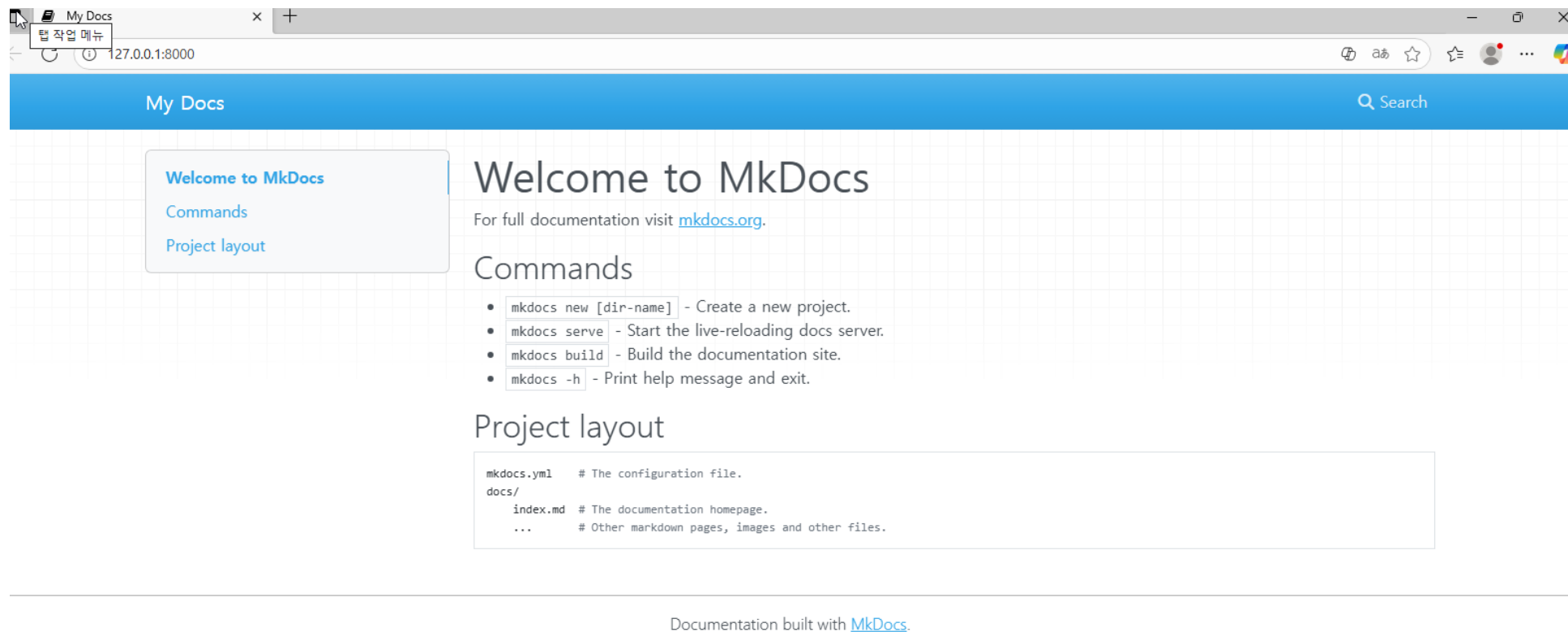
```
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
$ source c:/DEV/envs/py3_10_basic/Scripts/activate
(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
$ pip install --upgrade pip
c:\DEV\envs\py3_10_basic\Scripts\python.exe -m pip install --upgrade pip
(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
$ pip install mkdocs
(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
$ pip install mkdocs-material
(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
$ mkdocs new orderbean_docs
INFO    - Creating project directory: orderbean_docs
INFO    - Writing config file: orderbean_docs\mkdocs.yml
INFO    - Writing initial docs: orderbean_docs\docs\index.md
```

실습 5단계 -문서 관리

```
(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
$ cd orderbean_docs/
(py3_10_basic)
k8s@DESKTOP-R0EQ2U6 MINGW64 /c/DEV/Cursor_pro/OrderBean/orderbean_docs (backend)
$ mkdocs serve
INFO    - Building documentation...
INFO    - Cleaning site directory
INFO    - Documentation built in 0.70 seconds
INFO    - [16:49:54] Watching paths for changes: 'docs', 'mkdocs.yml'
INFO    - [16:49:54] Serving on http://127.0.0.1:8000/
INFO    - [16:50:00] Browser connected: http://127.0.0.1:8000/
```

실습 5단계 -문서 관리

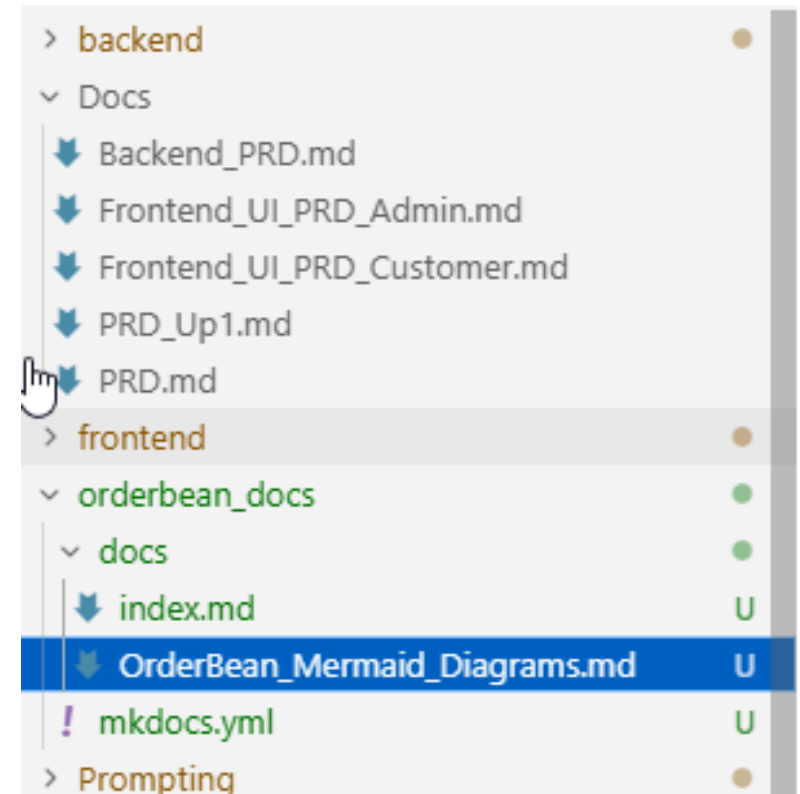
http://127.0.0.1:8000/



실습 5단계 -문서 관리

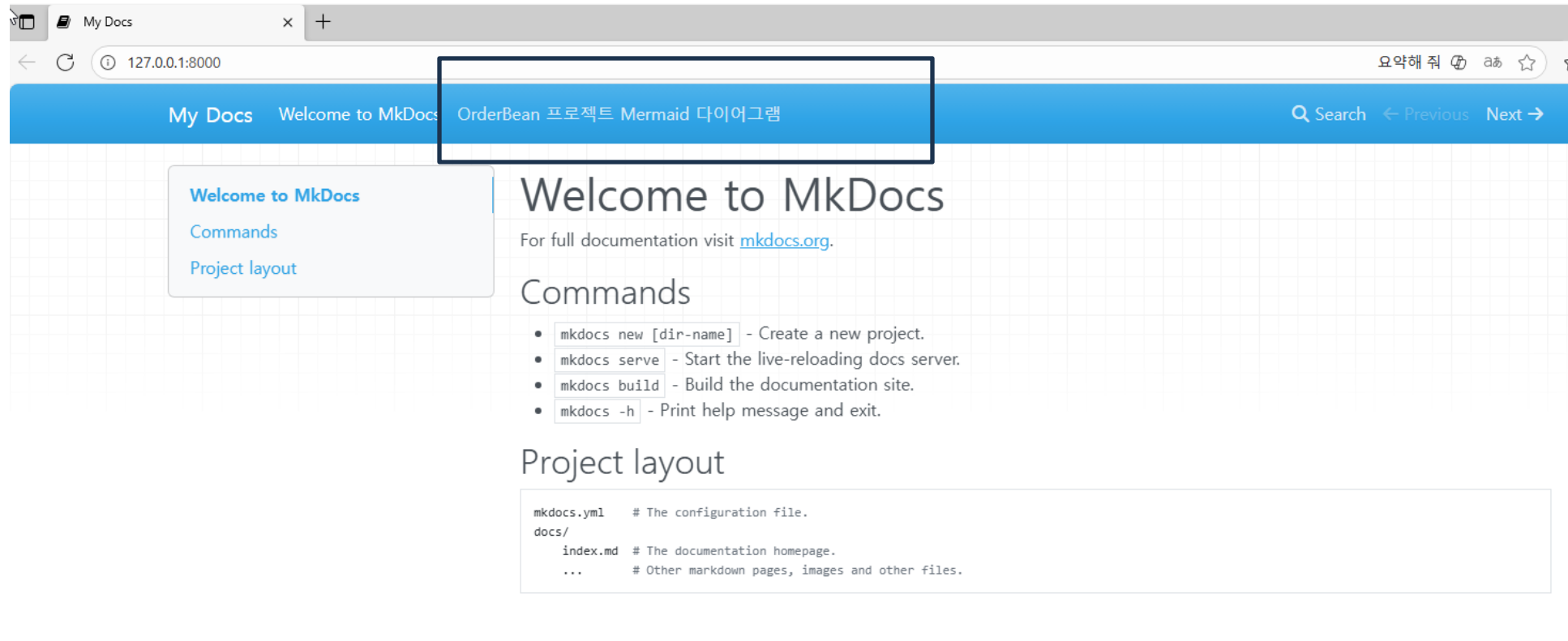
해당 폴더에 markdown 파일을 추가

OrderBean_Mermaid_Diagrams.md 복사



실습 5단계 -문서 관리

OrderBean 프로젝트 Mermaid 다이어그램 메뉴가 생성됨



실습 5단계 - 문서 관리

OrderBean 프로젝트 Mermaid 다이어그램 메뉴가 생성됨

The screenshot shows a web application interface with a blue header bar. The header contains 'My Docs', 'Welcome to MkDocs', and 'OrderBean 프로젝트 Mermaid 다이어그램'. On the right side of the header is a search bar and navigation links: 'Search', '← Previous', and 'Next →'.

The main content area is divided into two columns. The left column contains a sidebar menu with the following items:

- OrderBean 프로젝트 Mermaid 다이어그램
- 목차
- 1. 시스템 아키텍처
- 2. 데이터베이스 ERD
- 3. 고객 주문 플로우
- 4. 관리자 주문 관리 플로우
- 5. 백엔드 레이어 아키텍처
- 6. 프론트엔드 컴포넌트 구조
- 7. API 엔드포인트 구조
- 8. 상태 관리 흐름
- 9. 주문 상태 전이 다이어그램
- 10. 배포 및 개발 환경
- 11. 기술 스택 다이어그램
- 사용 방법
- 버전 히스토리
- 참고 문서

The right column displays the content for the selected menu item, 'OrderBean 프로젝트 Mermaid 다이어그램'. It includes the following information:

- 작성일: 2025년 11월 9일
- 프로젝트: OrderBean - 커피 주문 관리 시스템
- 버전: 1.0

Below this information is a section titled '목차' (Table of Contents) with the following links:

- [1. 시스템 아키텍처](#)
- [2. 데이터베이스 ERD](#)
- [3. 고객 주문 플로우](#)
- [4. 관리자 주문 관리 플로우](#)
- [5. 백엔드 레이어 아키텍처](#)
- [6. 프론트엔드 컴포넌트 구조](#)
- [7. API 엔드포인트 구조](#)
- [8. 상태 관리 흐름](#)

Below the table of contents is a section titled '1. 시스템 아키텍처' with the subtitle '전체 시스템 구성도'. It contains a Mermaid diagram snippet:

```
graph TD
  subgraph "Frontend Layer"
    A[React App<br/>TypeScript + Vite]
    A1[Customer Page]
    A2[Admin Page]
```

실습 5단계 -화면 깃 허브에 업로드하기

```
kz4ne@kz4network MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
```

```
$ git branch
```

```
* backend
```

```
frontend
```

```
main
```

```
requirements
```

```
kz4ne@kz4network MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
```

```
$ git add .
```

```
kz4ne@kz4network MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
```

```
$ git commit -m "API_FRONTEND_INTEGRATION_Finish ..."
```

```
[backend 6cc4c31] API_FRONTEND_INTEGRATION_Finish ...
```

```
...
```

```
kz4ne@kz4network MINGW64 /c/DEV/Cursor_pro/OrderBean (backend)
```

```
$ git push
```

```
Enumerating objects: 208, done.
```

```
.
```

```
To https://github.com/kznetwork/OrderBean.git
```

```
ae43949..6cc4c31 backend -> backend
```

실습 5단계 -화면 깃 허브에 업로드하기

☰

kznetwork / OrderBean

🔍 Type / to search

👤

📁

+

🕒

🔗

📧

<> Code

🔍 Issues

🔗 Pull requests

🕒 Actions

📁 Projects

📖 Wiki

🛡 Security

📈 Insights

⚙ Settings

👤 OrderBean

Public

📌 Pin

👁 Watch 0

🔗 Fork 0

★ Star 0

🔗 backend had recent pushes 38 seconds ago

Compare & pull request

🔗 backend

🔗 4 Branches

🔗 0 Tags

🔍 Go to file

⌵ Add file

<> Code

This branch is 6 commits ahead of, 1 commit behind main

🔗 Contribute

👤 kznetwork

API_FRONTEND_INTEGRATION_Finish ...

6cc4c31 · 1 minute ago

🕒 7 Commits

| | | |
|---------------------|--|--------------|
| 📁 Docs | feat: Add FastAPI backend development environment and d... | 18 hours ago |
| 📁 Prompting | API_FRONTEND_INTEGRATION_Finish ... | 1 minute ago |
| 📁 Report | API_FRONTEND_INTEGRATION_Finish ... | 1 minute ago |
| 📁 backend | API_FRONTEND_INTEGRATION_Finish ... | 1 minute ago |
| 📁 frontend | API_FRONTEND_INTEGRATION_Finish ... | 1 minute ago |
| 📁 res | API_FRONTEND_INTEGRATION_Finish ... | 1 minute ago |
| 📄 ADMIN_PAGE_FIX.md | API_FRONTEND_INTEGRATION_Finish ... | 1 minute ago |

👤 Monica

Instant

🔗

저장소 요약

가장 진보된 모델을 지원하여 저장소 내용을 빠르게 이해할 수 있도록 도와줍니다

🔗 이 저장소 요약

About

⚙

Author: 김대경, Reviewer : 전체 팀원 명

📖 Readme

📈 Activity

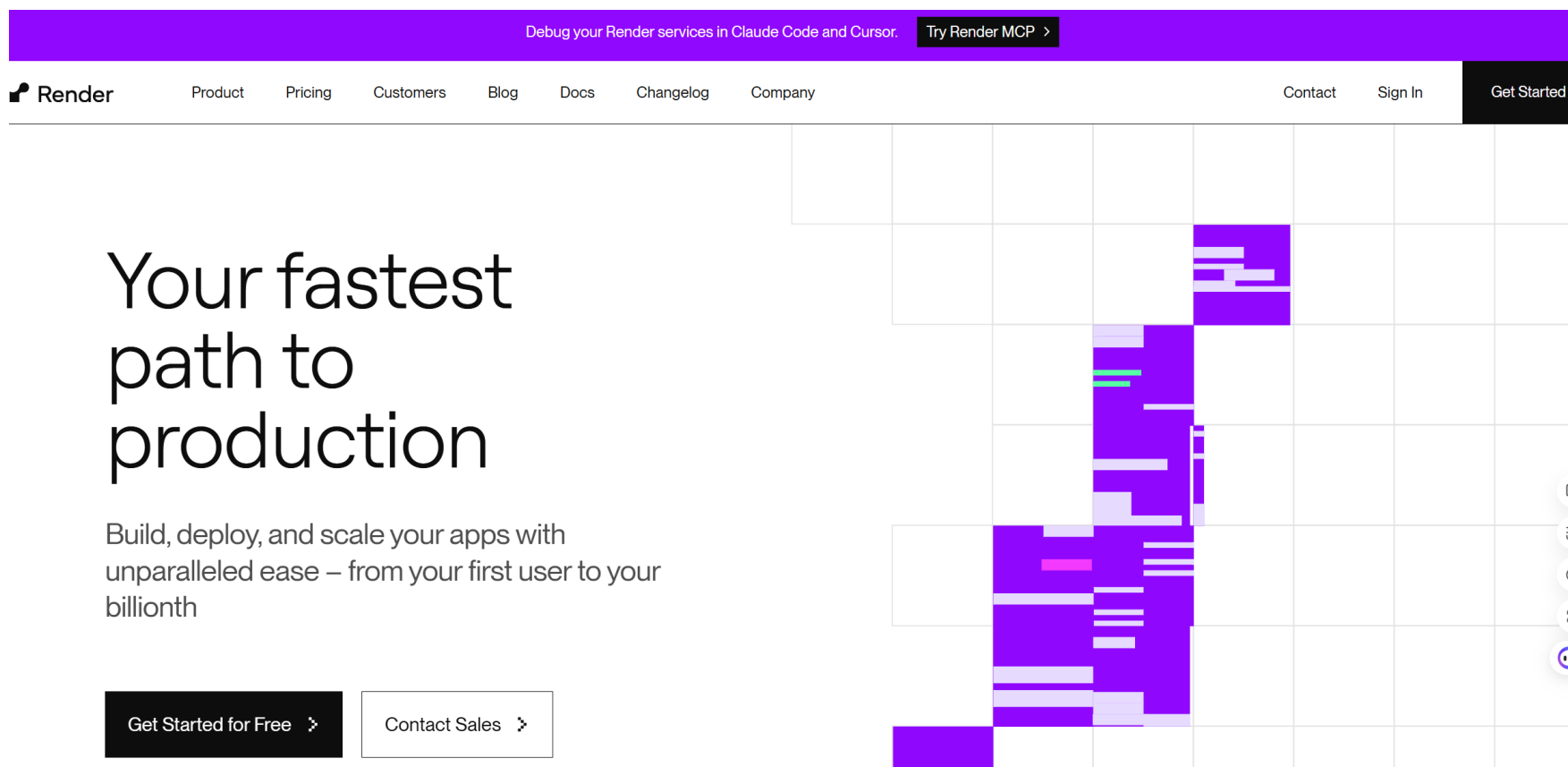
★ 0 stars

👁 0 watching

🔗 0 forks

실습 1단계 -Render 시작하기

<https://render.com/> > Sign In



실습 1단계 -Render 시작하기

깃헙 계정으로 Render 회원 가입

Sign In to Render



GitHub



GitLab



Bitbucket



Google

or

Email

your@email.com

Password

correct horse battery staple



Sign in

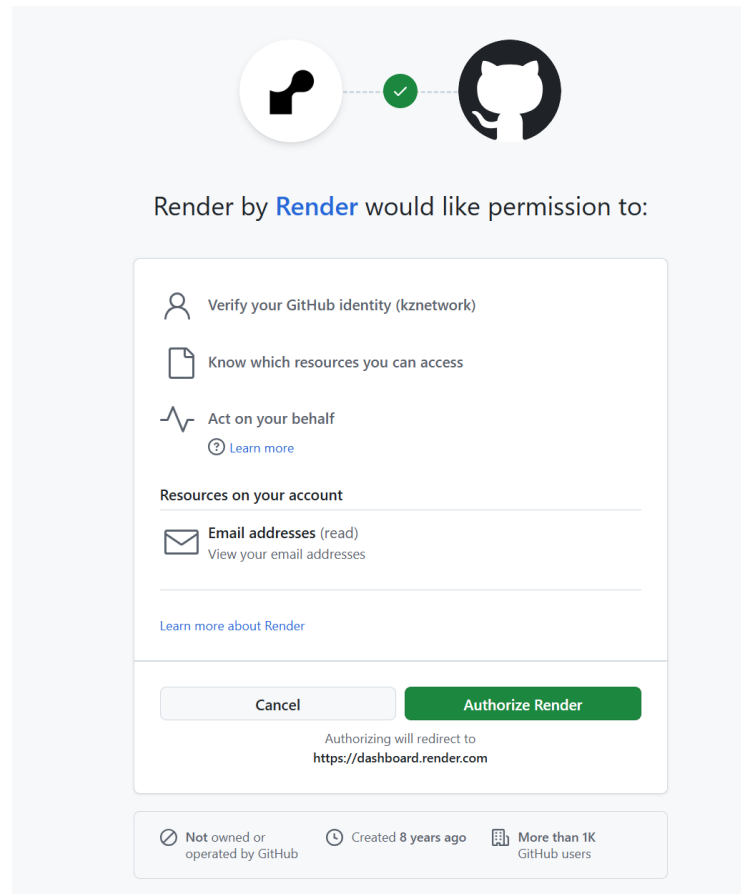
[Sign in with SSO](#)

Need an account? [Sign up](#)

Forgot your password? [Reset it](#)

실습 1단계 -Render 시작하기

깃헙 계정으로 Render 회원 가입



실습 1단계 -Render 시작하기

깃헙 계정으로 Render 회원 가입

Create an account

One last step — please confirm your email below.

Email

kz4network@gmail.com

By signing up you agree to our [terms of service](#) and [privacy policy](#).

Create Account

This site is protected by [hCaptcha](#). Its [Privacy Policy](#) and [Terms of Service](#) apply.

Almost there!

We've sent you an email at **kz4network@gmail.com**.

Please follow the instructions in the email.

Resend Verification Email

실습 1단계 -Render 시작하기

Google 계정으로 확인

Activate your Render account 받은편지함 x

◆ 이메일 요약

Render  <no-reply@render.com>
나에게 ▾

오후 2:50 (0분 전) ☆ 😊 ◀



Hi there,

Welcome to Render! Please verify your email address by clicking the button below:

Verify your email >

This link will expire in 24 hours.

If you did not sign up for a Render account, you can safely ignore this

..

실습 1단계 -Render 시작하기

Get Started

[↗ New empty project](#)

Deploy your code

Create a [static site](#), [web service](#), [private service](#), [background worker](#), or [cron job](#).

 GitHub

 GitLab

 Bitbucket

[↗ Deploy a public repo or existing image](#)

OR

Create a datastore

 Postgres
Relational data storage. Supports point-in-time recovery, read replicas, and high availability.

 Key Value
Managed Redis®-compatible storage. Ideal for use as a shared cache, message broker, or job queue.

실습 1단계 -Render 시작하기

Skip

 M My Workspace  Projects

Search  ^ K + New Upgrade Help K

Create a new Service

1 Choose service > 2 Configure > 3 Deploy

 **Static Sites**
Static content served over a global CDN. Ideal for frontend, blogs, and content sites.
[New Static Site →](#)

 **Web Services**
Dynamic web app. Ideal for full-stack apps, API servers, and mobile backends.
[New Web Service →](#)

 **Private Services**
Web app hosted on a private network, accessible only from your other Render services.
[New Private Service →](#)

 **Background Workers**
Long-lived services that process async tasks, usually from a job queue.
[New Worker →](#)

 **Cron Jobs**
Short-lived tasks that run on a periodic schedule.
[New Cron Job →](#)

 **Postgres**
Relational data storage. Supports point-in-time recovery, read replicas, and high availability.
[New Postgres →](#)

 **Key Value**
Managed Redis®-compatible storage. Ideal for use as a shared cache, message broker, or job queue.
[New Key Value Instance →](#)

[Skip](#)
Which to use?

실습 1단계 -Render 시작하기

앱 배포 화면

 My Workspace 

 Projects

 Blueprints

 Environment Groups

INTEGRATIONS

 Observability

 Webhooks

 Notifications

NETWORKING

 Private Links

WORKSPACE

 Billing

 Settings

Projects

Search  ^ K

+ New

Upgrade 

Help 

K

Overview

 Invite your team

+ Add new 

You haven't created any services yet.

Deploy in just a few minutes. Spin up web services, static sites, cron jobs, and more.

 **Deploy a Web Service** →
Dynamic web app. Ideal for full-stack apps, API servers, and mobile backends.

 **Deploy a Static Site** →
Static content served over a global CDN. Ideal for frontend, blogs, and content sites.

 **Create a Postgres database** →
Relational data storage. Supports point-in-time recovery, read replicas, and high availability.

 **Explore all service types** →

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

앱 배포 화면 > 우측 Add new > Postgre 선택

The screenshot shows the AWS Management Console interface. On the left is a navigation sidebar with sections: Projects (selected), Blueprints, Environment Groups, INTEGRATIONS (Observability, Webhooks, Notifications), NETWORKING (Private Links), and WORKSPACE (Billing, Settings). The main content area is titled 'Overview' and displays a message: 'You haven't created any services yet. Deploy in just a few minutes. Spin up web services, static sites, cron jobs, and more.' Below this are four cards: 'Deploy a Web Service' (Dynamic web app), 'Deploy a Static Site' (Static content served over a global CDN), 'Create a Postgres database' (Relational data storage), and 'Explore all service types'. On the right side of the console, there is a '+ Add new ^' button. A dropdown menu is open from this button, listing service types: Static Site, Web Service, Private Service, Background Worker, Cron Job, Postgres (highlighted with a red box), Key Value, Project, and Blueprint. A blue arrow points from the 'Add new' button to the 'Postgres' option in the dropdown.

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

OrderBean_db

New Postgres

Name

A unique name for your Postgres instance.

OrderBean_db

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

Plan Options

Plan Options



Instance Type

Set your database's RAM and CPU. You can change your instance type later.

- ☒ **Free**
For testing out Render Postgres

| | |
|--------------------|----------------|
| Free | 256 MB (RAM) |
| \$0 / month | 0.1 CPU |
| | 1 GB (Storage) |

- ☐ **Basic**
Reliability and performance for hobby projects. Starting at \$6 / month plus storage.

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

Deploy Postgres Instance

Monthly Total

Database instances are billed and prorated by the second.

Instance: **Free**

\$0 / month

Total

\$0 / month

 Deploy Postgres Instance

Cancel

Free

\$0 / month

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

Connections

Environment

OrderBean_db

Info

MONITOR

Logs

Metrics

MANAGE

Apps

Recovery

Connections

Hostname

An internal hostname used by your Render services.

dpg-d48phdggjchc73f199n0-a

Port

5432

Database

orderbean_db

Username

orderbean_db_user

Password

.....

Internal Database URL

.....

External Database URL

.....

PSQL Command

.....

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

External Database URL 복사

Environment

OrderBean_db

Info

MONITOR

Logs

Metrics

MANAGE

Apps

Recovery

Connections

Hostname

An internal hostname used by your Render services.

dpg-d48phdggjchc73f199n0-a

Port

5432

Database

orderbean_db

Username

orderbean_db_user

Password

.....

Internal Database URL

.....

External Database URL

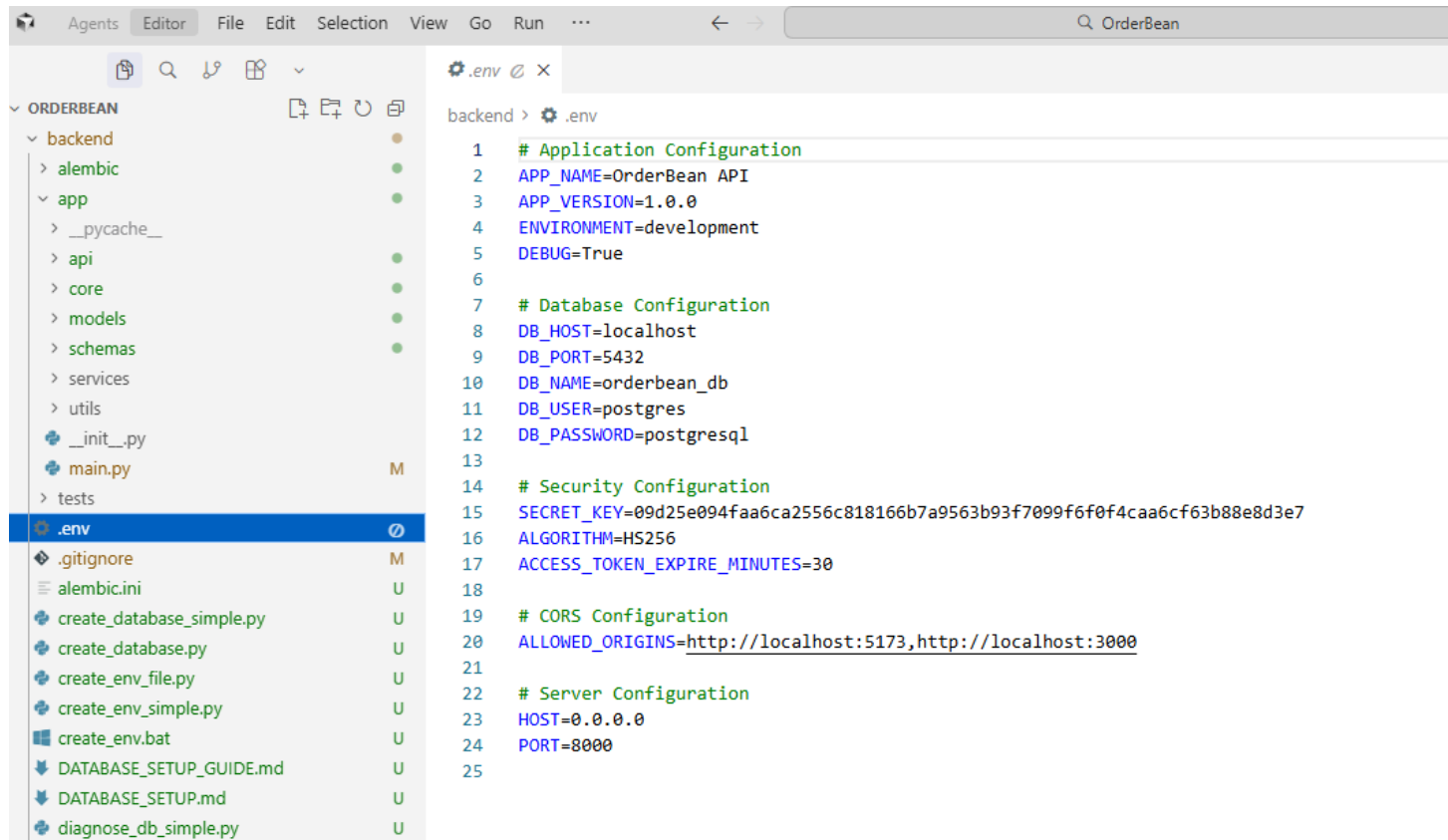
.....

PSQL Command

.....

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

`backend/.env` 파일



```
1 # Application Configuration
2 APP_NAME=OrderBean API
3 APP_VERSION=1.0.0
4 ENVIRONMENT=development
5 DEBUG=True
6
7 # Database Configuration
8 DB_HOST=localhost
9 DB_PORT=5432
10 DB_NAME=orderbean_db
11 DB_USER=postgres
12 DB_PASSWORD=postgresql
13
14 # Security Configuration
15 SECRET_KEY=09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7
16 ALGORITHM=HS256
17 ACCESS_TOKEN_EXPIRE_MINUTES=30
18
19 # CORS Configuration
20 ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000
21
22 # Server Configuration
23 HOST=0.0.0.0
24 PORT=8000
25
```

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

DB_HOST=localhost 를

DB_HOST=dpg-d48phdggjchc73f199n0-a.oregon-postgres.render.com 로 변경합니다.

실습 2단계 - PostgreSQL 데이터베이스 만들고 연결하기

DB_PORT=5432

DB_NAME=orderbean_db

DB_USER=postgres

DB_PASSWORD=postgresql 도 변경

DB_HOST=dpg-d48phdggjchc73f199n0-a.oregon-
postgres.render.com

DB_PORT=5432

DB_NAME=orderbean_db

DB_USER=orderbean_db_user

DB_PASSWORD=4hA>

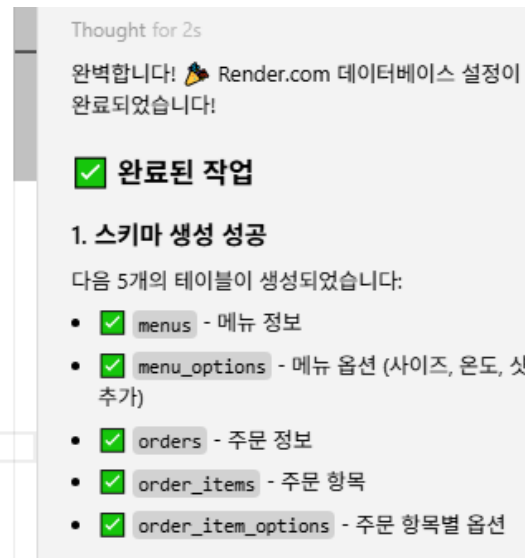
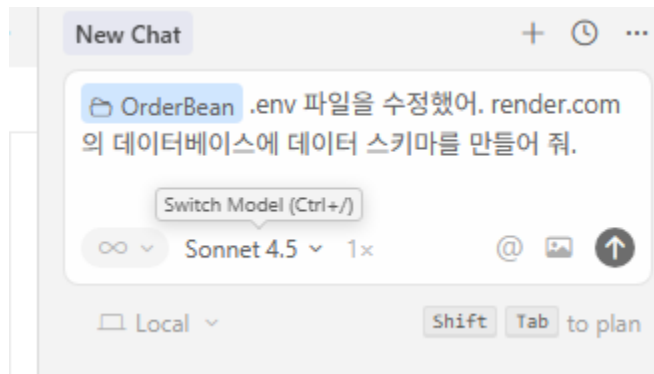
|CH0

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

Cursor.AI에게 스키마 생성을 요청합니다.

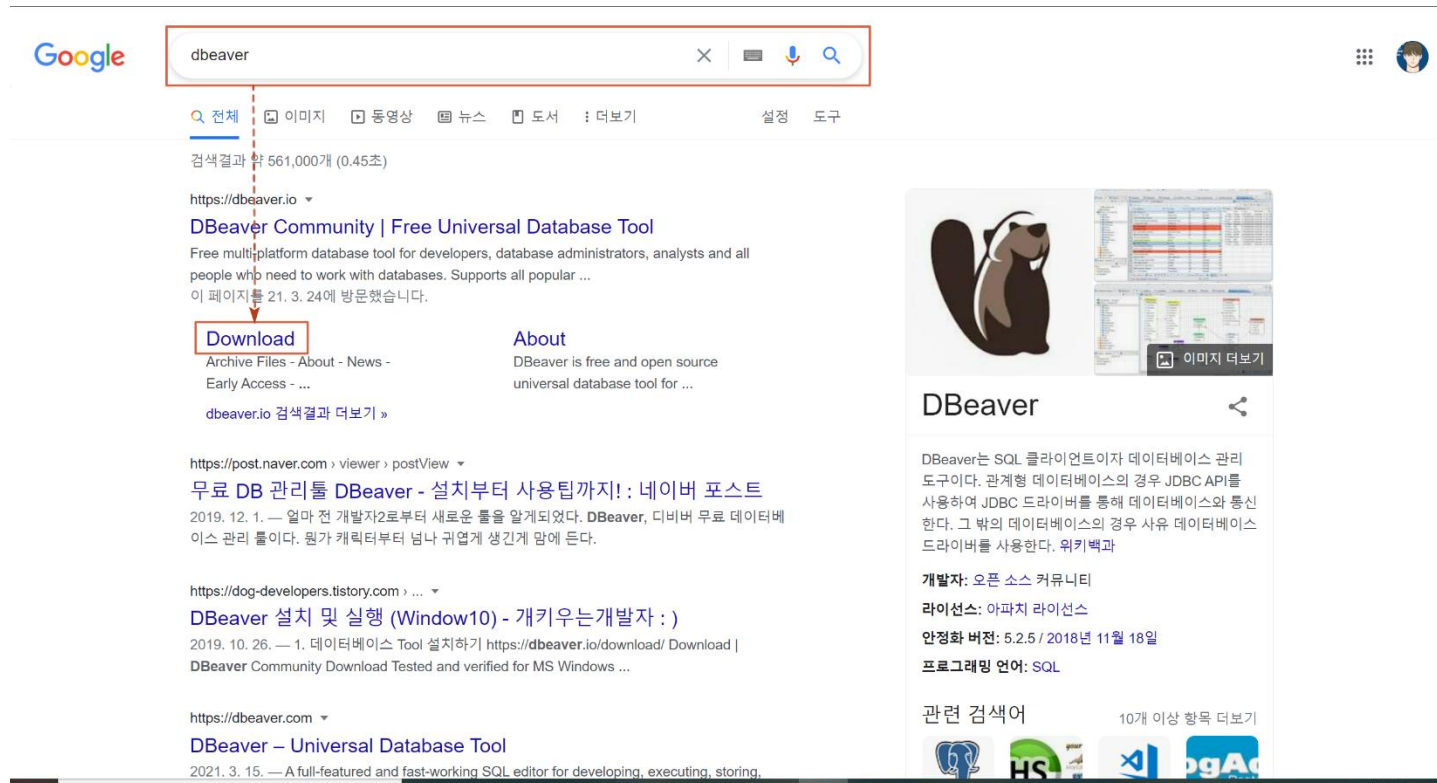
...

.env 파일을 수정했어. render.com의 데이터베이스에 데이터 스키마를 만들어 줘.



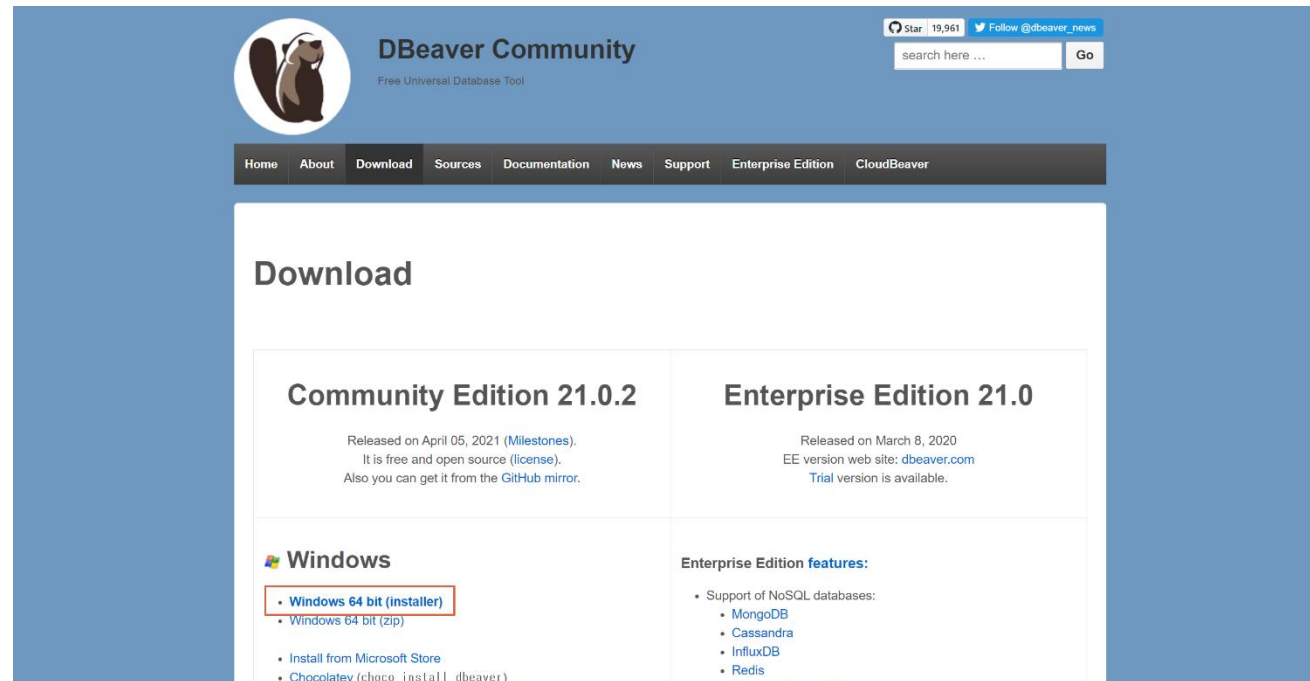
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

DBeaver Community 설치 및 PostgreSQL 연결
검색창에 dbeaver라고 검색합니다. 검색된 결과에서 Download를 클릭합니다.



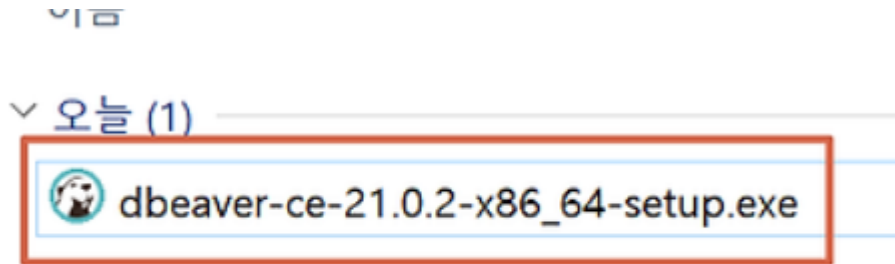
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

접속한 <https://dbeaver.io/download> 에서 자신의 OS에 맞는 설치 파일을 다운로드합니다. (여기서는 Windows 버전으로 진행 합니다.)



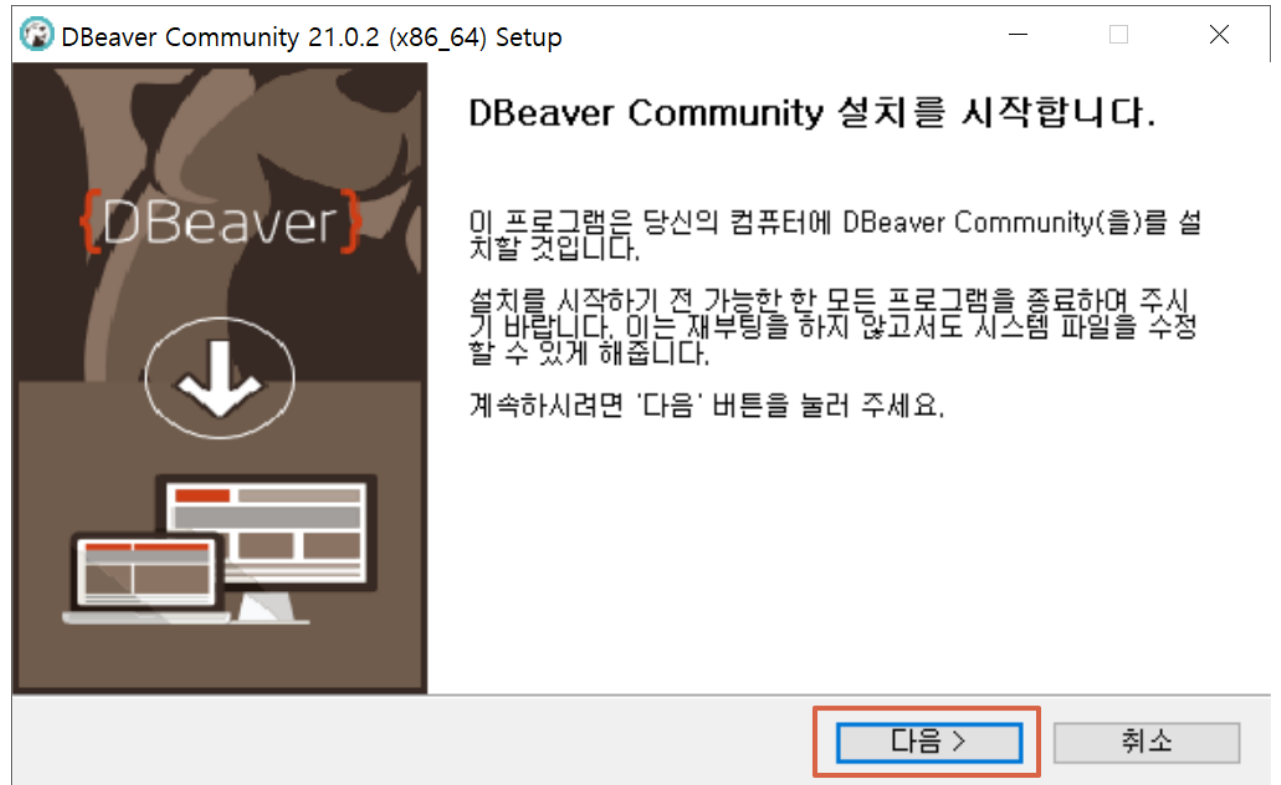
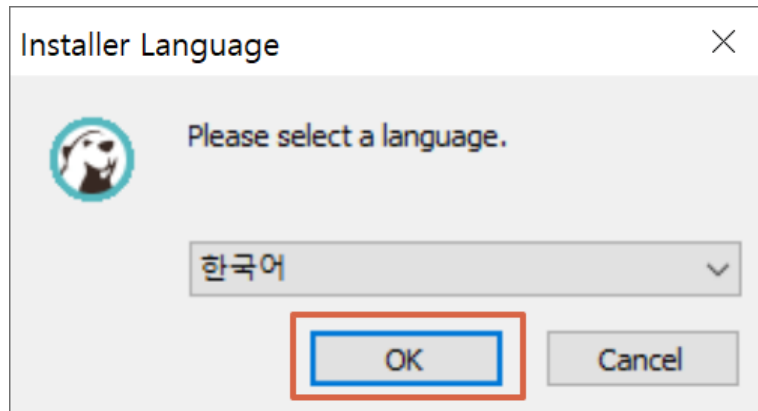
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

다운로드 폴더에서 다운로드된 설치 파일을 실행합니다.



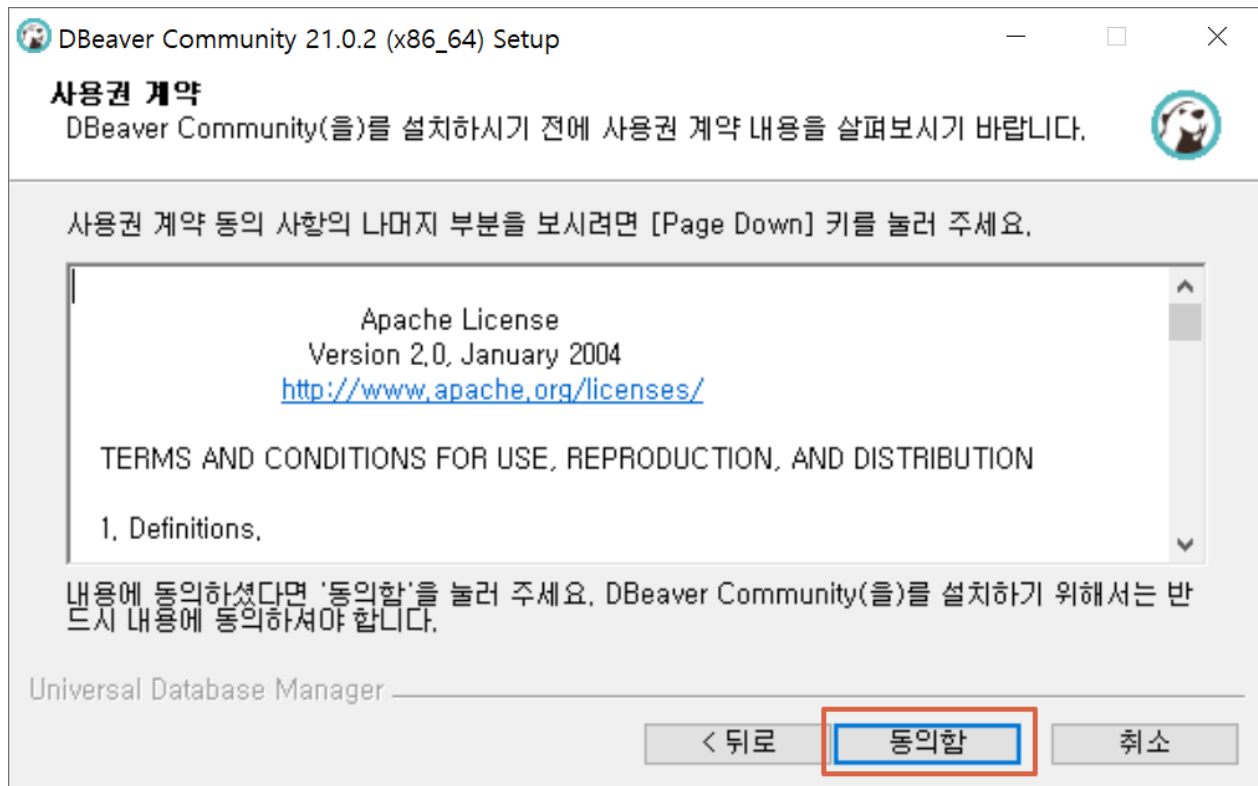
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

설치가 시작되면서 언어 선택창을 나타냅니다. 한국어로 선택하고 OK 버튼을 클릭합니다.



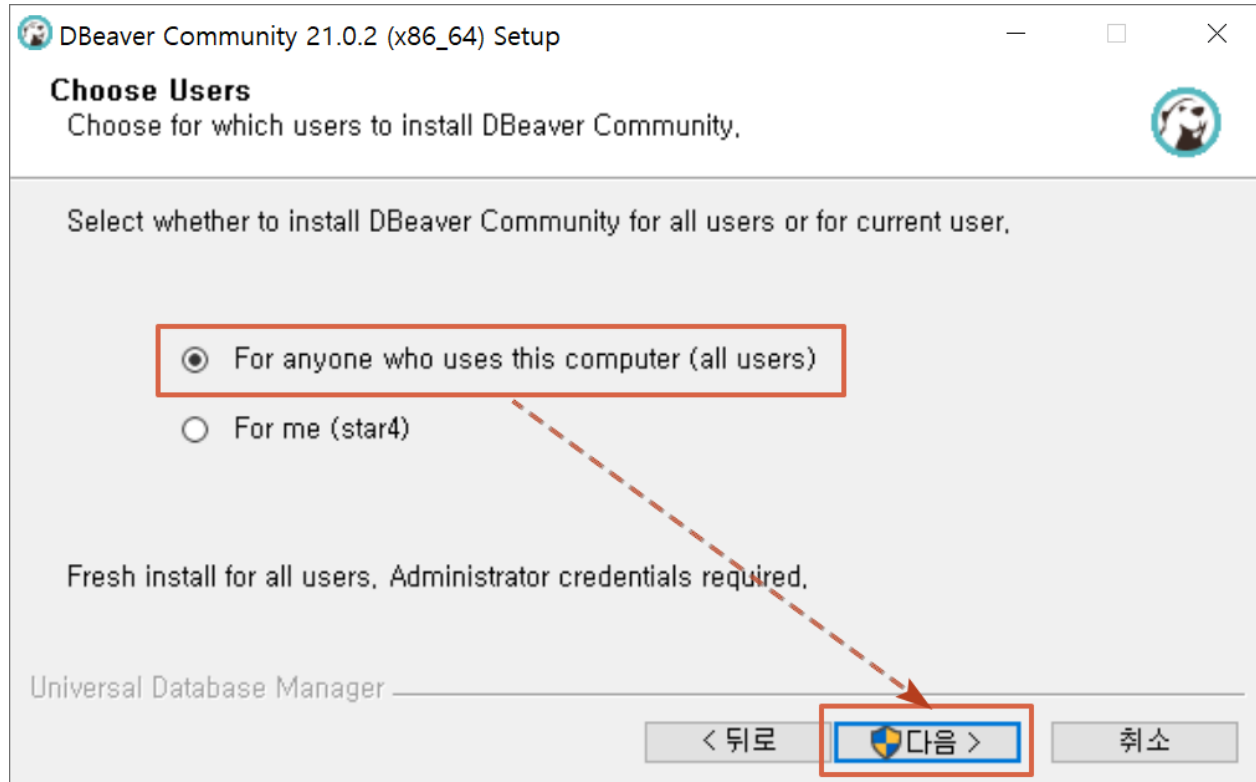
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

사용권 계약을 읽고 동의함 버튼을 클릭합니다.



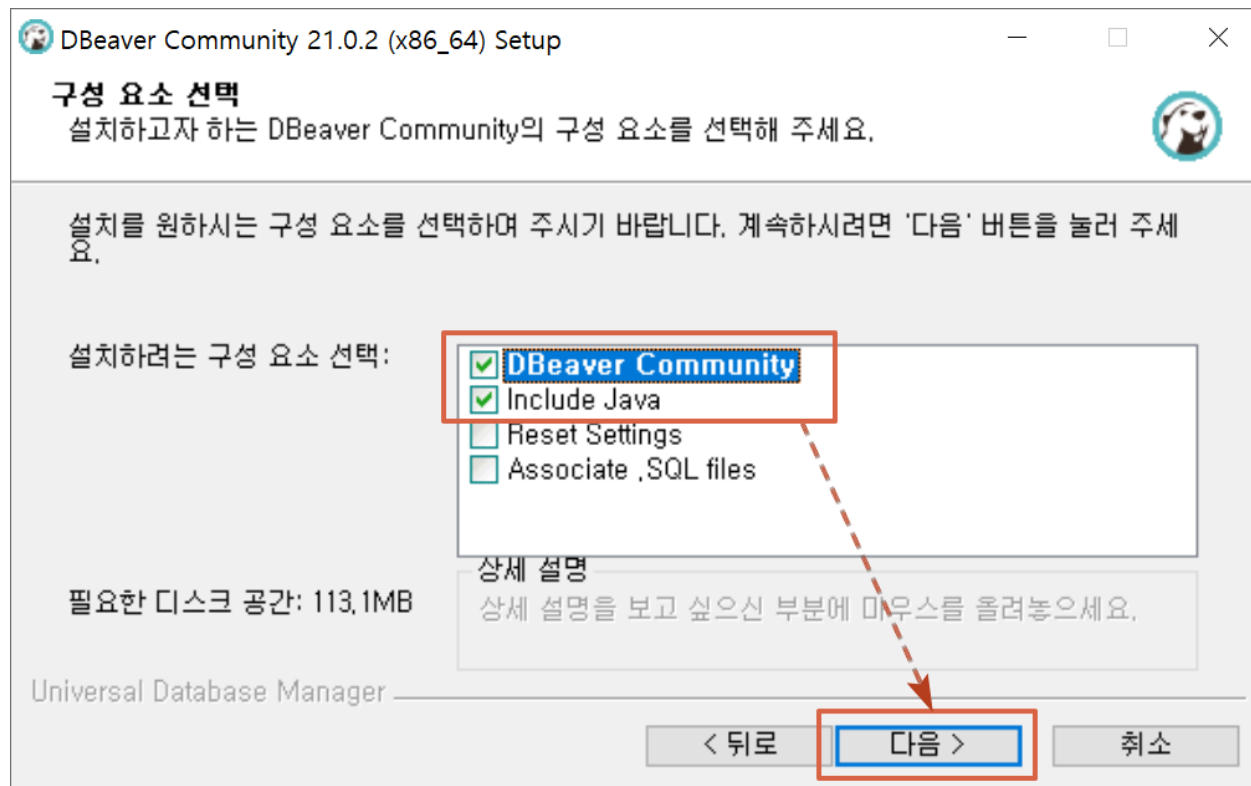
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

모든 사용자로 선택하고 다음 버튼을 클릭합니다.



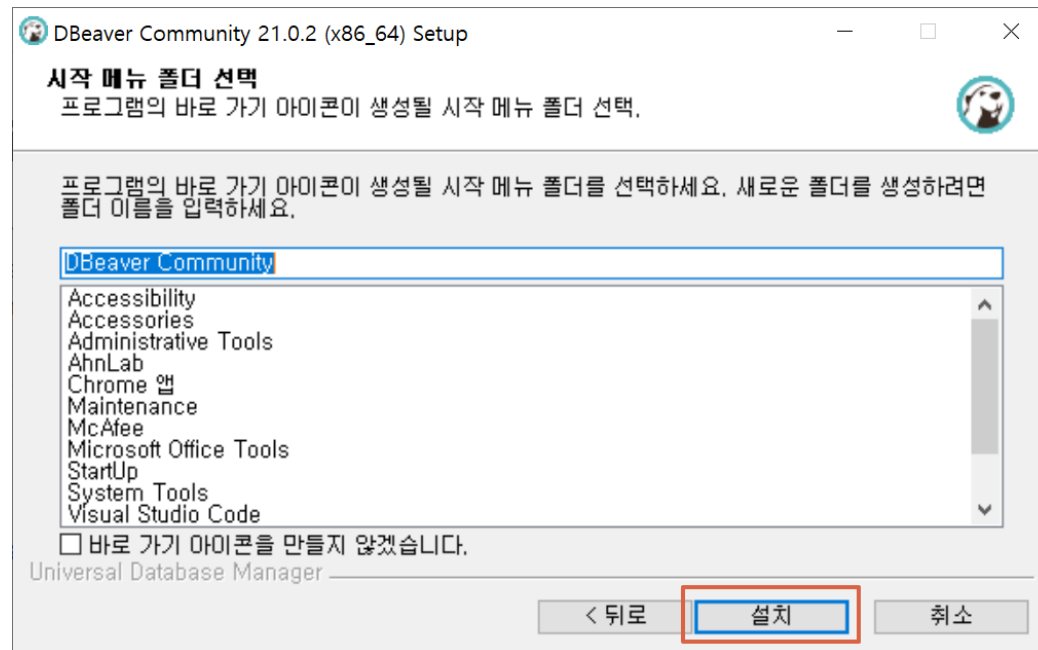
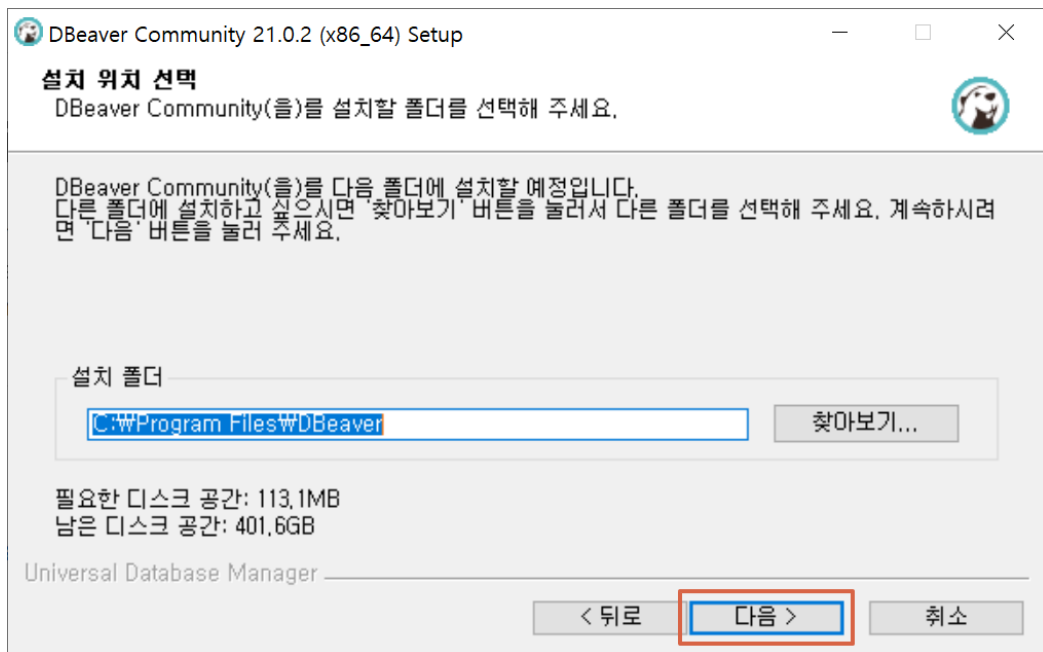
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

구성 요소 선택 화면에서 Default로 선택된 내용 확인 후 다음 버튼을 클릭합니다.



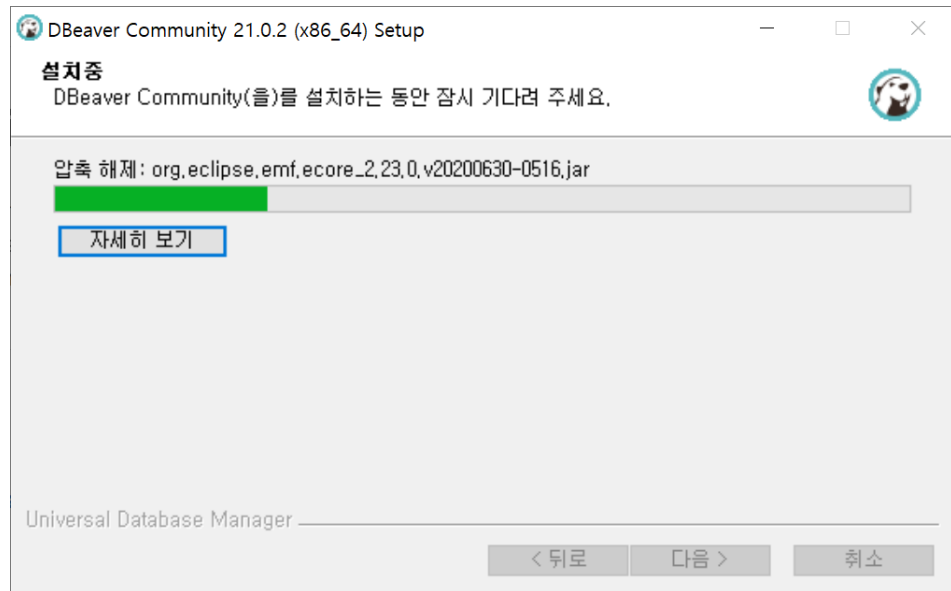
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

설치 위치 선택하면에서 Default 경로를 확인하고 다음 버튼을 클릭합니다.



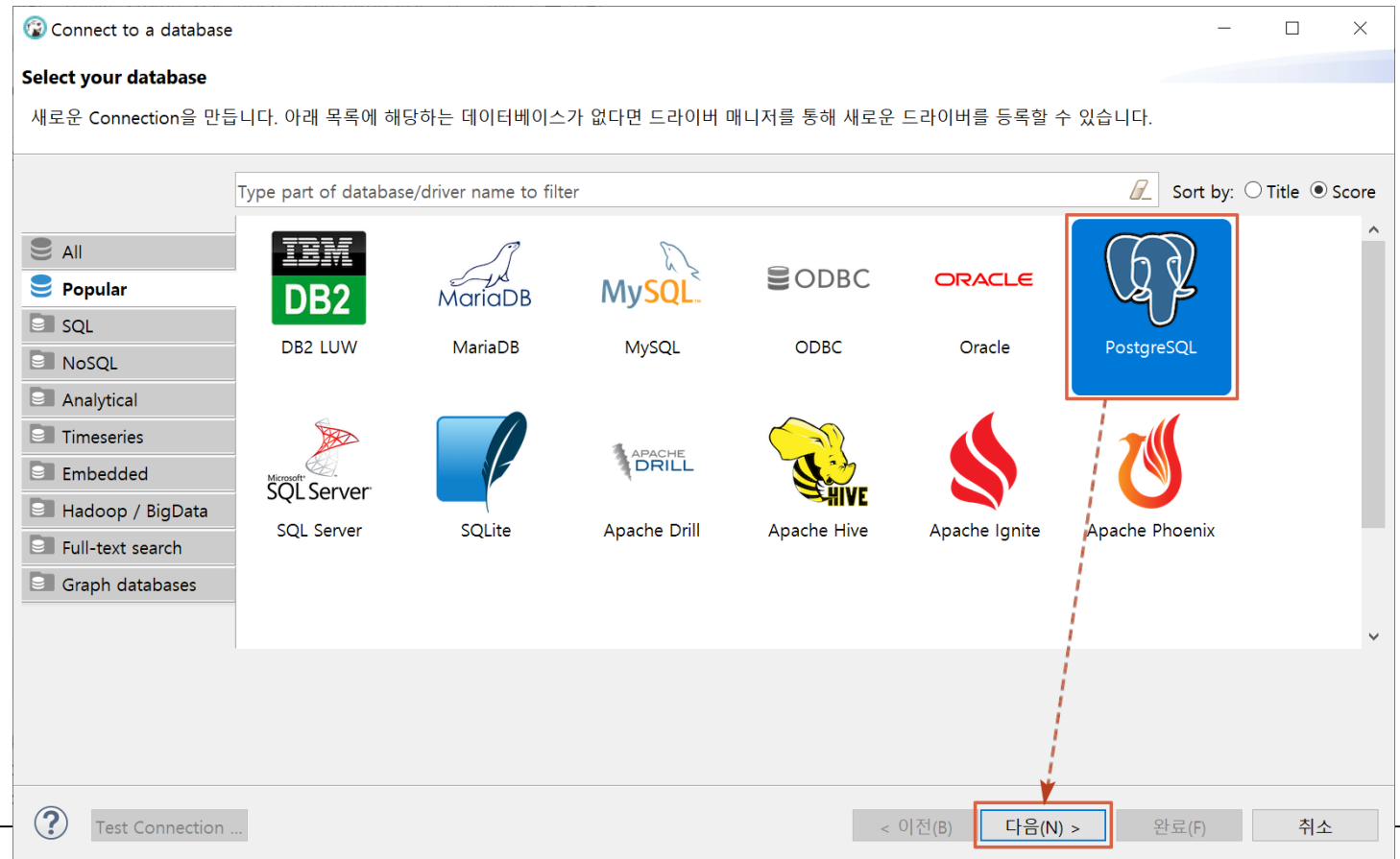
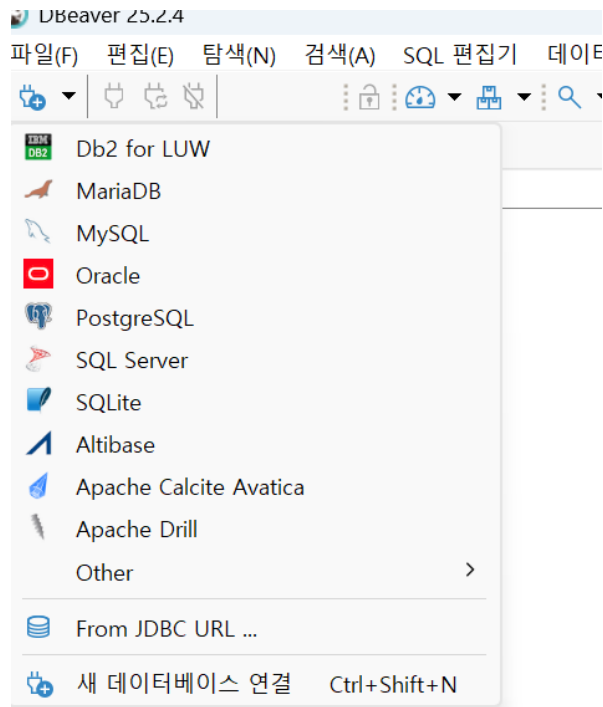
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

이제 설치가 진행됩니다.



실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

여러 종류의 database를 할 수 있으나 이번 예제에서는 PostgreSQL로 진행해 보겠습니다. PostgreSQL를 선택 후 다음 버튼을 클릭합니다.



실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

DB 연결 설정을 하는 창이 나타납니다. 연결할 DB의 Host, Port, Database, Username, Password 정보를 입력합니다.

- 1) Test Connection... 버튼을 클릭합니다.
- 2) Edit Driver Settings 버튼을 클릭합니다.
- 3) 완료 버튼을 클릭합니다.

Connect to a database

DB 연결 설정

PostgreSQL connection settings

Main Advanced Driver properties

Server

Connect by: ☒ Host ☐ URL

URL: jdbc:postgresql://dpg-d48phdggjchc73f199n0-a.oregon-postgres.render.com:5432/orderbean_db

Host: dpg-d48phdggjchc73f199n0-a.oregon-postgres.render.com Port: 5432

Database: orderbean_db ☐ Show all databases

Authentication

Authentication: Database Native

Username: orderbean_db_user

Password: Save password

Connection variables information

PostgreSQL Connection details (name, type, ...)

Driver name: PostgreSQL

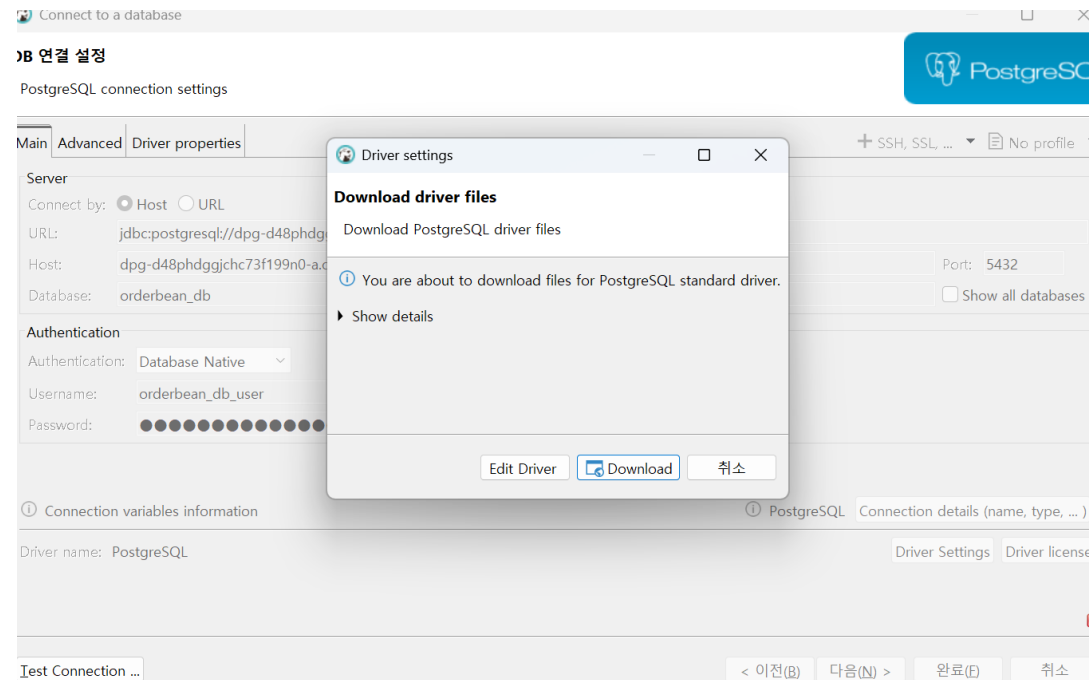
Driver Settings Driver license

Test Connection ...

< 이전(B) 다음(N) > 완료(F) 취소

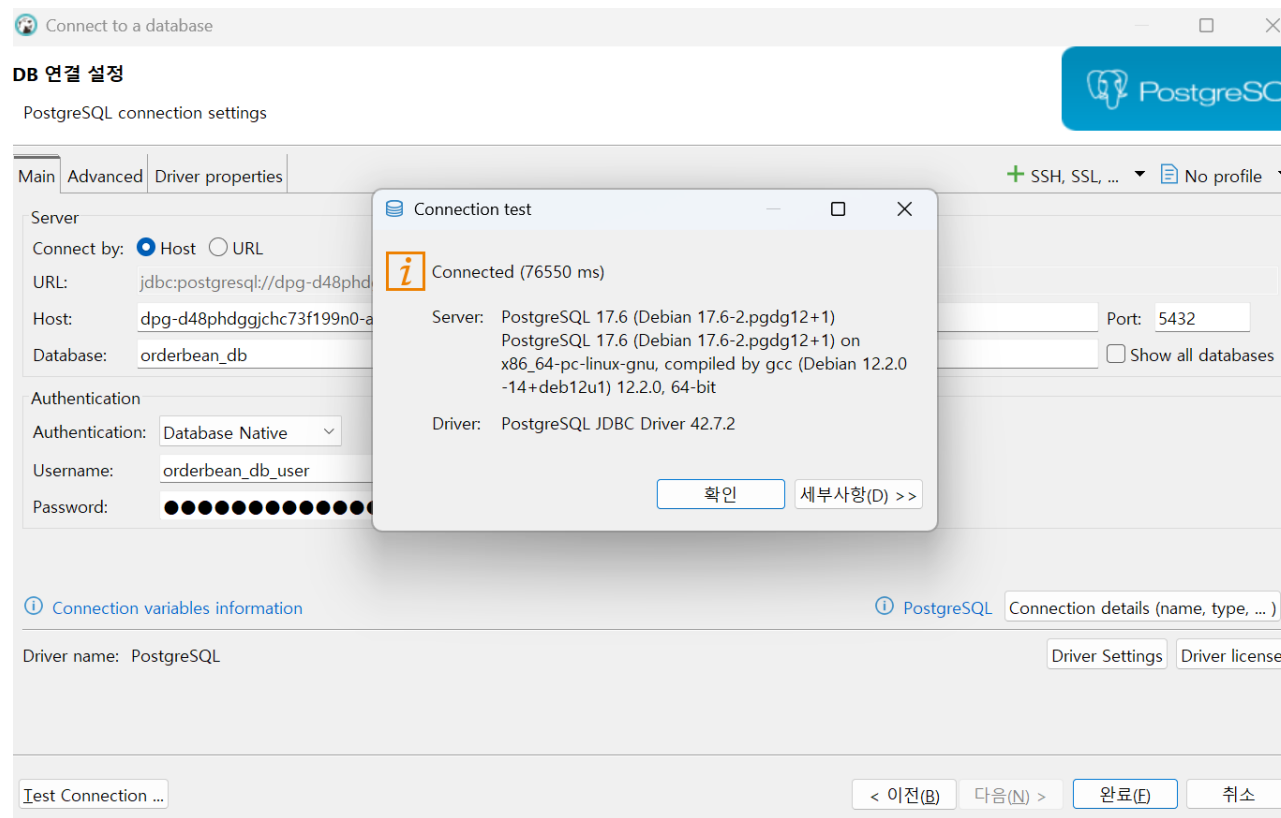
실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

선택한 database의 드라이버가 없으면 아래와 같이 자동으로 드라이버 다운로드 창이 뜨고 Download 버튼을 클릭하면 드라이버를 다운로드하고 끝나면 창이 닫힙니다.



실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

Test Connection 이 정상적으로 되면 아래와 같은 창이 뜨고, 확인 버튼을 클릭합니다. 완료버튼을 누릅니다.



실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

이제 database와 연결이 되었습니다. 아래와 같이 SQL 편집기를 새로 열고, SQL문을 작성한 후 실행 결과를 확인할 수 있습니다

The screenshot shows the DBeaver 25.2.4 interface. The top toolbar includes options for File (F), Edit (E), View (V), Check (C), SQL Editor (S), Database (D), Window (W), and Help (H). The main window displays a SQL script with the query `SELECT * FROM pg_catalog.pg_tables;`. Below the script, the results of the query are shown in a table with columns: schemaname, tablename, tableowner, tablespace, hasindexes, and hasrules. The table lists various system tables in the public schema, including pg_catalog, pg_statistic, pg_type, pg_foreign_table, pg_authid, pg_statistic_ext_data, pg_user_mapping, pg_subscription, pg_attribute, pg_proc, and pg_class. The status bar at the bottom indicates that 73 rows were fetched in 0.1s on 2025-11-11 at 08:43:13.

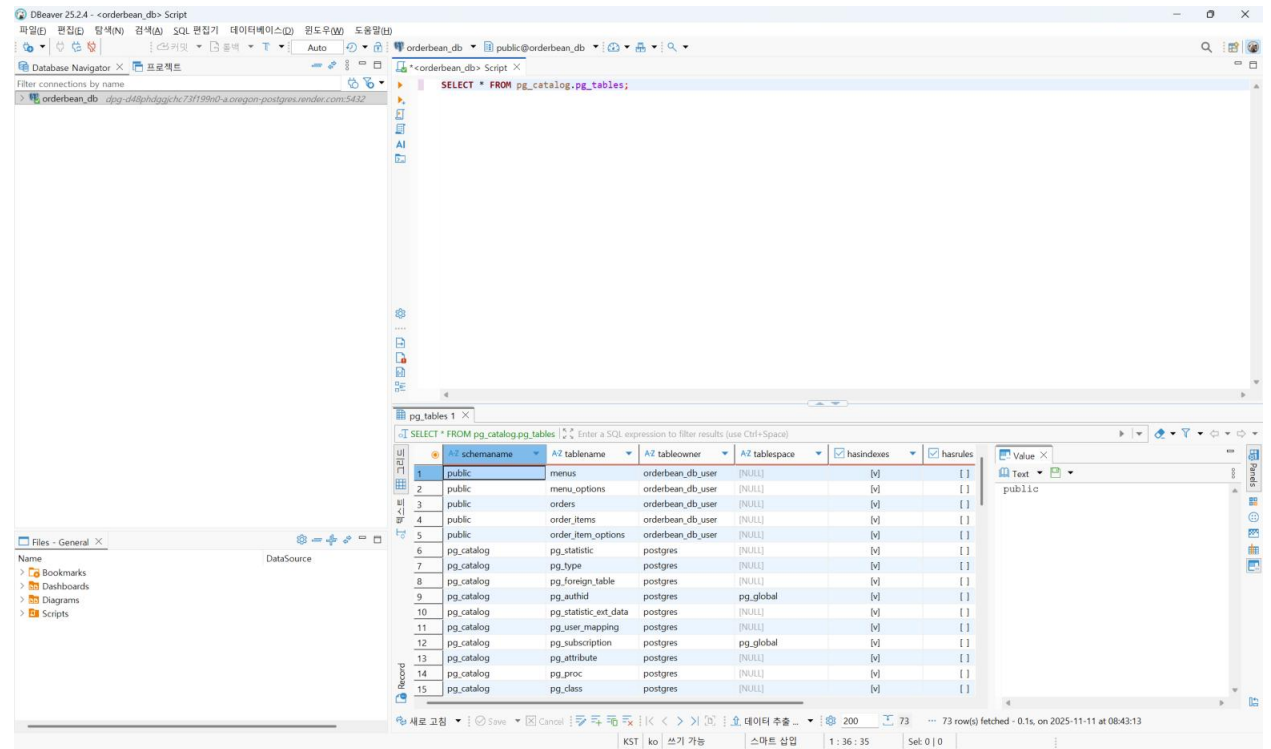
| | schemaname | tablename | tableowner | tablespace | hasindexes | hasrules |
|----|------------|-----------------------|-------------------|------------|------------|----------|
| 1 | public | menus | orderbean_db_user | [NULL] | [M] | [] |
| 2 | public | menu_options | orderbean_db_user | [NULL] | [M] | [] |
| 3 | public | orders | orderbean_db_user | [NULL] | [M] | [] |
| 4 | public | order_items | orderbean_db_user | [NULL] | [M] | [] |
| 5 | public | order_item_options | orderbean_db_user | [NULL] | [M] | [] |
| 6 | pg_catalog | pg_statistic | postgres | [NULL] | [M] | [] |
| 7 | pg_catalog | pg_type | postgres | [NULL] | [M] | [] |
| 8 | pg_catalog | pg_foreign_table | postgres | [NULL] | [M] | [] |
| 9 | pg_catalog | pg_authid | postgres | pg_global | [M] | [] |
| 10 | pg_catalog | pg_statistic_ext_data | postgres | [NULL] | [M] | [] |
| 11 | pg_catalog | pg_user_mapping | postgres | [NULL] | [M] | [] |
| 12 | pg_catalog | pg_subscription | postgres | pg_global | [M] | [] |
| 13 | pg_catalog | pg_attribute | postgres | [NULL] | [M] | [] |
| 14 | pg_catalog | pg_proc | postgres | [NULL] | [M] | [] |
| 15 | pg_catalog | pg_class | postgres | [NULL] | [M] | [] |

실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

이제 database와 연결이 되었습니다. 아래와 같이 SQL 편집기를 새로 열고, SQL문을 작성한 후 실행 결과를 확인할 수 있습니다

스키마 전체에 대한 테이블 조회 방법

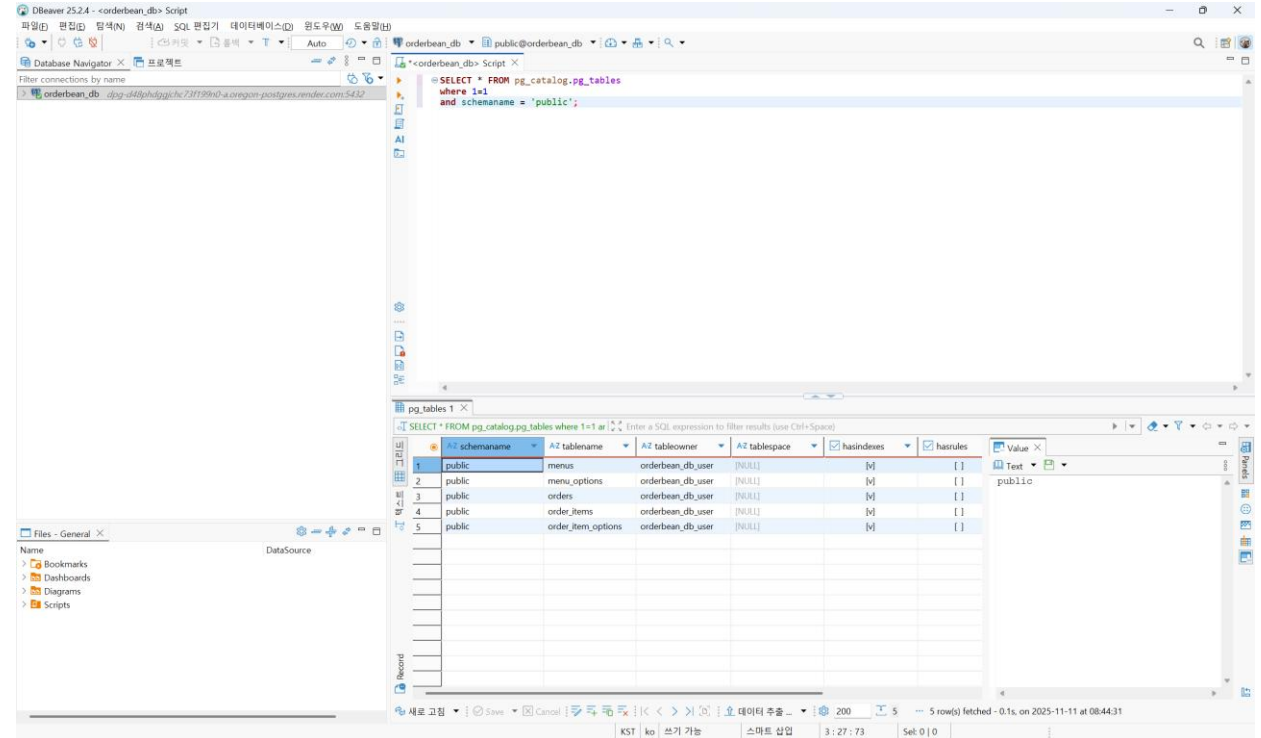
```
SELECT * FROM pg_catalog.pg_tables;
```



실습 2단계 – PostgreSQL 데이터베이스 만들고 연결하기

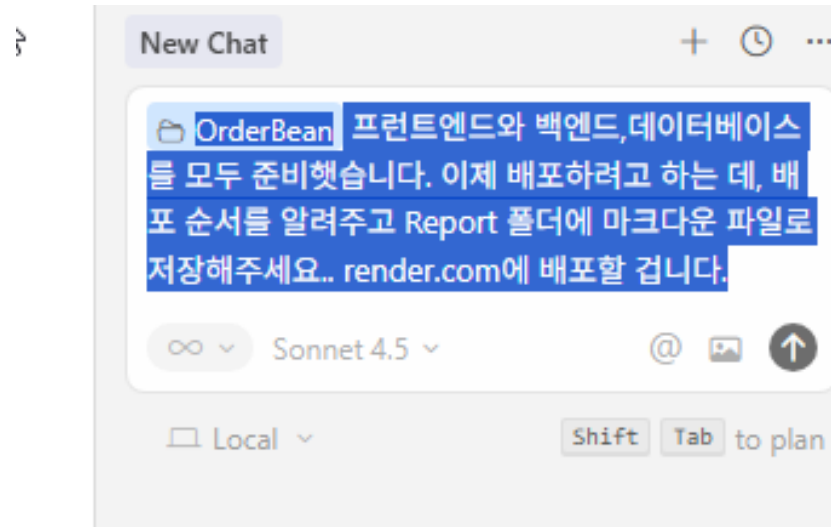
(기본)public 스키마에 대한 테이블 조회 방법

```
SELECT * FROM pg_catalog.pg_tables  
where 1=1  
and schemaname = 'public';
```



실습 3단계 - 백엔드 서비스 배포하기

@OrderBean 프론트엔드와 백엔드,데이터베이스를 모두 준비했습니다. 이제 배포하려고 하는 데, 배포 순서를 알려주고 Report 폴더에 마크다운 파일로 저장해주세요.. render.com에 배포할 겁니다.



실습 3단계 - 백엔드 서비스 배포하기

+Add new > Web Service

The screenshot shows the AWS Management Console interface for the 'Projects' section. The left sidebar contains navigation links: 'Projects' (selected), 'Blueprints', 'Environment Groups', and a section for 'INTEGRATIONS' including 'Observability', 'Webhooks', and 'Notifications'. The main content area is titled 'Overview' and 'Projects'. Under 'Projects', there is a card for 'My project' with a green status bar indicating 'All services are up and running', and a dashed box with the text '+ Create new project'. On the right side of the console, there are buttons for '+ New', 'Upgrade', and a help icon. Below these, there is a link 'Invite your team' and a dropdown menu labeled '+ Add new ^'. The dropdown menu is open, showing a list of service types: 'State Site', 'Web Service', 'Private Service', 'Background Worker', and 'Cron Job'. The 'Web Service' option is highlighted.

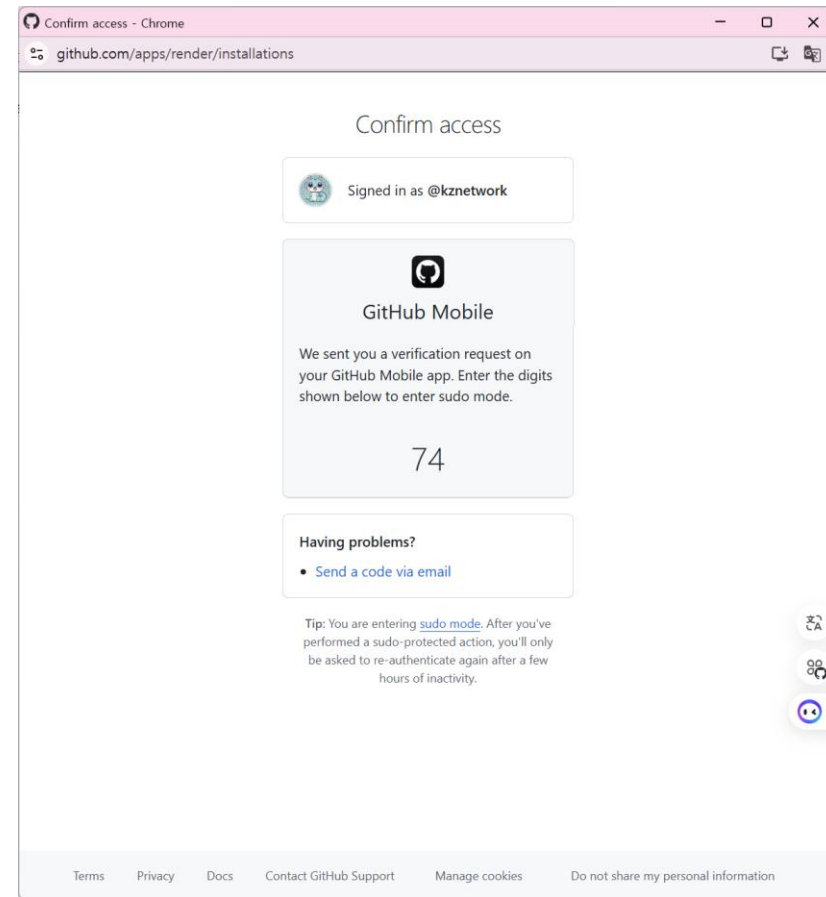
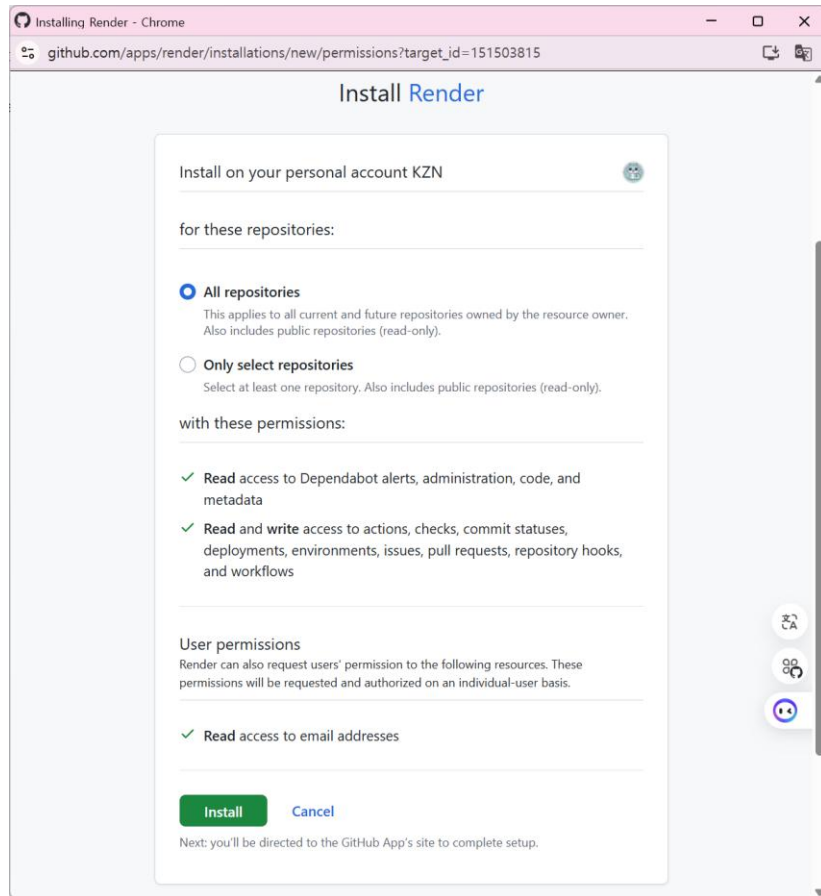
실습 3단계 - 백엔드 서비스 배포하기

Git Provide > GitHub

The screenshot shows the 'New Web Service' configuration interface. At the top, there's a navigation bar with 'My Workspace' and 'New Web Service' tabs. Below this, the 'Source Code' section is active, displaying three tabs: 'Git Provider' (highlighted with a blue box), 'Public Git Repository', and 'Existing Image'. The main content area shows a message 'No repositories found' with the instruction 'Connect your Git provider to deploy from your existing repositories.' Below this, there are buttons for 'GitLab' and 'Bitbucket'. A blue arrow points from the 'Git Provider' tab to a 'GitHub' button, which is also highlighted with a blue box. The text 'Configure your Git provider to give Render permission to access your repositories.' is positioned above the GitHub button.

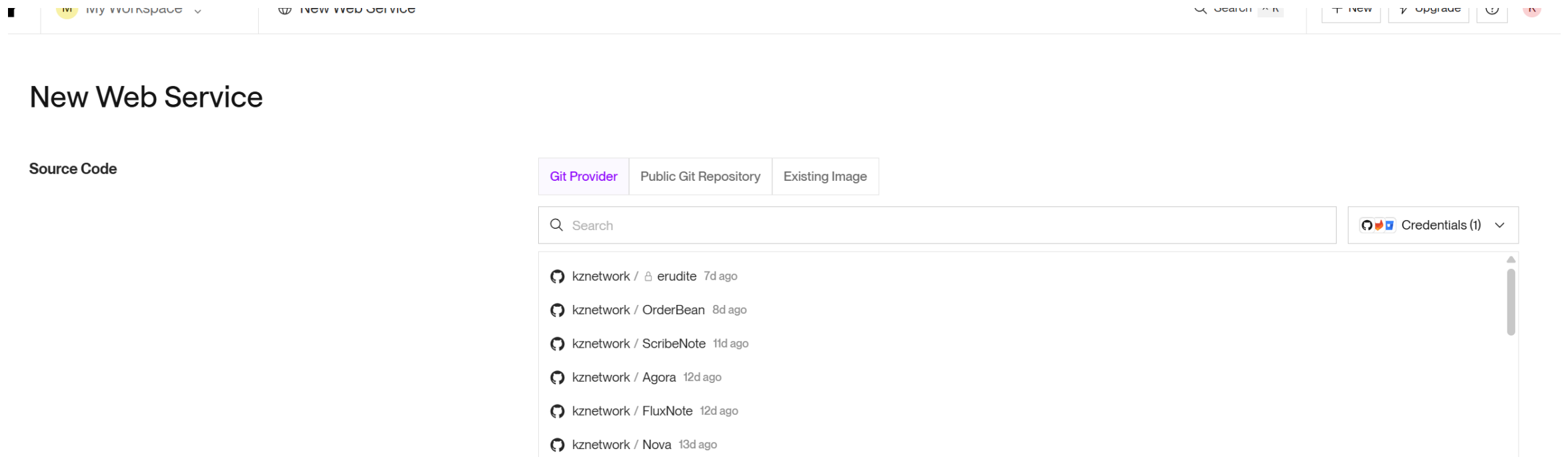
실습 3단계 - 백엔드 서비스 배포하기

Render에서 접근할 수 있도록 Install 클릭 > 깃 모바일 설치해 있어야 함.



실습 3단계 - 백엔드 서비스 배포하기

Render가 깃허브의 원격 저장소 목록을 가져와 화면에 표시해 줍니다. 배포해 사용할 원격 저장소를 클릭합니다.



실습 3단계 - 백엔드 서비스 배포하기

다른 서비스와 쉽게 구별할 수 있도록 Name > OrderBean-backend 로 , 나머지는 커서가 알려 주는 값을 입력해야 합니다.

```
## 2단계: 백엔드 (FastAPI) 배포
```

```
### 2-1. Web Service 생성
```

```
1. **Render Dashboard** → **New +** → **Web Service**
```

```
2. **Repository 연결**:  
- GitHub 계정 연동  
- OrderBean Repository 선택  
- Branch: `main` 또는 `master`
```

```
3. **서비스 정보 입력**:
```

```
```\nName: orderbean-backend\nRegion: Singapore (데이터베이스와 동일 지역 권장)\nBranch: main\nRoot Directory: backend\nRuntime: Python 3\n```
```

```
2-2. Build & Start 명령어 설정
```

```
Build Command:
```bash  
pip install -r requirements.txt  
```
```

```
Start Command:
```bash  
uvicorn app.main:app --host 0.0.0.0 --port $PORT  
```
```

## 실습 3단계 – 백엔드 서비스 배포하기

 My Workspace

 New Web Service

Q Search

+ New

Upgrade

?

K

**Select a service type**

Identify the right **service type** for your use case.

**Name**

A unique name for your web service.

**Project** Optional

Add this web service to a **project** once it's created.

**Language**

Choose the **runtime environment** for this service.

**Branch**

The Git branch to build and deploy.

**Region**

Your services in the same **region** can communicate over a **private network**. You currently have services running in **Oregon**.

**Root Directory** Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a **monorepo**.

**Build Command**

Render runs this command to build your app before each deploy.

**Start Command**

Render runs this command to start your app with each deploy.

Web Service

OrderBean-backend

Select a project...

/

Select an environment...

Python 3

main

Oregon (US West)

1 existing service

Deploy in a new region +

backend

backend/ \$ pip install -r requirements.txt

backend/ \$ uvicorn app.main:app --host 0.0.0.0 --port \$PORT

🔍

🌐

🔧

🔗

## 실습 3단계 – 백엔드 서비스 배포하기

### Instance Type > Free

#### Instance Type

##### For hobby projects

**Free**  
\$0 / month

512 MB (RAM)  
0.1 CPU

##### ⚠ Upgrade to enable more features

Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.

##### For professional use

For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

**Starter**  
\$9 / month

512 MB (RAM)  
0.5 CPU

**Standard**  
\$25 / month

2 GB (RAM)  
1 CPU

## 실습 3단계 - 백엔드 서비스 배포하기

### Environment Variables > Add from .env

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

NAME\_OF\_VARIABLE

value

+ Add Environment Variable

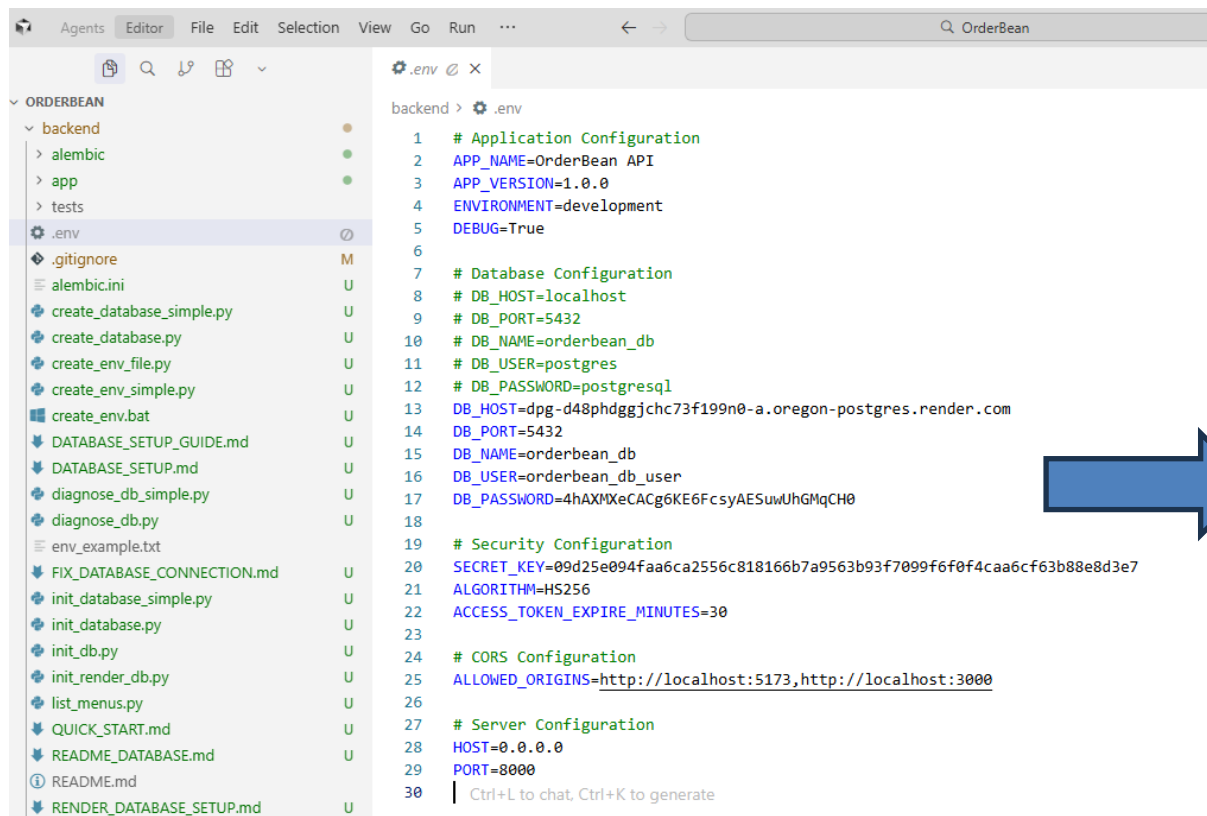
Add from .env

Generate



## 실습 3단계 - 백엔드 서비스 배포하기

Cursor.AI에서 .env 파일을 연 후, 그 안의 내용을 모두 선택해서 복사합니다.



```
1 # Application Configuration
2 APP_NAME=OrderBean API
3 APP_VERSION=1.0.0
4 ENVIRONMENT=development
5 DEBUG=True
6
7 # Database Configuration
8 # DB_HOST=localhost
9 # DB_PORT=5432
10 # DB_NAME=orderbean_db
11 # DB_USER=postgres
12 # DB_PASSWORD=postgresql
13 DB_HOST=dpg-d48phdggjchc73f199n0-a.oregon-postgres.render.com
14 DB_PORT=5432
15 DB_NAME=orderbean_db
16 DB_USER=orderbean_db_user
17 DB_PASSWORD=4hAXMXeCACg6KE6FcSyAESumUhGMqCH0
18
19 # Security Configuration
20 SECRET_KEY=09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7
21 ALGORITHM=HS256
22 ACCESS_TOKEN_EXPIRE_MINUTES=30
23
24 # CORS Configuration
25 ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000
26
27 # Server Configuration
28 HOST=0.0.0.0
29 PORT=8000
30
```

### Add from .env

Paste your .env contents to add multiple environment variables at once. Read the [docs](#) for correct syntax.

```
CORS Configuration
ALLOWED_ORIGINS=http://localhost:5173,http://localhost:3000

Server Configuration
HOST=0.0.0.0
PORT=8000
```

[Choose a file](#)

Add variables

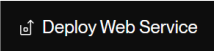
Cancel

## 실습 3단계 – 백엔드 서비스 배포하기

입력된 정보 확인 후, Deploy Web Service를 클릭합니다.

**Environment Variables**  
Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

|                             |       |   |
|-----------------------------|-------|---|
| APP_NAME                    | ..... | 🗑 |
| APP_VERSION                 | ..... | 🗑 |
| ENVIRONMENT                 | ..... | 🗑 |
| DEBUG                       | ..... | 🗑 |
| DB_HOST                     | ..... | 🗑 |
| DB_PORT                     | ..... | 🗑 |
| DB_NAME                     | ..... | 🗑 |
| DB_USER                     | ..... | 🗑 |
| DB_PASSWORD                 | ..... | 🗑 |
| SECRET_KEY                  | ..... | 🗑 |
| ALGORITHM                   | ..... | 🗑 |
| ACCESS_TOKEN_EXPIRE_MINUTES | ..... | 🗑 |

 Deploy Web Service Cancel

**Starter**  
\$9 / month

## 실습 3단계 – 백엔드 서비스 배포하기

<https://orderbean-backend.onrender.com> 은 프론트엔드 배포 시 필요

The screenshot shows the Render dashboard for a service named "OrderBean-backend". The left sidebar contains navigation links: Dashboard, OrderBean-backend, Events, Settings, MONITOR (Logs, Metrics), and MANAGE (Environment, Shell, Scaling, Previews, Disks, Jobs). The main content area shows the service details: "WEB SERVICE", "OrderBean-backend", "Python 3", "Free", and "Upgrade your instance →". It also displays the Service ID, the repository "kznetwork / OrderBean", and the build command "backend". The deployment status is "Failed" with the message "Exited with status 1 while building your code." and a link to "Upgrade now". The bottom section shows the live tail of the logs, which includes the message "[end of output]" and the timestamp "Nov 11 09:23:17 AM".

### 실습 4단계 - 프론트엔드 배포하기

Cursor.AI 에게 프론트엔드 배포 방법을 물어봅니다.

...

프론트엔드 코드를 Render에 배포하려고 합니다. 코드에서 수정할 부분과 배포 과정을 설명해주세요.

프론트엔드 코드를 Render에 배포하려고 합니다.  
드에서 수정할 부분과 배포 과정을 설명해주세요.

Switch Model (Ctrl+I)

## 실습 4단계 - 프론트엔드 배포하기

프론트엔드 수정이 다 끝나면, 깃허브로 푸시합니다.

```
🛠 배포 전 코드 수정

수정 사항: 없음! ✨

현재 코드는 이미 배포 준비가 잘 되어 있습니다:
- ✅ 환경 변수 사용
- ✅ 빌드 스크립트 설정
- ✅ API 서비스 구조화

필요한 것: 환경 변수 파일 생성

다음 3개의 환경 변수 파일을 생성하세요:

1. `.env.example` (예시용, Git 커밋)

```env
# API URL 설정
# 로컬 개발: http://localhost:8000
# 프로덕션: https://your-backend-api.onrender.com
VITE_API_URL=http://localhost:8000
```

2. `.env.development` (로컬 개발용)

```env
# 개발 환경 변수
VITE_API_URL=http://localhost:8000
```

3. `.env.production` (프로덕션용)

```env
# 프로덕션 환경 변수
# 백엔드 API URL (실제 백엔드 URL로 변경)
VITE_API_URL=https://orderbean-backend.onrender.com
```

** ⚠ 중요 **: `.env.production` 파일은 **Git에 커밋하지 마세요!** (gitignore 확인)

```

## 실습 4단계 - 프론트엔드 배포하기

### OrderBean-backend > New > Static Site

The screenshot shows the OnRender console interface. On the left is a sidebar with navigation links: Dashboard, OrderBean-backend, Events, Settings, MONITOR, and Loas. The main area displays details for the 'OrderBean-backend' service, including its ID, repository, and URL. A '+ New' button in the top right corner has a dropdown menu open, showing options like Static Site, Web Service, Private Service, Background Worker, Cron Job, and Postgres. A blue arrow points from the 'Static Site' option to the search results below. The search results show a list of repositories, with 'kznetwork / OrderBean' highlighted and a 'View repo' link next to it.

My Workspace ▾ OrderBean-backend ▾

← Dashboard

OrderBean-backend

Events

Settings

MONITOR

Loas

WEB SERVICE

**OrderBean-backend** Python 3 Free Upgrade your instance →

Service ID: srv-d4983amr433s73af0j0g

kznetwork / OrderBean backend

<https://orderbean-backend.onrender.com>

+ New Upgrade ?

Static Site

Web Service

Private Service

Background Worker

Cron Job

Postgres

Manual Deploy ▾

### New Static Site

Source Code

Git Provider Public Git Repository

Search

Credentials (1) ▾

kznetwork / erudite 8d ago

kznetwork / OrderBean 8d ago View repo

## 실습 4단계 - 프론트엔드 배포하기

### 4단계: 기본 설정

```

Name
orderbean-frontend
(또는 원하는 이름)

Region
Singapore (또는 가까운 지역)

Branch
main

Root Directory
frontend
(중요! 프로젝트 루트가 아님)

```

### 5단계: Build 설정

#### Build Command

```
```bash
npm install && npm run build
```
```

\*\*설명\*\*:

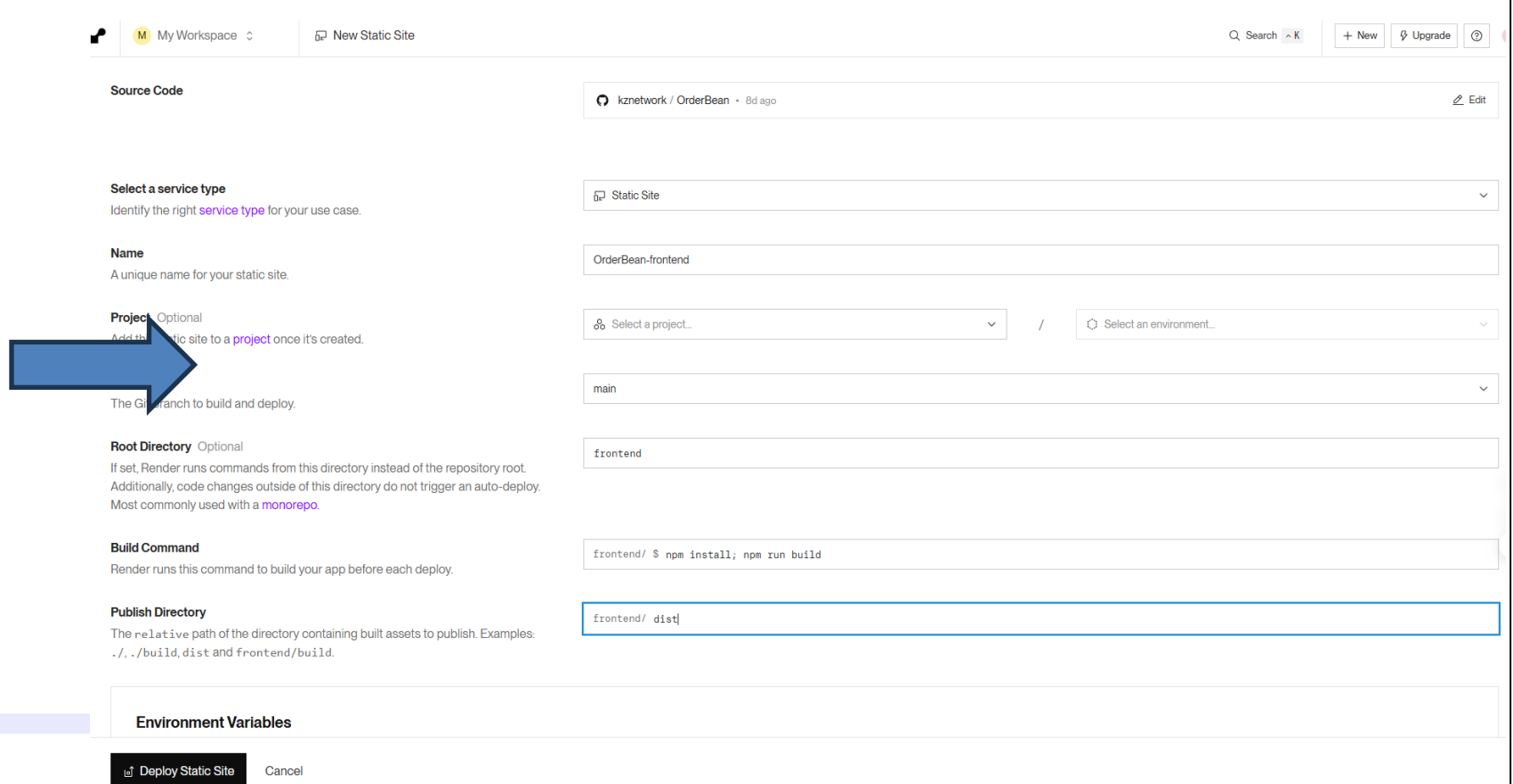
- `npm install`: 의존성 패키지 설치
- `npm run build`: TypeScript 컴파일 + Vite 빌드

#### Publish Directory

```
```bash
dist
```
```

\*\*설명\*\*:

Vite는 빌드 결과물을 `dist` 폴더에 생성합니다.



My Workspace New Static Site

Source Code

kznetwork / OrderBean · 8d ago

Select a service type

Identify the right service type for your use case.

Static Site

Name

A unique name for your static site.

OrderBean-frontend

Project

Optional

Add this static site to a project once it's created.

Select a project...

/

Select an environment...

main

frontend

Build Command

Render runs this command to build your app before each deploy.

frontend/ \$ npm install; npm run build

Publish Directory

The relative path of the directory containing built assets to publish. Examples: ../build, dist and frontend/build.

frontend/ dist

Environment Variables

Deploy Static Site Cancel

## 실습 4단계 - 프론트엔드 배포하기

My Workspace

OrderBean-frontend

Events

Settings

MONITOR

Metrics

MANAGE

Environment

Previews

Redirects/Rewrites

Headers

OrderBean-frontend

STATIC SITE

OrderBean-frontend

Service ID: srv-d498d0ogjchc73f98tn0

kznetwork / OrderBean main

<https://orderbean-frontend.onrender.com>

Manual Deploy

November 11, 2025 at 9:42 AM Failed

238b62b Main end ...

Exited with status 254 while building your code.

Read our docs for common ways to troubleshoot your deploy.

All logs Search

Nov 11 09:42:54 AM ==> Docs on specifying a Node.js version: https://render.com/docs/node-version

Nov 11 09:42:58 AM npm error code ENOENT



## 실습 5단계 - 유지보수

앱 배포 후, 수정 된 내용 깃 허브에 업로드하면 Render에서 자동으로 배포 됩니다. 만일 즉시 배포하고 싶은 경우, Manual deploy > deploy latest commi을 선택하면 됩니다.

The screenshot displays the Render dashboard for a service named 'OrderBean-backend'. The service is running on Python 3 and is currently on the 'Free' tier. A 'Manual Deploy' button is visible, which has been clicked to reveal a dropdown menu with the following options: 'Deploy latest commit', 'Deploy a specific commit', 'Clear build cache & deploy', and 'Restart service'. A warning message indicates that the free instance will spin down due to inactivity, potentially causing delays. The left sidebar provides navigation options, and the top bar shows the workspace and service name.

# Toy-project 예시 : ChatGPT 음성 기반 영어 학습 챗봇 만들기



## WB2\_Toy-project 예시 자료 참고해 작업할 수 있음

Toy-Projcet 예시.pptx

WB2\_Toy-project 예시-ChatGPT음성 비서 (Whisper + GPT + gTTS) - PRD v1.0

WB2\_Toy-project 예시-ChatGPT TDD 개발 - Cursor AI 단계별 프롬프트

# THANK YOU.

앞으로의 엔지니어는 단순한 '코더'나 '기계 조작자'가 아니라 뇌-기계 인터페이스를 통해 지식과 능력을 즉각 확장하는 존재(뉴로-인터페이스: Neuro Interface)가 될 수 있습니다.

- 🎯 목표 달성을 위한 여정이 시작됩니다.
- 🌟 궁금한 점이 있으시면 언제든지 문의해주세요!
- 🚀 함께 코더와 프롬프트 전문가로 성장해 나갑시다!

